GEMINI

Helix Constitution — Universal GEMINI.md

Field	Value
Revision	50
Created	2026-06-09
Last modified	2026-10-23T00:00:00Z
Status	active
Status summary	Amendment of 2026-10-23 — NEW §11.4.279–§11.4.286 (Jev decision-layer mandates) + §11.4.231 amendment paragraph, mirrored in lockstep; Amendment of 2026-10-21 (second) — §11.4.31 EXTENDED in place (a tracked, well-formed helix-deps.yaml in EVERY own-org repository incl. /submodules/* and /constitution/submodules/*, closed strings-only schema, root/hub coverage of own-org submodules, canonical engine scripts/deps_manifest/verify_helix_deps_manifests.sh with --scaffold, gate CM-HELIX-DEPS-MANIFEST-PRESENT implemented with a paired mutation test and wired into scripts/post_update_hook.sh against PROJECT_ROOT; user guide, manual, FAQ with diagrams). Prior round: Amendment of 2026-10-08 — §11.4.54 / §11.4.248 EXTENDED (not restated): the ticket-id key is per-project DATA (`ATM-NNN` is the default notation; engine key-grammar validation at mint/validate time and the atm_id -> ticket_id rename are owed engine work, not done; the WORDING is pending independent review

Field	Value
	and owner sight); paired with the codegraph_safe.sh MemAvailable-floor knob landing in the same change set (scripts/codegraph, no anchor text). Prior round: Amendment of 2026-10-12 (round 7, the last §11.4.276 budget round, closing the round-6 independent review NO-GO 0 blocking / 0 important / 4 minor / 1 nit, five findings only and no new structure: code paths are compared in one declared NORMALISED form — new (V3a): textual and symlink-free, `.` and empty segments dropped, `..` resolved, case-sensitive, segment by segment — and a scope-keyed row joins a contained path it equals, is an ancestor of or is a descendant of, while a scope that does not normalise is UNRESOLVABLE (X) instead of silently left out of the join (N29); the (F7.1) IDENTITY cell now reads 'an entry is in scope only when equal to (F1); an entry with no readable fingerprint is X' (N30); a missing stream position and an empty fingerprint are SEQUENCE DEFECTS, and every field of the input record is typed absent-able with the reference function and a field-by-field ABSENT enumerator proving each ABSENT cell (N27); the closure test resolves names per scope, whitelists the module body, forbids rebinding a builtin and probes a fresh import with the clock, environment and builtins instrumented (N28); a write-log mirror must be an in-scope record, now exercised by a cross-fingerprint mirror case (N31). Round 6 text follows — Amendment of 2026-10-12 (round 6, STRUCTURAL under §11.4.276(E), closing the round-5 independent review NO-GO 0 blocking / 1 important / 3 minor / 1 test-instrumentation minor / 1 nit; class SI a fourth time, so closed BY CONSTRUCTION: the (E.3)(c) FIELD TABLE is now GENERATED from the reference status function's ONE typed input record — every value the function reads is a named field of that record, every field is read, each carries SOURCE / VALID / ABSENT / ORDER / IDENTITY, and a test fails on an unlisted read, an ambient read, a missing column, an unread field or any byte of drift between this text and the generated table (N21); the rules once mixed into the table move to DERIVED VALUES (V1)-(V9); the record KIND, the contained CODE PATHS, store READABILITY as the runtime read, and the write-seam log are fields; creating a row and every hand-off are mirrored as `open` BIRTH records, so every entry is named from birth and a deleted row never drops it (N21); confirmation and seam records are VALID by role and order (N22); a status carries a VALID-UNTIL, the earliest expiry of a

Field	Value
	waiver it counts, and is re-evaluated at it (N23); a sequence defect holds only the fingerprints of the records involved, and a §11.4.268 repair with a recorded §11.4.66 decision quarantines it (N24). Round 5 text follows — Amendment of 2026-10-12 (round 5, STRUCTURAL under §11.4.276(E), closing the round-4 independent review NO-GO 0 blocking / 1 important / 4 minor / 1 nit; class SI again: the round-4 field list was open-ended and left entry IDENTITY and RECORD SCOPE unstated, so all three models shared an unstated last-declared-store-wins tie-break) — (E.3)(c)'s SINGLE AUTHORITATIVE SOURCE sentence is replaced by a CLOSED, numbered FIELD TABLE (F1)-(F12) that names every field the status reads and states that no unlisted fact (store declaration order, storage order, wall-clock time, a record label) ever decides a status, so a tie the table does not resolve is X; (F2) an entry is identified by its owning store and row key and has exactly one row carrying its class, an entry id matched in more than one row or in rows of different classes is X, and row keys are unique across stores (N15); absence-defined (L1)(L2)(L6)(L7) and polish (L5) entries own their tracked-item row, created on first match and never removed, so skipped and not-applicable resolve (N18); (F1) record scope is the delivered fingerprint except X's whole-stream sequence check (N16); only an entry's LATEST record can open it (N17); OWED-GATE-115/117 contracts and the paired-mutation list name the duplicate-row and store-order cases (N19). Round 4 text follows — Amendment of 2026-10-12 (round 4, STRUCTURAL under §11.4.276(E), closing the round-3 independent review NO-GO 0 blocking / 2 important / 4 minor; class named: status inputs with no closed entry universe and no single authoritative source per field) — (E.3) now names ONE source for every field the status reads: the closed ENTRY SET is the ledger join over every declared owning store UNION every entry id named by any disposition record for the fingerprint, so deleting a store row never resolves an entry (N9); ANY unreadable declared store, required or not, is an unresolvable input (N9); an entry's class comes only from its owning-store row, never from a label on a record, and a label that differs makes the record invalid (N10); a closed permitted-reason-code table covers all five dispositions, with free text moved to an evidence reference that closed and waived records must carry (N11); a record carries exactly one fingerprint and is never inherited, a scope-keyed

Field	Value
	write is mirrored once per already-delivered matching fingerprint and a later fingerprint gets no record from it (N12); OWED-GATE-115/116/117 contracts extended; enumerator rebuilt with a store model and a second independent implementation compared differentially (N13). Round 3 text follows — Amendment of 2026-10-12 (round 3, STRUCTURAL under §11.4.276(E), closing the round-2 independent review NO-GO 0 blocking / 2 important / 6 minor; class named: state-machine inputs with no ordering and no validity model) — (E.3) now models every status input as an EVENT RECORD in the §11.4.116 stream (fingerprint, single-writer sequence number, write time for audit only, seam-stamped writer role, closed kind set) and the status as a pure function of the valid records: a §11.4.185 confirmation counts only if it postdates every ledger record and a release-seam record only if it postdates that confirmation, so a reopen and its re-close void both (N1); a closed per-class permitted-disposition table (L7 §11.4.260 invariants and the L6 retest never skipped or waived; skip only with the topology-absent codes) and only the verifier seam may write a non-open disposition (N2); required vs optional stores (N3); `pending` computed from the inputs (N6); exhaustively enumerated (5.1 M evaluations, 0 failures, control needle on the round-2 model). Round 2 text follows — Amendment of 2026-10-12 (round 2, closing the round-1 independent review NO-GO 1 blocking / 4 important / 7 minor) — §11.4.230 EXTENDED in place with clause (E) (operator mandate 2026-10-12, `CRITICAL`, quick visible results; quoted exactly as typed with [sic], request R-2026-10-12-195857): a new scope's first-deliverable critical path is scheduled first while hardening starts in parallel (E.1); evidence is never traded for speed, §11.4.50 cited as written, and the permitted gaps are by construction exactly the ledger classes (E.2); every QA hand-off carries an open hardening ledger (read-only join; closed entry classes L1..L8 including the §11.4.40 retest and polish; closed dispositions open
Issues	none
Issues	—

Field	Value
sum-mary	
Fixed	§11.4.140 + §11.4.141 mirrors
Fixed sum-mary	Carrier created in lockstep with the Constitution.md §11.4.140 + §11.4.141 additions.
Con-tinu-ation	—

How inheritance works

This GEMINI.md is the **Gemini CLI carrier** of the Helix Constitution submodule. It is one of the four canonical agent context carriers (`constitution/CLAUDE.md` , `constitution/AGENTS.md` , `constitution/QWEN.md` , `constitution/GEMINI.md`) — these ARE the canonical root per §11.4.35, NOT consumer extensions. The full universal rule set lives in `constitution/Constitution.md` ; the sibling carriers (`CLAUDE.md` / `AGENTS.md` / `QWEN.md`) carry the same content for their respective agents. A consuming project's own repo-root `GEMINI.md` (if any) opens with the inheritance pointer and adds only project-specific rules; THIS file is the canonical carrier and does not.

For agents driven from a consuming project, the project's repo-root carrier SHOULD start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/GEMINI.md

All rules in `constitution/GEMINI.md` (and the `constitution/
        Constitution.md`
it references) apply unconditionally. The project-specific rules below
extend them.
```

Action-prefix recognition (§11.4.140 — LAYER-1)

The block below is the **canonical LAYER-1 action-prefix recognition instruction**, embedded VERBATIM (identical text) into every agent context carrier the constitution maintains. That textual identity across all four carriers IS the LAYER-1 universality: Gemini CLI reads this block via this carrier and applies the prefix expansion itself, so the action-prefix system works on Gemini CLI with zero extra setup.

> **§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09; arrow form 2026-07-02).** When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules` ; (3) execute the remainder of the prompt under the expanded instruction. **Five EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest>` (bare `::`), (2) `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), (3) `/ ACTION_NAME <rest>` (bare slash), (4) `/PREFIX::ACTION_NAME <rest>` (namespaced slash), (5) `ACTION_NAME ---> <rest>` (bare arrow) ≡ `PREFIX::ACTION_NAME ---> <rest>` (namespaced arrow). Thus `BACKGROUND :: x` ≡ `DEFAULT::BACKGROUND :: x` ≡ `/BACKGROUND x` ≡ `/DEFAULT::BACKGROUND x` ≡ `BACKGROUND ---> x` ≡ `DEFAULT::BACKGROUND ---> x` . `PREFIX` is an action NAMESPACE; the reserved default namespace is **`DEFAULT`** , and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar` , YAML `key: value` , URLs) in forms 1/2, the arrow-body separator `" ---> "` (one ASCII space on each side) in form

5, and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A` , re-scan, expand `B` , then the residual is the task); a leading `\` escapes the prefix for the `::` , arrow, and slash forms (`\BACKGROUND :: x` , `\BACKGROUND ---> x` , `\/BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/ host slash command (registry `slash_bare: auto` + `slash_conflicts: [..]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 5 forms) but is NOT registered is NEVER silently expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **`BACKGROUND`** expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). A second registered action **`REMINDER`** re-surfaces previously-scheduled, CRITICAL, status-UNCERTAIN work: FIRST verify the ACTUAL current status from captured evidence (never assume done or not-done — §11.4.6 no-guessing), THEN act on the delta — report the captured proof if genuinely complete, resume from the exact point if partial, action it NOW if not started, or surface the block per §11.4.66/§11.4.101 — always producing a status verdict (composes §11.4.6 / §11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.108 / §11.4.130 / §11.4.147).

Severity / handling-priority markers (§11.4.140 extension, 2026-07-16). Three further registered actions tag the REMAINDER of the prompt with a HANDLING PRIORITY (as `BACKGROUND` tags it "run in

> background"): `CRITICAL` (highest-priority / potentially release-blocking — maximum urgency + rigor ahead of lower-priority work, track it, auto-activate §11.4.102 systematic-debugging when an issue is involved, do NOT defer it), `IMPORTANT` (high-priority, above routine work but below CRITICAL — track it, full anti-bluff rigor), and `NOTE` (capture the remainder as durable CONTEXT — request-history §11.4.208 / §11.4.210 + persistent memory when a non-obvious project fact — applied when relevant, NOT an urgent action unless it contains an explicit action; §11.4.6 record only what the note says, never invent). All six grammar forms work (`CRITICAL: x` ≡ `CRITICAL :: x` ≡ `CRITICAL ---> x` ≡ `/CRITICAL x` ≡ `/DEFAULT::CRITICAL x`). Registering `NOTE` promotes `NOTE:` from an ordinary-prose NO-OP to a registered action (ordinary-prose single-colon examples are now `TODO:` / `WARNING:` / `FIXME:`).
>
> The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix` , `--ignore-prefix` , `--no-registry` , `--invent-expansion-OK` , `--single-layer-only` flag.

Token-efficiency mandate (§11.4.141)

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project producing user-facing interfaces (web, desktop, mobile, TUI) MUST use OpenDesign (https://github.com/nexu-io/open-design) as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes system for: (a) all color palette definitions supporting BOTH light and dark themes from project brand assets (§11.4.35), (b) typography scale and spacing, (c) component-level design tokens; if a desired UI pattern is not supported, extend OpenDesign upstream per §11.4.74 (extend-don't-reimplement); every UI component MUST ship light + dark theme

variants; elements MUST NOT overlap, fonts MUST NOT collide, labels MUST NOT overlay labels — any layout regression is a §11.4.162 violation; all UI changes MUST be covered by the project's standard test types including visual regression tests (before/after screenshots with per-pixel or perceptual-diff PASS/FAIL). Honest boundary (§11.4.6): OpenDesign governs design tokens and themes — it does NOT replace functional testing (§11.4.27), WCAG accessibility assertions (§11.4.107), nor the §11.4.48/§11.4.49 UI-driven and dual-approach testing methodology. Classification: universal (§11.4.17). Composes §11.4.74/.25/.27/.4(b)/.107/.48/.49/.35/.69/§1.1. Propagation gate `CM-COVENANT-114-162-PROPAGATION` + recommended gate `CM-OPENDESIGN-UI-SYSTEM` + paired §1.1 mutation.

§11.4.176 — Conflict-free multi-track work-division + exactly-once claim registry + capability-aware deadlock-proof device-lock (User mandate, 2026-07-02). Two mandatory multi-track arbitration layers, conflict-free by construction: (A) exactly-once work-item/logical-group claim registry (extends §11.4.147/§11.4.116; disjoint file-scope §11.4.58 L3 + worktree/CoW §11.4.167; no codebase loss §9.2/§11.4.113/§11.4.84/§11.4.41); (B) capability-aware deadlock-proof device-lock (§11.4.119 single-owner; all-or-nothing + non-blocking + TTL-reap breaks all 4 Coffman conditions; append-only JSONL + atomic snapshot §11.4.116); (C) universal/decoupled (§11.4.17/§11.4.28/§11.4.35) + evidence-based resource-tuning (§11.4.24/§11.4.6, never guessed). Classification: universal (§11.4.17). Gates CM-WORK-DIVISION-EXCLUSIVE-CLAIM + CM-DEVICE-LOCK-DEADLOCK-FREE + CM-COVENANT-114-176-PROPAGATION + paired §1.1 mutation. Composes §11.4.6 §11.4.17 §11.4.24 §11.4.28 §11.4.35 §11.4.41 §11.4.58 §11.4.84 §11.4.85 §11.4.103 §11.4.113 §11.4.116 §11.4.119 §11.4.147 §11.4.167 §12.6 §9.2 §11.4.176.

§12.11 — Maximal dynamic resource utilization for containerized builds (User mandate, 2026-07-03). The containerized build path (its OWN cgroup + own MemoryMax → OOM-kills ITSELF, never user.slice) MUST use the maximal SAFE fraction of host capacity, computed DYNAMICALLY per host: auto-detect nproc/MemTotal/RLIMIT_NPROC (never hardcode, §11.4.6), reserve explicit host headroom (cores+RAM), scale -j against BOTH the memory model AND the process rlimit (privileged nproc-raise for full -j; EAGAIN-safe -j otherwise), never break/bottleneck/wedge. NO fixed low -j for

the containerized path. The §12.6 60% ceiling + bare-metal -j cap REMAIN, SCOPED to user.slice-resident work — §12.11 does NOT weaken §12.6, it scopes it. Classification: universal (§11.4.17). Gate CM-MAXRES-DYNAMIC-BUILD + CM-COVENANT-12-11-PROPAGATION + paired §1.1 mutation. Composes §12.1 §12.2 §12.3 §12.6 §12.10 §11.4.6 §11.4.24 §11.4.133 §9.2 §11.4.35 §12.11.

§11.4.187 — Automatic multi-track ruler orchestration (User mandate, 2026-07-04; landed 2026-07-09). Compact summary: every project incorporating this constitution MUST ship its multi-track parallel-development orchestration as a UNIVERSAL, AUTOMATIC, OUT-OF-THE-BOX, INHERITED capability — not a per-project reimplementation — via a single conductor session that programmatically SPAWNS, DRIVES, RESUMES, MONITORS, and CONTROLS per-track headless workers. (1) **Headless spawn primitive** — workers launch via native headless `claude -p --output-format stream-json --verbose`; the worker's `session_id` is captured from the FIRST `init` event (never invented); success/failure is read from the terminal event's `result.is_error` field, NEVER the process exit code (a CLI can exit non-zero on a successful `result`, or zero on a failed one — exit-code-as-success-signal is a §11.4.6 guess); continuation of an existing worker uses `--resume <session_id>` invoked from the SAME cwd + env the spawn used (the storage/resume-collision gotcha). (2) **Per-subscription auth + rate-limit fallback** — every spawn `unset`s any ambient `ANTHROPIC_API_KEY` (the precedence trap that silently routes a "subscription" worker onto pay-per-token API billing) and sets the selected alias's `CLAUDE_CODE_OAUTH_TOKEN` / `CLAUDE_CONFIG_DIR`; on a captured rate-limit signature (`isApiErrorMessage` + `apiErrorStatus:429`, quota-exhausted vs transient distinguished by evidence, never guessed) the ruler REBINDS the affected track onto the next healthy alias (cross-track reuse permitted), falls back to a provider alias where configured, and bounded-parks (§11.4.101) rather than hanging when every alias is cooled. (3) **Crash-resilient ruler self-supervisor** — the ruler's OWN state (per-track alias/session_id/worktree/last-verdict) is persisted durably (write-temp-then-rename, NEVER tmpfs) so an external watchdog detects a dead ruler PID and re-hydrates every track's binding from the durable state + the §11.4.116 event stream — no work is silently lost to a ruler crash (§11.4.147 at the ruler layer). (4) **Idempotent bootstrap + auto-install + conductor-stays-home** — one idempotent

bootstrap command wires the whole system out of the box (cwd-hook install, config validation, reconciliation), auto-invoked on every constitution pull via the §11.4.164 `post_update_hook.sh` skill-register seam; the `conductor:`-designated alias/session is NEVER bound into a track worktree (the resolver returns nothing for it) so the conductor always runs from the main checkout. (5) **Project-agnostic engine, consumer-owned data** — the entire mechanism (config loader, alias→worktree resolver, cwd-hook, bind/fallback/cooldown orchestrator, session launcher, rate-limit monitor, supervisor) lives at `constitution/scripts/multitrack/`, inherited BY REFERENCE (§11.4.28(B)/§11.4.177), carrying NO project literal; every consuming project supplies its own per-host `config/multitrack/<hostname>.yaml` (tracks/mounts/serials/aliases/conductor) as DATA, never as an engine edit. Anti-bluff (§11.4/§11.4.108): every piece carries four-layer coverage (pre-build gate + on-host test + paired §1.1 mutation + captured evidence per §11.4.69), and each fix declares a runtime signature verified on a clean host — never a claim the mechanism "should work." Honest boundary (§11.4.6): this anchor mandates the MECHANISM; genuine per-subscription quota-isolation proof requires live tokens exercising real rate-limit boundaries (an operator-scheduled live acceptance window) — a mock-driven GREEN proves the spawn/resume/fallback CONTRACT, never live-quota behaviour, and the two are never conflated or substituted for each other. Classification: universal (§11.4.17). Composes §11.4.20 (subagent-driven-by-default) / §11.4.28 (submodule decoupling + dependency layout) / §11.4.58 (parallel-development PWU pipeline) / §11.4.70 (subagent-driven execution default) / §11.4.101 (autonomous-decision-over-blocking — the bounded-park-on-all-cooled behaviour) / §11.4.103 (continuous parallel-stream routine) / §11.4.111 (resolve-by-stable-name — config by serial, never by index) / §11.4.116 (real-time conductor↔framework sync channel) / §11.4.119 (single-resource-owner partitioning) / §11.4.126 (default autonomous-loop mode) / §11.4.128 (always-on device-recording — the analogous host-safety discipline) / §11.4.131 (standing session-resumption file) / §11.4.147 (crashed-agent respawn-until-complete — the ruler-self-supervisor is its ruler-layer instantiation) / §11.4.164 (constitution auto-propagation hook seam) / §11.4.167 (feature work-stream lifecycle) / §11.4.176 (multi-track work-division + exactly-once claim + device-lock) / §12.6/§12.8 (host-safety memory + concurrency ceilings — the spawn/launch pool is budget-gated). Propagation gate `CM-COVENANT-114-187-`

`PROPAGATION` (literal `11.4.187`) + recommended gates `CM-MULTITRACK-ENGINE-IN-CONSTITUTION` (the generic ruler scripts present under `constitution/scripts/multitrack/`, carrying no project literal, `bash -n` + `sh -n` clean per §11.4.67) / `CM-MULTITRACK-AUTOINSTALL-WIRED` / `CM-MULTITRACK-BOOTSTRAP-IDEMPOTENT` / `CM-MULTITRACK-BIND-ON-START` / `CM-MULTITRACK-FALLBACK-MONITOR` / `CM-MULTITRACK-HEADLESS-SPAWN` / `CM-MULTITRACK-RULER-SUPERVISOR` + paired §1.1 mutation (strip the literal → propagation gate FAILs; re-introduce a project literal into the engine, or downgrade a rate-limit rebind to exit-code-based success detection → the corresponding mechanism gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.187. Non-compliance is a release blocker. No escape hatch — no `--skip-multitrack`, `--manual-multitrack-only`, `--no-auto-install`, `--per-project-reimpl-OK`, `--exit-code-is-success-signal` flag.

§11.4.190 — Website engineering-quality mandate: every project website MUST be fully responsive + completely SEO-optimized + uniquely OpenDesign-authored + bleeding-edge enterprise-quality, each PROVEN with captured evidence (User mandate, 2026-07-10). Verbatim operator mandate: "every website we create inside our projects MUST BE fully responsive - working on all browsers, all operating systems, all platforms and all devices and screen sizes! Every Website MUST BE completely SEO optimized! Layouts and templates done MUST BE unique and everything fully designed from OpenDesign! Make sure this applies to our project Website as well! Everything must be bleeding-edge enterprise quality, stunning visual look and feel, innovative, creative and unique! There MUST BE no false or faulty result or bluff of any kind anywhere!" Every website / web-UI surface a project ships — INCLUDING this project's OWN website — MUST satisfy AND PROVE with captured physical evidence (§11.4.5/§11.4.69/§11.4.107/§11.4.170 — never a bare claim, §11.4.6) ALL of: **(A) FULL RESPONSIVENESS** — correct, usable rendering across ALL major browser engines (Chromium/Firefox/WebKit), all operating systems, all device classes (desktop/laptop/tablet/phone/large-display) and the full range of screen sizes/orientations/pixel-densities; NO broken layout / overflow / clipping / overlap / off-screen or unusable control at ANY breakpoint. **(B) COMPLETE SEO OPTIMIZATION** — semantic HTML + correct document outline; per-page unique `<title>` + meta description; Open Graph + Twitter cards; canonical URLs; structured data (schema.org / JSON-

LD); `robots` + XML sitemap; crawlable / indexable markup; WCAG AA accessibility + Core Web Vitals performance (both feed ranking). **(C) UNIQUE OpenDesign-AUTHORED templates/layouts** — EVERY layout + template designed FROM the OpenDesign design-token system (§11.4.162), NOT generic / off-the-shelf / templated; visually unique, innovative, creative, brand-distinct. **(D) BLEEDING-EDGE ENTERPRISE VISUAL QUALITY** — stunning, polished, professional look-and-feel in BOTH light AND dark themes (§11.4.162 tokens). **(E) ANTI-BLUFF PROOF** — NONE of (A)–(D) may be CLAIMED without captured physical evidence: responsiveness PROVEN by device-independent host-rendered screenshots across the real breakpoint × engine matrix + an OCR/vision layout oracle (no overlap / label-over-label / clip / overflow / off-screen); SEO PROVEN by an automated audit (Lighthouse SEO + structured-data validation + meta / OG / sitemap / robots presence) meeting a defined score floor; visual quality PROVEN by the §11.4.170 device-independent host-rendered pixel proof per screen × state × {light,dark}; uniqueness PROVEN by OpenDesign-token provenance. A website reported responsive / SEO-optimized / unique / enterprise-quality WITHOUT these captured proofs is a §11.4 PASS-bluff at the web-UI layer. §11.4.190 is the strict WEB-quality expansion of §11.4.162 (OpenDesign — it applies §11.4.162's design-token mandate to the full responsive + SEO + enterprise-quality web surface) and binds §11.4.170's device-independent host-rendered pixel proof as the correctness oracle; sibling of §11.4.168 (exported-document independent visual validation) at the web-UI layer. Classification: universal (§11.4.17) — references no project-specific hardware; the consuming project supplies its concrete breakpoint × engine matrix, SEO score-floor, and OpenDesign token set per §11.4.35. Composes §11.4.5/§11.4.6/§11.4.69/§11.4.107/§11.4.162/§11.4.168/§11.4.170. Propagation gate `CM-COVENANT-114-190-PROPAGATION` (literal `11.4.190`) + recommended gates `CM-WEBSITE-RESPONSIVE-PROVEN` (host-rendered screenshots across the breakpoint × engine matrix + layout oracle — no overlap/clip/overflow/off-screen) / `CM-WEBSITE-SEO-OPTIMIZED` (automated SEO audit meets the score floor + structured-data / meta / OG / sitemap / robots present) / `CM-WEBSITE-OPENDESIGN-UNIQUE` (every layout/template's OpenDesign-token provenance, no generic/off-the-shelf template) + paired §1.1 mutation (report a website responsive/SEO/unique/enterprise-quality with NO captured proof, OR ship a generic non-OpenDesign template, OR skip the light/dark host-rendered

pixel proof → the recommended gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.190. Non-compliance is a release blocker. No escape hatch — no `--skip-responsive-proof`, `--seo-optional`, `--generic-template-OK`, `--visual-quality-later`, `--claim-without-render` flag.

§11.4.191 — Work-to-track/branch binding enforcement: a logic-group's work can NEVER be committed OR dispatched onto the wrong track/branch (research-derived, 2026-07-10). Compact summary: every logical group of workable items binds to EXACTLY ONE canonical `(track, branch)` recorded ONCE in the single source of truth (the §11.4.93 workable-items DB `logic_groups.destination` = authoritative branch + a `canonical_track` column + a `group_paths` file-scope manifest), and NO change belonging to a group may LAND on, nor be DISPATCHED to, a track/branch other than that group's canonical one — the file-scope generalisation of §11.4.181/§11.4.182 from branch-NAME consistency to work-PLACEMENT consistency (forensic FACT 2026-07-10: scrcpy/Remote-app work — group `mistiq-vader`, canonical `feature/mistiq-vader`/T2 — was committed onto `main`/T1; §11.4.181's `guard-branch-consistency.sh` never fired because committing onto an existing branch is not a feature-branch CREATE, and no gate checked which FILES land on which BRANCH). (1) **Single source of truth** — the `logic_group → (branch, track, path-globs)` binding is recorded ONCE in the DB and LOOKED UP, never re-invented (a commit/dispatch placing a group's file-scope on a non-canonical branch/track is a §11.4.6 no-guessing violation). (2) **Preventive hook (§11.4.109/§11.4.177)** — a PreToolUse guard `guard-work-track-binding.sh` (inherited BY REFERENCE, never copied) BLOCKS (exit 2) a git-commit-class Bash call whose to-be-committed file set contains a path owned by a group whose canonical branch/track ≠ the current checkout, AND BLOCKS an Agent/Task dispatch whose §11.4.182 label track/branch ≠ the canonical (track, branch) of the ticket's group; fail-closed on unreadable registry, `# guardrails:allow <reason>` audited escape. (3) **Detective gate (§11.4.110/§11.4.108)** — `CM-WORK-TRACK-BINDING-ENFORCED` asserts, on the current branch's own added commits (`merge-base(HEAD,main)..HEAD`), that every `group_paths`-owned file's group destination == current branch (and canonical_track == current track when pinned) — the actual enforcement boundary regardless of how the commit was

made, the runtime signature being FAIL-on-wrong-placement / PASS-on-canonical-placement on a clean checkout. (4) **Risk-free reconciliation (§9.2/ §11.4.113/§11.4.84)** — a mis-placed change is reconciled by MERGING its commits onto the group's canonical branch (union, no commit lost, no `-s ours` /reset), ff-only push to all upstreams (NEVER force-push), then removing it from the wrong branch only after the merge is confirmed on all mirrors. Classification: universal (§11.4.17) — references no project hardware; the consuming project supplies its `(track, branch)` map, `group_paths` seed, and commit-wrapper wiring per §11.4.35. Composes §11.4.6/.28/.29/.35/.58/.75/.84/.93/.95/.109/.110/.113/.119/.167/.176/.177/.178/.181/.182/ §9.2. Propagation gate `CM-COVENANT-114-191-PROPAGATION` (literal `11.4.191`) + recommended gate `CM-WORK-TRACK-BINDING-ENFORCED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.191. Non-compliance is a release blocker. No escape hatch — no `--allow-cross-track-commit` , `--skip-work-binding` , `--commit-any-branch-OK` , `--dispatch-any-track-OK` , `--no-file-scope-check` flag.

§11.4.192 — Continuous multi-track auto-backfill: a FREE track MUST be IMMEDIATELY re-assigned its next-highest-priority domain work-set, never left idle while its domain has actionable items (User mandate, 2026-07-11). Verbatim operator mandate: "once a Track is free, we shall assign next set of workable items to it ... full continuous multi-track development ... no stopping ... this is MANDATORY". Compact summary: the moment any track's worker COMPLETES its current work-set (its item(s) closed with anti-bluff evidence, or merged), that FREE track MUST be IMMEDIATELY assigned the NEXT-highest-priority actionable work-set from its OWN domain (its §11.4.191 logic_group / canonical branch) — a track is NEVER left idle while its domain still holds actionable workable items, and the conductor/ruler CONTINUOUSLY re-assigns with NO stopping (full continuous multi-track development); PROMOTES the §11.4.103(B) ≥3-parallel-background-streams-with-auto-backfill invariant into the multi-track (§11.4.176/§11.4.178/§11.4.179/§11.4.187) layer (§11.4.103(B) backfills a STREAM the moment it is done; §11.4.192 backfills a TRACK the moment its worker is done, keyed to that track's canonical (track, branch) domain per §11.4.191). (1) **WHAT** — on any track-worker completion the conductor/ruler (§11.4.187) MUST, per that track's domain, look up the NEXT actionable work-set, CLAIM it exactly-once

(§11.4.176 exactly-once claim registry, no two tracks claiming one item), and DISPATCH it as the track's new §11.4.182-labelled work stream — without waiting for a scheduled wake or an operator prompt. (2) **PRIORITY** — the next work-set is the track domain's highest-priority actionable item (§11.4.42/ §11.4.72 priority order, audio-first where in scope), never an arbitrary pick. (3) **SOURCE-SIDE ADVANCES EVEN WHEN VALIDATION IS GATED** — when the next item's on-device VALIDATION is blocked on a pending build / device availability (§11.4.108 clean-target / §11.4.185 manual-QA / operator-gated flash), the track MUST still advance the item's SOURCE-SIDE work (investigation, RED-test §11.4.115, source patch, pre-build gate + paired §1.1 mutation, docs) and honestly §11.4.3/§11.4.52 SKIP-with-reason ONLY the genuinely device-gated validation portion — a build/device gate on the validation step is NOT a reason to idle the whole track. (4) **HONEST BOUNDARY (§11.4.6)** — a track idles ONLY when its domain GENUINELY has zero remaining actionable workable items OR every remaining item is externally blocked with no source-side advancement possible (§11.4.94(A)/ §11.4.97/§11.4.101 idle-only-when-blocked closed set); "the merges are done" / "the current item is done" / "nothing visible right now" is NEVER a valid idle reason while the domain still holds progressable items (a §11.4.94+§11.4.97 violation); an unavoidable per-item block PARKS one item, it does NOT stop the track (§11.4.101 — the track backfills its next non-blocked domain item). Classification: universal (§11.4.17) — references no project hardware; the consuming project supplies its track→domain map + priority source per §11.4.35. Composes §11.4.42/.58/.72/.94/.97/.101/.103/.167/.176/.178/.179/.182/.187. Propagation gate `CM-COVENANT-114-192-PROPAGATION` (literal `11.4.192`) + recommended gate `CM-CONTINUOUS-MULTITRACK-BACKFILL` (on a track-worker completion the ruler re-assigns that track's next-highest-priority domain work-set; a track with a completed worker AND remaining actionable domain items but no new dispatch → the gate FAILs) + paired §1.1 mutation (let a free track sit idle while its domain holds an actionable item, OR treat "merges done" as a valid idle reason → the gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.192. Non-compliance is a release blocker. No escape hatch — no `--allow-idle-track`, `--skip-backfill`,

`--stop-multitrack`, `--merges-done-is-idle`, `--defer-track-reassign` flag.

§11.4.193 — Anti-blind-typing mandate: every UI interaction MUST be "seen" and "understood" via OCR/vision/screenshot proof, NEVER blind-typed (User mandate, 2026-07-13). Verbatim operator mandate: "No blind typing of any kind anywhere!!!"; "Login to apps and services MUST BE done with understanding and seeing the whole UI ALWAYS!!!"; "blind typing is STRICTLY FORBIDDEN, and instead everything MUST BE seen and understood in coordination with proper models and means we have incorporated in HelixQA!!!". Blind typing — sending keystrokes/text/input events to a UI surface WITHOUT first capturing AND understanding what is on screen (OCR / vision / uiautomator / accessibility-tree / screenshot) AND confirming the result after each interaction — is STRICTLY FORBIDDEN everywhere and always, for every agent/tool/automation/platform/context. It is LITERAL bluff: claiming "credentials entered"/"login completed" without visual proof the keystrokes landed on the intended fields/screen producing the intended result is a §11.4 PASS-bluff at the UI-interaction layer (a credential typed into the wrong field, or a WebView that silently swallowed input, = end-user feature broken while the agent reported success). (1) **SEE-BEFORE-YOU-TYPE** — before ANY `input text` / `input keyevent` / `adb shell input` / uiautomator gesture, capture the current screen state (screencap + Tesseract OCR ≥150 DPI, OR uiautomator dump grep, OR accessibility-tree dumpsys, OR HelixQA VisionEngine frame per §11.4.159/§11.4.160) AND confirm the intended target element is visible AND focused — the capture IS the §11.4.5/§11.4.69 `touch_input` evidence. (2) **VERIFY-AFTER-YOU-TYPE** — after EVERY interaction step, capture a NEW screen state + confirm the intended change (5 `input text` + 1 final screenshot = blind typing). (3) **OCR/VISION ORACLE MANDATORY** — Tesseract ≥150 DPI / uiautomator-dump grep / HelixQA VisionEngine / accessibility-tree dumpsys. (4) **WEBVIEW** — near-empty uiautomator hierarchy → OCR fallback per §11.4.117; if even OCR is infeasible (secure/DRM-blanked surface captures black) → the login is operator_attended-only, SKIP-with-reason per §11.4.3, NEVER blind-type-and-claim-success. (5) **CREDENTIAL-SAFE OCR** — redact credential characters from OCR output + logs per §11.4.10; evidence = the redacted screenshot / OCR text. (6) **ANTI-BLUFF** — every login/interaction PASS cites a captured-evidence artefact containing pre-interaction state + post-interaction state +

OCR/vision verification of the expected change; self-validated per §11.4.107(10) with a golden-good + golden-bad fixture pair. Classification: universal (§11.4.17). Composes §11.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.117 / §11.4.159 / §11.4.160 / §11.4.10 / §11.4.52 / §11.4.3. Propagation gate `CM-COVENANT-114-193-PROPAGATION` (literal `11.4.193`) + recommended gate `CM-NO-BLIND-TYPING` + paired §1.1 mutation (strip OCR verification before an `input text` → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.193. Non-compliance is a release blocker. No escape hatch — no `--blind-typing-ok`, `--skip-ocr-verification`, `--assume-input-landed`, `--login-without-proof`, `--type-without-seeing` flag.

§11.4.195 — Branch-structure governance: taxonomy + merge-after-live-QA + flavor/product non-merge (User mandate, 2026-07-14). Verbatim operator rules: branch taxonomy `feat/<NAME_OF_FEATURE>` / `product/<NAME_OF_PRODUCT>` / `flavor/<NAME_OF_FLAVOR>`; tracks on FEATURE branches or `main` MUST — after full confirmation + validation + verification + full LIVE MANUAL TESTING (§11.4.185) — MERGE to `main` COMPLETELY (all submodules + main repo), ff-only, NEVER force-push (§11.4.113), no commit lost (§9.2); tracks with an entirely-new FLAVOR/PRODUCT do ALL work on a `flavor/` AND/OR `product/` branch (both may exist) which does NOT merge to `main` the same way; commit + push ALL branches to ALL upstreams (§2.1), ff-only. Every controlled branch belongs to one of THREE canonical taxonomy classes, each with a distinct integration lifecycle: **(A) Taxonomy** (extends §11.4.181/§11.4.167(D)) — `feat/<slug>` = FEATURE (merges to `main` after live-manual-QA, clause B); `product/<slug>` = PRODUCT line + `flavor/<slug>` = FLAVOR line (do NOT merge to `main` the standard way, clause C); `<slug>` lowercase snake/kebab (§11.4.29); the canonical branch NAME is minted ONCE + used IDENTICALLY across main repo + every touched owned submodule (§11.4.181), recorded once in the §11.4.176 claim registry / §11.4.93 `logic_group → branch` map + LOOKED UP never re-invented (§11.4.6); the prefix STRINGS are operator-canonical, a project already on a different feature-prefix (`feature/…`) is compliant so long as it is the ONE canonical name per §11.4.181, reconciled by the §11.4.181 risk-free MERGE-onto-canonical path (ff-only, no commit lost, never a rename that drops commits). **(B) FEATURE/main → merge-after-live-QA** — before

merging to `main` satisfy IN ORDER automated four-layer (§11.4.4(b)) + §11.4.40 full-suite retest THEN full LIVE MANUAL TESTING by the QA team (§11.4.185 FINAL human gate, automated GREEN necessary NOT sufficient); only after §11.4.185 may the merge proceed + it MUST be COMPLETE (main repo + every touched owned submodule) ff-only onto latest `main` (§11.4.113), NEVER force-push, NO commit lost / NO history rewrite (§9.2/§11.4.41 step 3), pushed to ALL upstreams (§2.1); binds the §11.4.185 gate as an explicit precondition of the §11.4.167(I)/§11.4.188 approval-gated back-merge. **(C) FLAVOR/PRODUCT lines → do-not-merge-to-main-the-same-way** — an entirely-new flavor/product line does ALL work on a `product/<NAME>` AND/OR `flavor/<NAME>` branch (both may coexist), a divergent line with its own release lifecycle that does NOT flow back to `main` via clause B; STILL obeys §11.4.113 (no force-push) / §9.2 (no commit lost) / §2.1 (push all branches all upstreams ff-only) / §11.4.188 (MAY merge `main` INTO itself to stay close — trunk-into-line permitted; only line-into-trunk is gated/withheld). **(D) Push discipline** (§2.1/§11.4.113) — commit + push ALL branches (feature/product/flavor) to ALL upstreams ff-only, NEVER force-push; new-branch creation is inherently ff (§9.2) + non-disruptive from an existing HEAD, permitted in a dirty tree under §11.4.84 provided no commit is in-flight. Honest boundary §11.4.6 — governs branch STRUCTURE + integration DIRECTION, does NOT prove any branch's work correct (still §11.4.108 runtime-signature + §11.4.40 retest + §11.4.185 manual-QA). Classification: universal (§11.4.17) — strict generalisation of §11.4.181 + §11.4.167(D)/(I) + §11.4.188 + §11.4.185. Composes §2.1/§9/§9.2/§11.4.6/§11.4.17/§11.4.29/§11.4.35/§11.4.40/§11.4.41/§11.4.84/§11.4.93/§11.4.113/§11.4.167(D)/(I)/§11.4.176/§11.4.178/§11.4.181/§11.4.185/§11.4.188/§11.4.191. Propagation gate `CM-COVENANT-114-195-PROPAGATION` (literal `11.4.195`) + recommended gate `CM-BRANCH-TAXONOMY-CLASS` (every controlled branch prefix ∈ `feat/` | `product/` | `flavor/` |reserved `main` / `master`; a `product/` | `flavor/` branch is NOT the source of a merge onto `main`; a `feat/` | `main` back-merge cites a §11.4.185 manual-QA confirmation) + paired §1.1 mutation (a `product/*` merged onto `main`, OR a `feat/*` back-merge with no §11.4.185 citation → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.195. Non-compliance is a release blocker. No escape hatch — no `--any-branch-prefix-OK`, `--merge-product-to-main`,

`--skip-live-qa-before-merge`, `--force-push-branch`, `--partial-submodule-merge` flag.

§11.4.196 — Native-alias-first priority + per-alias real-signal limit/subscription tracking + auto-rebind-on-recovery + resource-detection-by-real-identity-not-substring-match (User mandate, 2026-07-14). Verbatim operator mandate: use ALL operational claude-NATIVE aliases FIRST — as long as ≥1 native alias is operational (NOT session-rate-limited, NOT weekly-limit-reached, NOT expired-subscription) — before ANY provider alias; track per alias the session rate-limit / weekly-limit / subscription expiry+renewal from REAL signals (never faked, §11.4.6) + record when each becomes operational AGAIN; the moment a higher-priority native recovers, rebind tracks to it; and fix the host-budget guard footgun where a `pgrep -f` substring-match false-REFUSED every worker spawn. The multi-track alias-binding + rate-limit/subscription-tracking layer of the §11.4.187 ruler orchestration MUST implement ALL of: **(A) NATIVE-ALIAS-FIRST PRIORITY** — every OPERATIONAL native (`claudeN`) alias is selected before ANY provider, UNCONDITIONALLY, while ≥1 native is operational; the guarantee is STRUCTURAL — a native/provider CLASS partition scanned native-class-completely-before-provider-class (§11.4.111 resolve-by-CLASS-not-by-file-position), so a provider NEVER wins while a non-cooled native exists regardless of roster order; within a class the operator-mandated order (native equal-capability, then providers deepseek→xiaomi→opencode→kimi→…) is authoritative decision-DATA (§11.4.28) never re-invented (§11.4.6). **(B) PER-ALIAS REAL-SIGNAL LIMIT + SUBSCRIPTION TRACKING** — "operational?" per alias from REAL captured signals, NEVER faked/guessed (§11.4.6): the §11.4.187 captured 429 signature (`isApiErrorMessage` + `apiErrorStatus:429` / `api_error_status:429`) sub-classified into reason-CLASS closed set `{session | weekly | subscription}` from real markers (reset/weekly-limit/session-limit); each limited alias records its CLASS + an operational-again EPOCH with a DIFFERENT window per class — session (self-heals at reset, short), weekly (~until the weekly reset, long), subscription (INDEFINITE — operational-again UNKNOWN until a REAL renewal signal, parked with a far-future sentinel; §11.4.6 NEVER guesses a renewal date, an operator/config supplies it). A limited alias is NEVER auto-selected until its epoch passes (session/weekly) or a real renewal/recovery clears it (subscription). **(C) AUTO-USE-ON-**

RECOVERY — the moment a higher-priority native recovers (window elapsed / renewal marked), rebind tracks UPWARD to it (a `promote`-class op keyed off a comparable priority-rank: native class before provider, then config order), PRESERVING each track's worktree AND device leases (leases key on the STABLE track id — the alias changes, not the track; §11.4.119/§11.4.187); idempotent. **(D) RESOURCE-DETECTION BY REAL IDENTITY, NOT SUBSTRING-MATCH (RB-02 guard fix)** — a "heavy work in flight" guard MUST identify a real resource by its ACTUAL process command line read from the OS (`/proc/<pid>/cmdline`), NEVER a bare `pgrep -f REGEX` substring match that false-matches a process merely MENTIONING a token — a pattern-CARRIER (a process quoting the whole alternation pattern; a `claude` /worker whose prompt embeds the token; a launcher/monitor/test), a same-tool ENGINE process, or the guard's own process/parent (the §12.12 self/sibling pgrep footgun — forensic FACT 2026-07-13: the guard false-REFUSED EVERY §11.4.187 spawn on a host with ZERO real builds because a live `claude` worker's prompt quoted the pgrep pattern); the guard re-reads each matched PID's real cmdline + excludes those carrier/engine/worker/self classes; an UNREADABLE/vanished cmdline is conservatively counted as the real resource (REFUSE — safe reversible default, §11.4.101/§12.8), so a genuine build is still caught. **(E) ANTI-BLUFF (§11.4/§11.4.108)** — four-layer coverage each (pre-build gate + on-host test + paired §1.1 mutation + captured evidence §11.4.69) with §11.4.115 RED-polarity + §11.4.50 determinism; honest boundary §11.4.6 — mandates the MECHANISM + REAL-signal derivation of "operational", genuine live per-subscription quota-isolation proof still needs live tokens (operator-scheduled acceptance window), never conflated with a mock/hermetic-contract GREEN. Classification: universal (§11.4.17). Composes §11.4.187/.182/.176/.111/.101/.103/.6/.108/.115/.119/ §12.12/§12.6/§12.8. Propagation gate `CM-COVENANT-114-196-PROPAGATION` (literal `11.4.196`) + recommended gate `CM-ALIAS-PRIORITY-LIMIT-TRACKING` (native-first CLASS partition; per-alias reason-class + operational-again tracking; auto-rebind-on-recovery; build-guard resolves by real cmdline not bare substring) + paired §1.1 mutation (swap native/provider priority bases so a provider outranks a native → recommended gate FAILs; flip the guard's strict-cmdline-filter to the pre-fix bare `pgrep` → its RB-02 test FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.196. Non-

compliance is a release blocker. No escape hatch — no `--provider-before-operational-native`, `--fake-limit-state`, `--guess-renewal-date`, `--no-rebind-on-recovery`, `--substring-match-build-guard`, `--skip-real-signal-classification` flag.

§11.4.197 — Research / kicked-off-work completion mandate: every research effort MUST be driven to full, wired, verified completion or explicitly closed — never left un-wired in the backlog (User mandate, 2026-07-15). Verbatim operator mandate: "all research work we start MUST BE fully finished, completely confirmed, validated and verified! It MUST NOT happen for it to stay at the backlog uncompleted and un-wired! Loss of requirements, failure to incorporate them is FORBIDDEN!!! Everything MUST BE fully completed! No exceptions!" Every research effort / kicked-off improvement / design doc / spike / prototype / PoC / partially-landed feature a project STARTS MUST be driven to FULL completion OR explicitly evidence-backed CLOSED — there is no third "sitting un-wired in the backlog" state; "we researched/designed/started it" is NEVER terminal, and a `docs/research/*` doc / design doc / spike / half-wired feature that is neither COMPLETED-and-wired nor explicitly-closed is a §11.4.197 violation of §11.4 PASS-bluff severity at the requirements-integrity layer (the requirement was accepted, work began, then it silently evaporated); loss of requirements + failure to incorporate started work are FORBIDDEN. **COMPLETED** = (1) implemented + (2) WIRED (integrated AND used by default, NOT merely present behind a dead flag / uncalled function / unreferenced module / never-selected path — mere-presence-without-wiring is the §11.4.124 dead-code + §11.4.108 layer-2/3 SOURCE→ARTIFACT→RUNTIME gap) + (3) confirmed/validated/verified with captured physical evidence §11.4.5/§11.4.69/§11.4.107 on a clean deployment §11.4.108 runtime-signature (never metadata-only/config-only/absence-of-error/grep-without-runtime §11.4/§11.4.1) + (4) tracked to a terminal Status §11.4.15/§11.4.33. **Explicitly CLOSED** = deliberately terminated with a documented evidence-backed reason from the closed vocabulary — §11.4.90 Obsolete / §11.4.112 structurally-impossible / §11.4.122 operator-confirmed drop — NEVER silent abandonment ("no time"/"lower priority" = §11.4.21 Operator-blocked or a still-open tracked item, NOT a close). **Mechanical tracker binding (§11.4.93):** every research effort / design doc / kicked-off improvement MUST map to ≥1 tracked workable item (ATM-NNN §11.4.54) whose completion the loop enforces; a research doc with NO implementing item, or a

tracked item stalled un-wired past the cadence, is a violation; the autonomous loop (§11.4.87/.94/.97/.103/.126/.192) MUST treat every un-wired research effort as an actionable queue item — driving it to COMPLETED or explicitly CLOSED is exactly the "keep working until the queue is genuinely empty" contract. Honest boundary §11.4.6 — "fully completed" is bounded by wired+verified OR evidence-backed-closed, NOT "every speculative idea ships"; every STARTED effort reaches a documented terminal state, proven never assumed. Classification: universal (§11.4.17). Composes §11.4.8/.87/.94/.97/.103/.118/.123/.124/.150/.192/.90/.112/.122/.21/.93/.54/.108/.15/.33/.126. Propagation gate `CM-COVENANT-114-197-PROPAGATION` (literal `11.4.197`) + recommended gate `CM-RESEARCH-COMPLETION-TRACKED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.197. Non-compliance is a release blocker. No escape hatch — no `--research-may-stay-unwired`, `--skip-completion-tracking`, `--abandon-without-close`, `--present-but-unwired-is-done`, `--backlog-research-OK`, `--lose-requirement-OK` flag.

§11.4.198 — Default working mechanisms: multi-alias native-first orchestration AND heavy token-optimization are ALWAYS-ON defaults for single- and multi-track work (User mandate, 2026-07-15). Operator mandate: multiple claude-toolkit aliases (native-first) AND heavy token-optimization MUST be the DEFAULT mechanisms, always used with single- and multi-track development and with single or multiple working agents. Two proven mechanisms promoted to STANDING always-on defaults, engaged automatically for EVERY configuration (single- OR multiple-agent, single- OR multi-track), never a per-request opt-in: **(A) MULTI-ALIAS NATIVE-FIRST ORCHESTRATION** — the claude-toolkit multi-alias mechanism (every OPERATIONAL claude-native alias before ANY provider per §11.4.196 native-alias-first priority + per-alias real-signal limit/subscription tracking + auto-rebind-on-recovery, driven by the §11.4.187 automatic multi-track ruler) MUST be the default execution substrate for ALL work, NOT only the multi-track case; a SINGLE-track/SINGLE-agent session STILL binds its worker through the same native-first alias-priority mechanism (best operational native + rebind-on-recovery), a MULTI-track/multiple-agent config uses it across every track (§11.4.176 exactly-once claim + §11.4.182 track+branch+alias label); running on an arbitrary/hardcoded/provider-first alias when an operational native exists, or bypassing the mechanism because "it is just one agent", is a violation. **(B)**

HEAVY TOKEN-OPTIMIZATION — the §11.4.141 token-efficiency discipline (compact-anchor layout with full-one-hop-away, targeted reads over whole-file dumps, subagent context isolation §11.4.20/§11.4.70, evidence-hash references not evidence dumps) MUST apply ALWAYS — every read/dispatch/doc-edit — for single- AND multi-track, single OR multiple agents; token-heavy patterns (re-reading whole large files when a targeted slice suffices, dumping full evidence into context, monolithic non-subagent execution of parallelisable work) when a token-efficient path exists are violations. Honest boundary §11.4.6 — "default" = auto-engaged without an operator request; it does NOT weaken host-safety, the multi-alias/multi-agent fan-out stays under the §12.6 60% memory ceiling + §12.12 thread-headroom + §11.4.58 parallel-agent cap (contending/redundant agents are NOT "the default", §11.4.183 maximum-USEFUL-throughput); §11.4.198 BINDS the two mechanisms as always-on, it does NOT redefine them (mechanics live in §11.4.196/§11.4.187/§11.4.141). Classification: universal (§11.4.17). Composes §11.4.141/.187/.196/.182/.176/.183/.103/.58/.20/.70/.126/§12.6/§12.12. Propagation gate `CM-COVENANT-114-198-PROPAGATION` (literal `11.4.198`) + recommended gate `CM-DEFAULT-MECHANISMS-ALWAYS-ON` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.198. Non-compliance is a release blocker. No escape hatch — no `--single-agent-skips-alias-priority`, `--provider-first-default`, `--token-optimization-optional`, `--heavy-tokens-OK`, `--not-default-mechanism` flag.

§11.4.199 — Exact-reproduction-sequence mandate: when a working reproduction exists, the investigation MUST use ITS exact sequence — a deviating repro proves NOTHING (research-derived, 2026-07-15). Forensic FACT (2026-07-15): an investigation concluded a defensive "cover" mechanism "never engages" (and the defect it guards was therefore unfixed) — the conclusion was WRONG because the hand-rolled reproduction drove the target with a DIFFERENT input than the existing test (a HOME key where the test sent a media-pause key), so it never reached the PRECONDITION the mechanism keys off; re-driven with the TEST'S EXACT sequence the mechanism provably ENGAGED. Every investigation / debugging pass / validation that reproduces a defect or exercises a code path MUST: (1) **use the existing reproduction's EXACT sequence** when a working reproduction already exists (a test/harness/recorded sequence that demonstrably reaches

the defect) — same inputs, same order, same timing, same preconditions, same topology (§11.4.3); a hand-rolled approximation is NOT a substitute; (2) **a deviating variant MUST FIRST prove it reaches the precondition** with captured evidence before ANY conclusion is drawn from it; (3) **a repro that misses the precondition proves NOTHING** — it is NOT evidence of absence (§11.4.6), and concluding "defect present"/"defect absent"/"the mechanism never engages" from it is a §11.4/§11.4.1 bluff at the INVESTIGATION layer (a FAIL-bluff when it wrongly condemns working code, a PASS-bluff when it wrongly clears broken code); (4) **record the exact driving sequence as captured evidence** so it is replayable + auditable, every deviation explicit + justified + precondition-proven. STRENGTHENS §11.4.102 (Phase-2 reproduce) / §11.4.146 (reproduce-first) / §11.4.115 (RED on the broken artifact) — they mandate THAT you reproduce, §11.4.199 mandates HOW: with the sequence PROVEN to reach the defect. Classification: universal (§11.4.17). Composes §11.4.1/.3/.5/.6/.7/.102/.107/.115/.123/.135/.146. Propagation gate `CM-COVENANT-114-199-PROPAGATION` (literal `11.4.199`) + recommended gate `CM-EXACT-REPRO-SEQUENCE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.199. Non-compliance is a release blocker. No escape hatch — no `--approximate-repro-OK`, `--skip-precondition-proof`, `--hand-rolled-repro-suffices`, `--absence-proves-absence`, `--deviating-sequence-conclusion-OK` flag.

§11.4.200 — Non-targetable deploy/flash tooling MUST isolate exactly ONE eligible target and VERIFY-AFTER-WRITE on the INTENDED target (research-derived, 2026-07-15). Forensic FACT (2026-07-15): a flashing tool with no device-selection argument AUTO-SELECTED a device from the bus, silently wrote the freshly-built image to the WRONG board (a sibling still attached), printed SUCCESS, and the INTENDED board kept its OLD build — every downstream test then validated a target that had NEVER received the artifact (the §11.4.108 ARTIFACT→RUNTIME layer silently broken while every tool reported green). Whenever a deploy / flash / install / write tool CANNOT address a specific target, the procedure MUST: (1) **targetability is a PRECONDITION** — EITHER pass an explicit STABLE selector (serial/UUID/stable id per §11.4.111, never an enumeration ordinal) OR GUARANTEE exactly ONE eligible target is present at the moment of the write (siblings disconnected/powered-down/excluded), the enumerated eligible-target set

captured as evidence (size == 1); (2) **the tool's own success message is NOT proof** — an "ok"/exit-0 proves only that bytes reached the device the TOOL selected, NEVER the device the OPERATOR intended (§11.4.6); (3) **VERIFY-AFTER-WRITE on the intended target** — read back the deployed artifact's IDENTITY (build id / version / checksum / fingerprint) FROM the intended target and assert it equals the artifact just written; a mismatch or the OLD identity is a FAIL, never a warning (the §11.4.108 ARTIFACT→RUNTIME assertion applied to the deployment step); (4) **no validation on an unverified target** — any test run against a target whose deployed-artifact identity was NOT verified is INVALID and any PASS is a §11.4 PASS-bluff. Honest boundary §11.4.6 — isolation + read-back prove the INTENDED target holds the INTENDED artifact, NOT that the artifact is correct (§11.4.108/.40/.130). Classification: universal (§11.4.17). Composes §11.4.6/.46/.108/.111/.119/.130/.133/.139. Propagation gate `CM-COVENANT-114-200-PROPAGATION` (literal `11.4.200`) + recommended gate `CM-DEPLOY-TARGET-ISOLATED-AND-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.200. Non-compliance is a release blocker. No escape hatch — no `--auto-select-target-OK`, `--trust-tool-success-message`, `--skip-post-write-verify`, `--validate-unverified-target`, `--multiple-targets-on-bus-OK` flag.

§11.4.201 — Every guard/gate MUST assert the REAL condition: a false-positive refusal is a FAIL-bluff, a false-negative pass is a PASS-bluff (research-derived, 2026-07-15). Forensic FACT (2026-07-15, two independent instances in one cycle): (a) a host-budget guard used a bare `pgrep -f '<pattern>'` and REFUSED every worker spawn on a host with ZERO real builds because a live worker's own command line merely QUOTED the pattern (the §11.4.196(D)/§12.12 carrier footgun); (b) a host-safety pre-flight refused a build on a "swap full" reading that did not reflect a real memory-pressure condition, clearing only after the operator manually cycled swap — BOTH guards refused on a PROXY signal that was NOT the real condition. Every GUARD / GATE / PRE-FLIGHT (host-safety, resource-budget, readiness, lock, capability, availability) MUST assert the REAL condition it CLAIMS, resolved from the AUTHORITATIVE source (the real `/proc/<pid>/cmdline`, the real cgroup/meminfo pressure metric, the real lock-holder liveness via `kill -0`, the real device identity), NEVER from a proxy signal that something which is

NOT the condition can satisfy: (1) **a FALSE-POSITIVE REFUSAL is a FAIL-bluff (§11.4.1)** — blocking work when the condition is ABSENT is exactly as forbidden as passing while a defect is present (it halts real work AND teaches operators/agents to bypass guards); (2) **a FALSE-NEGATIVE PASS is a PASS-bluff (§11.4)**; (3) **SELF-VALIDATED (§11.4.107(10))** — every guard ships a golden-TRUE fixture (condition present → guard FIRES) AND a golden-FALSE fixture INCLUDING the known CARRIER/decoy case (condition absent but proxy signal present → guard MUST NOT fire), both wired into the meta-test; a guard firing on its golden-FALSE fixture is ITSELF the defect; (4) **conservative-safe default on an unresolvable signal (§11.4.101/§12)** — refuse the destructive/high-blast-radius action and say so HONESTLY ("could not resolve the condition" is NOT a false positive; claiming a condition never verified IS one, §11.4.6); (5) **every refusal reports its RESOLVED evidence** (real pid+cmdline, real metric+threshold, real lock holder) so a false positive is diagnosable in one step. GENERALISES §11.4.196(D) (resource-detection-by-real-identity — to EVERY guard) + §11.4.180 (liveness PROVEN before reaping — to every guard's condition). Classification: universal (§11.4.17). Composes §11.4.1/.6/.101/.107(10)/.109/.111/.180/.196(D)/§12/§12.6/§12.8/§12.12. Propagation gate `CM-COVENANT-114-201-PROPAGATION` (literal `11.4.201`) + recommended gate `CM-GUARD-ASSERTS-REAL-CONDITION` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.201. Non-compliance is a release blocker. No escape hatch — no `--proxy-signal-OK`, `--substring-match-guard`, `--false-positive-refusal-acceptable`, `--unvalidated-guard`, `--refuse-without-evidence` flag.

EXTENSIONS landed 2026-07-15 (existing anchors strengthened — full text in `Constitution.md`). §11.4.89(G) subagent long-op discipline + conductor-owns-inherently-long-ops: a SUBAGENT MUST NOT run a single synchronous op exceeding the runtime's no-progress stream watchdog (~5 min) — it backgrounds + polls (each poll IS the progress the watchdog observes); inherently-long ops (full builds / full suites / device flashes) are the CONDUCTOR's to run (no such watchdog); a stall-killed subagent is a CRASH per §11.4.147 (respawned, never absorbed as a completion); watchdog budget read from the runtime, never guessed (§11.4.6). **§11.4.106(F)** write-seam hook enforcement: the doc/DB sync MUST be enforced at the COMMIT seam (hook refuses a commit whose staged set touches a chain source while its exports/

DB are stale) + the BUILD seam (pre-build gate) + the CONSTITUTION-PULL seam (§11.4.164 post-update hook) — a stale CONTINUATION (§12.10) / session-resumption file (§11.4.131) / memory artefact while live state moved is a violation; honest boundary §11.4.6 — seam hooks give eventual-consistency-at-every-write, NOT literal real-time (that needs a watch daemon, carried as a tracked §11.4.197 upgrade path, never claimed as shipped). **§11.4.147(e)** API-QUOTA / RATE-LIMIT EXHAUSTION is a FIRST-CLASS CRASH CLASS: an agent killed mid-work by a weekly/session/subscription quota boundary is `crashed` (never `complete`) — record the REAL limit signal + reason-class per §11.4.196(B), REBIND to the next OPERATIONAL alias per §11.4.196(A)/(C) and RESPAWN there (bounded-PARK per §11.4.101 only when EVERY alias is cooled), RESUME from the killed agent's captured transcript + partial state at its EXACT last point, and the §11.4.87/.94/.97/.126 loop done-condition MUST NOT read satisfied while any quota-killed entry is non-`complete`. **§11.4.196(F)** CONFIGURED ≠ IN USE: forensic FACT — the alias orchestrator was installed with a valid config yet its binding registry was EMPTY while THREE live workers ran outside it, so `promote` rebound ZERO tracks; EVERY spawn path MUST bind THROUGH the orchestrator, and the mechanism's readiness is its RUNTIME SIGNATURE (§11.4.108) — **the binding-registry rows MUST EQUAL the live-worker set** — an empty/partial registry alongside live workers is a FALSE-READY state and claiming "ready/in use" on config alone is a §11.4/§11.4.6 orchestration-layer bluff.

§11.4.202 — Reporting directives: a report MUST auto-create a fully-populated, fully-synced workable item — never a prose acknowledgement (User mandate, 2026-07-15). Verbatim operator mandate: introduce reporting directives `ISSUE` / `BUG` / `TASK` that create a proper workable item filled with all details, with the WHOLE workable-items system automatically up to date and fully in sync — the main DB, all documents, and all external tracking systems; universal, reusable on every project, fully decoupled, recognized immediately by every project that pulls the constitution. Every governed project MUST expose REPORTING DIRECTIVES — the §11.4.140 registered actions **`BUG`** (Type=Bug), **`TASK`** (Type=Task), and **`ISSUE`** (generic report, CLASSIFIED into the §11.4.16 closed set) — such that a plain-language report from the operator is TURNED INTO a real, tracked, fully-synced workable item, automatically, every time. A report that is discussed, acknowledged, answered in prose, or "noted" WITHOUT landing a tracked item is a §11.4.202 violation of

§11.4 PASS-bluff severity at the requirements-intake layer: the requirement was accepted and then silently evaporated (the §11.4.197 loss-of-requirements failure, at its entry point). **(1) GRAMMAR (§11.4.140).** The three directives are ordinary registry rows, so EVERY §11.4.140 form works out of the box (`BUG :: <text>` , `DEFAULT::BUG :: <text>` , `/DEFAULT::BUG <text>` , `BUG ---> <text>`), PLUS a SIXTH form this anchor adds — the **single-colon form** **`NAME: <text>`** (`ISSUE: subtitles are late`), the shape operators actually type. The single-colon form is **REGISTERED-ACTION-ONLY by construction**: a single-colon token that is NOT a registered action is a NO-OP (an ordinary prompt), NEVER an ASK and NEVER a sub-system shortcut — because ordinary English prose (`TODO:` , `WARNING:` , `FIXME:`) shares that shape (`NOTE:` is now a registered §11.4.140 severity marker), and routing it to the §11.4.66 clarify path would question every such sentence (§11.4.6 honest boundary: the other five forms keep their ASK-on-unknown behaviour — their shapes do not occur in prose). Host-command collisions are handled by the EXISTING §11.4.140 conflict mechanism (`slash_bare: auto` + `slash_conflicts: [..]`), which already satisfies the operator's explicit-prefix requirement: a bare `/NAME` that collides with a built-in (e.g. the documented Claude Code built-in `/bug`) is NOT honored, while the namespaced `/DEFAULT::NAME` form is ALWAYS unambiguous — and the `NAME:` / `NAME ::` / `NAME --->` forms are never affected by any host command. **(2) THE ITEM.** The created item MUST carry, on ALL surfaces: a valid Status (§11.4.15, `Queued` at intake) + a Type from the CLOSED set {Bug | Feature | Task} (§11.4.16 — `ISSUE` is a REPORTING CHANNEL, never a fourth type; inventing one is a violation) + a stable auto-incremented id (§11.4.54), and a COMPREHENSIVE structured description (§11.4.148 D2 / §11.4.171): what / affected-scope / reproduction / acceptance. An undetermined section is recorded as an explicit `UNKNOWN:` gap — NEVER invented (§11.4.6). For `ISSUE` , the type is classified from the report's content and stated as FACT; if the content does not determine it, the operator is ASKED (§11.4.66 / §11.4.105) BEFORE creation; ONLY when running autonomously where asking is impossible (§11.4.101) may the §11.4.16 lowest-stakes ambiguity default `Task` be used — and then the defaulted classification MUST be recorded verbatim in the item and surfaced for reclassification, never silently asserted. **(3) THE FULL SYNC (the load-bearing half).** Creating the item is NOT sufficient: the directive MUST drive the WHOLE workable-items system

into sync, in one shot — (a) the item lands in the SQLite single-source-of-truth (§11.4.93 / §11.4.95); (b) EVERY derived document is regenerated FROM the DB — the open + closed trackers, their summaries, and their `.md` / `.html` / `.pdf` / `.docx` siblings (§11.4.12 / §11.4.53 / §11.4.65 / §11.4.106); (c) the item is pushed to EVERY configured external tracker (§11.4.148 D5). **(4) NEVER A FAKED PUSH (§11.4.10 / §11.4.6 / §11.4.3).** A tracker whose credentials or whose client are absent is SKIPPED with an honest machine-readable reason (`credentials_absent` — names of the unset variables ONLY, never a value; `tracker_client_absent` — PENDING-OPERATOR-INPUT). A tracker PASS may be recorded ONLY after a real push command really exited 0. Fabricating, assuming, or silently omitting a push is a §11.4 bluff at the integration layer; a missing tracker NEVER blocks the item from landing in the DB + docs (the report is never lost because a mirror is unreachable). **(5) DECOUPLED ENGINE, CONSUMER-OWNED DATA (§11.4.28 / §11.4.177).** The item-creation + full-sync engine lives in the constitution submodule (`constitution/scripts/reporting/report_item.sh`), is inherited BY REFERENCE (never copied), and carries ZERO project literals; every project-specific value — DB path, id prefix, sync command, tracker commands + their required env vars — comes from a CONSUMER-OWNED config file (DATA, never an engine edit). The engine FAILS CLOSED with an actionable message when that config is absent — it never guesses a project's paths (§11.4.6). **(6) ANTI-BLUFF (§11.4 / §11.4.107(10)).** The mechanism ships four-layer coverage: a pre-build gate (`CM-REPORTING-DIRECTIVES`) whose FUNCTIONAL invariant SOURCES and RUNS the parser (never a grep-only assertion, §11.4.108), a paired §1.1 mutation test that proves each invariant is genuinely load-bearing, and an end-to-end suite run against a REAL SQLite DB with REAL binaries (no mocks, §11.4.27) that asserts a real row lands with the right Type/Status/id, that the docs are regenerated FROM the DB, and that an absent tracker SKIPs honestly — self-validated with a golden-good + golden-bad + negative-control fixture set (an oracle that passes its golden-bad fixture is itself the bluff). Classification: universal (§11.4.17) — the consuming project supplies its DB path, id prefix, sync command, and tracker bindings per §11.4.35. Composes §11.4.140 (grammar) / §11.4.93 / §11.4.95 (DB SSoT) / §11.4.15 / §11.4.16 / §11.4.54 (status + type + id) / §11.4.148 / §11.4.171 (comprehensive description) / §11.4.106 / §11.4.12 / §11.4.53 / §11.4.65 (doc

sync) / §11.4.10 (credentials) / §11.4.66 / §11.4.105 (clarify, never guess) / §11.4.101 (autonomous decision) / §11.4.197 (a started requirement never evaporates) / §11.4.28 / §11.4.177 (decoupling) / §11.4.164 (auto-registration on constitution pull) / §11.4.107(10) / §1.1. Propagation gate `CM-COVENANT-114-202-PROPAGATION` (literal `11.4.202`) + recommended gate `CM-REPORTING-DIRECTIVES` (registry declares ISSUE/BUG/TASK + the single-colon form is registered-only; the engine exists, is parse-clean, and is project-literal-free; the three directives really expand at runtime while prose does not; the honest-skip reasons exist and a tracker PASS is gated on a real exit 0) + paired §1.1 mutation (strip an action row, flip the single-colon form to non-registered-only, strip the parser branch, remove the prose NO-OP branch, make the engine fake a tracker push, or couple the engine to one project → the gate FAILs; strip the literal → the propagation gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.202.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--report-without-item`, `--prose-acknowledgement-OK`, `--skip-item-creation`, `--skip-tracker-sync`, `--fake-tracker-push`, `--invent-a-fourth-type`, `--hardcode-project-in-engine` flag.

§11.4.207 — Instant multi-stream resume engine: whole-fleet continuation state is a durable, content-addressed, atomically-committed snapshot resumed in O(changed) reads, not an O(history) re-read (User mandate, 2026-07-15). Compact summary: the continuation mechanism (§12.10 / §11.4.127 / §11.4.131 / §11.4.205) is EXTENDED — never replaced — by a mechanical engine so a fresh session resumes ALL work (every Track, main stream, and agent) at once, INSTANTLY and at token cost O(changed streams), by reading ONE snapshot manifest + one compact blob per stream, NEVER re-reading the full handoff history. (1) **Content-addressed Merkle store** — each stream's canonical state bytes are named by their sha256 (dedup: unchanged streams share a hash → a global fleet snapshot costs O(changed)); a snapshot is ONE manifest referencing every stream's content-hash (§11.4.205(4)). (2) **O(streams) resume, O(history) NEVER on the resume path** — `RestoreAll` reads the manifest + one blob per stream; the append-only event ledger is audit + lost-update recovery only, never read to resume (§11.4.127). (3) **Tamper-evident + deterministic** — every blob is integrity-checked on read (bytes MUST hash to their id) and a deterministic

round-trip (re-serialize must byte-match), no wall-clock in the hashed content, verified by a SELF-VALIDATING oracle: golden-good PASS + golden-bad detected FAIL + negative-control (a legitimately-older-but-valid snapshot) PASS — the false-positive guard proving the verifier distinguishes a LAGGING copy from a TAMPERED one (§11.4.201/§11.4.107(10)/§11.4.206(3)). (4) **Durable + atomic + single-writer + crash-safe** — atomic ref write (temp→fsync→rename→dir-fsync, §11.4.205(6)), append-only ledger (§11.4.205(6)), single-writer-per-stream refusal (§11.4.206), advisory lock with provably-stale reap (`kill -0` liveness, never steals a live lock, §11.4.180/§9.2). (5) **Project-agnostic engine, consumer-owned data** — the engine (`github.com/vasic-digital/continuum` , GitLab mirror) carries ZERO project literals, fails closed rather than guess a store path (§11.4.6/§11.4.177), and is inherited BY REFERENCE as a depth-1 submodule under the §11.4.28(C) carve-out (`constitution/submodules/continuum/` , `helix-deps.yaml` , zero own-org deps); every consumer supplies its store path / actor / stream naming as DATA. Anti-bluff (§11.4/§11.4.108): the whole engine ships four-layer coverage (unit + integration + e2e + stress + chaos §11.4.85, race-clean) with a measured resume Metric (`blobs_read == 1 + streams` is the O(streams) proof) and the self-validating oracle as captured evidence (§11.4.5/§11.4.69/§11.4.107); honest boundary (§11.4.6): it snapshots the state an agent REPORTS, not its execution image (not a CRIU/durable-execution runtime, §11.4.112), and `est_tokens` is a documented `bytes/4` heuristic, not a tokenizer count. Classification: universal (§11.4.17). Composes §12.10 / §11.4.127 / §11.4.131 / §11.4.205 / §11.4.116 / §11.4.147 / §11.4.180 / §11.4.201 / §11.4.206 / §11.4.107(10) / §11.4.28 / §11.4.177 / §11.4.85 / §9.2. Propagation gate `CM-COVENANT-114-207-PROPAGATION` (literal `11.4.207`) + recommended gate `CM-CONTINUUM-RESUME-ENGINE-PRESENT` (the engine present under `constitution/submodules/continuum/` , `go test -race ./...` green, `continuum selfcheck` = good=PASS/bad=FAIL/negctrl=PASS, zero project literals) + paired §1.1 mutation (strip the literal → propagation gate FAILs; downgrade the resume path to read the ledger, or the oracle to pass its golden-bad → the mechanism gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.207. Non-compliance is a release blocker. No escape hatch — no `--resume-reads-history` , `--skip-integrity-check` ,

`--unvalidated-oracle-OK`, `--hand-typed-resume-state`, `--per-project-resume-reimpl` flag.

§11.4.208 — Operator-request-history document: every project maintains a project-local, always-in-sync ledger of every operator request/prompt (User mandate, 2026-07-15). Verbatim operator mandate: introduce a request-history document capturing per operator request/prompt — full content, accepted-timestamp with explicit timezone, the track that processed it, the alias that took the workable item, and the model + effort used — always in sync, no false data, no bluff; the operator's load-bearing correction (§11.4.35/§11.4.17): the RULE is universal (→ constitution submodule), the request-history DOCUMENT itself is PROJECT-LOCAL (never placed into the constitution). Every governed project MUST maintain a single project-local, append-only, newest-first **operator-request-history document** at a fixed project-declared path (e.g. `docs/requests/history.md`; §11.4.35, never moved without a §11.4.66 operator decision), each entry carrying EXACTLY: (1) **Request content** (verbatim where archived, else a faithful complete summary, marked which); (2) **Accepted (when)** — date + time + TIMEZONE stated EXPLICITLY on every entry (consuming project declares its default zone §11.4.35; date-only ⇒ `time UNKNOWN`, never fabricated); (3) **Track** (§11.4.176/.178/.182); (4) **Alias** (§11.4.182/.196); (5) **Model + effort**. (A) **NO INVENTION (§11.4.6)** — an unrecoverable field is recorded literally `UNKNOWN`, never guessed; a fabricated timestamp/alias/model is a §11.4/§11.4.1 bluff at the requirements-record layer. (B) **HONEST RECONSTRUCTION BOUNDARY** — pre-mandate sessions are best-effort reconstructed from the durable dated record with unrecoverable fields marked `UNKNOWN` and the boundary stated explicitly, never silently implied complete. (C) **ALWAYS IN SYNC + EXPORTED** — §11.4.44 revision header, §12.10 append-on-every-new-prompt, §11.4.65/§11.4.73 four-format export (`.md` + `.html` + `.pdf` + `.docx` where applicable, siblings ≥ `.md`). (D) **KEEP-APPLYING MECHANISM** — a helper script and/or a `UserPromptSubmit`-class hook appends a newest-first row per NEW prompt at capture time (track/alias derived deterministically §11.4.182, honest `?` / `UNKNOWN`); a helper landed WITHOUT the auto-capture hook is honestly a partial mechanism (hook wiring = tracked §11.4.197/§12.10 follow-up, never claimed as auto-capture it does not perform). (E) **COMPLEMENTS, NEVER REPLACES** any per-session no-loss ledger (§11.4.197/.202), the workable-items DB SSoT (§11.4.93/.95), or the §12.10/§11.4.131 resumption artefacts;

requirement loss at intake is FORBIDDEN (§11.4.197). (F) **RULE-UNIVERSAL / DOCUMENT-PROJECT-LOCAL (§11.4.35)** — the RULE lives in the constitution submodule; the document + its path + default timezone are consumer-owned project DATA (§11.4.28 decoupling), NEVER placed into the constitution. Classification: universal (§11.4.17). Composes §11.4.6/.35/.44/.65/.73/.118/.131/.176/.178/.182/.196/.197/.202/.93/.95/§12.10. Propagation gate `CM-COVENANT-114-208-PROPAGATION` (literal `11.4.208`) + recommended gate `CM-REQUEST-HISTORY-DOC-PRESENT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.208. Non-compliance is a release blocker. No escape hatch — no `--request-history-optional`, `--skip-request-log`, `--no-timezone-stamp`, `--invent-missing-field`, `--request-history-in-constitution`, `--prompt-without-history-entry` flag.

§11.4.209 — Code-review MUST run on the Opus model at xhigh effort as PRIMARY substrate, with a genuine-unavailability Sonnet fallback — no Fable, no escalation tier (User mandate, 2026-09-26, supersedes 2026-09-15 and 2026-07-15; amended 2026-10-04 to add the Sonnet-fallback clause). Verbatim operator mandate (2026-09-26, `CRITICAL`): "Main model to use is Sonet, reviews and other heavy stuff have to be done by Opus instead of Fable!!! Fix this permanently on all levels!" **Verbatim operator mandate (2026-10-04, `CRITICAL`), amending clause (A) — adds a genuine-unavailability fallback, never changes the primary substrate:** "If we do not have access to Opus model for some reason but only to Sonnet, we MUST USE what we have! Reviews MUST BE done. Ideally with Opus model, however Sonet MUST BE used if Opus for any reason cannot be used! Add this / extend into main constiotution Submodule and apply the rule asap so nothing is blocked with review which is not performed yet! We MUST BE maximally efficient! Work MUST BE performed with all means we have at particular moment!" Every mandatory independent code-review the constitution requires — the §11.4.125 code-review-agent gate before pre-build + main build, the §11.4.142 universal every-change-review (no exception), the §11.4.194 exhaustive all-scenario/all-angle review, iterated to a zero-finding GO per §11.4.134 — MUST be dispatched on the **Opus model at `xhigh` effort** as PRIMARY substrate, whatever the size or complexity of the reviewed change. (A) **PRIMARY Opus-xhigh + genuine-unavailability Sonnet fallback (amended 2026-10-04) — no escalation tier, no Fable** — Fable is NOT

permitted in any role, ever (the 2026-09-15 Fable-escalation clause and the 2026-07-15 Fable-first mandate remain WITHDRAWN, kept verbatim as lineage in `Constitution.md`); no model other than Opus or the sanctioned Sonnet fallback below substitutes, and no lower effort for Opus itself is acceptable; when Opus-xhigh is genuinely unavailable (a captured FACT §11.4.6, never a guessed unavailability) the review MUST be dispatched on **Sonnet** (at the highest effort the dispatch path can set) rather than blocked — the sole sanctioned fallback, per the verbatim 2026-10-04 mandate above: reviews MUST be done with whatever capable means are actually available, Opus ideally, Sonnet when Opus genuinely cannot be used; the Opus-unavailability FACT and the Sonnet substitution MUST both be recorded honestly in the review's captured evidence (§11.4.5/.69) — a Sonnet-fallback review is NEVER silently presented as if it ran on Opus; a review is NEVER left undone / blocked solely because Opus is unavailable while Sonnet is reachable (§11.4.183 maximal multi-agent utilization). ONLY when BOTH Opus and Sonnet are genuinely unavailable (each established as its own captured FACT per §11.4.6, never guessed) is the review BLOCKED and deferred — the fact recorded, the owed review tracked (§11.4.197/§11.4.21); `xhigh` is REQUIRED on whichever substrate actually runs WHERE THE DISPATCH PATH CAN SET IT (the §11.4.194 all-angle depth is unreachable lower) — where it cannot (the pre-existing §11.4.231(F.2) harness gap), clause (C)'s honesty rule governs: the gap is recorded, never silently claimed as xhigh unverified. (B) **Independence preserved (§11.4.70/.20)** — the reviewing model (Opus or the sanctioned Sonnet fallback) is the structurally-separated reviewer, never the author; §11.4.209 sets the review's MODEL + EFFORT, not its independence / iterate-until-GO (§11.4.134) / captured-evidence (§11.4.5/.69/.107). (C) **Honest boundary (§11.4.6)** — a review claimed done that ran on Fable, Haiku, or any model other than Opus or the sanctioned Sonnet fallback, or below `xhigh`, or on Sonnet with no recorded Opus-unavailability fact, is a §11.4 bluff at the review-substrate layer; the review's captured evidence records which model + effort actually performed it (where the dispatch path cannot set or report effort, §11.4.231(F.2), that gap is recorded instead of a claimed `xhigh`). (D) **Values already burned in a public repository (§11.4.10)** — any plaintext credential value that has appeared in documentation, a spec, or commit history is compromised from the moment of commit; an illustration never embeds a real-looking username/

password pair without stating it is compromised-by-disclosure. STRENGTHENS §11.4.125/.142/.194/.134 — does NOT replace them; it pins the review's execution substrate, now a PRIMARY-plus-sanctioned-fallback pin rather than a single-model pin. Classification: universal (§11.4.17). Composes §11.4.125/.134/.142/.194/.6/.10/.20/.21/.70/.183/.196/.197/.231(A — the `sonnet` rung this fallback draws on). Propagation gate `CM-COVENANT-114-209-PROPAGATION` (literal `11.4.209`) + recommended gate `CM-CODE-REVIEW-OPUS-XHIGH` (amended 2026-10-04: now accepts model=Opus+xhigh OR model=Sonnet carrying a captured Opus-unavailability fact; FAILs on Fable/Haiku/any other model, or on Sonnet with no recorded fact; gate-code = separate work item, NOT claimed shipped §11.4.6) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.209. Non-compliance is a release blocker. No escape hatch beyond the sanctioned (A) Sonnet fallback (itself requiring a captured Opus-unavailability FACT, never a guess) — no `--review-any-model`, `--review-lower-effort-OK`, `--skip-opus`, `--non-xhigh-review`, `--escalate-to-fable-or-haiku`, `--sonnet-fallback-without-recorded-fact`, `--leave-review-blocked-while-sonnet-reachable` flag.

§11.4.210 — Zero-loss request/prompt intake: every operator request/prompt MUST be mechanically CAPTURED, TRACKED, and PROCESSED — never skipped, ignored, avoided, or lost (User mandate, 2026-07-15). Verbatim intent: "every request/prompt we issue MUST ALWAYS be respected, taken into account, executed and processed! No avoiding, ignoring, skipping or any form of loss!" Every operator request/prompt MUST be (a) CAPTURED in the §11.4.208 request-history ledger MECHANICALLY at intake via a WIRED verified-runtime-state `UserPromptSubmit` auto-capture hook (NOT a manual step, NOT a "tracked follow-up" — §11.4.210 PROMOTES §11.4.208(D)'s hook from partial-follow-up to MANDATORY per the §11.4.205 enforced-not-advisory pattern; forensic FACT 2026-07-15: an entire release-drive session's prompts went un-recorded because the hook was left un-wired); (b) TRACKED → a §11.4.202 workable item / §11.4.197 completion, or an explicit evidence-backed no-op-with-reason; (c) PROCESSED — respected/executed, never dropped. Loss at ANY of the three stages is a §11.4 PASS-bluff at the requirements-intake layer of §11.4.197's severity class. Docs-chain-bound (§11.4.106), gated (§11.4.75 — hook verified-installed in the live hook path + content-matched,

ledger fresh); the autonomous loop (§11.4.126/.87/.94/.97/.103) processes every captured request (done-condition never satisfied while any captured request is un-processed, §11.4.147(e)); honest §11.4.6 (UNKNOWN never invented). Classification: universal (§11.4.17). Composes §11.4.197/.202/.208/.205/.106/.126/.75/.6/.147(e). Propagation gate `CM-COVENANT-114-210-PROPAGATION` (literal `11.4.210`) + recommended gate `CM-REQUEST-CAPTURE-HOOK-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.210. Non-compliance is a release blocker. No escape hatch — no `--skip-request-capture`, `--manual-ledger-OK`, `--hook-optional`, `--drop-unprocessed-request`, `--ledger-may-be-stale` flag.

§11.4.211 — Merge-conflict resolution during main→feature/product/flavor merges MUST run on the Opus model at xhigh effort as PRIMARY substrate, with a genuine-unavailability Sonnet fallback — no Fable (User mandate, 2026-07-16; amended 2026-09-26; amended again 2026-10-04 to add the Sonnet-fallback clause). Verbatim operator mandate (2026-09-26, `CRITICAL`), superseding the model choice of the 2026-07-16 mandate: "Main model to use is Sonet, reviews and other heavy stuff have to be done by Opus instead of Fable!!! Fix this permanently on all levels!" **Verbatim operator mandate (2026-10-04, `CRITICAL`), amending clause (A) — adds a genuine-unavailability fallback, never changes the primary substrate:** "If we do not have access to Opus model for some reason but only to Sonnet, we MUST USE what we have! Reviews MUST BE done. Ideally with Opus model, however Sonet MUST BE used if Opus for any reason cannot be used! Add this / extend into main constiotution Submodule and apply the rule asap so nothing is blocked with review which is not performed yet! We MUST BE maximally efficient! Work MUST BE performed with all means we have at particular moment!" The 2026-07-16 mandate ("During the merge of main branch to all feature/product/flavor branches of tracks 2 - 4, if conflicts happen they MUST BE resolved using Fable model with xhigh effort! If Fable is not available, then with Opus with xhigh!") remains WITHDRAWN as to its model choice and kept as lineage; its scope and its `xhigh` requirement stand. Any merge-CONFLICT resolution performed while integrating the canonical trunk (`main` / `master`) into a feature / product / flavor branch — the §11.4.188 regular main→feature merge cadence, the §11.4.195 branch-taxonomy integration, the §11.4.41 merge-first pipeline, the §11.4.167(D) trunk-into-

stream sync AND the §11.4.167(I) approval-gated back-merge — MUST be carried out on the **Opus model at `xhigh` effort** as PRIMARY substrate; the merge-resolution analogue of §11.4.209, pinning it for the DECISION-MAKING of conflict resolution (which side to keep, how to UNION both sides without loss, marker removal, semantic reconciliation of divergent edits). **(A) PRIMARY Opus-xhigh + genuine-unavailability Sonnet fallback (amended 2026-10-04) — no Fable:** Fable is NOT permitted in any role, ever, and no lower effort for Opus itself is acceptable; when Opus-xhigh is genuinely unavailable (not configured / rate-limited / unreachable, established as FACT §11.4.6 — never a guessed unavailability) the conflict resolution MUST be carried out on **Sonnet** (at the highest effort the dispatch path can set) rather than blocked — the sole sanctioned fallback, per the verbatim 2026-10-04 mandate above; the Opus-unavailability FACT and the Sonnet substitution MUST both be recorded honestly in the merge's captured evidence (§11.4.5/.69) — a Sonnet-resolved merge is NEVER silently presented as if Opus resolved it. ONLY when BOTH Opus and Sonnet are genuinely unavailable (each established as its own captured FACT per §11.4.6, never guessed) is the resolution BLOCKED and deferred — the fact recorded, the merge NOT completed, the branch left quiescent with no conflict marker committed; `xhigh` REQUIRED on whichever substrate actually runs WHERE THE DISPATCH PATH CAN SET IT (a careless resolution silently drops a commit or commits a marker) — where it cannot (the pre-existing §11.4.231(F.2) harness gap), clause (D)'s honesty rule governs: the gap is recorded, never silently claimed as xhigh unverified. **(B) Scope:** the operator named tracks 2–4 (`feature` / `product` / `flavor`); GENERALISES per §11.4.17 to EVERY controlled main→(feature|product|flavor) merge on EVERY track that hits a conflict. **(C) Safety preserved:** sets the MODEL + EFFORT of the conflict-resolution work only, BOUNDED BY §11.4.113 (NEVER force-push) / §9 / §9.2 (no commit lost, hardlinked pre-op backup before any large/risky merge) / §11.4.41 step 3 (ZERO conflict markers committed, ZERO file silently dropped, union of both sides preserved) / §11.4.188(4) anti-bluff (post-merge smoke GREEN + marker-scan empty + no-lost-commit) / §11.4.84 (quiescent-only) / §11.4.37/§11.4.71 (fetch-first). **(D) Honest boundary (§11.4.6):** a merge whose conflicts were resolved on Fable or any model other than Opus or the sanctioned Sonnet fallback, or below `xhigh`, or on Sonnet with no recorded Opus-unavailability fact, is a §11.4 bluff at the merge-

resolution-substrate layer — the merge's captured evidence records which model + effort resolved the conflicts (and, when blocked, the FACT that both Opus and Sonnet were unavailable). Where the dispatch path cannot set or report the effort (the §11.4.231(F.2) harness gap), the evidence records that gap honestly and the merge is never claimed to have run at `xhigh` on an unverified basis. STRICT EXTENSION of §11.4.209 to merge-CONFLICT resolution; STRENGTHENS §11.4.188/§11.4.195/§11.4.41 by pinning the substrate of their conflict-resolution step. Classification: universal (§11.4.17). Composes §11.4.209/.183/.188/.195/.41/.113/§9/§9.2/.6/.167(D)(I)/.84/.37/.71/.196/.231(A — the `sonnet` rung this fallback draws on). Propagation gate `CM-COVENANT-114-211-PROPAGATION` (literal `11.4.211`) + recommended gate `CM-MERGE-CONFLICT-OPUS-XHIGH` (amended 2026-10-04: now accepts model=Opus+xhigh OR model=Sonnet carrying a captured Opus-unavailability fact; FAILs on Fable/any other model, or on Sonnet with no recorded fact; gate-code = separate work item, NOT claimed shipped §11.4.6) + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.211. Non-compliance is a release blocker. No escape hatch beyond the sanctioned (A) Sonnet fallback (itself requiring a captured Opus-unavailability FACT, never a guess) — no `--merge-conflict-any-model`, `--skip-opus-merge-resolution`, `--non-xhigh-merge-conflict`, `--resolve-conflict-lower-effort-OK`, `--escalate-to-fable`, `--sonnet-fallback-without-recorded-fact`, `--leave-merge-blocked-while-sonnet-reachable` flag.

§11.4.212 — Main README is the canonical starting-point / entry point for ALL project documentation — no doc may be an orphan unreachable from README (User mandate, 2026-07-16). Verbatim operator mandate: "Manual tests catalog must have linking all through the main README as the starting point of this and any other Project documentation! This fact and mandatory rule MUST be added into the constitution." The main README (`README.md` at the repository root) is the canonical STARTING POINT / entry point for ALL project documentation, for THIS project and EVERY project incorporating this constitution. Every project document in the §11.4.65 export scope — INCLUDING the manual-tests catalog (the §11.4.185 manual-QA testing catalog), the §11.4.153 per-feature Status / Status_Summary docs, the Issues / Fixed trackers + their summaries (§11.4.12 / §11.4.53 / §11.4.57), and every guide / research note / changelog / script companion doc — MUST be

reachable / linked (DIRECTLY or TRANSITIVELY) from the main README. No project document may be an ORPHAN — a doc that no link-path from README reaches is a §11.4.212 violation. STRICT GENERALISATION of §11.4.57 (which mandated a `Tracked-Items + Status Documents` doc-link section covering the tracker / status set) to EVERY §11.4.65-scope document, so the README is the single navigable root of the project's documentation tree. **(A) Reachability = a link path.** README → guide → sub-doc is fine (transitive); the requirement is that SOME path from README reaches every in-scope doc, not that every doc is linked directly at top level. **(B) Always-sync (§11.4.59 / §11.4.106 docs-chain).** A newly-added §11.4.65-scope doc that no README-reachable path links is a §11.4.212 violation the moment it lands; the manual-tests catalog specifically MUST be linked from the README (the operator's load-bearing example). **(C) Honest boundary (§11.4.6).** Reachability is a link-graph property (every doc FINDABLE from README), NOT a claim about each doc's CORRECTNESS or freshness (those stay §11.4.44 / §11.4.106 / the doc's own gates). **(D) Composition with §11.4.57.** §11.4.57's `Tracked-Items` section remains the mandated README landing block for the tracker / status set; §11.4.212 adds that EVERY OTHER in-scope doc is likewise reachable — §11.4.57 is the specialisation, §11.4.212 the generalisation. Classification: universal (§11.4.17) — the consuming project supplies its README path, its manual-tests-catalog path, and its §11.4.65 doc-set enumeration per §11.4.35. Composes §11.4.57 / §11.4.59 / §11.4.65 / §11.4.106 / §11.4.153 / §11.4.185 / §11.4.44 / §11.4.12 / §11.4.53. Propagation gate `CM-COVENANT-114-212-PROPAGATION` (literal `11.4.212`) + recommended gate `CM-README-DOC-ENTRYPOINT-COMPLETE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.212. Non-compliance is a release blocker. No escape hatch — no `--orphan-doc-OK`, `--readme-not-entrypoint`, `--skip-readme-link`, `--doc-unreachable-OK` flag.

§11.4.213 — FEATURE research-scheduling directive: recognized via all §11.4.140 forms, SCHEDULES (never synchronously executes) a deep, enterprise-grade research + implementation-planning effort as a tracked workable item (operator-directed mandate, 2026-07-16). Compact summary: extends the §11.4.202 reporting-directive family with a fourth registered action `FEATURE` — recognized via ALL SIX §11.4.140 grammar forms (`FEATURE :: x` / `DEFAULT::FEATURE :: x` / `/FEATURE x` / `/DEFAULT::FEATURE x` /

`FEATURE ---> x` / the single-colon `FEATURE: x`, registered-action-only so lowercase prose like "feature: ..." never expands and is never asked about, §11.4.6) — that SCHEDULES rather than synchronously executes a deep research + implementation-planning effort: the directive creates a Type=Task / Status=Queued workable item (§11.4.15/.16/.54) by INVOKING the SAME §11.4.202 `report_item.sh` engine (never a duplicate implementation of the item-creation/DB-sync/tracker-push machinery — a second divergent tracker-push path would itself be a bluff-surface regression) and appends it to a durable `docs/requests/feature_queue.md` queue (mirroring `BACKGROUND`'s `docs/requests/background_queue.md` pattern) so a scheduled request is never dropped; the item's comprehensive description (§11.4.148/.171) embeds the FULL research mandate as its acceptance-defining work program — deep web research on best incorporation (§11.4.8/.99/.150), systematic-debug of every enumerated weak-spot/gap/danger-zone with a risk-free rock-solid design per gap (§11.4.102), always enterprise-grade/bleeding-edge/innovative, exhaustive documentation down to lines-of-code + micro-POCs + diagrams + SQL + templates (§11.4.65/.73), reuse-first investigation of vasic-digital/HelixDevelopment components kept decoupled (§11.4.28/.74/.177), full test-type + Challenges + HelixQA planning (§11.4.27/.169), fine-grained phased/task/subtask breakdown, enterprise-scalability + max-performance planning, OpenDesign UI/UX wireframes where applicable (§11.4.162/.190), full CodeGraph integration (§11.4.78/.79/.80), and fully-synced workable items across the SQLite SSoT + every connected external tracker (§11.4.93/.95/.148 D5), honestly SKIPPING any absent tracker (§11.4.10) — NEVER a faked push. The multi-day research itself is EXECUTED by the standing autonomous loop (§11.4.87/.94/.97/.103/.126) when it claims the item, driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under §11.4.197 — never left un-wired in the backlog; "scheduled" is never reported as "done" (§11.4.6). Anti-bluff four-layer coverage: pre-build gate `CM-FEATURE-DIRECTIVE` (FUNCTIONAL — sources + RUNS the §11.4.140 parser, never grep-only, §11.4.108) + paired §1.1 mutation + a real-DB end-to-end evidence trail (temp SQLite DB, no mocks, §11.4.27). Classification: universal (§11.4.17). Composes §11.4.140/.202/.197/.8/.99/.150/.102/.65/.73/.28/.74/.177/.27/.169/.162/.190/.78/.79/.80/.93/.95/.1
Propagation gate `CM-COVENANT-114-213-PROPAGATION` (literal `11.4.213`) + recommended gate `CM-FEATURE-DIRECTIVE` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.213. Non-compliance is a release blocker. No escape hatch — no `--feature-runs-synchronously`, `--skip-feature-queue`, `--feature-without-research-mandate`, `--reimplement-tracker-push`, `--hardcode-project-in-engine`, `--skip-item-creation` flag.

EXTENSIONS landed 2026-07-17 (existing anchors strengthened — full text in `Constitution.md`; research-derived from a consuming project's status-without-custody forensics). Forensic FACTs (genericised, all measured 2026-07-17): 576 tracked items, 182 claiming a done/ready status, **173 (95%) with NO registered regression guard** — an operator manually sampled 6 done-claiming items and found **6/6 broken**, the expected draw of that arithmetic, not bad luck; recorded reopen-per-recorded-fix **52%** (published elite <10%), itself an UNDERCOUNT because recurrences re-entered the tracker as NEW ids and statuses moved with ZERO history rows (one item: 4 Reopened events, 0 terminal events); the standing guard suite's exit code read ONLY its FAIL count — measured live on a release-candidate target: **61 registered guards → PASS 0 / FAIL 5 / PENDING 56 → exit 0 "not blocked"** — and the release-tag tooling consulted the guard suite **ZERO times**, so to the release gate "never verified" and "verified good" were the same colour; the project's own history selected the discriminator — every fix whose done-claim was preceded by an on-device RED→GREEN polarity flip that HELD recorded zero reopens, and every fix confirmed at source-green bounced. **No new anchor number was minted**: §11.4.115 and §11.4.146 already said what the operator mandates and did not bind because they are prose consulted by agents, not conditions checked by seams — the deliverable is the MECHANICAL BINDING (the §11.4.205 test applied: "if I documented this and nobody implemented the hook, would anything catch it?"). **§11.4.115(F) — machine-written verdict pairs + guard-viability:** RED and GREEN are harness-written verdict files, never prose — carrying the item id, guard identity, polarity, exit code, the target's artifact fingerprint READ FROM THE TARGET AT RUN TIME, iterations (≥3, §11.4.50), and evidence files whose CLASS matches the defect's layer (a source-grep transcript can never satisfy a user-visible/pixel/audio class — the wrong-layer oracle dies by construction); GREEN requires a prior RED with exit ≠ 0 on the PRE-fix artifact and a DIFFERENT fingerprint (identical fingerprints prove the fix was never deployed); a guard never observed FAILing on the genuinely-broken artifact is

unvalidated instrumentation and mints no verdicts (a guard structurally unable to FAIL — or to PASS — satisfies nothing); the canonical §1.1 mutation for a landed fix is the fix-commit's REVERT, and a mutation whose diff only deletes the literal strings the gate greps for is a refused tautology. Recommended gate `CM-GUARD-VERDICT-MACHINE-WRITTEN`. **§11.4.135 (verdict-coverage extension) — ABSENCE of a verdict blocks exactly as a FAIL does:** the suite's exit semantics MUST incorporate COVERAGE, never only the FAIL count; the release seam refuses the tag while `uncovered = registered ∧ topology-present ∧ no-verdict-for-the-candidate-fingerprint` is non-empty OR any FAIL exists; the release-tag tooling MUST consult the suite/verdict store mechanically; REQUIRES-REBUILD PENDING is honest on intermediate artifacts and self-clearing on the candidate (re-run on the candidate, or the item leaves the release's done-set — both forced, neither a permanent exemption); topology-absent guards are exempt ONLY via a checked-in target-pool topology map and MUST be enumerated in the release changelog as honest gaps (§11.4.3/§11.4.118) — never silent; a naive "PENDING always blocks" is REJECTED as a §11.4.201 FAIL-bluff; pre-existing unguarded backlogs adopt via a ONE-TIME monotone-decrease ratchet snapshot so day one is green and the disease cannot spread. Recommended gate `CM-RELEASE-VERDICT-COVERAGE` (self-tested with golden-good / golden-bad / negative-control per §11.4.205/§11.4.107(10)). **§11.4.146(D3) — status-custody mechanical binding:** a done/ready status write is REFUSED without the file chain `registry row keyed by the item's EXACT stable id (checked-in alias map for legacy keys — a guard keyed to a non-item id fails the join and satisfies nothing) → executable registered guard → §11.4.115(F) RED+GREEN verdict pair → class-matched evidence` — prose appears nowhere in the chain; Seam A = the workable-items engine (§11.4.93) refuses the write (project-agnostic per §11.4.28, custody paths are consumer DATA per §11.4.35); Seam B = a FULL-TABLE sweep gate on every pre-build run (not a diff check — catches raw-DB writes that bypass Seam A); Seam C = the §11.4.135 release seam; §11.4.205 construction order — the executable sweep + its golden-bad meta-fixture (a temp SSoT overlay with a chain-less terminal item; the meta-test RUNS the sweep and requires FAIL, and asserts the sweep is WIRED) land in the SAME commit as any governance text citing them. Recommended gate `CM-STATUS-CUSTODY`. Honest boundary (§11.4.6), carried explicitly: the binding proves no

status outruns its evidence and the REPORTED defect cannot silently return under the guard's conditions — it does NOT prove the feature bug-free (family dedup §11.4.186 remains), does NOT make a miscalibrated oracle honest (§11.4.107(10) goldens are necessary, not sufficient), does NOT replace the §11.4.185 manual-QA sufficiency gate, and makes target availability a HARD dependency of status progress BY DESIGN (a visible block replaces a silent pass). All Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their registry, verdict-store, sweep, ratchet, and topology-map paths as DATA per §11.4.35. Propagation gates for the literals `11.4.115` / `11.4.135` / `11.4.146` are unchanged and stay GREEN; the three recommended mechanism gates + paired §1.1 mutations are gate-code = separate work item.

EXTENSIONS + NEW ANCHORS landed 2026-07-17 (round 2 — the carrier/measurement-integrity harvest). Research-derived from one cycle's forensics in a consuming project, where EVERY false finding had ONE shape: **a token that MENTIONS X was matched as X, or an instrument returned silence that was read as data.** Genericised FACTs: a removal's completeness was measured by counting references to the removed component and the count matched the AUDIT-TRAIL COMMENTS documenting the removal (a completed removal read as failed); a pattern's escapes were eaten by an intervening shell so the far side searched a LITERAL and matched nothing (a false "the component is missing" that would have implied the target could not boot — it had booted); `\|` under an extended-regex dialect is a LITERAL pipe so a mandated-fields check searched a string nobody wrote and nearly reported the fields absent (all were present); a `check | head` pipeline returned the PAGER's exit status so a parse-check reported "clean ✓" three times without checking anything; a line-start-anchored pattern missed an indented, colour-escaped line and read a present verdict as no-verdict; a binary extractor split a multi-byte character and reported a present string as absent. §11.4.196(D)'s process-carrier and §12.12's self-match had ALREADY anchored this shape TWICE and it kept recurring — that recurrence is the argument for the general rule. The self-demonstrating instance: a mutation-residue scan run over the governance repo flagged the very anchor that DEFINES the mutation markers, because that anchor quotes them in its rule text (proven a carrier, not residue, by an identical HEAD-vs-worktree count). **The fix, proven in-session: the CONTROL NEEDLE** — an artifact check

returned 0 hits; instead of reporting absence it ran a string KNOWN present through the SAME path, got 1 hit, and only THEN reported the zero as real.

EXTENSION — §11.4.201 (6)(7)(8): scope widened to EVERY load-bearing measurement + carrier-vs-thing + control needle + metric-validation. (6) clauses (1)-(5) bind guards (decision seams that refuse/allow) with a refusal-shaped taxonomy; (6)-(8) bind **every measurement whose result is load-bearing** (grep/count/parse/exit-code/scan/metric/artifact-inspection) with a MEASUREMENT-shaped taxonomy — a **FALSE-NULL** (a zero read as absence when the instrument could not see) and a **FALSE-MATCH** (a hit on a carrier) are §11.4/§11.4.1 bluffs at the measurement layer; the null is more dangerous because a broken instrument and a clean artifact return the identical quiet zero. (7)(a) MATCH STRUCTURE, NOT SUBSTRING — distinguish the THING from a CARRIER that MENTIONS it. (7)(b) **A NULL IS NOT EVIDENCE until a CONTROL NEEDLE proves the instrument can see** — same instrument, same path, same artifact, a known-present needle MUST return non-null; needle absent ⇒ the instrument is blind and the zero says NOTHING (report the blindness, never the absence §11.4.6). **The needle MUST share the certified query's LOAD-BEARING FEATURES** (same dialect constructs / anchoring / quoting / encoding): a bare-literal needle certifies only the layers a bare literal crosses — the (7)(c) dialect FACT refutes the naive form directly, since a literal needle sails through the same tool+path and would 'certify' a zero from a query still searching a string nobody wrote, whereas a needle carrying the SAME alternation dies with it. Earned inference is therefore BOUNDED: needle found ⇒ the path can see the needle's QUERY CLASS ⇒ a zero from that same class is evidence; a zero from a query using features the needle never exercised needs a needle of its own class. Generalises §11.4.115(F)'s validate-the-detector from guards to every measurement, PER-QUERY — **golden fixtures cannot substitute** (§11.4.146(D3): goldens necessary-not-sufficient), an instrument can pass every fixture on the authoring host and be blind on the real path. (7)(c) **THE PATH IS PART OF THE INSTRUMENT** — the tool PLUS every layer the query crosses (wrapper-shell quoting, regex dialect, pipeline exit status, anchoring, encoding); each returns a CLEAN CONFIDENT WRONG answer without crashing, so §11.4.1's script-bug clause (which covers crashes-into-FAIL) never fires and §11.4.67's parse-check (parse-time only) cannot see it — the control needle is the ONLY mechanism that tests the path. (8) **A METRIC MUST BE VALIDATED AGAINST THE**

DEFINITION OF DONE — construct the correct end-state and evaluate the metric there; if the correct end-state does not move it to target, the metric is INVALID and MUST NOT gate work (a metric the correct outcome REFUTES will, if enforced, drive work AWAY from correctness); honest boundary §11.4.6 — this constitution already ships one such gating metric (§11.4.135's adoption ratchet, blocking at the RELEASE-TAG seam) whose validation against the definition of done is OWED, not claimed, and which every consuming project operating a ratchet MUST TRACK as a §11.4.197 item (stated as the obligation it is — this anchor asserts NO project has already filed it; asserting an unverified tracker state would be the §11.4.6 violation the clause forbids). **Interim precedence:** until that validation lands the §11.4.135(5) ratchet KEEPS GATING — clause (8) does NOT disarm it (an unvalidated ratchet blocking a release is bounded + visible + operator-resolvable; disarming the only adoption mechanism would silently re-open the unguarded-backlog hole §11.4.135 closes — the §11.4.101 reversible-safe choice); clause (8) binds every NEW metric from this anchor forward, and the ratchet's validation as owed work — never as a licence to switch it off. Gate contract landed in the canonical file: `CM-GUARD-ASSERTS-REAL-CONDITION` EXTENDED to the measurement layer ((i) a reported absence cites a class-matched control-needle result; (ii) the needle's class matches the certified query's load-bearing features) + NEW recommended sibling `CM-METRIC-VALIDATED-AGAINST-DONE` (every gating/ranking metric carries a recorded definition-of-done evaluation reaching target) + paired §1.1 mutations (bare-literal needle on a dialect-dependent query → (ii) FAILs; needle stripped → (i) FAILs; metric whose correct end-state moves AWAY from target → FAILs). Gate-code = separate work item — the contract is landed, the code is NOT claimed shipped (§11.4.6).

§11.4.214 (NEW) — Recurrence-links-not-mints: a defect that returns MUST reopen its existing item, never enter as a new id. FACT: recurrences of already-"fixed" defects re-entered as BRAND-NEW ids (≥5 chains, one three deep), so the ORIGINAL's reopen counter never incremented — and the §11.4.55 `reopens_count`, the exact signal §11.4.132(d)/§11.4.189 use to rank the most-fragile work for the deepest scrutiny, **stays quiet precisely on the items that keep breaking**; the recorded 52% reopen rate was a KNOWN UNDERCOUNT for this reason; ~15 items mapped to ~9 real root causes, so "fix all 15" would have forked duplicate/conflicting fixes on one cause. Before minting a NEW id from ANY intake path, check for an existing item describing

the SAME defect → RESOLVE the match THROUGH its `duplicate-of` links to the CANONICAL CHAIN HEAD (§11.4.90's closure vocabulary includes `duplicate-of`, and clause (5) MANUFACTURES such items — reopening one would resurrect a non-canonical copy beside its live canonical item, forking one defect into two and crediting the §11.4.55 signal to the WRONG item), then LINK it, and REOPEN (§11.4.34) **iff the resolved head is in a TERMINAL state**; multi-match ⇒ resolve every match to its head and act on the SINGLE survivor, >1 distinct head or a broken/circular link ⇒ UNDECIDED (ask / mint-with-link, never guess §11.4.6). **Open-original case:** a matched item still OPEN gets LINK-ONLY — 'reopen an open item' is undefined in the §11.4.15 status machine and §11.4.34's `Reopened` presupposes a closure to demote FROM (§11.4.7); the reopen path exists to correct the specific lie 'closed as fixed and it is not', and an open item is not telling it. **§11.4.202 precedence (explicit, never left to a consumer):** §11.4.202's mandate is that no report is ever LOST — SATISFIED IN FULL by a SAME-DEFECT report landing as link + (terminal-only) reopen; it forbids the report evaporating into prose, it does NOT require a fresh id per report. §11.4.214 refines WHICH item a report lands on; §11.4.202 owns THAT it lands; DISTINCT/UNDECIDED verdicts run §11.4.202's mint path unchanged. (1) EVERY intake path binds (reporting directives §11.4.202, AI-detected test/gate failures, regression-guard verdicts, cycle re-discovery, operator manual testing — §11.4.34's own reason vocabulary enumerates them; wiring dedup ONLY into the reporting channel under-covers exactly the machine-driven paths that recur most). (2) Key on ticket else normalised `(subject, scope)` — NEVER a bare subject substring (§11.4.186). (3) **A wrong merge silently DELETES a real defect — so `distinct-but-similar` is a FIRST-CLASS verdict**: SAME-DEFECT ⇒ link+reopen, DISTINCT ⇒ mint (correct + expected), UNDECIDED ⇒ ask (§11.4.66/.105) or mint-WITH-a-candidate-duplicate-link; autonomous default (§11.4.101) is mint-with-link over merge — a spurious id is cheap + recoverable, a wrongly-merged defect is LOST (the asymmetry decides). (4) Self-validated oracle (§11.4.107(10)/§11.4.186): golden-good + golden-bad + a **negative-control** (two genuinely DISTINCT same-subject items that MUST NOT merge) — an oracle without it is a false-merge engine. (5) Id stability preserved (§11.4.54 — reopen the original, never renumber/reuse/retire; an already-minted duplicate is LINKED via §11.4.90 duplicate-of, never silently deleted). (6) The restored signal IS the point — an undercounting reopen

counter is a §11.4/§11.4.1 bluff at the PRIORITISATION layer: it reports the fragile set as stable and aims the deepest scrutiny at the wrong items. Honest boundary §11.4.6 — dedup makes the signal TRUE, not the item correct, and a same-symptom recurrence with a different root cause is real (§11.4.102 decides; the linked+reopened item is where that is recorded, never a reason to fork). Gate `CM-RECURRENCE-LINKS-NOT-MINTS` (every CONFIRMED-duplicate link whose target was TERMINAL at link time has a reopen event on that target) — **its own scope is §11.4.201-bound**: it MUST NOT fire on an OPEN-target link (LINK-only is correct there) nor on a clause-(3) UNDECIDED candidate-duplicate link (un-reopened BY DESIGN, and the mandated autonomous default), or it becomes exactly the §11.4.201(1) false-positive refusal this same round harvests against; its golden-FALSE set MUST include both + paired §1.1 mutation. Why NEW: §11.4.55 CONSUMES the count, §11.4.34 presupposes the status is already `Reopened`, §11.4.54 forbids only one-id-two-items, §11.4.186's DEDUP needs DIVERGENT metadata and gates export/sync/commit not INTAKE, and §11.4.202's operative text, read WITHOUT this anchor, reads as mint-every-time — the reading the precedence paragraph above corrects (they are NOT in conflict: §11.4.202 owns THAT a report lands, §11.4.214 refines WHICH item it lands on).

§11.4.215 (NEW) — A doc that BINDS work MUST live, tracked, in the repository where that work happens. FACT: the document DEFINING a binding acceptance gate was UNTRACKED and lived in an out-of-tree scratch directory belonging to a different checkout — so the gate could not be honoured (unreadable), reviewed (no diff/history/authorship), version-controlled (no revision/rollback), or reconciled when it contradicted the source of truth. A binding artifact that lives nowhere is a rumour with an imperative mood. (1) **BINDING ⇒ TRACKED, and un-tracked ⇒ NOT BINDING** — a binding claim sourced from an untracked/out-of-tree doc MUST NOT be enforced; land the doc or decline the gate, never obey a citation of it. (2) IN THE REPOSITORY THE WORK HAPPENS IN — a sibling checkout / scratch dir / chat message / issue comment is NOT tracked for this purpose (the test: can someone who clones ONLY that repo read the binding text at its declared path?); multi-repo binding docs are inherited BY REFERENCE from a tracked shared dependency (§11.4.28/§11.4.177), never copied, never out-of-tree. (3) **The prior question §11.4.212 cannot ask** — §11.4.212 ENUMERATES the in-repo scope and checks reachability, so a doc in no checkout is never enumerated and its gate

cannot FAIL; §11.4.212 answers "is it findable?", §11.4.215 answers "is it here at all?" (once in-repo, §11.4.212/.44/.65/.73/.106 all attach as normal — this is their precondition, not their competitor). (4) **The generalisation of §11.4.95** (whose entire content is "authoritative ⇒ tracked, NOT a build artefact, IS authoritative source data" for ONE named artifact), exactly as §11.4.212 was minted as the generalisation of §11.4.57. §11.4.11 is the counterweight and stays intact — logs/forensics/scratch remain out-of-tree and untracked by DEFAULT; **binding is what pulls a document into the repository**, and a doc nobody cites as binding is governed by §11.4.11, not this anchor. Honest boundary §11.4.6 — in-repo presence proves AVAILABLE + ATTRIBUTABLE + VERSION-CONTROLLED, never CORRECT/current (§11.4.44/.106/.186), and never well-seated (§11.4.120's seam clause — whose forensic case was carried by this SAME out-of-tree document: two defects, one artifact, which is itself the argument that an unreviewable binding doc is where bad gates hide). Gate `CM-BINDING-DOC-IN-REPO` + paired §1.1 mutation.

All Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their intake wiring, id scheme, normalisation function, and binding-doc paths as DATA per §11.4.35. Propagation gates `CM-COVENANT-114-214-PROPAGATION` + `CM-COVENANT-114-215-PROPAGATION` (literals `11.4.214` / `11.4.215`); the literals `11.4.201` / `11.4.112` / `11.4.120` / `11.4.69` are unchanged and their propagation gates stay GREEN. All recommended mechanism gates + paired §1.1 mutations = gate-code, a separate work item.

DESIGN-METHODOLOGY MANDATES landed 2026-07-22 (§11.4.216–§11.4.223 — full text in `Constitution.md`; proving project: the Helix Thready MVP design package at `docs/public/research/mvp/design/`). The complete design methodology built and verified end-to-end in Helix Thready is codified as eight universal anchors so every governed project inherits it out of the box. All Classification: universal (§11.4.17) — consumers supply brand values, platform lists, paths, and submodule pointers as DATA per §11.4.35. Propagation gates `CM-COVENANT-114-216-PROPAGATION` … `CM-COVENANT-114-223-PROPAGATION` (literals `11.4.216` – `11.4.223`) + recommended mechanism gates + paired §1.1 mutations = gate-code, a separate work item.

§11.4.216 — Canonical machine-readable design-token source (User mandate, 2026-07-22). Compact summary: every UI-shipping project maintains its design tokens as ONE canonical machine-readable source — CSS custom properties, light in `:root {}`, dark via the `[data-theme="dark"]` override (composing the sanctioned dark mechanisms) — and EVERY surface binds via `var(--…)` or a GENERATED binding produced by codegen FROM that file, never a hand-kept copy (hand-copies drift silently); NO INVENTED VALUES (§11.4.6 applied to design): every color/size/duration traceable to captured evidence — brand colors to a recorded eyedrop/measurement of the canonical brand asset (proving project: brand `#B6E376` = mean of 1,625,855 logo pixels), accent pairs to MEASURED WCAG ratios against their PAIRED background (6.03:1 light / 13.56:1 dark), structural values to the documented shared scale; a hex resolving to no token is a violation — use the nearest token, never mint one; the token source is the white-label seam (§11.4.217 closed override set); not-yet-built codegen bridges are [OPEN]-tracked (§11.4.223), never claimed shipped. Composes §11.4.6/.162/.170/.190/.197/.217/.223. Propagation gate `CM-COVENANT-114-216-PROPAGATION` + recommended gate `CM-DESIGN-TOKEN-SINGLE-SOURCE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.216. Non-compliance is a release blocker. No escape hatch — no `--hand-copied-palette-OK`, `--inline-hex-OK`, `--invent-token-value`, `--light-only-tokens`, `--platform-keeps-own-palette` flag.

§11.4.217 — OpenDesign brand contract: 9-section DESIGN.md + tokens.css twin, disjoint color roles, AA-pinned accents, locked attribution footer (User mandate, 2026-07-22). Compact summary: every UI-shipping product carries under `design/opendesign/` (or its declared equivalent §11.4.35) a `DESIGN.md` in the VERIFIED OpenDesign 9-section schema (H1 + `> Category:` blockquote immediately after the H1; number-prefix-parsed sections; a "Font labels for catalog extraction" block; real hex; CSS variables wrapped in `:root {}` with dark via `[data-theme="dark"]`; real component CSS with `prefers-reduced-motion` targeting specific elements, never a global `*`; `:focus-visible` on every interactive component — format verified against the tool's CURRENT source per §11.4.99, never from memory) + the machine-readable `tokens.css` twin (the §11.4.216 canonical file or a compiled view; prose and tokens never diverge); THREE

DISJOINT color roles by contract — brand DECORATIVE only (never body text), accent INTERACTIVE + AA-PINNED (measured ≥ 4.5:1 per mode), semantic STATE never overridden by any brand (a brand color can never mask an error signal); white-label = a CLOSED override set (brand tokens, accent pair, logo, slogan) with accent overrides SERVER-VALIDATED to WCAG AA or REJECTED carrying the measured ratio + an AA-passing suggestion; structural/neutral/semantic tokens NOT account-overridable; the organization attribution footer is LOCKED — present on every surface, never removed even fully white-labeled. Composes §11.4.6/.99/.162/.190/.216/.220/.223. Propagation gate `CM-COVENANT-114-217-PROPAGATION` + recommended gate `CM-OPENDESIGN-BRAND-CONTRACT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.217. Non-compliance is a release blocker. No escape hatch — no `--skip-brand-contract`, `--prose-only-design-md`, `--brand-as-text-OK`, `--unvalidated-accent-override`, `--remove-attribution-footer`, `--semantic-tokens-brandable` flag.

§11.4.218 — Living design-library catalogue + per-cell-provenance reusability matrix (User mandate, 2026-07-22). Compact summary: every UI-shipping project maintains a LIVING component catalogue rendering EVERY component × declared state × light/dark from SELF-CONTAINED artifacts (zero external requests — no CDNs, vendored libs inline, honest font fallbacks), token-bound (§11.4.216), with REAL interactive behaviors (real dialog/sort/toggle; `prefers-reduced-motion` honored per §11.4.221) — proving project: 14 groups, ~38 components, ≈118 state cells, ≈236 rendered states, both themes auditable via a real persisted toggle; PLUS a components × platforms reusability matrix where EVERY cell carries closed-set provenance — VERIFIED (re-checked at source, dated) / PROPOSED ([DEFAULT — adjustable]) / ASSUMED (labeled inference); an unmarked cell reads ASSUMED; a scaffold cell claiming VERIFIED is a §11.4 PASS-bluff at the design-planning layer; the matrix is read BEFORE implementing any component on any platform. Spec'd-but-unrendered components/states are tracked gaps (§11.4.223). Composes §11.4.6/.162/.170/.216/.221/.223. Propagation gate `CM-COVENANT-114-218-PROPAGATION` + recommended gate `CM-DESIGN-LIBRARY-LIVING-CATALOGUE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.218. Non-compliance is a release blocker. No escape hatch — no

`--cdn-in-catalogue`, `--single-theme-catalogue`, `--mocked-behaviors-OK`, `--unmarked-matrix-cell`, `--scaffold-claims-verified` flag.

§11.4.219 — Per-app screen catalogues: full-IA coverage, per-platform chrome, honesty markers (User mandate, 2026-07-22). Compact summary: every shipped surface (web / mobile / desktop / TUI / marketing / …) has a screen catalogue covering EVERY node of its information architecture (an IA node with no catalogue screen is a tracked gap, never silently skipped); per-platform CHROME VARIANTS rendered where one design serves several hosts (mobile OS chrome toggle, per-OS window chrome, terminal-width constraints); each screen SELF-CONTAINED, token-bound (§11.4.216), light + dark, populated with REALISTIC product data and a per-screen state contract (loading/empty/error/populated); UNBUILT or UNDECIDED behavior is HONESTLY marked — `[GAP: id]` / `[OPEN: id]` (§11.4.223), labelled placeholders, inline notes, disabled controls — NEVER a designed-as-if-real fake flow (a fake flow presented as settled is a §11.4 PASS-bluff at the product-design layer). Proving project: 13 web + 11 mobile (Android/iOS chrome toggle) + desktop shell + 10 TUI + 3 marketing screens, every unbacked flow marked. Composes §11.4.6/.162/.170/.185/.190/.216/.218/.223. Propagation gate `CM-COVENANT-114-219-PROPAGATION` + recommended gate `CM-SCREEN-CATALOGUE-FULL-IA` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.219. Non-compliance is a release blocker. No escape hatch — no `--partial-ia-coverage`, `--one-chrome-fits-all`, `--fake-flow-OK`, `--lorem-ipsum-content`, `--single-theme-screens` flag.

§11.4.220 — Open-first design tooling: self-hosted PenPot primary via the PenPot submodule; proprietary tools import/export only; in-repo open-format sources (User mandate, 2026-07-22). Compact summary: the collaborative design platform is an open-source self-hostable tool — PenPot as the standing default — run SELF-HOSTED and consumed ONLY via the dedicated reusable PenPot submodule (working name `vasic-digital/PenPot`, final repository name pending — the generic referent is normative meanwhile) carrying ALL PenPot mechanisms/utilities (deployment recipe, RPC client, access-token minting, import tooling, browser-hydration automation, MCP wiring), itself consuming the Containers submodule (§11.4.76, rootless

§11.4.161) and DECLARING that dependency in its §11.4.31 `helix-deps.yaml`; projects MUST NOT hand-roll PenPot deployment/client code (§11.4.28/§11.4.74 — extend the submodule upstream); the submodule carries mechanisms ONLY, never project design content. Proprietary tools (Figma or equivalent) are OPTIONAL import/export/review bridges, NEVER the source of truth — no design value may exist only inside a proprietary tool/cloud; design sources live IN-REPO in open, diffable formats (SVG masters, CSS tokens, JSON manifests/IR/variables payloads, Lottie JSON, self-contained HTML — §11.4.215 applied to design). Tooling capability claims are RUNTIME-PROBED (§11.4.201/§11.4.99) with a headless-vs-operator-interactive matrix + an honest-limits section — a cloud push claimed without a real credentialed run is a fabrication. Operational rules (research-verified): the HEADLESS write path is the token-authenticated RPC API — access tokens (minted under the `enable-access-tokens` backend flag, handled per §11.4.10) are the MANDATED automation auth mechanism, never interactive OAuth relays; API-created PenPot files REQUIRE a browser hydration pass before handoff (thumbnails/ text layout materialize in the browser runtime, not the write API) — an unhydrated API-created file handed off as render-ready is a §11.4 PASS-bluff at the design-handoff layer. Composes §11.4.6/.10/.28/.31/.69/.74/.76/.99/.161/.162/.201/.215/.216/.223. Propagation gate `CM-COVENANT-114-220-PROPAGATION` + recommended gate `CM-OPEN-DESIGN-TOOLING-PRIMARY` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.220. Non-compliance is a release blocker. No escape hatch — no `--figma-is-source-of-truth`, `--cloud-only-design`, `--proprietary-format-source`, `--hand-rolled-penpot-stack`, `--oauth-relay-automation`, `--skip-hydration-pass`, `--unprobed-tooling-claim`, `--rootful-design-stack` flag.

§11.4.221 — Motion discipline: token-bound timing + machine-readable manifest + reduced-motion static fallbacks (User mandate, 2026-07-22). Compact summary: every duration/easing binds to the §11.4.216 motion tokens (an ad-hoc timing literal bypassing the token layer violates §11.4.6); every playable animation ships a STATIC reduced-motion fallback DERIVED FROM ITS OWN KEYFRAMES (poster frame for loopers, final frame for one-shots) in an open format, named per animation in a versioned MACHINE-READABLE motion manifest (id, loop mode, poster, reduced-motion mode, `reducedMotionFallback`) — a deliberate instant-cut is an EXPLICIT null-

with-reason, never a silent absence; `prefers-reduced-motion: reduce` honored on EVERY surface via targeted component gates (never a global `*`), swapping in the manifest-named fallback; NO animation is claimed working without a REAL render in a real runtime (parse + structural schema + rendered geometry with progression assertions — the §11.4.107(10) analyzer discipline applied to motion assets), unexercised runtimes [OPEN]-tracked. Proving project: 6 hand-authored Lottie animations + 5 keyframe-derived fallback SVGs + manifest v2, lottie-web/jsdom + headless-Chromium validated, mobile runtimes honestly [OPEN]. Composes §11.4.6/.107(10)/.162/.170/.216/.218/.223. Propagation gate `CM-COVENANT-114-221-PROPAGATION` + recommended gate `CM-MOTION-TOKEN-BOUND-REDUCED-FALLBACK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.221. Non-compliance is a release blocker. No escape hatch — no `--ad-hoc-duration`, `--no-reduced-motion-fallback`, `--global-star-motion-kill`, `--manifest-optional`, `--unrendered-animation-claimed-working` flag.

§11.4.222 — Design-completion export wave: a design is not done until the exports land, honestly (User mandate, 2026-07-22). Compact summary: a design (system / brand / screen set / change wave) is NOT done until its EXPORT WAVE runs with verified outputs — (1) per-platform token codegen regenerated from the canonical source (or [OPEN]-tracked); (2) the FULL icon/asset raster matrix, documented per OS surface and locked 1:1 to its generator, produced by an HONEST-OUTPUT pipeline: every candidate renderer PROBE-TESTED with a content-verified probe render before use (§11.4.201 — presence ≠ correctness), fall-through on probe failure, absent tools SKIP-with-note (§11.4.3/§11.4.69), NEVER a placeholder/blank/faked output, every raster verified non-degenerate (size floor + non-blank; multi-resolution bundles opened + verified per §11.4.38) — proving project: 83 real rasters, zero faked; (3) both-theme screen renders (deterministic theme forcing, every render verified non-blank) + the bound design book; (4) hand-off bundles with per-leg HONEST statuses (produced / deferred-operator-only / impossible-with-reason); (5) the §11.4.65/§11.4.73 doc twins regenerated. A design wave reported complete without its export wave — or with any faked/unverified export output — is a §11.4 PASS-bluff at the design-delivery layer. Composes §11.4.3/.6/.38/.65/.69/.73/.201/.216/.219/.220. Propagation gate `CM-COVENANT-114-222-PROPAGATION` + recommended gate `CM-DESIGN-EXPORT-`

`WAVE-COMPLETE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.222. Non-compliance is a release blocker. No escape hatch — no `--skip-export-wave`, `--unprobed-renderer`, `--placeholder-raster-OK`, `--single-theme-renders`, `--fake-bundle-status`, `--stale-doc-twins` flag.

§11.4.223 — Provenance-marker discipline for design docs + central open-item registry (User mandate, 2026-07-22). Compact summary: every design document carries inline provenance markers from a closed, project-declared vocabulary whose MINIMUM CORE is `[VERIFIED]` (with evidence/check date) / `[OPERATOR]` / `[DEFAULT — adjustable]` / `[OPEN: id]` (extensions: `[RESEARCH]`, `[ASSUMED]`, `[BUILD-NEW]`, `[GAP: id]`, `[IN-HOUSE: …]`, `[CONSTITUTION §x]` per §11.4.35); an unmarked load-bearing claim reads UNVERIFIED, and an assumption presented as verified is a §11.4.6 violation of §11.4 PASS-bluff severity at the design-documentation layer; every unresolved decision is an `[OPEN: id]` with a stable, never-renumbered id (§11.4.54 discipline) owned by a named file; the design area index maintains the SINGLE CANONICAL open-item registry, TREE-VERIFIED in both directions (every tree id in the registry, no registry ghosts — the §11.4.186 consistency discipline applied to design opens), closures citing the closing revision, narrowed items recording exactly what closed and what remains; registry items map to tracked workable items (§11.4.15/§11.4.54/§11.4.93) so the §11.4.197 completion mandate applies — no design open is silently abandoned. Proving project: 41 registered ids across nine families, tree-verified, 2 closed with citations. Composes §11.4.6/.15/.44/.54/.61/.93/.123/.186/.197/.212. Propagation gate `CM-COVENANT-114-223-PROPAGATION` + recommended gate `CM-DESIGN-DOC-PROVENANCE-MARKERS` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.223. Non-compliance is a release blocker. No escape hatch — no `--unmarked-claims-OK`, `--assumption-as-verified`, `--unregistered-open-item`, `--registry-optional`, `--silently-abandon-open` flag.

§11.4.224 — Test-first (TDD) for ALL work, not only fixes + a minimum code-coverage floor that is NECESSARY-never-sufficient (User mandate, 2026-07-22). Verbatim operator mandate: "Any work we do MUST START by writing the test! We MUST FULLY comply and follow the TDD - The test driven

development! It MUST BE applied for any change, implementation, wiring, tests, challenges and any other form of codebase we write! Bash scripts as well! Coverage with TDD MUST BE close to 100%, not less than 85% !!!" **The gap closed:** §11.4.43's operative text binds FIXES only ("Every fix MUST follow the 5-step TDD-fix workflow"), so a NEW feature / wiring / gate / Challenge / BASH SCRIPT inherited no test-first obligation; and §11.4.27/ §11.4.169's "100%" is test-TYPE breadth (which KINDS of test exist per feature × platform), stating no CODE-coverage figure anywhere — §11.4.224 closes exactly those two holes and claims nothing else as new. **(A) TEST-FIRST BINDS ALL WORK** — every work product with an executable surface (change, feature, implementation, wiring, refactor, gate, §1.1 mutation, Challenge, test-harness code, AND bash/shell scripts): the test is written FIRST, run FIRST, and OBSERVED TO FAIL for the right reason before the implementation exists (RED on the broken artifact per §11.4.115 where one exists; RED against the absent behaviour where the work is new); a test authored AFTER its implementation proves only that it AGREES with the code, never that it CATCHES the defect (the §11.4.43 PASS-bluff generalised). Per-work-class test named explicitly (§11.4.6): bash/shell script → an executing test through its REAL invocation path asserting exit status + output + state delta, NEVER a `bash -n` parse-check alone (parse-cleanliness is §11.4.67's separate obligation and proves nothing about behaviour); a pre-build GATE → its paired §1.1 mutation IS the test-first artifact, authored and observed to make the gate FAIL BEFORE the gate is trusted (§11.4.115(F) — observation-before-trust, NOT authorship order); a Challenge → its scored assertion against captured evidence (§11.4.5/§11.4.69); a behaviour-bearing config/data change (any committed data an executing consumer reads to decide behaviour) → an executing test that drives that CONSUMER through the changed value and asserts the changed observable behaviour. **(B) CODE-COVERAGE FLOOR ≥ 85%, ~100% TARGET**, applying to the executable codebase INCLUDING bash/shell (the operator's "Bash scripts as well!"). **(C) NECESSARY, NEVER SUFFICIENT — anti-gaming (§11.4.201(8)/§11.4/§1.1):** MEASURED FACT (in-session, 2026-07-22) — a line-granular instrument reported an `if/else` line COVERED while the `else` branch was NEVER taken, and an assertion-free test raises the number identically to a proving one; therefore the REAL bar remains catch-its-own-negation proven by a paired mutation (a line executed by an assertion-free test is NOT covered in the sense that matters), the floor is

a MINIMUM ON A PROXY, and citing "≥ 85% covered" as proof a change works is a §11.4 PASS-bluff at the metric layer; the floor NEVER stands alone (composes §1.1 + §11.4.5/§11.4.69/§11.4.107 captured evidence + §11.4.108 runtime signature). **(D) METRIC VALIDATION (§11.4.201(8)) — SPLIT VERDICT, honestly stated:** directional validation PERFORMED 2026-07-22 (at the correct end-state — every behaviour test-driven, every test catching its negation — the implementing lines ARE executed by their tests, so the metric DOES reach target; the floor is VALID as a NECESSARY condition); the CONVERSE is REFUTED by the clause-(C) measured gameability fact; and the EMPIRICAL per-corpus calibration of the 85% threshold is explicitly OWED, NOT claimed — every consuming project enforcing the floor MUST track it as a §11.4.197 item (this anchor asserts no project has already filed it — asserting an unverified tracker state would be the §11.4.6 violation the clause prevents); INTERIM PRECEDENCE mirroring §11.4.201(8)'s own §11.4.135-ratchet precedent — the floor KEEPS GATING while the calibration is owed (an uncalibrated floor blocking a merge is bounded, visible, operator-resolvable; disarming it silently re-opens the untested-code hole — the §11.4.101 reversible-safe choice), never a licence to switch it off. **(E) HOW MEASURED, per language, honestly (§11.4.6/§11.4.35):** the consuming project supplies its per-language instrument as DATA; the universal rule is that an instrument MUST exist and MUST actually be RUN — a floor nobody can measure is an unmeasurable gate and itself a §11.4.201 bluff, and a percentage nobody measured is a §11.4.6 fabrication. MEASURED host reality (2026-07-22): Go built-in coverage (`go test -coverprofile` + `go tool cover`) + `gcov` PRESENT, but `kcov` / `bashcov` / `bats` / `shellspec` / `shunit2` ALL PROBED ABSENT for bash (`shellcheck` IS present but is a LINTER, not a coverage instrument, and satisfies nothing here). PROVEN-FEASIBLE zero-tooling bash mechanism, measured in-session never asserted: `PS4='+COV:${BASH_SOURCE##*/}:${LINENO}:' bash -x <script>` emits executed line numbers, and executed ÷ executable (non-blank, non-comment) source lines yields line coverage — a probe correctly identified an uncovered function body, and subshell bodies ARE traced. HONEST LIMITS (measured, stated as fact): LINE coverage NOT branch coverage (the clause-(C) same-line `if/else` counted covered with one branch never taken); `set +x` regions and traps unaccounted — a FLOOR-measuring instrument only, so branch-level assurance requires a real tool (e.g. `kcov`) or an honestly-recorded shortfall.

Where NO instrument exists for a language: INSTALL one, or record an §11.4.3/§11.4.69 SKIP-with-reason + a tracked §11.4.197 item — NEVER an invented percentage, NEVER a silently-dropped floor. EXCLUSION-LIST FENCE (amendment, remediation round of 2026-07-22 — the §11.4.135 exemption pattern applied to the coverage corpus): a corpus exclusion is legal ONLY via a CHECKED-IN exclusion list, every entry justified from the closed class set {generated-code (output of a committed generator per §11.4.77) | vendored-third-party | non-shipping-fixtures-and-golden-assets} and enumerated as honest gaps never silent; excluding FIRST-PARTY executable code additionally REQUIRES a tracked §11.4.197 item naming the plan to bring it into scope — an unlisted, unjustified, or un-tracked-first-party exclusion VOIDS the measured figure (a consumer that excludes most of its first-party executable code and truthfully measures ≥ 85% on the remainder has satisfied the floor in letter and voided it in substance — the silent-evasion channel this fence closes). **(F) HONEST BOUNDARY / SCOPE (§11.4.6):** "all work" = work products with an EXECUTABLE surface; a pure prose/governance/ documentation edit has no executable surface and is NOT coverage-scoped (its discipline is §11.4.44/§11.4.65/§11.4.106/§11.4.212 + the §11.4.142 universal review — demanding a coverage percentage there would be a §11.4.201(1) false-positive refusal), BUT a governance change that SHIPS an executable artifact (gate, hook, script, mutation) is FULLY in scope for (A) and (B); test-first PRECEDES §11.4.108 four-layer verification / §11.4.40 full-suite retest / §11.4.185 manual-QA, never replaces them; and the CODE-coverage axis is DISTINCT from §11.4.27/§11.4.169's TEST-TYPE breadth — both apply, neither substitutes, satisfying one is never evidence for the other. Classification: universal (§11.4.17) — the consuming project supplies its per-language instruments, corpus scope, CHECKED-IN clause-(E)-fenced exclusion list, and threshold calibration per §11.4.35. Composes §11.4/.1/.5/.6/.27/.35/.43/.50/.66/.67/.69/.77/.85/.107/.108/.110/.115/.122/.135/.142/.146/.169/.18
§1.1. Propagation gate `CM-COVENANT-114-224-PROPAGATION` (literal `11.4.224`) + recommended gates `CM-TEST-FIRST-ALL-WORK` + `CM-COVERAGE-FLOOR` (contract amended, remediation round of 2026-07-22: the measured numerator counts ONLY RED-CAPABLE tests — every contributing test carries a recorded failing-first run OR a paired mutation observed to make it FAIL, zero-assertion contributors rejected by construction, sampling-based mutation verification an acceptable stated implementation — so ONE real RED

test plus assertion-free padding to 85% FAILs the gate; and the gate FAILs on any exclusion that is unlisted, outside the clause-(E) closed class set, or first-party without its tracked §11.4.197 item; brownfield adoption is DELIBERATELY UNDEFINED — the owed gate-code item MUST surface the adoption question to the OPERATOR per §11.4.66 with concrete options {immediate hard floor | one-time §11.4.135-style monotone-decrease ratchet | per-corpus phase-in | changed-code-only with a scheduled full-corpus deadline} BEFORE first enforcement, recording the answer as consumer DATA — never an invented ratchet, a §11.4.122-class decision the operator owns) + paired §1.1 mutations; gate-code = a separate work item — the CONTRACT is landed, the gate CODE is NOT claimed shipped (§11.4.6). **Canonical authority:** constitution submodule `Constitution.md` §11.4.224. Non-compliance is a release blocker. No escape hatch — no `--test-after-OK`, `--skip-red-first`, `--bash-exempt-from-tests`, `--coverage-not-applicable`, `--below-floor-OK`, `--coverage-proves-correct`, `--unmeasured-percentage-OK`, `--assertion-free-test-counts` flag.

§11.4.225 — Scheduler-quota burst-throttling telemetry + interactive-scope isolation from bursty fleets (research-derived, 2026-07-23). Compact summary: progressive / bursty / load-correlated interactive degradation MUST be diagnosed from the scheduling scope's THROTTLING ACCOUNTING — on cgroup v2, `cpu.stat` `nr_throttled` / `throttled_usec` as DELTAS over a measured window under load, as a fraction of enforcement periods, WITH a quiet-phase control — NEVER from averages: quota enforcement is per-period, so a workload far below quota ON AVERAGE can still throttle a large fraction of periods and stall every co-resident process, while CPU average / load average / RSS / thread counts all read HEALTHY (their quiet zeros are §11.4.201(6) FALSE-NULLS; forensic FACT 2026-07-22: a session scope at quota 9.6 CPU/100ms on a 64-CPU host, average demand 4.35 — UNDER quota — yet 16.9% of periods throttled during active phases, 0% quiet, worsening with session age, the co-located terminal-multiplexer server freezing ≥100ms per throttled period into multi-second interactive stalls); an "averages look healthy" verdict without throttle-accounting deltas is a §11.4.6 guess, and the same discipline binds every quota'd scheduler (container CPU limits, VM caps/steal). DESIGN half: an interactive-latency-sensitive process (terminal server / TUI / editor / operator shell) MUST NOT share a fixed-quota NO-BURST scope with a bursty agent/

build fleet — grant a measured burst allowance, isolate the interactive process in its own scope, or both; quota values are HOST-ADAPTIVE (computed from detected capacity per §12.11, §11.4.6 never hardcode — sibling FACT: a FIXED 2-CPU default quota on a 64-core host → 18.8% of periods throttled, 1757s forced idle, fixed by host-adaptive computation proven with a four-quadrant polarity test reading the LIVE cgroup back per §11.4.108); the THIRD orthogonal host-exhaustion axis beside §12.6 memory + §12.12 threads — no axis's instrument substitutes for another's. Honest boundary §11.4.6 — throttle telemetry proves WHERE periods were lost, not that throttling is the only degrader (RTT floors / GC load ranked separately in the founding investigation), and the design half never licenses removing a quota host policy requires (§12). Classification: universal (§11.4.17). Composes §11.4.6/.24/.102/.108/.128(1)/.201/§12.6/§12.8/§12.11/§12.12. Propagation gate `CM-COVENANT-114-225-PROPAGATION` (literal `11.4.225`) + recommended gates `CM-QUOTA-THROTTLE-TELEMETRY` + `CM-INTERACTIVE-SCOPE-NOT-QUOTA-STARVED` + paired §1.1 mutations (averages-only verdict on a progressive-degradation issue → telemetry gate FAILs; fixed low quota literal on a detected many-core host, or terminal server co-located with the fleet in a no-burst quota scope → isolation gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.225. Non-compliance is a release blocker. No escape hatch — no `--averages-prove-healthy`, `--skip-throttle-telemetry`, `--fixed-quota-default-OK`, `--interactive-in-fleet-scope-OK`, `--no-burst-needed`, `--skip-quiet-phase-control` flag.

EXTENSIONS landed 2026-07-23 (the systematic-debugging / measurement-semantics harvest — full text in `Constitution.md`; every lesson verified against captured evidence or re-demonstrated under control conditions before codifying, §11.4.6). §11.4.115(G) — defect-identity binding for CONSTRUCTED reproductions: a RED test's PRECONDITION must be TRACEABLE to the reported defect's real evidence (the reporter's sequence §11.4.199, a captured failure trace, or a probe proving the precondition occurs in the reported environment), recorded with the RED verdict as a precondition-provenance field { `observed` | `constructed` }; when NO working repro exists and the precondition is CONSTRUCTED, the fix mints at most DEFENSIVE HARDENING — labelled as such, NEVER closing the

reported item (it stays OPEN §11.4.197); systematic-debugging refutation of every mechanism a pending fix addresses REVOKES its done-claim on the spot (forensic FACT 2026-07-22: an exit-hang fix nearly shipped on a hand-made-lock-holder RED→GREEN; real-PTY §11.4.199-shape re-driving refuted every addressed mechanism — 6/6 clean exits — and the change was honestly re-labelled in-source "defensive hardening ... NOT a fix for the reported exit-hang"); strengthens (A) from artifact-synthetic to MECHANISM-synthetic; gate `CM-RED-PRECONDITION-DEFECT-TRACEABLE` (a `constructed` provenance on a defect-CLOSING claim FAILs; the same provenance on an honest hardening item PASSes — the §11.4.201(1) false-positive guard) + paired §1.1 mutation. **§11.4.201(9)(10)(11)(12)** — measurement-semantics quartet: (9) FIELD-IDENTITY + UNITS + SEMANTIC CLASS {capacity|occupancy|rate|cumulative|high-water} verified against the authoritative source definition before ANY summary-read metric gates or propagates — CAPACITY≠OCCUPANCY is the named canonical instance (forensic FACT: `rcv_wnd` 181120, a window-CAPACITY ceiling, misread as a "~1.4s standing send queue"; real Send-Q a few hundred bytes `app_limited`; propagated into THREE agent briefs + operator reports before the raw field was re-read, then publicly withdrawn — a downstream consumer of an unverified metric INHERITS the bluff); (10) OBSERVER DECONTAMINATION — an instrument contributing to its own measured population (probe sockets in a socket census, sampler load in a load profile) identifies + EXCLUDES its footprint AND re-proves the decontaminated instrument still sees via a (7)(b) control needle (FACT: TIME-WAIT 68–80 raw → 4–5 decontaminated, ~16× observer-induced, while the same scan still caught a real +264MB/1→93-socket step); composes §11.4.128(1) observer-effect budget + §11.4.119 single-resource-owner for measurement arms; (11) ARTIFACT-USABILITY-NOT-PREREQUISITE-PRESENCE — a readiness/install/health gate probes the ARTIFACT through its real invocation path (§11.4.224(A)), never a build prerequisite's presence (FACT: a `command -v <compiler>` gate turned a host holding a WORKING prebuilt into a false "cannot launch" exit-1 — "[ok] verified" and "[FAILED] cannot launch" in ONE transcript, review-graded BLOCKING; the proxy fails BOTH directions — clause (1) and (2) at once); (12) the SHELL-INSTRUMENT FOOTGUN CHECKLIST — NEW standing reference `docs/guides/shell_instrument_footguns.md` (inherited by reference §11.4.28/§11.4.177): exec-wrapper classes (`timeout` / `nice` / `env`) cannot run shell functions/

aliases (rc=127 false refusal), `pgrep -f` self/carrier match (§11.4.196(D)/§12.12), delimiter-keyed extractors TRUNCATING the unit under test into a FALSE PASS, `bash -x` xtrace BLINDED by the code under test's `exec ... 2>/dev/null` (countermeasure `BASH_XTRACEFD`), and a background watchdog subshell spawned INSIDE a `$(...)` command-substitution — it inherits the substitution's pipe WRITE-END, and an early disarm (`kill`) ORPHANS its `sleep` grandchild which keeps that write-end OPEN, so the `$(...)` BLOCKS for the FULL timeout budget after the real work completed instantly, every verdict and exit code CORRECT (countermeasure: redirect the watchdog subshell's fds away from the pipe — `( … ) >/dev/null 2>&1 &` — or have the probe write to a file) — all five DEMONSTRATED under control conditions 2026-07-23; every shell-touching §11.4.142/§11.4.209 review consults it, every shell-instrument zero stays bound by the (7)(b) class-matched needle; `CM-GUARD-ASSERTS-REAL-CONDITION` EXTENDED with invariants (iv) metric-semantics-verified / (v) census-decontaminated / (vi) artifact-probed-not-prerequisite / (vii) checklist-consulted + paired §1.1 mutations (gate-code = separate work item). **§11.4.67(6)** — sourced-function `exec` redirection-scoping: a command-less `exec` applies EVERY redirection on its line to the CURRENT shell PERMANENTLY, so in sourced/library functions an fd-manipulation `exec` MUST brace-scope its side redirections (`{ exec 9>>"$f"; } 2>/dev/null`); a bare `exec N>file ... 2>/dev/null` in sourced code is a review defect + greppable gate class (forensic FACT 2026-07-22: a sourced lock helper's bare `exec 9>>lock 2>/dev/null` silenced the operator's interactive-shell stderr for the shell's LIFETIME — one provider launch left the terminal unable to print any error — while the paired unlock had used the correct brace form all along; marker-probe proven pre/post-fix); gate `CM-SHELL-EXEC-REDIRECTION-SCOPED` (brace-scoped forms and `exec`-with-command exempt — the §11.4.201(1) false-positive guard) + paired §1.1 mutation. **§11.4.142** — teaching forensic FACT added (2026-07-22): a transport-hardening change was committed AND pushed to `main` BEFORE any independent review (honestly flagged); the post-hoc §11.4.209-substrate review returned NO-GO with ONE BLOCKING finding — an unsafe security-relevant default, already public on every mirror — plus 2 Important + 4 Minor, and remediation consumed a full fix-forward cycle a pre-push review would have folded into the original change; the rule needed NO change (§11.4.142/.125/.134/.209 already bind it with no carve-out) — the

incident is cited as its newest forensic anchor; remediation fix-forward per §11.4.113, never a history rewrite. ALREADY-COVERED verdicts this round: measurement-arm single-resource-ownership + observer-effect budget (§11.4.119 + §11.4.128(1) — the decontamination instance landed as §11.4.201(10), not a new anchor); review-before-push as a rule (bound in full by §11.4.142 family — teaching FACT only). All Classification: universal (§11.4.17) — every toolchain literal genericised; consumers supply scopes, quotas, instruments, and paths as DATA per §11.4.35. The literals `11.4.115` / `11.4.201` / `11.4.67` / `11.4.142` are unchanged — their propagation gates stay GREEN.

§11.4.226 — Evidence-class-at-closure + standing detection pressure: machinery presence does NOT predict whether a fix holds — the EVIDENCE CLASS at closure (runtime vs source) under real DETECTION PRESSURE does; prose does not bind, seams do (research-derived, 2026-07-23). Compact summary: the decisive finding of the 2026-07-22 reopen/first-touch root-cause forensics, confirmed across four independent evidence lines (internal forensics; a 76-source external literature pass; one cross-project corpus WITHOUT tracking machinery; one WITH exemplary machinery): **every fix whose done-claim was preceded by an on-target RED→GREEN polarity flip that held recorded ZERO reopens; every fix confirmed only at source-green bounced** (runtime-evidence corpus: median attempts-to-stick 1, ~0.3% corrected reopen — a floor under weak detection pressure; source-green corpus: 52% recorded, itself an undercount; the healthy corpus's ONLY bouncing family was a wrong-layer echo-assertion; 69 PENDING-BUILD claim-moves; the last audited release entered manual QA with 1/25 items user-layer-validated — the operator was the FIRST user-layer oracle most claims ever met, so first-touch rediscovery was the deterministic pipeline output, not bad luck). The mandate — the STRICT FORMALIZATION of §11.4.115(F)/§11.4.146(D3)/§11.4.108: (1) closed evidence-class RANKING {runtime > artifact > source} with a per-defect-layer floor at every closure seam — a user-visible/runtime defect can NEVER close on artifact/source-class evidence (source-on-source stays legal, the §11.4.201(1) false-refusal guard); (2) class proven by MACHINE FIELDS never a label (runtime ⇒ target fingerprint read FROM the target + runtime observable; artifact ⇒ path + content hash; source ⇒ ref; label-without-fields = SHAPE-INCOMPLETE, unreadable = BLIND); (3) ANTI-ECHO — a "runtime observable" that IS a grep/

source-scan transcript is WRONG-LAYER, refused (a grep can never observe a pixel; the pixel-defect-by-five-greps closure dies at this seam); (4) per-status evidence PRECONDITIONS — every §11.4.15/§11.4.33 status carries a checked precondition, not only the terminal write (FACT: a penultimate "ready"-class status parked 79 items — 42% of the done-claiming population — as design-complete-but-UNVERIFIED; the operator's 6/6-broken sample was drawn from exactly that set); (5) FIX-KIND honesty — a state-only repair/mitigation cannot close a defect item as fixed (sibling-corpus FACT: 11 recurrence chains, median 3 / max 7 attempts, dominated by state-only "fixes"); (6) STANDING DETECTION PRESSURE — pressure is a CAUSAL INPUT to every measured quality rate (a rate without its pressure regime is incomparable, §11.4.6; the 52%-vs-~0.3% split is explained by pressure + closure class, not machinery; the healthy corpus's one recurrence surfaced at 24 days = the next SWEEP not the next use): every registered, topology-present guard carries a FRESHNESS CONTRACT (verdict on the CURRENT artifact fingerprint within a declared staleness budget); the never-executed/stale/over-budget set feeds a STANDING risk-ordered re-run queue (most-reopened-first §11.4.189, then stalest-first) drained by the zero-idle loop (§11.4.94/§11.4.97) as regular development-cycle work — the §11.4.135 release seam is the blocking BACKSTOP, never the only executor (FACT: two standing guards, finally executed during the forensics, immediately emitted the latent FAILs — one reproduced the operator's loudest defect 3/3; registration was treated as coverage while execution was nobody's job); topology-absent guards enumerated never queued, an empty registry is BLIND never "all fresh". Binding: applies during EVERY regular development cycle; CHECKED through every code-review per §11.4.194(6); on every constitution pull the §11.4.32 sweep + §11.4.164 hook analyze + apply the additions and violations are tracked + cleared (§11.4.15) — no parallel mechanism. Honest boundary §11.4.6: a measured correlation with a stated mechanism and zero counter-cases — NOT a randomized trial (partially-censored exposure carried openly); fabricated machine fields stay closed by §11.4.115(F) harness-written verdicts + the §11.4.135 candidate-fingerprint join (layered, not eliminated); oracle calibration stays §11.4.107(10); the layer taxonomy is authored data (mislabelling is the §11.4.194(6)(a) review class's detective target); re-running a weak guard proves little — oracle strength stays §1.1/§11.4.115(F), orthogonal discovery stays §11.4.85/§11.4.118. Reference mechanisms (proven, NOT

wired §11.4.205): SOL-01/02/04/09 RED-first POCs with golden-good/golden-bad/negative-control fixtures under `docs/research/quality/solutions/` (65/65 GREEN, deterministic second iteration) — no force until their seams are wired by tracked §11.4.197 work items. Classification: universal (§11.4.17) — consumers supply layer bindings, evidence stores, verdict registries, and staleness budgets as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-226-PROPAGATION` (literal `11.4.226`) + recommended gates `CM-EVIDENCE-CLASS-AT-CLOSURE` + `CM-GUARD-FRESHNESS-SCHEDULER` + paired §1.1 mutations (close a runtime-layer item on source evidence / label-without-fields / echo-observable / stale guard with no queue entry → the gates FAIL; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6). **Canonical authority:** constitution submodule `Constitution.md` §11.4.226. Non-compliance is a release blocker. No escape hatch — no `--source-green-closes-runtime-defect`, `--label-is-class`, `--echo-observable-OK`, `--registration-is-coverage`, `--rate-without-pressure-regime`, `--mitigation-closes-defect`, `--skip-freshness-queue` flag.

§11.4.227 — Governance-corpus self-custody: the rule corpus is bound by the same custody it imposes — every named gate implemented-or-registered-deferral under a monotone ratchet, and propagation gates count anchor BLOCKS (exactly-once + lockstep-identical), never bare literals (research-derived, 2026-07-23). Compact summary: measured on THIS corpus 2026-07-22 (commands + control needles preserved in the root-cause analysis): 10,735 canonical lines, 220 distinct anchors; **413 named CM-* gates — 241 (58%) with no implementation in either canonical gate site — plus 85 literal "gate-code = separate work item" deferrals**; the custody seam designed 2026-07-17 was still prose on 2026-07-22 (the corpus's highest-value remediation fell into its own gap within five days); the fix-confirmation anchors predate the failures they failed to prevent by five-plus weeks — while **every fully-mechanised rule in the record either prevented or exposed its failure mode** (the PENDING-semantics + release verdict-coverage seams landed within a day of diagnosis and now block): the gap is not rule quality, it is the prose-to-seam conversion rate. The SHAPE defect: the propagation-gate family — the only family that runs everywhere — asserts literal-presence ≥ 1, mechanically rewarding restatement over implementation ("the corpus measures its own health in words present, and so it grows words");

consequences measured: two anchors duplicated verbatim-divergently in ALL FOUR mirrors while every presence-gate stayed green (the F7 incident); one anchor NUMBER heading two different mandates (the §11.4.140/§11.4.141 collision, independently re-detected by the block-integrity instrument — resolution stays an operator-owned re-mint); two canonical entry-point mandates contradicting (the mandated resumption file unreachable from the mandated README); external corroboration: mandated-checklist adoption without workflow embedding moved nothing across 101 hospitals (the Ontario null). The mandate: **(A) NAMED-GATE LEDGER + RATCHET** — every `CM-*`-class token named in the corpus is, at every build, either IMPLEMENTED (token in an EXECUTABLE gate site; prose carriers never count §11.4.201(7)(a)) or covered by a REGISTERED DEFERRAL row pointing at a tracked §11.4.197 item; silent gate debt refused; the unimplemented count is MONOTONE-DECREASING (validated against the definition of done per §11.4.201(8) — at the correct end-state it is 0); a vanished gate NAME without an explicit removal citation FAILs (the delete-names-to-lower-the-count gaming channel closed; repeals stay legal + visible, the §11.4.166 pattern); **an anchor's "done" state is its SEAM landing, not its TEXT landing** — a rule authored without its seam is in §11.4.205's worse-than-nothing state, and the ledger makes that state visible, counted, and shrinking; brownfield baseline = operator §11.4.66 decision (the §11.4.224(E) adoption-fence pattern), never an invented ratchet. **(B) ANCHOR-BLOCK INTEGRITY** — every propagation-class gate counts BLOCK-STARTS (structural line-anchored heading patterns), never bare literals: exactly ONE block per anchor per governance file (0 = absence, >1 = DUPLICATION, FAIL naming anchor + file); content-hash EQUALITY across the declared §11.4.157 lockstep mirror set (divergent copies FAIL naming both files; compact-vs-full variance ACROSS layers stays legitimate — canonical-vs-mirror drift is the §11.4.186 family's job); mid-body citations are CARRIERS and never count; ZERO blocks extracted = BLIND-OR-EMPTY never clean (§11.4.201(6)); an anchor NUMBER heading two mandates is a COLLISION and FAILs (§11.4.54 never-reuse applied to anchor ids); the extractor captures full dotted ids so a sub-anchor never prefix-matches its parent (a measured §11.4.201(1) false positive in the founding instrument, fixed RED-first). **(C)** semantic contradiction between two DIFFERENT anchors stays undecidable-by-hashing — only the decidable projections are mechanized (fenced scopes §11.4.112(5), seam declarations §11.4.120), the

rest is §11.4.142/§11.4.194 review territory, stated never claimed; this anchor adds the enforcement-ratio INSTRUMENT, not a growth ban. Binding: the ledger regenerates + diffs on every governance edit + every build (regular development cycles, never an on-demand audit); review-checked per §11.4.194(6)(c); on every constitution pull the §11.4.32 sweep + §11.4.164 hook run the ledger + block-integrity checks and violations are tracked + cleared. Reference mechanisms (proven, NOT wired §11.4.205): SOL-05 + SOL-06 RED-first POCs under `docs/research/quality/solutions/` (SOL-05's live run reproduced the 413-gate census exactly; SOL-06's live run confirmed the four mirrors clean post-F7 + re-detected the 140/141 collision) — no force until wired by tracked work items. Classification: universal (§11.4.17) — consumers (and this governance repo itself) supply gate-site lists, deferral registries, and lockstep mirror sets as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-227-PROPAGATION` (literal `11.4.227` — itself REQUIRED block-start-shaped per (B)) + recommended gates `CM-GATE-LEDGER-RATCHET` + `CM-ANCHOR-BLOCK-INTEGRITY` + paired §1.1 mutations (name a gate with no implementation + no deferral / delete a name without citation / duplicate a block / diverge two lockstep copies / re-mint a number → the gates FAIL; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6). **Canonical authority:** constitution submodule `Constitution.md` §11.4.227. Non-compliance is a release blocker. No escape hatch — no `--name-gate-without-seam`, `--silent-gate-debt-OK`, `--delete-name-to-lower-count`, `--presence-gte-1-suffices`, `--duplicate-anchor-OK`, `--mirror-drift-OK`, `--reuse-anchor-number` flag.

EXTENSION landed 2026-07-23 (round 2 — the reopen/first-touch root-cause harvest; full text in `Constitution.md`). §11.4.194(6) — the quality-harvest review classes: every code-review (§11.4.142 universal / §11.4.125 gate / §11.4.209 Opus-xhigh substrate, iterated to GO per §11.4.134) MUST additionally CHECK, per reviewed change: (a) **closure-evidence class vs defect layer** (§11.4.226) — a terminal/ready-class status moving on evidence below the defect layer's floor, a user-visible property "verified" by file-existence + greps, an echo-assertion, or a state-only repair closing a defect as fixed is a FINDING; (b) **needled measurements** — every load-bearing measurement in the change/its tests satisfies §11.4.201(6)–(12), and NO count was converted into a finding without reading the underlying lines — a COUNT IS A LEAD, THE

LINES ARE THE FINDING (forensic FACT: twice in one measured cycle a count was read as state — a "mixed half-written doc" that was actually coherent, and an inflated FAIL total that counted its own summary line); (c) **governance-and-gate integrity** (§11.4.227) — a change NAMING a gate lands its implementation or registered deferral in the same change; a governance edit preserves anchor-block integrity; (d) **reviewer-authored adversarial mutations** — the author's self-authored mutation set is NECESSARY but NOT SUFFICIENT (forensic FACT, from the dated forensic corpus cited in the 2026-07-22 root-cause analysis: an author's six self-authored mutations were ALL caught while an independent reviewer's FIRST THREE different mutations ALL sailed through, each restoring a user-visible defect); the reviewer MUST attempt ≥1 mutation the author did not write (or record honest infeasibility per §11.4.3) — a guard whose reviewer-authored mutation survives is unvalidated instrumentation per §11.4.115(F) and a FINDING. The literal `11.4.194` is unchanged — its propagation gate stays GREEN.

§11.4.228 — Cross-agent extension lifecycle mandate: per-platform compatibility, source tracking, documentation with compatibility matrix, and auto-wiring on constitution pull (research-derived from Extensions Catalog forensics, 2026-07-24). Compact summary: every governed extension (Skill, MCP, ACP, LSP, plugin) MUST (A) ship a per-platform compatibility declaration (Claude Code, OpenCode, Gemini CLI, Qwen Code minimum) with closed vocabulary `{SUPPORTED|UNTESTED|UNSUPPORTED|CONFIG-ONLY}`, NEVER cause loops/broken behavior in any platform, and carry a per-platform liveness test §11.4.169; (B) be traceable to its exact canonical origin (repo + commit SHA + path, recorded in `EXTENSION_SOURCE.yaml` — symlinks preferred over copies, blind copies without provenance are §11.4.124 dead-code-in-waiting), with a per-project extensions catalog (`docs/extensions/EXTENSIONS_CATALOG.md`) as the single source of truth, §11.4.86 roster/corpus-backed fingerprint; (C) the extensions catalog MUST carry a per-platform compatibility matrix + liveliness-testing evidence per extension × platform per §11.4.69 `feature_class=extension_loading`, four-format export per §11.4.65, auto-synced via §11.4.106 docs-chain; (D) the §11.4.164 `post_update_hook.sh` is EXTENDED to auto-detect all governed agent platforms, symlink constitution skills into each (`.claude/skills/` , `.opencode/skills/` , platform

equivalents), register MCP servers into each platform's config, self-validate, and be idempotent — closing the gap where 4 constitution skills exist in source but are un-wired to any platform (the §11.4.108 source-present-runtime-absent gap). Honest boundary §11.4.6 — wiring guarantees REACHABILITY, never runtime acceptance. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-228-PROPAGATION` + recommended gates `CM-EXTENSION-PER-PLATFORM-COMPAT` / `CM-EXTENSION-SOURCE-TRACKED` / `CM-EXTENSION-CATALOG-DOCUMENTED` / `CM-CONSTITUTION-EXTENSIONS-AUTO-WIRED` + paired §1.1 mutations. Full text in `Constitution.md` §11.4.228.

§11.4.229 — Live in-session task/todo tracker MUST always be up to date and fully in sync with the real work state — never stale, never completed-shown-as-pending nor pending-shown-as-done (User mandate, 2026-07-25). Verbatim operator mandate: "make sure it is ALWAYS up to date fully in sync! CRITICAL: This MUST BE one of mandatory rules / constraints! Add it with all relevant details into the constitution Submodule and relevant related files." The agent's LIVE IN-SESSION TASK/TODO TRACKER — the harness-provided task list / `TodoWrite`-class todo tool / any equivalent moment-to-moment execution-state surface the runtime exposes DURING a session — MUST ALWAYS be kept up to date and FULLY IN SYNC with the real live work state: never stale, never showing completed work as pending, never showing pending / blocked / abandoned work as done or in-progress; updated the MOMENT any tracked task's state changes (started → in-progress, completed → done, blocked, abandoned, split, re-scoped, newly discovered), never batched to a later checkpoint, never reconstructed from memory at the end, never left reflecting a superseded plan — a live tracker that lags the real work is a §11.4 PASS-bluff at the execution-transparency layer (an operator or resuming conductor per §11.4.116 makes decisions on a false picture of what is done / in-flight / owed). (A) every state-change updates it IMMEDIATELY (exactly one in-progress where the surface models that; no completed-marked-pending, no pending-marked-done, no ghost task for dropped work — dropped work is closed with a reason); (B) NEVER stale, NEVER inverted (§11.4.6 — the tracker is captured FACT, falsely-done = PASS-bluff / falsely-pending = §11.4.1 FAIL-bluff, drift corrected the moment observed); (C) DISTINCT artifact that COMPLEMENTS never REPLACES §12.10 (committed CONTINUATION doc) / §11.4.131 (standing committed session-resumption FILE) / §11.4.97 (operator-facing milestone progress

cadence) / §11.4.106(F) (doc/DB sync at commit/build/constitution-pull write-seams) / §11.4.202 / §11.4.208 (workable-item DB + request-history ledgers) — those bind committed/cross-session/operator-report/tracked-work artefacts, NONE binds the live in-session tracker itself; it is the in-session analogue of §12.10's always-current mandate applied to the harness task list, the truth those artefacts are materialised FROM; (D) honest boundary §11.4.6 — where no such surface exists this is honestly N/A (§11.4.3 SKIP-with-reason, latent-binding per §11.4.96), and "in sync" = reflects the real work state as the agent knows it now, not omniscience about unobserved work. Classification: universal (§11.4.17). Composes §12.10 / §11.4.131 / §11.4.97 / §11.4.106(F) / §11.4.126 / §11.4.116 / §11.4.202 / §11.4.208 / §11.4.6 / §11.4.1. Propagation gate `CM-COVENANT-114-229-PROPAGATION` (literal `11.4.229`) + recommended gate `CM-LIVE-TASK-TRACKER-IN-SYNC` + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6). No escape hatch — no `--tracker-may-lag`, `--batch-todo-updates`, `--stale-task-list-OK`, `--reconstruct-todos-at-end`, `--skip-task-tracker-sync`, `--pending-may-be-done`, `--completed-may-stay-pending` flag. Full text in `Constitution.md` §11.4.229.

§11.4.230 — Parallelized-pipeline methodology: build↔validate overlap + affected-stages-only re-run + parallel multi-device testing/recording + fan-out-subagents-always + incremental delivery granularity with a declared QA-build cadence + a quick visible first deliverable with a hardening status and ledger tracked to production readiness (research-derived, 2026-07-26; clause (D) added 2026-10-04; clause (E) added 2026-10-12).

Forensic motivation (genericised): the sequential pipeline shape — build FULLY completes → THEN validation begins → on ANY discovery, restart the WHOLE cycle from scratch → all devices tested one-at-a-time — serialises the operator's wall-clock onto build latency + one-device-at-a-time evidence gathering, so a multi-hour containerised build followed by serial per-device validation and a from-scratch restart-on-discovery makes time-to-a-manual-QA-capable-release a multiple of what the genuinely-parallel work requires. §11.4.58's PWU pipeline already overlaps Stage 1 of round N+1 with Stages 4–5 of round N; §11.4.89 already backgrounds long tests; §11.4.103 already mandates ≥3 auto-backfilled parallel streams; §11.4.119 already partitions shared hardware by single-owner; §11.4.128/§11.4.144 already mandate

always-on recording + availability-following. §11.4.230 BINDS those into one operating discipline and adds the five invariants they did not each state on their own: (A) the build↔validate CONCURRENCY seam + affected-stages-only re-run-on-discovery, (B) all-devices-concurrent + worker-pool recording/processing, (C) fan-out-subagents as the standing default, (D) incremental delivery granularity + a declared QA-build cadence (added 2026-10-04), (E) a quick visible first deliverable ordered first on the critical path + a machine-computed per-deliverable hardening status and ledger tracked to production readiness (added 2026-10-12). It is the parallelization capstone the way §11.4.103 is the continuous-parallel-stream capstone — a strengthening bind, never a weakening.

(A) PIPELINE-OVERLAP PARALLELIZATION. Validation/verification work MUST run CONCURRENTLY WITH the build wherever a validation stage does NOT depend on the build's own not-yet-produced artifact — never deferred until the build finishes when it could have run alongside it. Deploy the moment the build is done (build-completion is the deploy trigger, per §11.4.121 no-commit-while-build-writes-tracked-artifacts + §11.4.108 artifact-must-exist ordering — the deploy waits for the artifact, the deploy-independent validation does not). Concretely, WHILE a build runs, the following run in parallel per the §11.4.96 SAFE-during-build catalogue: pre-build source-layer gates + meta-test mutation sweeps + coverage/clash checks (§11.4.110), the §11.4.125/§11.4.142/§11.4.194 independent code-review on the batch's SOURCE, doc/DB sync (§11.4.106), and on-device validation of the PRIOR build (device-independent of the in-flight build, §11.4.119). On any NON-huge-blocker discovery, the cycle MUST: (1) bump the version code (a new, distinctly-identifiable candidate artifact — never re-use the failing artifact's id, §11.4.111 stable-name + §11.4.200 target-identity); (2) fix at the proven root cause (§11.4.102 systematic-debugging first, §11.4.115 RED-on-the-broken-artifact); (3) RE-RUN ONLY THE AFFECTED STAGES, PARALLELIZED — the stages whose inputs the fix changed (its own targeted validation FIRST per §11.4.130, then the affected regression set), NOT a serial restart-from-scratch of the entire cycle. The explicit efficiency target is a **4–5× reduction in time-to-a-manual-QA-capable-release** versus the fully-sequential shape — a measured target, tracked as build-and-validate wall-clock deltas (§11.4.24 build-resource stats), never an assumed one (§11.4.6). **HONEST BOUNDARY (§11.4.6 / §11.4.108 / §11.4.120):** the CURRENT build's ARTIFACT-layer (§11.4.108 layer 2) and

RUNTIME-ON-CLEAN-TARGET-layer (layer 3) validation CANNOT precede its own artifact — a gate whose precondition is the build's output belongs at a post-build seam, never overlapped onto authorship (§11.4.120 seam-placement; §11.4.69 `artifact_not_yet_built`). "Concurrent with the build" therefore means the deploy-INDEPENDENT stages overlap; the CURRENT-artifact runtime validation begins the moment that artifact exists, in parallel across devices per (B). **"Affected-stages-only re-run" is bounded to NON-huge-blocker discovery**: a HUGE BLOCKER (release-blocking-severity defect / regression invalidating the cycle / blast radius reaching the batch's other fixes) still triggers the §11.4.129 STOP → fix-all → full-RESTART protocol (restart, never resume) — §11.4.230(A) does NOT weaken §11.4.129, §11.4.40 full-suite-retest-before-tag, §11.4.130 validate-the-fix-first, §11.4.132 risk-ordered validation, or §11.4.185 manual-QA-final; it narrows the from-scratch restart to exactly the cases those anchors require it, and parallelises everything else.

(B) PARALLEL MULTI-DEVICE TESTING + RECORDING/PROCESSING. Every available, reachable test device MUST be exercised CONCURRENTLY — one validation stream per device, never one-device-at-a-time when the devices are independent — each stream the SINGLE EXCLUSIVE OWNER of that device's exclusive resources per §11.4.119 (the owner drives; any concurrent stream targeting the same device is read-only), so per-device evidence is never cross-contaminated (§11.4.119 evidence-integrity). Every device is ALWAYS-RECORDED per §11.4.128 (non-invasive, side-effect-free, deterministic layout) and AVAILABILITY-FOLLOWED per §11.4.144 (a dropped device logs an honest offline event + reconnect-and-resume, never a silent corpus hole, never a faked continuity). Recording CAPTURE + ANALYSIS/PROCESSING MUST be fanned out through a BOUNDED WORKER-POOL (a Go worker-pool or platform equivalent) so N devices' recordings are captured + analysed in parallel and per-device evidence never serialises onto a single processor — the pool bounded by §12.6 memory + §12.12 thread headroom. Per-device PASS cites its own captured-evidence artefact (§11.4.5/§11.4.69/§11.4.107) on a CLEAN/fresh deployment (§11.4.108 runtime-signature, §11.4.200 verify-deployed-artifact-identity-per-device); the highest-risk / most-reopened cases (§11.4.132/§11.4.189) get the deepest scrutiny FIRST on every device. **HONEST BOUNDARY (§11.4.6):** genuinely-shared exclusive resources (one HDMI/audio sink, one media-playback path) serialise per

§11.4.119 single-owner-over-time even across devices; "all devices concurrent" means independent devices, never two owners of one exclusive resource.

(C) FAN-OUT-SUBAGENTS-ALWAYS. Subagent fan-out (§11.4.20/§11.4.70 subagent-driven; §11.4.187 multi-track ruler; §11.4.192 auto-backfill) is the STANDING DEFAULT for EVERY genuinely-parallelizable effort — batch investigation, test-authoring, gate-authoring, per-device validation (B), per-PWU development (§11.4.58), doc/research work — not a per-request opt-in and not reserved for "big" tasks. A parallelizable effort executed monolithically in the foreground when a fan-out path exists is a §11.4.230(C) violation (the §11.4.94/§11.4.97/§11.4.103 zero-idle discipline applied to HOW work is dispatched, not only WHETHER it runs). **HONEST BOUNDARY (§11.4.6 / §11.4.183):** "always fan out" is BOUNDED by (1) genuine parallelizability (contending/redundant/serially-dependent subagents are NOT fan-out — they are waste, and spawning them is NOT maximization per §11.4.183 maximum-USEFUL-throughput); (2) §12.6 60% memory ceiling; (3) §12.12 `RLIMIT_NPROC` thread headroom; (4) §11.4.58 ≤6 concurrent-agent cap; (5) §11.4.119 single-resource-owner (two subagents may not both drive one exclusive device); (6) critical-state sequencing (commits/pushes/tags stay serial per §11.4.88/§11.4.113). The bar is maximum USEFUL parallel throughput, never agent count; fan-out that exceeds a host-safety ceiling or contends a single resource is a violation, not compliance.

(D) INCREMENTAL DELIVERY GRANULARITY + DECLARED QA-BUILD CADENCE — a long-running spec/feature ships every already-proven-correct increment as it clears its own bar, never withholding delivery until the WHOLE spec/feature is done (operator mandate, 2026-10-04, `CRITICAL`; research-derived; added 2026-10-04). Verbatim operator mandate (2026-10-04): "All work we do, especially SpecKit specs, gets incorporated right away. Fully deterministic validation/verification tests of all supported test types relying on machine-produced evidence get incorporated too, aiming to cover everything critical. All intensive testing (gates, checks, fixing, polishing) keeps running in BACKGROUND continuously until fully done. IN PARALLEL we keep building specs one by one, phase by phase, story by story, task by task. We get fast delivery results and AS SOON AS POSSIBLE builds that can be handed to QA teams for manual testing RIGHT AWAY, while in parallel all testing/fixing/polishing/improvement continues exactly as it would

anyway. We do NOT wait for all of that to finish first — that can take days [...]. QA reports discovered issues; background agents keep working until everything is absolutely flawless. As background work progresses and fixes/improvements land in the main codebase, we keep producing REGULAR builds periodically delivered to QA teams for ongoing full manual testing. It is critical that background agents keep ALL workable items, documentation, and other materials in sync so nothing ever goes out of date." Forensic anchor (genericised, FACT): in one consuming project a multi-phase spec had RED tests drafted for every phase and many source slices independently reviewed to a source-correctness GO, yet nothing reached QA for days, because delivery was implicitly held at the WHOLE-SPEC boundary while ONE slice's iterate-to-GO review loop ran past forty rounds — the finished, proven work of every other slice sat undelivered. The GAP this clause closes, stated precisely so it is not restated elsewhere (§11.4.227): clause (A) overlaps validation with the build of ONE candidate; §11.4.235(A) makes a batch build-eligible on source-correctness alone; §11.4.235(D) makes that eligibility PER SLICE inside a batched review — but none of them obliges an ELIGIBLE increment to actually be DELIVERED before the rest of its spec completes, none sets a delivery CADENCE, none says how an incomplete spec stays safe to ship, and none binds the hand-off to tracker truth. Build-eligibility that is never exercised is the same withheld delivery under a different name. ALL of the following hold:

(D.1) THE DELIVERY UNIT IS THE INCREMENT, NOT THE SPEC. A spec / feature / epic decomposed into phases / stories / tasks (vertical, independently testable slices per the §11.4.148 SLICING extension — Small + Testable) MUST integrate each increment into its INTEGRATION BRANCH — the trunk for `main`-targeted work, or the `feat/` / `product/` / `flavor/` branch (§11.4.195) the increment actually lives on for branch-scoped work — and make it part of the next QA deliverable (D.3) cut from that branch, the moment THAT increment holds its OWN source-correctness GO per §11.4.235(A)/(D). When an increment exists on MULTIPLE coexisting integration branches — for example both a `product/*` AND a `flavor/*` branch per §11.4.195(A) — EACH branch is independently obligated under (D.3)'s own per-branch cadence: the increment's delivery obligation is evaluated separately per branch it lives on, never as a single ambiguous "the branch". A QA build MAY be cut from a `feat/` / `product/` / `flavor/` branch. For a `feat/` branch that QA build IS the live-manual-testing vehicle §11.4.195(B) requires BEFORE the

branch may merge to trunk, so the feature-branch QA build is the PRECURSOR to trunk-landing, never a consequence of it; a `product/` / `flavor/` line, which never merges to trunk the standard way (§11.4.195(C)), receives its cadence builds from its own branch. Clause (D) relaxes neither §11.4.195(B) (no `feat/` merge to trunk before live manual QA) nor §11.4.195(C), and a later merge of the branch to trunk still follows §11.4.195 / §11.4.188. Withholding an already-build-eligible increment solely because OTHER increments of the same spec are unfinished, or because its own or another increment's `test-instrumentation` / `process-doc` loops (§11.4.235(D)) are still open, is a §11.4.230(D) violation. An increment WITHOUT its own source-correctness GO is not deliverable — incremental delivery is never a licence to ship unreviewed or unproven work.

(D.2) INCOMPLETE-SPEC, SAFE DELIVERABLE. Shipping an increment while its spec is incomplete MUST leave the deliverable correct on every path a user or QA tester can reach: EITHER the increment is a complete, independently valuable vertical slice, OR the unfinished surface is LATENT — unreachable by construction, behind a release toggle (P. Hodgson, "Feature Toggles", martinfowler.com, 2017: "Release Toggles allow incomplete and untested codepaths to be shipped to production as latent code which may never be turned on"), or behind branch-by-abstraction (trunkbaseddevelopment.com: a technique "to effect a 'longer to complete' change in the trunk"). A toggle is a PERMITTED mechanism, never a mandated one (consistent with §11.4.265's own statement that it mandates none), and it is inventory with a carrying cost (Hodgson: "Savvy teams view their Feature Toggles as inventory which comes with a carrying cost"): every release toggle carries a tracked §11.4.197 removal item and a consumer-declared expiry (§11.4.35), and a toggle past its expiry is a §11.4.261 finding. A toggle that needs to outlive its expiry — for example because its latent path's (D.4) hardening loop legitimately runs many rounds — is extended ONLY through a §11.4.271 waiver (rostered non-producer authoriser, a new unelapsed expiry, a named tracked item), never by a silent expiry edit, by enabling the still-unproven path, or by deleting finished work. A latent path is NOT an advertised capability (§11.4.266) until it is enabled and proven.

(D.3) A DECLARED QA-BUILD CADENCE, NOT A ONE-SHOT. Every project declares as DATA (§11.4.35) its QA-build trigger: `per-increment` (each build-

eligible increment landing on its integration branch, D.1), `periodic` (a fixed interval — the release-train pattern; SAFe's "Develop on Cadence. Release on Demand" separates the development rhythm from the release decision), or `whichever-first`; a project that declares nothing gets `per-increment`, the safe default (§11.4.101). A `periodic` or `whichever-first` interval MUST be recorded as DATA together with its written justification (§11.4.35) and stated in every QA hand-off record (D.5), so an interval long enough to defeat this clause is visible rather than silent; this clause sets no numeric maximum — any such cap is an operator decision (§11.4.66). Whenever an integration branch (D.1) holds build-eligible content not yet in a QA build cut from that branch and the declared trigger has fired, a QA deliverable MUST be cut from that branch and handed to QA for ongoing manual testing (§11.4.185). Builds COALESCE: a QA build already in flight covers every increment that had landed on its branch before that build started, and increments landing WHILE it is in flight roll into the NEXT build rather than each demanding a fresh one — the obligation is that eligible content never waits past the next build, not one build per increment. A build that FAILS, or is REFUSED at §11.4.236, covers NOTHING — its increments roll into the next build. "Completed" in this clause and in its two amended gate contracts (D's recommended gates, below) means HANDED TO QA: a build provides coverage only once it has genuinely been handed to QA for manual testing, never merely once the build process itself exits successfully. Each such deliverable is a §11.4.235(B) cycle boundary with its own monotonic version increment (§11.4.151), passing §11.4.236 readiness on its own candidate fingerprint, and built once then promoted (§11.4.264). The §11.4.236 "mandated validation" for a cadence build is AT MINIMUM the source-correctness GO plus the pre/post-build gates of §11.4.235(A), PLUS every operator-declared blocker bound to the QA-deploy seam per §11.4.236 and any §11.4.239 critical-invariant scenarios the increment touches — NOT the open background-hardening backlog, which by §11.4.235(A) never gates a build. "Deliverable" is project-defined DATA (§11.4.35): a binary, a container, a device image — or, for a dev-tooling / governance feature, the feature's own demonstrable, testable state at a fingerprinted commit; no artifact form is implied by this clause.

(D.4) CONTINUOUS BACKGROUND HARDENING THAT NEVER BLOCKS THE NEXT INCREMENT. Test-instrumentation hardening, gate / check / mutation authoring, polishing, and the fixes for QA-reported findings on

ALREADY-delivered increments run as standing background streams (§11.4.103 / §11.4.192 / clause (C)), each a tracked item (§11.4.197) driven to its own zero-finding GO (§11.4.134 — not relaxed), landing on its integration branch (D.1) as it reaches GO and riding the NEXT cadence build cut from that branch. They never gate a LATER increment's build-eligibility, with two exceptions that follow from existing anchors rather than relax them: an open `source-defect` finding (§11.4.235(D)) in code the later increment touches or depends on is a `source-defect` of that later increment too; and a huge blocker still triggers §11.4.129 STOP → fix-all → full restart.

(D.5) NEVER-STALE TRACKING IS A PRECONDITION OF EVERY HAND-OFF, NOT AN AFTERTHOUGHT. With many increments and many background streams in flight at once, the existing sync machinery — §11.4.106(F) write-seam doc/DB sync, §11.4.229 live tracker, §11.4.202 / §11.4.208 / §11.4.210 intake and request history, §11.4.148 item integrity, §11.4.197 completion — applies at EVERY increment landing and EVERY cadence build (cited, not restated). Each QA hand-off records the deliverable's fingerprint and the exact set of tracked items it contains; a hand-off whose tracker or documents disagree with the deliverable (an item marked done whose fix is not in that build, or a fix in the build that no item records) is a §11.4 PASS-bluff at the delivery layer.

(D.6) QA FINDINGS ON EARLY INCREMENTS ARE STILL COVERAGE ESCAPES. Earlier, more frequent QA builds surface defects earlier; they do NOT make manual QA the discovery mechanism. Every defect manual QA finds on an increment build triggers the §11.4.238(C) coverage-escape audit and a new automated check, exactly as for a release candidate.

(D) honest boundary, research grounding, and gates. Honest boundary (§11.4.6): clause (D) changes WHEN and HOW OFTEN proven-correct work reaches QA; it does NOT change what gates a release candidate. It does not weaken §11.4.129 (huge-blocker full restart), §11.4.40 (full-suite retest before a release TAG — a cadence QA build is not a tag), §11.4.185 (manual-QA-final terminal gate for a release), §11.4.236 (readiness verdict per candidate), §11.4.264 / §11.4.265 (build once and promote; progressive delivery), §11.4.134 (every finding of every layer still driven to GO), or §11.4.235(D)'s conservative-safe default (an unclassifiable finding is a `source-defect` and does block). It does not prove any increment correct — that stays §11.4.108 /

§11.4.235. Research grounding, stated honestly: DORA's "Working in small batches" capability (dora.dev) — "break them down into work units that can be completed independently and in short timeframes", with dark launching / feature toggles named as the way to deploy incomplete features hidden from users; Hodgson's toggle taxonomy and toggle-as-inventory discipline (quoted in D.2); branch-by-abstraction (quoted in D.2); SAFe's "Develop on Cadence. Release on Demand" (D.3); and GitHub Next's 2026 "Continuous AI" / Agentic Workflows technical preview, which its principal researcher calls "the agentic evolution of continuous integration" — scheduled or event-triggered agent loops for recurring work such as CI-failure investigation and documentation-drift fixing, cited here only as industry direction for (D.4)/(D.5), not as an adopted mechanism. No single published methodology was found that combines per-increment QA delivery with continuously running agent hardening streams; that composition is this clause's own synthesis, not a claim of prior art. Classification: universal (§11.4.17) — consumers supply the cadence trigger, interval, toggle mechanism and expiry policy, and the deliverable form as DATA per §11.4.35. Composes §11.4.101 / §11.4.103 / §11.4.106(F) / §11.4.129 / §11.4.134 / §11.4.148 / §11.4.151 / §11.4.185 / §11.4.188 / §11.4.192 / §11.4.195 / §11.4.197 / §11.4.202 / §11.4.208 / §11.4.210 / §11.4.229 / §11.4.235(A)(B)(D) / §11.4.236 / §11.4.238 / §11.4.239 / §11.4.261 / §11.4.264 / §11.4.265 / §11.4.266 / §11.4.271. Recommended gates `CM-INCREMENT-DELIVERED-NOT-SPEC-WITHHELD` (FAIL: an increment holding its source-correctness GO is absent from every completed-and-handed-to-QA or in-flight QA build from its integration branch (D.1) after the declared trigger fired while no build from that branch is in flight — coalescing per (D.3) so an increment waiting on an in-flight build's successor is never a FAIL, and a build that FAILED or was REFUSED at §11.4.236 counts as NEITHER completed NOR in-flight so it provides no coverage and the increment rolls to the next build; FAIL, the §11.4.201(1) dual guard: an increment WITHOUT that GO appears in a QA build) + `CM-QA-BUILD-CADENCE-DECLARED` (FAIL: an integration branch (D.1) holds build-eligible content that no completed-and-handed-to-QA or in-flight QA build from that branch covers, the declared trigger has fired, and no QA build from that branch is in flight; coalescing per (D.3) — a build in flight covers every increment that landed before it started and later increments roll into the NEXT build, so an in-flight build is never a FAIL and the gate never demands one build per increment; a build that FAILED or was REFUSED at

§11.4.236 covers NOTHING and does not satisfy this gate; FAIL also: a `periodic` / `whichever-first` interval with no recorded justification; an absent declaration resolves to `per-increment`, never to "no cadence") + `CM-RELEASE-TOGGLE-TRACKED-EXPIRY` (FAIL: a release toggle with no tracked removal item or no expiry, or past its expiry) + `CM-QA-HANDOFF-TRACKER-IN-SYNC` (FAIL: a hand-off record whose item set disagrees with the deliverable's contents) + paired §1.1 mutations (withhold an eligible increment until spec completion; ship an increment lacking its GO; delete the cadence declaration and assert a "no cadence" reading; add a toggle with no expiry; mark an item done whose fix is absent from the build → each gate MUST FAIL; golden-FALSE: an increment shipped on the first trigger after its GO, behind a tracked unexpired toggle, with a matching hand-off record → none fires). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227); the four gates are registered as tracked deferrals in `constitution/scripts/gates/gate_ledger_deferrals.tsv`. No escape hatch — no `--withhold-until-spec-complete`, `--hardening-gates-next-increment`, `--no-qa-cadence`, `--untracked-release-toggle`, `--stale-tracker-at-handoff`, `--manual-qa-as-discovery` flag.

(E) QUICK VISIBLE FIRST DELIVERABLE, THEN HARDENING TRACKED TO PRODUCTION READINESS — A FAST DELIVERABLE IS ALWAYS LABELLED AS FAST, NEVER PASSED OFF AS FINISHED, AND NEVER BOUGHT WITH WEAKER EVIDENCE (operator mandate, 2026-10-12, `CRITICAL`; added 2026-10-12). Verbatim EXCERPT of the operator mandate (2026-10-12, `CRITICAL`): the closing part of operator request R-2026-10-12-195857, quoted exactly as typed, every spelling error preserved uncorrected [sic]. The earlier part of the same message sets unrelated bug-fix priorities; the whole message is recorded under that id in the consuming project's §11.4.208 operator-request ledger (in the reference project, `docs/requests/history.md`). The excerpt: "Follow our main development rule: do it all first, build and deploy images, keep testing in background!!! It is ABSOLUTELY critical do create, generate, fix and polish it all and deliver flashing imases fastest possible and then keep testing, fixing and polishing in parallel everything further! We MUST ALWAYS HAVE QUICK VISIBLE RESUTLS!!! This MUST BE part of the constitution (root constiotution Submopdule) and properly added! After quick delivery then in parallel we MUST properly track progress on testing, fixing and polishing until we get to full

production readiness!!! Maximall paralelization and use of agents / subagents is mandatory so we can deliver fastest and best results! Do additional in depth analisys and planning in order for this method to work flawlessly for this cutting-edge enterprise project! All checks, validation, verification and testing MUST BE done ALWAYS fully deterministically, with no false or faulty results and with no bluff or AI slop of any kind!" Typos as typed, reproduced uncorrected [sic]: `imases`, `RESUTLS`, `constiotution`, `Submopdule`, `Maximall`, `paralelization`, `analisys`. Normalised reading (an aid only, never a substitute for the verbatim text): deliver flashable images as fast as possible, keep testing, fixing and polishing in parallel; quick visible results always; record this in the root constitution submodule; after the quick delivery, track the progress of testing, fixing and polishing in parallel until full production readiness; maximal parallelisation and use of agents and subagents; in-depth analysis and planning; every check, validation, verification and test fully deterministic, with no false or faulty result and no bluff. (The project nouns in the quote are the operator's own example; the rule below is generic and consumers supply those nouns as DATA per §11.4.35.) The GAP this clause closes, stated precisely so it is not restated elsewhere (§11.4.227): clause (D) makes every source-proven increment deliverable on a declared cadence and keeps the tracker true at hand-off (D.5); clause (C) and §11.4.183 make fan-out the default; §11.4.235(A)/(E) build, flash and live-verify a GO'd group; §11.4.236 refuses a hand-off without a candidate-fingerprinted verdict — but none of them (i) orders a new scope's critical path so that its FIRST deliverable is not queued behind deliverable-independent work, (ii) records HOW HARDENED a delivered build is, (iii) joins the hardening debt still open against a delivered fingerprint into one ledger, (iv) tracks that debt's trajectory to production readiness with a direction rule, a stall rule and a starvation rule, or (v) defines what "quick" means as a measurement (measured 2026-10-12 with a control-needled census of the canonical file: zero hits for `delivered-unhardened` and `hardening ledger` (the single bare `unhardened` hit is an unrelated §11.4.276 forensic carrier); positive control `incremental delivery` hits inside clause (D); negative control, a fabricated token, zero). A fast deliverable whose record cannot be told apart from a finished one is a §11.4 PASS-bluff at the delivery-status layer; a fast deliverable that skipped any seam is a §11.4 PASS-bluff at the evidence layer. ALL of the following hold:

(E.1) FIRST-DELIVERABLE CRITICAL PATH, SCHEDULED FIRST. When a scope starts (a spec, a declared group of tracked items, or any consumer-declared delivery scope — DATA per §11.4.35), the work that makes its FIRST increment build-eligible and deliverable is the CRITICAL PATH and is scheduled FIRST: the increment's source fix; its source-correctness review to GO (§11.4.235(A)/(D), on the §11.4.209 substrate, bounded by §11.4.276); the pre/post-build gates and every operator-declared QA-deploy blocker §11.4.236 requires for that candidate; the build; the flash; and the §11.4.235(E) live sanity and per-item verdicts. Deliverable-independent work — every entry class of the (E.3)(b) ledger that the seams of (E.2) do not require for a QA build — MUST NOT be placed ahead of that path. It STARTS IN PARALLEL from the first moment of the scope (clause (C), §11.4.103, §11.4.183, §11.4.192) as background streams that never sit on the critical path. "Quick" therefore re-orders WHEN hardening COMPLETES relative to the first deliverable; it never decides WHETHER hardening runs and never makes hardening wait to START. The first deliverable MUST contain at least one increment holding its own source-correctness GO (D.1) and MUST exercise a user-visible behaviour of that increment on the target (the "visible" in the mandate) — an empty, trivial or no-behaviour-change build delivered only to stop a clock is a §11.4.6 fabrication. A consumer's own stricter precondition on building (for example a mandatory specification-and-review workflow that must reach GO before any build) still applies in full and is part of the critical path, never shortened by this clause.

(E.2) EVIDENCE IS NEVER TRADED FOR SPEED — QUICK IS NEVER UNVERIFIED. A quick deliverable passes EXACTLY the same seams, with the same strength of evidence, as any other QA build; all of these bind unchanged and are cited, not restated: the source-correctness GO (§11.4.235(A)/(D), §11.4.209, §11.4.134, §11.4.276); §11.4.108 artifact-layer and runtime-on-clean-target-layer evidence for the delivered fingerprint; §11.4.236 readiness; the §11.4.235(E) per-device live sanity and per-item verdicts; §11.4.5 / §11.4.262 machine-created evidence; §11.4.50 exactly as that anchor states it — "Every test that PASSes MUST have been executed N times (default N=3, N=10 for cycle-validation suites) against the same firmware MD5 + same device + same topology and produced IDENTICAL PASS in every iteration." — and §11.4.201(6)-(12) / §11.4.273 control-needled nulls. Nothing in this clause lowers, skips, defers, samples or narrates any of these. A deliverable that

reached QA by weakening one of them is not a quick deliverable — it is a §11.4 PASS-bluff, and §11.4.236 refuses it. What a quick deliverable MAY still lack is EXACTLY the set of entries the (E.3)(b) ledger can hold — the ledger's entry classes are the one and only list of permitted gaps, so the two lists are identical by construction — except that a seam above is never a gap: a contained item with neither a live verdict nor an honest per-item SKIP-with-reason blocks the hand-off itself, and an (L1) entry is `open` in a delivered ledger only when a later live FAIL reopens it. Every permitted gap is LABELLED in the deliverable's status and ledger per (E.3), never silently absent.

(E.3) EVERY DELIVERABLE CARRIES A MACHINE-COMPUTED HARDENING STATUS, ITS OPEN HARDENING LEDGER, AND THE EVENT RECORDS BOTH ARE COMPUTED FROM. Every QA hand-off record (clause (D.5)) MUST carry, for the delivered fingerprint, (a) the HARDENING STATUS, computed from (b) the OPEN HARDENING LEDGER and (c) the EVENT RECORDS defined below. (b) THE LEDGER is a READ-ONLY JOIN over the stores that already own each fact, NEVER a new parallel or hand-maintained store (§11.4.227, §11.4.233(F) reuse). Its entry classes form a CLOSED list: (L1) each contained item with no passing §11.4.235(E.3) live verdict on this fingerprint; (L2) per contained item, the tests still owed — the §11.4.4(b) four layers, the §11.4.27 / §11.4.169 test types and the paired §1.1 mutations (future-regression instrumentation, additional test types, mutation hardening); (L3) each open §11.4.235(D) `test-instrumentation` / `process-doc` finding against contained code; (L4) each open §11.4.261 finding touching contained code; (L5) each tracked polish item recorded against a contained item; (L6) the §11.4.40 full-suite retest, until it has passed on this fingerprint; (L7) each §11.4.260 production-readiness invariant that is not green for a contained component; (L8) each §11.4.271 waiver covering contained code. Every entry carries its own tracked §11.4.197 item id (§11.4.54) — an entry with no tracked id is itself a §11.4.261 finding — and exactly one DISPOSITION from the CLOSED set { `open` | `closed` | `skipped` | `not-applicable` | `waived` }: `closed` — the entry's own closing evidence exists for this fingerprint; `skipped` — an honest per-item SKIP-with-reason whose code is one of the two TOPOLOGY-ABSENT codes of the consumer's §11.4.69 closed reason set, `hardware_not_present` or `topology_unsupported`, never invented (§11.4.3); `not-applicable` — the consumer has no tier for that entry, recorded with the §11.4.69 code `feature_disabled_by_config` (for (L1) this

is exactly the §11.4.235(E.4) N/A consumer with no device-targeted flash/deploy entry point and no bound deploy ledger); `waived` — covered by a waiver that is valid under §11.4.271 at computation time (an elapsed or invalid waiver makes the entry `open`); `open` — every other case, and every case whose input cannot be resolved. PERMITTED DISPOSITIONS PER CLASS — a closed table; a disposition outside its class's row is INVALID: (L1) `open` | `closed` | `skipped` | `not-applicable` — a live verdict is an (E.2) seam and is never waived; (L2) `open` | `closed` | `skipped` | `not-applicable` | `waived`; (L3), (L4) and (L5) `open` | `closed` | `waived` — a finding or a polish item never disappears as absent topology; (L6) `open` | `closed` | `not-applicable` — the §11.4.40 retest is never skipped and never waived; (L7) `open` | `closed` | `not-applicable` — an unmet §11.4.260 invariant blocks release and is never skipped or waived, and `not-applicable` means only that the invariant has no subject in that component; (L8) `open` | `closed` | `waived` — `closed` once the waived debt is paid and the waiver retired. `skipped`, `not-applicable` and `waived` entries count as resolved for the status but are NEVER hidden: they are listed with their reason and tracked id, counted per class in every announcement, and enumerated as honest gaps in the release notes (§11.4.135, §11.4.118). (c) THE EVENT RECORDS. Every fact the status reads is an EVENT RECORD in the existing append-only event stream (§11.4.116 — reused, never a second store), carrying: the fingerprint; a SEQUENCE NUMBER assigned by the stream's single writer at append time and strictly increasing within the stream; the wall-clock write time, recorded for audit and NEVER used for ordering, because clocks skew (§11.4.6); the WRITER ROLE, stamped by the seam that performs the write and never supplied by the writer itself (§11.4.240(B), capability not instruction); the record KIND from the closed set { `disposition` (an entry id, a disposition, a reason code and an evidence reference; a class label, if the record carries one, is informational only), `qa-confirmation` (the §11.4.185 act), `release-seam-pass` }; and its payload. Every write to an owning store — creating a row, changing an entry's disposition, deleting a row — is recorded in that store's WRITE-SEAM LOG (§11.4.106(F)), and the seam mirrors at the same write, as a `disposition` record, every write that creates an entry's row or changes its disposition: a creation is mirrored as an `open` BIRTH record with reason code `open-debt`, so every entry is named by a record from birth and a later deletion of its row never removes it from the entry set (V5). The log entry of

each mirrored write names the stream position of its mirror record (F7.4). A record carries exactly ONE fingerprint and is never inherited by another fingerprint: a write to a FINGERPRINT-KEYED store is mirrored once, for that fingerprint; a write to a SCOPE-KEYED store is mirrored once per already-delivered fingerprint whose hand-off record (D.5) names a contained item or code path that the written row matches, at the moment of the write, so a later re-close reaches every older fingerprint it matches; a fingerprint delivered AFTER that write acquires no record from it — instead, at that fingerprint's hand-off, the delivering seam writes an `open` BIRTH record for that fingerprint for every row the join matches at that moment (logged as a `birth` write, (F7.3)), so its matching rows are named from delivery and stay `open` until the verifier seam re-checks their closing evidence against that fingerprint and writes a record for it. WRITER ROLES form the closed set { `verifier-seam` | `qa-confirmer` | `release-seam` | `producer` }, and the ALLOWED WRITERS are: a `disposition` record setting `open` — any role, because a downward write is always safe (§11.4.201(4)); a `disposition` record setting any other disposition — the `verifier-seam` only, never the producer of the contained work (§11.4.240, §11.4.249); a `qa-confirmation` record — the `qa-confirmer` only; a `release-seam-pass` record — the `release-seam` only. FIELD TABLE — the status reads exactly the fields below and nothing else. The table is the field-by-field projection of the status function's ONE typed input record: every value the function reads is a named field of that record or of a record nested in it, every field below is read, and each carries five columns — SOURCE, the one place its value comes from; VALID, what makes it usable; ABSENT, what happens when it is missing, unreadable or outside its set; ORDER, whether it orders anything; IDENTITY, how one instance is told from another. An implementation conforms only when its own input record projects to this table byte for byte. No unlisted fact — the order in which stores are declared or read, the order in which records are stored, a wall-clock time other than (F10), or a label on a record other than the comparison of (F6.10) — ever decides a status, so a tie this table does not resolve is never broken: it is X. (F1) `fingerprint` — SOURCE: the hand-off record (D.5): the delivered fingerprint; VALID: non-empty; ABSENT: X; ORDER: none; IDENTITY: exact equality; a record or row is in scope only when its own fingerprint equals it. (F2) `contained_items` — SOURCE: the hand-off record (D.5): the contained tracked §11.4.197 item ids; VALID: a set, possibly empty; ABSENT: X; ORDER:

none — a set; IDENTITY: the item id. (F3) `contained_paths` — SOURCE: the hand-off record (D.5): the contained code paths; VALID: a set, possibly empty, of repository-relative code paths, each of which normalises (V3a); ABSENT: X; a contained path that does not normalise (V3a) is X; ORDER: none — a set; IDENTITY: the normalised path (V3a): two paths are one path exactly when their normalised forms are equal. (F4) `store_decls` — SOURCE: the consumer's owning-store declaration (§11.4.35), one declaration per store; VALID: at least one declaration; store names unique; ABSENT: X; no declaration at all, or two declarations of one store, is X; ORDER: none — declaration order carries no meaning; IDENTITY: the store name (F4.1). (F4.1) `store_decls[].decl_store` — SOURCE: the declaration; VALID: non-empty; ABSENT: X; ORDER: none; IDENTITY: the store's name, unique among the declarations. (F4.2) `store_decls[].required` — SOURCE: the declaration; VALID: true or false; ABSENT: X; ORDER: none; IDENTITY: belongs to its declaration. (F4.3) `store_decls[].keying` — SOURCE: the declaration; VALID: exactly one of `fingerprint-keyed`, `scope-keyed`, `not-fingerprint-bound`; ABSENT: X, and a value outside that set is X; ORDER: none; IDENTITY: belongs to its declaration. (F5) `store_reads` — SOURCE: the read of each declared store performed at computation time — a runtime fact, never the declaration; VALID: exactly one read per declared store; ABSENT: a declared store with no read, or a read of an undeclared store, is X; ORDER: none — read order carries no meaning; IDENTITY: the store name (F5.1). (F5.1) `store_reads[].read_store` — SOURCE: the read; VALID: names a declared store; ABSENT: X; ORDER: none; IDENTITY: matches (F4.1). (F5.2) `store_reads[].readable` — SOURCE: the read itself, shown with a §11.4.273 control needle that the store answered; VALID: true only when the needle was found; ABSENT: false, or no needle result, is X; ORDER: none; IDENTITY: belongs to its read. (F5.3) `store_reads[].rows` — SOURCE: the rows the read returned; VALID: each row carries (F5.3.1)–(F5.3.3); ABSENT: a readable store may return none and then contributes no entry, unless it is required (X); ORDER: none — row order carries no meaning; IDENTITY: the row key (F5.3.1). (F5.3.1) `store_reads[].rows[].key` — SOURCE: the row; VALID: matched in at most one row across every declared store (V4); ABSENT: X; ORDER: none; IDENTITY: the entry id: the tracked §11.4.197 item id for (L1), (L2), (L5), (L6) and (L7), the finding id for (L3) and (L4), the waiver id for (L8). (F5.3.2) `store_reads[].rows[].cls` — SOURCE: the row; VALID:

exactly one of (L1)–(L8); ABSENT: X, and a value outside that set is X; ORDER: none; IDENTITY: the entry's class; never taken from a record. (F5.3.3) `store_reads[].rows[].scope` — SOURCE: the row; VALID: a fingerprint in a fingerprint-keyed store; in any other store, a tracked item id or a repository-relative code path; ABSENT: X; in a scope-keyed or not-fingerprint-bound store, a scope that is not an item id in (F2) and does not normalise (V3a) is UNRESOLVABLE: X, never silently left out of the join; ORDER: none; IDENTITY: decides whether the row is matched for the fingerprint (V3). (F6) `records` — SOURCE: the existing §11.4.116 append-only event stream, read at computation time; VALID: readable; ABSENT: X; ORDER: none of its own — only (F6.2) orders, never storage order; IDENTITY: the stream position (F6.1). (F6.1) `records[].rid` — SOURCE: the stream, assigned on append; VALID: unique in the stream; ABSENT: missing, or shared by two records: a SEQUENCE DEFECT (V2); ORDER: none — a position never orders; IDENTITY: the record's identity, named by write-log mirrors (F7.4) and repairs (F9.1). (F6.2) `records[].seq` — SOURCE: the stream's single writer, assigned on append; VALID: an integer, unique in the stream; ABSENT: missing or shared: a SEQUENCE DEFECT (V2); ORDER: the ONLY ordering key; IDENTITY: none. (F6.3) `records[].kind` — SOURCE: the record; VALID: `disposition`, `qa-confirmation` or `release-seam-pass`; ABSENT: any other value: the record is never valid and decides nothing; ORDER: none; IDENTITY: none. (F6.4) `records[].role` — SOURCE: the stamp of the seam that performed the write, never the writer; VALID: `verifier-seam`, `qa-confirmer`, `release-seam` or `producer`; ABSENT: any other value: the record is not valid; ORDER: none; IDENTITY: none. (F6.5) `records[].fp` — SOURCE: the record; VALID: exactly one fingerprint; ABSENT: missing, empty or unreadable: a SEQUENCE DEFECT for every fingerprint (V2); ORDER: none; IDENTITY: the record is in scope only when equal to (F1), and is never inherited by another fingerprint. (F6.6) `records[].entry` — SOURCE: the record (`disposition` only); VALID: matches exactly one matched row (V4); ABSENT: the record is not valid; ORDER: none; IDENTITY: names the entry by its row key (F5.3.1). (F6.7) `records[].disp` — SOURCE: the record (`disposition` only); VALID: in the closed disposition set and in its class's permitted row; ABSENT: the record is not valid; ORDER: none; IDENTITY: none. (F6.8) `records[].code` — SOURCE: the record (`disposition` only); VALID: in its disposition's permitted

reason-code row; ABSENT: the record is not valid; ORDER: none; IDENTITY: none. (F6.9) `records[].ref` — SOURCE: the record (`disposition` only); VALID: present when the code requires it; for `waived` it names the waiver (F8.1); ABSENT: the record is not valid when its code requires it; ORDER: none; IDENTITY: none. (F6.10) `records[].label` — SOURCE: the record; VALID: informational; only compared with the entry's class (V6); ABSENT: nothing — a record need not carry one; ORDER: none; IDENTITY: never a source of class; a label that differs from the class makes the record not valid. (F7) `write_log` — SOURCE: each owning store's write-seam log (§11.4.106(F)), which records every mirroring obligation the seam creates; VALID: readable; ABSENT: X; ORDER: none; IDENTITY: (F7.1) and (F7.2). (F7.1) `write_log[].w_fp` — SOURCE: the log entry; VALID: a non-empty fingerprint; ABSENT: missing, empty or unreadable: X; ORDER: none; IDENTITY: an entry is in scope only when equal to (F1); an entry with no readable fingerprint is X. (F7.2) `write_log[].w_key` — SOURCE: the log entry; VALID: an entry id; ABSENT: X for an in-scope entry (F7.1); ORDER: none; IDENTITY: the entry the write concerns. (F7.3) `write_log[].op` — SOURCE: the log entry; VALID: `birth` (the entry's row matched at hand-off or created after it) or `disposition` (a change of its disposition); ABSENT: X for an in-scope entry (F7.1); ORDER: none; IDENTITY: none. (F7.4) `write_log[].mirror` — SOURCE: the log entry, written by the seam that appended the mirror record; VALID: the stream position (F6.1) of an in-scope `disposition` record naming (F7.2), with disposition `open` for a birth; ABSENT: missing or not matching, for an in-scope entry (F7.1): X; ORDER: none; IDENTITY: points at (F6.1). (F8) `waivers` — SOURCE: the §11.4.271 waiver register, read at computation time; VALID: readable; ABSENT: X; ORDER: none; IDENTITY: (F8.1). (F8.1) `waivers[].wid` — SOURCE: the waiver register; VALID: unique in the register; ABSENT: X; ORDER: none; IDENTITY: named by a `waived` record's evidence reference (F6.9). (F8.2) `waivers[].authorised` — SOURCE: §11.4.271 evaluation of the waiver (rostered non-producer authoriser, tracked item); VALID: true or false; ABSENT: not authorised; ORDER: none; IDENTITY: belongs to its waiver. (F8.3) `waivers[].expiry` — SOURCE: the waiver; VALID: a time; ABSENT: the waiver is not valid (§11.4.271 requires an expiry); ORDER: compared only with (F10); never orders records; IDENTITY: belongs to its waiver. (F9) `repairs` — SOURCE: the §11.4.268 stream-integrity repair records; VALID: readable;

an empty list means no repair; ABSENT: X; ORDER: none; IDENTITY: none. (F9.1) `repairs[].rids` — SOURCE: the repair record; VALID: stream positions (F6.1); ABSENT: the repair quarantines nothing; ORDER: none; IDENTITY: names records by (F6.1). (F9.2) `repairs[].decision` — SOURCE: the repair record: a reference to its recorded §11.4.66 operator decision; VALID: present; ABSENT: the repair quarantines nothing; ORDER: none; IDENTITY: none. (F10) `now` — SOURCE: the computing seam's clock at computation time; VALID: a time; ABSENT: X; ORDER: compared only with (F8.3); never orders records; IDENTITY: none. DERIVED VALUES — rules over the fields above, never inputs of their own. (V1) RECORD SCOPE — the IN-SCOPE records are the records not quarantined by a repair (V2) whose fingerprint (F6.5) equals (F1); a record carrying any other fingerprint is an input to nothing except the sequence check (V2). (V2) SEQUENCE CHECK AND REPAIR — a SEQUENCE DEFECT is a record that is not quarantined and has no sequence number or no stream position, shares its sequence number or its stream position with another record that is not quarantined, or has a missing, empty or unreadable fingerprint; it is X for every fingerprint carried by a record involved in it (for a missing, empty or unreadable fingerprint, for every fingerprint) and for no other fingerprint, so a malformed record of one fingerprint never holds another. RECOVERY: a §11.4.268 stream-integrity repair record carrying a reference to its recorded §11.4.66 operator decision QUARANTINES the records it names (F9.1); a quarantined record is an input to nothing, and a repair with no decision reference quarantines nothing. Because a quarantine removes a record from every rule, it can raise a status; that is why it needs the recorded decision, and every quarantine is listed in every announcement. (V3) JOIN — a row is MATCHED for the fingerprint when its store is fingerprint-keyed and its scope (F5.3.3) equals (F1), or its store is scope-keyed or not-fingerprint-bound and its scope either equals an item id in (F2) or, normalised (V3a), equals, is an ancestor of, or is a descendant of a normalised path in (F3), so a finding recorded against a directory joins every contained path below it and a finding recorded against a file joins the contained directory that holds it. (V3a) PATH NORMALISATION — one textual rule for every code path, in (F3) and in a row's scope alike, and never a file-system lookup, so a symbolic link is never resolved: split the path on `/` ; drop every empty segment and every `.` segment; a `..` segment removes the segment before it; the normalised path is the remaining segments joined by `/` , and a path with no segment left (`.`

or `./`) is the repository root, an ancestor of every path. A path that is empty, starts with `/` , or has a `..` segment with no segment before it does not normalise and is UNRESOLVABLE. Comparison is exact and case-sensitive, segment by segment: `src/a` is an ancestor of `src/a/b` but not of `src/ab` and not of `SRC/a/b` , and `\` is an ordinary character. A row in a scope-keyed or not-fingerprint-bound store whose scope is not an item id in (F2) and does not normalise is X, never silently left out of the join; a contained path that does not normalise is X. (V4) ENTRY IDENTITY — an entry is identified by its row key and has exactly ONE matched row, which carries its class: for an entry defined by the ABSENCE of a fact ((L1), (L2), (L6), (L7)) and for a polish item ((L5)) that row is its tracked §11.4.197 item row (§11.4.54) in the consumer's tracked-item store, created when the join first matches the entry and never removed; for (L3) and (L4) it is the finding's row in its findings store; for (L8) it is the waiver's row in its waiver store. A key matched in more than one row — in one store or in several, with equal or different classes — is X, so row keys MUST be unique across every declared owning store (the tracked §11.4.197 item id is such a key). (V5) ENTRY SET — the closed union of every matched row's key and every entry id named by any in-scope `disposition` record, valid or not; removing a row from an owning store is never a resolution — only a valid record is — and because every entry is named by its birth record, a deleted row never removes its entry. (V6) CLASS — read from the entry's one matched row (V4), never from a label on a record; an entry named by a record but with no matched row has no class, so no record for it is valid and it is `open` . (V7) LATEST RECORD of an entry — its in-scope `disposition` record with the highest sequence number. (V8) WAIVER VALIDITY — a waiver (F8) is valid when it is authorised under §11.4.271 (F8.2) and the computation time (F10) is before its expiry (F8.3). (V9) VALID-UNTIL — the earliest expiry (F8.3) among the waivers named by the latest valid `waived` records the status counts as resolving an entry; none when it counts none. PERMITTED REASON CODES PER DISPOSITION — a closed table; a code outside its disposition's row is INVALID, and free text never sits in the code, only in the evidence reference: `open` — `open-debt` (also for a reopen); `closed` — `closing-evidence` , with an evidence reference naming the entry's own closing evidence for this fingerprint; `skipped` — `hardware_not_present` | `topology_unsupported` ; `not-applicable` — `feature_disabled_by_config` ; `waived` — `waiver` , with an evidence

reference naming the §11.4.271 waiver. A `disposition` record is VALID only when it is in scope (V1), its entry id matches exactly one matched row (V4) and so has a class (V6), any class label it carries equals that class, its writer role is allowed for its disposition, its disposition is permitted for that class, its reason code is permitted for its disposition, it carries the evidence reference its code requires, and, for `waived`, the waiver its evidence reference names is valid (V8). A `qa-confirmation` record is VALID exactly when it is in scope, its writer role is `qa-confirmer`, and its sequence number is greater than L (below). A `release-seam-pass` record is VALID exactly when it is in scope, its writer role is `release-seam`, and its sequence number is greater than that of the latest in-scope `qa-confirmation` record. A record of any other kind is never valid. A record that is not valid is never silently dropped: it is shown, it is a §11.4.261 finding, and a `disposition` record that is not valid makes its entry `open` when it is the entry's latest record (V7; below); an earlier invalid record decides nothing. (a) THE STATUS takes exactly one value from the CLOSED set { `delivered-unhardened` | `delivered-hardened-awaiting-qa` | `production-ready` }. It is a PURE FUNCTION of the FIELD TABLE's fields and of nothing else: never typed or chosen by hand (§11.4.6, §11.4.262), and never dependent on the order in which records are stored, nor on wall-clock time except through §11.4.271 waiver expiry, which can only lower it. It carries its VALID-UNTIL (V9) and a `pending` field naming what still holds it below `production-ready`, drawn from the closed set { `ledger`, `manual-qa`, `release-seam`, `not-fingerprint-bound`, `unresolvable-input` }. Its inputs are derived from the records as follows. X = at least one input cannot be resolved: a field whose ABSENT column says X is missing, unreadable or outside its set — in particular a missing delivered fingerprint, contained-item set or contained-path set, a contained path that does not normalise (V3a), no store declaration or two declarations of one store, a declared store with no read or with an unreadable read (F5.2) whether it is required or not, a read of an undeclared store, or an unreadable event stream, write-seam log, waiver register or repair list; an owning store that the consumer declares REQUIRED holds no matched row for this fingerprint (a store that is not required, such as a waiver store that may legitimately be empty, contributes zero entries when it is readable and empty, and its readability is shown with a control needle per §11.4.273); a SEQUENCE DEFECT involving this fingerprint (V2); a key matched in more than one row (V4); a row in a scope-keyed or not-fingerprint-

bound store whose scope is not an item id in (F2) and does not normalise (V3a); a write-log entry for this fingerprint whose mirror (F7.4) is not an in-scope `disposition` record naming that entry (with disposition `open` for a birth); or a matched row with no `birth` write-log entry for this fingerprint. The DISPOSITION of an entry is the disposition of its LATEST `disposition` record (V7) when that record is valid, and `open` when it is not valid or when the entry has no record. L = the highest sequence number of any in-scope `disposition` record (V1), valid or not, or 0 when there is none. O = the number of entries in the ENTRY SET (V5) whose disposition is `open`. Q = the LATEST in-scope `qa-confirmation` record (V1) is valid, so its sequence number is greater than L. Hence ANY ledger record written after a confirmation — a reopen, and equally the later re-close of that reopen — VOIDS the confirmation, and a new §11.4.185 confirmation is required after the ledger's final record. S = Q holds and a valid `release-seam-pass` record exists, so its sequence number is greater than that of the latest `qa-confirmation` record. Hence voiding the confirmation voids every seam record that relied on it. The release seam MUST refuse to write the record unless O = 0, Q holds and B does not. B = at least one owning store is labelled not-fingerprint-bound (a consumer declaration, below). TRANSITION TABLE, evaluated top to bottom, first matching row wins, so it is total and deterministic over every combination of X, O, Q, S, B: row 1 — X → `delivered-unhardened`, pending `unresolvable-input` (§11.4.201(4) conservative-safe default, never upward); row 2 — O > 0 → `delivered-unhardened`, pending `ledger`; row 3 — O = 0 and not Q → `delivered-hardened-awaiting-qa`, pending `manual-qa`; row 4 — O = 0, Q, and (not S or B) → `delivered-hardened-awaiting-qa`, with pending containing `release-seam` exactly when S does not hold and `not-fingerprint-bound` exactly when B holds; row 5 — O = 0, Q, S and not B → `production-ready`, pending empty. Hence `delivered-hardened-awaiting-qa` means exactly that every automated and hardening entry is resolved by a valid record and only human or release acts remain, and `production-ready` is reachable only after a §11.4.185 confirmation that postdates the ledger's final record and a release-seam record that postdates that confirmation, both for the same fingerprint. A later record that reopens an entry, or changes any disposition at all, recomputes the status downward by the same table, and the status stays below `production-ready` until a new confirmation and a new seam record follow; no status is ever sticky.

FRESHNESS: every owning store is one of (i) FINGERPRINT-KEYED — its facts are keyed by the candidate fingerprint (verdict stores, live-verdict records); (ii) SCOPE-KEYED — its facts are keyed by tracked item id or code path and carry a per-record write time (the workable-items database, the §11.4.261 findings ledger), joined through the contained-item set and contained code paths that the hand-off record (D.5) names; or (iii) NOT-FINGERPRINT-BOUND — it can supply neither, and it MUST carry that label explicitly in the consumer's store declaration (§11.4.35). A status is FRESH only while its ledger snapshot includes — by sequence number, never by wall-clock time — the latest event record for every matching record of every owning store (for a fingerprint-keyed store, every record for this fingerprint; for a scope-keyed store, every record for the contained items and code paths). It is recomputed at every announcement, at every write to an owning store (§11.4.106(F) write-seam discipline, §11.4.229), and at its VALID-UNTIL (V9), because a waiver that expires is not an event record: a status is FRESH only while the time is before its VALID-UNTIL, so an expired waiver lowers it on time instead of leaving the older value announced. A status announced from an older snapshot, a different fingerprint, or at or after its VALID-UNTIL is STALE and is itself a §11.4.261 finding. A not-fingerprint-bound store can never be proven fresh for a fingerprint, so it holds the status below `production-ready` (row 4) until the store is bound; its entries are still joined and shown. ANNOUNCEMENT: the status, its `pending` field and the per-class, per-disposition ledger counts MUST be shown wherever the deliverable is announced — the hand-off record, the release or deploy notes, the §11.4.97 progress report, and the §11.4.259 README badge row — and every announced status carries the fingerprint, the ledger-snapshot time it was computed from, its VALID-UNTIL, and every quarantine (V2) it relied on. While the newest deliverable of any open scope is `delivered-unhardened`, or its status is stale or unknown, the §11.4.259 production-readiness gauge MUST NOT read GREEN. "Quick" is never presented as "production ready" (§11.4.6, §11.4.259).

(E.4) TRACKED CONVERGENCE TO PRODUCTION READINESS — DIRECTION, STALL, STARVATION AND THE LOOP. For every open scope each cadence build appends a convergence record to the existing append-only event stream and snapshot (§11.4.116 — reused, not reinvented) carrying the fingerprint, the computed status and `pending` field, the ledger counts per

class and disposition from (E.3)(b), and the delta against the previous deliverable of the same scope; the trajectory is reported in every §11.4.97 progress report. DIRECTION RULE — a scope's `open` count MUST NOT grow between consecutive deliverables except by entries that NEW increments brought in; growth with no new increment means hardening is going backwards and is surfaced as a §11.4.261 finding. STALL RULE — the `open` count failing to decrease across N consecutive deliverables that brought in no new increment is a STALL under §11.4.267(D), escalated, re-tiered under §11.4.231, or routed to a recorded §11.4.66 decision, never left as a silent background stream; N is consumer DATA (§11.4.35, §11.4.267(E)) and never a constitutional literal (§11.4.267(E) admits only an operator-mandated ceiling recorded in its own anchor), this N is distinct from every other N the constitution leaves to consumers (for example the §11.4.235(E.2) consecutive-DEFERRED bound), and a scope bound by this clause with no declared N is a CONFIG ERROR reported as such, never a silently unbounded stall window. STARVATION RULE — a scope with O > 0 MUST have at least one live hardening stream dispatched against it (§11.4.232(A) registry row, §11.4.147 agent registry) whenever host headroom exists under clause (C)'s bounds; O > 0 with zero live hardening streams while headroom exists is STARVATION and a §11.4.94 / §11.4.97 zero-idle violation, never an acceptable "the deliverable is out, hardening can wait" state. Background hardening keeps running under clauses (C)/(D.4), §11.4.126, §11.4.183 and §11.4.192 until the scope reaches `production-ready`. LOOP DONE-CONDITION: the §11.4.126 done-condition is NOT satisfied while any delivered scope of the current release has O > 0 (unless every open entry is itself blocked on a recorded external dependency under §11.4.101 / §11.4.21). A scope at `delivered-hardened-awaiting-qa` — O = 0, so only the §11.4.185 human confirmation or the release-seam record remains — IS a recorded external block for that scope's done-condition, because those acts are the operator's and the release seam's by design (§11.4.185, §11.4.40); for a non-release scope (§11.4.126(B)), which has no release seam, reaching that status with `pending` naming only `manual-qa` or `release-seam` satisfies the scope's done-condition. An external block NEVER idles the loop: the loop keeps progressing every other actionable item, the pending human act is surfaced in every §11.4.97 report until it is recorded, and the status recomputes the moment it is (§11.4.94, §11.4.97, §11.4.101).

(E.5) PARALLELISM IS MAXIMAL AND BOUNDED. The critical path of (E.1) and the hardening streams of (E.4) run concurrently by default under clause (C) and §11.4.183, inside clause (C)'s own honest boundary, which is cited here, unchanged and not restated. The one rule this clause adds: where the critical path and a hardening stream contend for one exclusive resource (§11.4.119 / §11.4.176), the critical path holds it until the first deliverable is handed off, and contention is never resolved by skipping a seam of (E.2).

(E.6) WHAT "QUICK" MEANS — MEASURED, NEVER GATING. "Quick" is a MEASUREMENT, never a target that weakens evidence. Each scope records (a) its wall-clock TIME-TO-FIRST-DELIVERABLE, from scope start to the first hand-off that passes §11.4.236 and carries its (E.3) status, and (b) per increment, its GO-TO-HAND-OFF latency, from its source-correctness GO to the first hand-off that contains it, both as §11.4.24-style statistics. They are TREND metrics only. Under §11.4.201(8) they are validated against the definition of done for DIRECTION — a shorter time that still passes (E.2) is better, and a time achieved by failing (E.2) does not count at all — but they are NEVER used to gate, rank, or reward, because a gating speed metric invites exactly the gaming (E.1) forbids. They are reported next to the `open` count and the convergence trajectory of (E.4), never instead of them.

(E.7) FAILURE MODES THIS CLAUSE NAMES, EACH WITH ITS DETECTOR. (1) STALE STATUS — a status announced from an outdated ledger snapshot or a different fingerprint, or at or after its VALID-UNTIL, or a not-fingerprint-bound store left unlabelled (detector: `CM-DELIVERY-STATUS-FRESH`); (2) STATUS INFLATION — a status hand-set, outside the closed set, different from the transition-table value, computed over an entry set that omits an entry named by a `disposition` record or while a declared owning store is unreadable, computed from a value that is not a FIELD TABLE field, by an implementation whose input record does not project to the FIELD TABLE byte for byte, or by breaking a tie the table leaves to X (for example an entry id with rows in two stores resolved by store declaration order), `production-ready` computed with no release-seam record for the fingerprint, a confirmation or seam record counted although a later ledger record voids it, or a GREEN gauge over an unhardened deliverable (detector: `CM-DELIVERY-HARDENING-STATUS-COMPUTED`); (3) UNTRACKED OR HIDDEN DEBT — a ledger kept as a hand-maintained store, an entry with no tracked id or no disposition, a

disposition outside its class's permitted row, a disposition validated against a class label on the record instead of the entry's owning-store class, a reason code outside its disposition's permitted row, a non-`open` disposition written by any role other than the verifier seam, a `skipped` entry whose reason is not one of the two topology-absent codes, a resolved-but-disclosed entry missing from an announcement, or a contained item with no live verdict that is absent from the ledger (detector: `CM-HARDENING-LEDGER-TRACKED`); (4) HARDENING STARVED OR NEVER CONVERGING — a direction-rule growth, an un-escalated stall, an undeclared N, a starved scope, or a loop reported done while a scope of the current release has O > 0 without a recorded external block (detector: `CM-HARDENING-CONVERGENCE-MONOTONE`); (5) DELIVERED WITHOUT EVIDENCE OR WITHOUT LABELS — a quick hand-off missing any (E.2) seam, a first deliverable with no source-GO'd increment or no user-visible behaviour, or a hand-off with no status and ledger attached (detector: `CM-QUICK-DELIVERY-EVIDENCE-UNWEAKENED`). Each detector is itself a guard under §11.4.201: it ships golden-TRUE and golden-FALSE-with-carrier fixtures, takes the conservative-safe default on an unresolvable signal, and prints its resolved evidence on every refusal.

(E.8) HONEST BOUNDARY (§11.4.6). Clause (E) re-orders WHEN deliverable-independent hardening COMPLETES relative to a scope's first deliverable, and makes that deliverable's remaining debt visible, tracked and converging. It weakens none of the following: §11.4.5 / §11.4.262 / §11.4.50 deterministic machine evidence; §11.4.108 four-layer evidence; §11.4.209 / §11.4.134 / §11.4.276 review; §11.4.236 readiness; §11.4.235(E) live verification and its device eligibility and lease rules; §11.4.185 manual QA; §11.4.40 full-suite retest before a TAG (a quick deliverable is a QA build, not a tag); §11.4.129 huge-blocker full restart; §11.4.239 critical-invariant failure-path scenarios; or a consuming project's own stricter delivery precondition. A `skipped`, `not-applicable` or `waived` disposition is an honest, disclosed gap, never a PASS (§11.4.3, §11.4.271). It does not prove any deliverable correct — that stays §11.4.108 / §11.4.235 — and the four-to-five-times wall-clock target of clause (A) stays a measured target, never an assumed one. A status of `delivered-unhardened` is an honest statement of debt, never permission to stop paying it.

(E) gates. Classification: universal (§11.4.17) — consumers supply the scope definition, the N stall bound, the owning-store declarations (each store's keying class from (E.3), including any explicit not-fingerprint-bound label), the polish-item marker, and the badge and report locations as DATA per §11.4.35. Composes §11.4.3 / §11.4.4(b) / §11.4.5 / §11.4.6 / §11.4.24 / §11.4.27 / §11.4.40 / §11.4.50 / §11.4.54 / §11.4.66 / §11.4.69 / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.106(F) / §11.4.108 / §11.4.116 / §11.4.118 / §11.4.119 / §11.4.126 / §11.4.129 / §11.4.134 / §11.4.135 / §11.4.147 / §11.4.169 / §11.4.176 / §11.4.183 / §11.4.185 / §11.4.192 / §11.4.197 / §11.4.201 / §11.4.209 / §11.4.227 / §11.4.229 / §11.4.231 / §11.4.232 / §11.4.233 / §11.4.235(A)(D)(E) / §11.4.236 / §11.4.239 / §11.4.259 / §11.4.260 / §11.4.261 / §11.4.262 / §11.4.267 / §11.4.271 / §11.4.273 / §11.4.276 / §1.1. Recommended gates, each with its FAIL conditions and its §11.4.201(1) golden-FALSE: `CM-DELIVERY-HARDENING-STATUS-COMPUTED` (FAIL: a QA hand-off record with no status or no `pending` field; a status or `pending` value outside its closed set; a status differing from the transition-table value recomputed from the (E.3)(b) join; `production-ready` with no release-seam record for the fingerprint, with O > 0, without the §11.4.185 confirmation, or with a not-fingerprint-bound input; a release-seam record written while O > 0 or before the §11.4.185 confirmation; a confirmation counted whose sequence number is not greater than that of every ledger record for the fingerprint, or a seam record counted whose sequence number is not greater than that of the latest confirmation (the reopen-then-re-close replay); a `pending` value naming an act whose valid record already exists; a status that changes when records are re-stored in another order or read at another wall-clock time; an entry set that omits an entry named by any `disposition` record for the fingerprint (for example after its store row was deleted); a status computed while any declared owning store, required or not, is unreadable; a record counted for a fingerprint it does not itself carry; an entry id that matches more than one row, or rows of different classes, resolved to one class (by store declaration order, by the record, or otherwise) instead of X; a status that changes when the owning stores are declared in another order; a status that reads a value that is not a FIELD TABLE field, or an implementation whose input record does not project to the FIELD TABLE byte for byte; a matched row with no birth record for the fingerprint, or a write-log entry whose mirror is missing, resolved as anything but X; a sequence defect of another fingerprint holding this one; a quarantine

without a recorded §11.4.66 decision; an unresolvable input resolved upward; the §11.4.259 gauge GREEN while the newest deliverable of an open scope is `delivered-unhardened`, stale or unknown; golden-FALSE: a `delivered-hardened-awaiting-qa` hand-off with O = 0, all-resolved entries including disclosed `skipped` and `not-applicable` ones, no §11.4.185 confirmation, pending `manual-qa` and an AMBER gauge fires nothing) + `CM-DELIVERY-STATUS-FRESH` (FAIL: an announced status whose ledger-snapshot time precedes the latest write to any matching record of any owning store; an announced status at or after its VALID-UNTIL, or one counting a `waived` entry with no VALID-UNTIL; a fingerprint differing from the deliverable announced; an announcement carrying no fingerprint or no snapshot time; an owning store with no declared keying class; a scope-keyed write not mirrored to every already-delivered fingerprint whose hand-off record names a matching contained item or code path; golden-FALSE: a status recomputed after the last matching write, with one explicitly labelled not-fingerprint-bound store holding it at `delivered-hardened-awaiting-qa`, fires nothing) + `CM-HARDENING-LEDGER-TRACKED` (FAIL: a ledger entry with no tracked §11.4.197 item id or no disposition; an absence-defined ((L1), (L2), (L6), (L7)) or polish ((L5)) entry with no tracked-item row, or a tracked-item row removed while its entry is in the entry set; an invalid `disposition` record that is not its entry's latest record treated as opening the entry; a disposition outside its closed set or outside its class's permitted-disposition row (for example an (L7) entry `skipped`); a disposition validated against a class label carried by the record instead of the entry's owning-store class; a reason code outside its disposition's permitted row, or a `closed` or `waived` record with no evidence reference; a non-`open` disposition record whose writer role is not `verifier-seam`; a `skipped` entry whose reason is not `hardware_not_present` or `topology_unsupported`; a `waived` entry whose waiver is elapsed or invalid under §11.4.271; a `skipped`, `not-applicable` or `waived` entry absent from an announcement's counts; a ledger implemented as a hand-maintained store instead of a join over the owning stores; a contained item with no passing live verdict that is absent from the ledger; golden-FALSE: a join-computed ledger whose every entry carries a tracked id, a valid disposition and appears in the announcement fires nothing) + `CM-HARDENING-CONVERGENCE-MONOTONE` (FAIL: a scope's `open` count grows between consecutive deliverables beyond the entries new increments brought in; N consecutive no-new-increment

deliverables without a decrease and with no §11.4.267(D) escalation record; a bound scope with no declared N; O > 0 with zero live hardening streams while host headroom exists; the §11.4.126 loop reported done while a delivered scope of the current release has O > 0 with no recorded external block; golden-FALSE: a scope whose `open` count shrinks each no-new-increment deliverable with one live hardening stream, and a scope at `delivered-hardened-awaiting-qa` pending only `manual-qa` treated as externally blocked while the loop continues other work, fire nothing) + `CM-QUICK-DELIVERY-EVIDENCE-UNWEAKENED` (FAIL: a hand-off lacking the §11.4.236 candidate-fingerprinted verdict; a source GO not recorded on the §11.4.209 substrate or its sanctioned recorded fallback; a §11.4.235(E) per-item verdict replaced by narrative; a first deliverable with no increment holding its own source GO or exercising no user-visible behaviour; a hand-off with no status or ledger attached; golden-FALSE: a first deliverable carrying every required verdict and an honestly non-empty ledger fires nothing) + `CM-CANONICAL-MIRROR-BODY-LOCKSTEP` (scope: every anchor whose mirrors carry the FULL block rather than a compact or pointer form, as §11.4.230 does; FAIL: the canonical `Constitution.md` block body, its heading marker normalised, differs from any full-block mirror body; named golden-BAD fixtures: (i) a canonical-only edit followed by regeneration and (ii) the same edit applied consistently to every full-block mirror with `Constitution.md` untouched — both pass every existing gate today; golden-FALSE: a compact or pointer mirror, which §11.4.227(B) treats as a legitimate cross-layer variance, fires nothing) + paired §1.1 mutations (hand-set a status to `production-ready`; compute `production-ready` without a release-seam record; replay an old ledger snapshot after an owning-store write; drop one entry from the ledger join, or drop a `skipped` entry from an announcement; inject `open` growth with no new increment, or remove every live hardening stream from a scope with O > 0; strip the §11.4.236 verdict from a quick hand-off; replay a reopen and a re-close after the release-seam record with no new confirmation and seam record; mark an (L7) entry `skipped`, or let a `producer`-role record close an entry; delete the store row of an entry whose latest record is `open`, or make a declared non-required store unreadable; label a `waived` record with a class that permits `waived` while the entry's owning-store class does not; give one entry id rows of different classes in two stores and resolve it by either declaration order; drop the tracked-item row of an (L1) entry that holds a valid

`skipped` record; delete the row of an (L6) entry whose birth record is missing from the stream; make the status read a value that is not a field of its input record; let a malformed record of another fingerprint hold this one; count a waiver after its expiry; edit the §11.4.230 body in `Constitution.md` only and regenerate; edit it in every full-block mirror with `Constitution.md` untouched — each MUST make its gate FAIL). Gate code is a separate work item, NOT claimed shipped (§11.4.6 / §11.4.227); the six gates are registered as tracked deferrals `OWED-GATE-115` .. `OWED-GATE-120` in `constitution/scripts/gates/gate_ledger_deferrals.tsv`. No escape hatch — no `--quick-skips-review`, `--quick-skips-readiness-verdict`, `--hand-set-delivery-status`, `--unlabelled-quick-build`, `--stale-status-OK`, `--quick-is-production-ready`, `--untracked-hardening-debt`, `--hidden-skip-OK`, `--hardening-can-wait`, `--speed-metric-gates`, `--stop-hardening-after-delivery`, `--qa-wait-idles-loop` flag.

(Process note for future extenders: §11.4.230's next free clause letter is (F); never reuse (A)–(E).)

Classification: universal (§11.4.17) — a platform-neutral parallelization discipline reusable by ANY project that builds an artifact, validates it across multiple targets, and records device evidence; the consuming project supplies its concrete device set + serials, build/flash tooling, recording layout, and worker-pool implementation per §11.4.35. Composes §11.4.20 / §11.4.24 (wall-clock stats prove the 4–5× target) / §11.4.40 / §11.4.58 (the PWU pipeline this binds + extends) / §11.4.69 / §11.4.70 / §11.4.88 / §11.4.89 / §11.4.94 / §11.4.96 (SAFE-during-build catalogue) / §11.4.97 / §11.4.102 / §11.4.103 / §11.4.108 (artifact/runtime layer ordering — the honest boundary) / §11.4.110 / §11.4.115 / §11.4.119 (single-resource-owner — B's load-bearing partition) / §11.4.120 (seam-placement — A's honest boundary) / §11.4.121 / §11.4.125 / §11.4.126 / §11.4.128 (always-on recording — B's evidence source) / §11.4.129 (huge-blocker full-restart — the un-weakened carve-out) / §11.4.130 / §11.4.132 / §11.4.144 (availability-following — B's drop-handling) / §11.4.183 (maximum-USEFUL-parallelism — C's honest boundary) / §11.4.185 (manual-QA-final — the un-weakened terminal gate) / §11.4.187 / §11.4.192 / §11.4.200 (per-device deployed-artifact verification) / §12.6 / §12.12. Propagation gate `CM-COVENANT-114-230-PROPAGATION` (literal `11.4.230` across the consumer fleet) + recommended gates `CM-PIPELINE-OVERLAP-VALIDATION` (a build that

runs with zero deploy-independent validation stages running concurrently when SAFE-catalogue work is queued → FAIL; a NON-huge-blocker discovery that restarts the whole cycle from scratch instead of re-running only the affected stages → FAIL) + `CM-PARALLEL-DEVICE-WORKER-POOL` (available independent devices validated serially instead of concurrently, OR recording capture/analysis serialised onto one processor when a worker-pool path exists → FAIL) + `CM-FAN-OUT-SUBAGENTS-DEFAULT` (a genuinely-parallelizable effort run monolithically in the foreground when a fan-out path exists AND all host-safety ceilings have headroom → FAIL) + paired §1.1 meta-test mutations (per §1.1; gate-code = separate work item, NOT claimed shipped §11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.230.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--sequential-build-then-validate`, `--restart-whole-cycle-on-discovery`, `--serial-device-testing`, `--serialise-recording-processing`, `--no-fan-out`, `--monolithic-parallelizable-work` flag exists.

§11.4.231 — Nano-precision model-tier selection: lightest-capable-model-first with dynamic mid-task escalation (User mandate, 2026-07-26, `CRITICAL`).

Verbatim operator mandate: every work effort MUST use the LIGHTEST CAPABLE model, escalating tier only as complexity rises — a nano-precision, per-task model-tier choice, NOT "everything on the heaviest model." Forensic motivation (genericised, §11.4.6): dispatching every task — a trivial doc-sync, a one-line status read, a mechanical format fix — on the heaviest available model wastes capacity + token budget + wall-clock that a lighter tier would spend identically well, while the OPPOSITE failure (a hard multi-factor root-cause investigation dispatched on a tier too light to reason through it) produces exactly the shallow, guessed, or wrong-conclusion work §11.4.6 / §11.4.102 / §11.4.194 forbid. Neither "always heaviest" nor "always lightest" is compliant; the correct discipline is a PER-TASK, EVIDENCE-BASED tier choice that tracks the task's own complexity, re-evaluated as that complexity becomes known — the nano-precision selection this anchor mandates.

(A) LIGHTEST-CAPABLE-FIRST + ESCALATE-ON-COMPLEXITY. Every dispatched work effort — main-stream or subagent (§11.4.20/§11.4.70), single-

track or multitrack (§11.4.187/§11.4.192) — is assigned the LIGHTEST model tier from the closed, capability-ordered ladder `sonnet < opus` genuinely sufficient to complete it correctly, with **`sonnet` the DEFAULT working tier** and `opus` for every review and other heavy work (operator mandate 2026-09-26, `CRITICAL`, verbatim: "Main model to use is Sonet, reviews and other heavy stuff have to be done by Opus instead of Fable!!! Fix this permanently on all levels!"). The review/merge-conflict-resolution substrate is a SEPARATE, already-pinned Opus-xhigh tier per clause (E), not an extra rung of this ladder; `fable` is not a rung and is not a permitted dispatch tier; `haiku` is excluded per clause (D.1). Tier selection is a DECIDABLE heuristic, not a guess (§11.4.6) — evaluated against, at minimum: (i) **task scope** (single mechanical edit / focused single-file change / multi-file-or-multi-component change); (ii) **reasoning depth** (a lookup-and-apply action vs. a multi-factor root-cause investigation per §11.4.102/§11.4.194); (iii) **blast radius** (a change touching one isolated file vs. one crossing shared state, hardware safety §11.4.133, or governance/gate seams §11.4.227); (iv) **novelty** (a previously-solved, pattern-matched task vs. a genuinely new problem with no established precedent). Closed-set tier bindings (illustrative, non-exhaustive — the consuming project supplies its own precise mapping per §11.4.35): **`sonnet`** (the default) — trivial/mechanical AND moderate work (simple greps/searches, doc-sync regeneration, status/field reads, format/lint fixes, boilerplate scaffolding, a focused single-component debug, a bounded single-file implementation, straightforward research or audit reads); **`opus`** — complex and other heavy work, including every review (architecture/design decisions, multi-factor reasoning per §11.4.194(1), cross-feature blast-radius analysis per §11.4.92 Pass 2/3, hard root-cause investigation per §11.4.102 that resists a first-pass hypothesis). A task whose complexity is genuinely UNDECIDABLE from available evidence starts at the NEXT tier UP from the best available estimate (the safe-reversible default per §11.4.101) rather than guessed downward.

(B) DYNAMIC MID-FLIGHT SWITCHING. Tier assignment is NOT a one-time, start-of-task decision — it is RE-ASSESSED continuously as the task's real complexity becomes known during execution. A task DISPATCHED as simple that PROVES harder mid-flight (an unexpected multi-factor interaction surfaces, a root cause resists the first two hypotheses, the blast radius turns out to cross a governance/hardware-safety seam) MUST ESCALATE to the

next-heavier tier for its remainder — continuing on an under-powered tier past the point complexity is KNOWN is a clause-(D) false economy, not compliance. Conversely, a task DISPATCHED as heavy that DECOMPOSES into genuinely light, bounded, low-blast-radius subtasks (mechanical file edits following an already-designed pattern, doc regeneration, per-item status reads) dispatches THOSE subtasks on a LIGHTER tier via subagent fan-out (§11.4.20/§11.4.70/§11.4.183) rather than running every subtask on the parent's heavier tier by default — capability-matching applies at EVERY dispatch granularity, not only the top-level task.

(C) ALWAYS-ON COMPOSITION, NEVER A PER-REQUEST OPT-IN. Nano-precision tier selection is a STANDING DEFAULT mechanism, engaged automatically for EVERY work effort from the first prompt of a session onward (mirrors §11.4.126 default-autonomous-loop / §11.4.198 default-mechanisms-always-on) — applied TOGETHER WITH, never as a substitute for: §11.4.141 (heavy token-optimization — a lighter model dispatched with a bloated context is not the efficiency this anchor targets); §11.4.183 (maximum-USEFUL-parallelism — tier-matching each parallel stream, not merely running more streams on one fixed tier); §11.4.187/§11.4.192 (the multi-track ruler orchestration + continuous auto-backfill — each track's worker is tier-assigned per this anchor, not defaulted to one blanket tier project-wide); and §11.4.196/§11.4.198 (native-alias-first priority + always-on default mechanisms — this anchor governs WHICH TIER within whichever alias/account is selected, composing with, never replacing, §11.4.196's ALIAS-layer priority). None of these five composed anchors is weakened, duplicated, or re-specified by §11.4.231 — this anchor adds the TIER axis to their existing ALIAS/PARALLELISM/TOKEN axes.

(D) HONEST BOUNDARY (§11.4.6) — "LIGHTEST CAPABLE" MEANS CAPABLE, NOT CHEAPEST-REGARDLESS. "Lightest capable" is a CONJUNCTION, not a discount: a tier choice that fails to actually complete the task correctly — producing a shallow investigation, a guessed root cause, an under-verified fix, or ANY form of bluff per the §11.4 anti-bluff covenant — is NOT "lightest capable," it is UNDER-POWERED, and under-powering a task that then fails, bluffs, or requires costly rework is a FALSE ECONOMY and a §11.4 violation of equal severity to any other PASS-bluff (the token/time saved at dispatch is spent many times over in re-investigation, re-review, and the risk

of a shipped defect). Capability determinations are made from the clause-(A) decidable heuristic + observed task outcomes — EVIDENCE-BASED, never guessed (§11.4.6): "this always works fine on `sonnet`" without a captured basis is exactly the guess §11.4.6 forbids, and a tier's genuine capability for a task class is established by the same anti-bluff discipline (§11.4.5/§11.4.69/§11.4.107) that governs every other claim in this constitution. The ladder binds the DISPATCHER's tier ASSIGNMENT: a dispatcher MUST NOT assign any tier outside `sonnet < opus` — never `fable` (2026-09-26 mandate), and never `haiku` where clause (D.1) applies. A serving model the platform selects for a turn WITHOUT a dispatcher assignment is recorded honestly in the §11.4.182 label (whatever id the platform reports, never re-labelled); it is neither a dispatch-tier violation by the dispatcher nor a licence to assign that model, and review/merge-conflict scopes run on Opus-xhigh per clause (E) regardless.

(D.1) `haiku` PROHIBITED WHERE IT CANNOT RECEIVE ITS GOVERNING CONTEXT — measured fact (2026-09-26). In the reference consuming repository a `haiku` subagent dispatch failed before doing any work with "Prompt too long (~421k tokens vs 200k limit)" (request id `req_011CfSBXm7Kn6enyUT1UEaWd`, recorded in that repository's agent registry `docs/requests/agent_registry.jsonl`): the governance context every dispatched agent must receive there exceeds the `haiku` tier's context window, so a `haiku` dispatch cannot carry its binding context and `haiku` is PROHIBITED as a dispatch tier in that repository. A consuming project MAY re-admit `haiku` as a rung below `sonnet` ONLY on its OWN captured evidence (§11.4.6/§11.4.35) that a `haiku` dispatch in that project receives its full governing context and completes its task correctly — never by assumption, and never by trimming the governing context to make it fit. A `haiku` dispatch that fails this way is recorded honestly as a failed dispatch (§11.4.147), never silently retried on the same tier.

(E) HARD PINS OVERRIDE THIS DEFAULT FOR THEIR SCOPES. §11.4.231 sets the DEFAULT working tier for NON-review, non-merge-conflict-resolution work. It does NOT relax, override, or compete with the two existing hard model-substrate pins: §11.4.209 (every mandatory independent code-review — §11.4.125/§11.4.142/§11.4.194 iterated to GO per §11.4.134 — runs on **Opus at `xhigh` effort** as PRIMARY substrate, with a genuine-unavailability SONNET fallback, amended 2026-10-04 — no Fable, no Haiku, no other

fallback model; BLOCKED only when BOTH Opus and Sonnet are genuinely unavailable, NEVER a lower tier or any other model) and §11.4.211 (every main→feature/product/flavor merge-CONFLICT resolution runs on the SAME Opus-xhigh-primary-plus-Sonnet-fallback substrate, blocked only when both Opus and Sonnet are genuinely unavailable). Those two anchors are STRICTER, SCOPE-SPECIFIC pins that this anchor's general ladder does not reach — review and merge-conflict-resolution are NOT eligible for sonnet/opus-by-complexity dispatch; they run on their pinned substrate ALWAYS, regardless of how "simple" the reviewed change or the conflict appears. A review or merge-conflict resolution dispatched on anything other than the §11.4.209/§11.4.211 Opus-xhigh-primary-plus-sanctioned-Sonnet-fallback substrate (amended 2026-10-04) is a violation of THOSE anchors, not a valid application of §11.4.231's general ladder.

**(F) EFFORT-TIER SELECTION — the SAME nano-precision discipline applied to the reasoning-EFFORT axis (operator IMPORTANT mandate, 2026-07-26; verbatim: "Besides dynamic determining proper models to use, we MUST decide the effort as well and switch between the effort settings same as we do now for chosen model! Labeling MUST BE extended for this details as well. Example: (T1/main - claude4 - opus - high)."). ** Every dispatched work effort is assigned not only a model tier per clauses (A)-(E) but ALSO the LIGHTEST reasoning-EFFORT setting from the closed ladder `low < medium < high < xhigh < max` genuinely sufficient to complete the task correctly — chosen by the SAME clause-(A) decidable heuristic (scope / reasoning-depth / blast-radius / novelty), NOT a guess (§11.4.6). Illustrative bindings (non-exhaustive; the consuming project supplies its precise mapping per §11.4.35): `low` / `medium` — trivial / mechanical work (doc-sync regeneration, status / field reads, format / lint fixes, boilerplate); `high` — moderate-to-complex reasoning (focused single-component debug, multi-factor analysis per §11.4.102 / §11.4.194); `xhigh` / `max` — the HARDEST verification / merge-conflict / architecture work. (F.1) DYNAMIC MID-TASK ESCALATION — effort, like model tier per clause (B), is RE-ASSESSED continuously as the task's real complexity becomes known; a task dispatched at a lower effort that PROVES harder mid-flight MUST ESCALATE its effort for the remainder (continuing under-effort past the point complexity is KNOWN is the clause-(D) false economy), and a heavy task decomposing into light bounded subtasks dispatches THOSE at a lighter effort — effort-matching

applies at EVERY dispatch granularity, not only the top-level task. (F.2) HONEST HARNESS BOUNDARY (§11.4.6, load-bearing — do NOT overstate the mechanism) — effort-tiering is REAL where the execution path exposes an effort control and a DOCUMENTED CAPABILITY-GAP where it does not: the current Claude Code **Agent tool** (subagent dispatch) exposes a `model` parameter but **NO `effort` parameter**, so a subagent's effort is NOT presently settable via that path (the §11.4.182 `<effort>` label field degrades honestly to `?` there); the **Workflow tool's `agent()`** DOES take an `effort` argument, so effort IS settable there — the label reports whichever path is live, never fakes one. This is the SAME aspirational-vs-harness state the clause-(E) §11.4.209 / §11.4.211 Opus-`xhigh` pins already occupy: the pins mandate the effort, and whether a dispatch path can SET it is a harness capability reported honestly, never faked (§11.4.182). (F.3) HARD-PIN FLOOR PRESERVED — clause (E)'s hard pins are ALSO effort pins: every §11.4.209 mandatory independent code-review AND every §11.4.211 merge-conflict resolution runs at `xhigh` effort on Opus as PRIMARY substrate (or, on a captured genuine Opus-unavailability FACT, on the §11.4.209/§11.4.211 sanctioned Sonnet fallback, amended 2026-10-04 — Fable and Haiku remain forbidden in every role), and this general effort ladder NEVER tiers those two scopes below `xhigh` where the dispatch path can set it, regardless of how "simple" the reviewed change or the conflict appears — a review / merge-conflict resolution run below `xhigh` on a path that COULD set it, or run on a model other than Opus or the sanctioned Sonnet fallback, is a §11.4.209 / §11.4.211 violation, not a valid application of this ladder. (F.4) HONEST BOUNDARY (§11.4.6) — "lightest capable effort" is a CONJUNCTION, not a discount: an effort setting that yields a shallow / guessed / under-verified result is UNDER-POWERED, a FALSE ECONOMY, and a §11.4 violation of PASS-bluff severity, exactly as an under-powered MODEL tier is per clause (D); capability determinations are evidence-based, never guessed (§11.4.6).

Classification: universal (§11.4.17) — a platform-neutral model-capacity-matching discipline reusable by any project dispatching work across a tiered model roster; the consuming project supplies its own precise tier↔task-class mapping and its own model-name ladder per §11.4.35 (the `sonnet` / `opus` ladder with Sonnet as the default, the clause-(D.1) `haiku` prohibition, and the Opus-xhigh pins are this project's current concrete instantiation, not a universally-fixed vocabulary — a consumer with a differently-named tier roster

maps its own ladder onto the SAME lightest-capable-first + dynamic-escalation + always-on-composition + honest-boundary + hard-pin-override structure, re-admitting a lighter rung only on its own captured evidence per clause (D.1)). Composes §11.4.6 (no-guessing — capability determinations are evidence-based) / §11.4.20 / §11.4.70 (subagent-driven dispatch — the granularity at which tier-matching applies) / §11.4.92 (blast-radius analysis feeds the clause-(A) heuristic) / §11.4.101 (safe-reversible default on undecidable complexity — escalate up, never guess down) / §11.4.102 / §11.4.194 (reasoning-depth signals for the heuristic) / §11.4.126 / §11.4.141 / §11.4.183 / §11.4.187 / §11.4.192 / §11.4.196 / §11.4.198 (the always-on composition set, clause (C)) / §11.4.209 / §11.4.211 (the hard-pinned review/merge-conflict substrate this anchor does NOT touch, clause (E)). Propagation gate `CM-COVENANT-114-231-PROPAGATION` (literal `11.4.231` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the consumer fleet per §11.4.227(B)) + recommended gate `CM-MODEL-TIER-LIGHTEST-CAPABLE` (a dispatched effort whose logged/declared model tier exceeds the clause-(A) heuristic's indicated tier for a task later confirmed trivial-and-successful-at-the-lighter-tier is a WARN/candidate finding — never a hard FAIL on tier choice alone, since tier-adequacy is only provable in retrospect per clause (D); the gate instead HARD-FAILS the two decidable violations: (i) a review or merge-conflict-resolution effort NOT run on the §11.4.209/§11.4.211 pinned substrate, and (ii) a task whose complexity was PROVEN mid-flight [an escalation event was logged, or a downstream failure/bluff was traced back to it] that did NOT escalate tier before continuing, and (iii) a dispatch assigned `fable`, or assigned `haiku` where clause (D.1) applies) + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped — §11.4.6; mutation plan: (1) dispatch a code-review labeled as running on `sonnet` instead of Opus-xhigh → gate MUST FAIL; (2) inject a logged mid-task complexity-escalation event with NO corresponding tier-escalation record → gate MUST FAIL; (2b) record a dispatch assigned `fable`, or assigned `haiku` where clause (D.1) applies → gate MUST FAIL; (3) golden-FALSE fixture: a task that stays on its initially-assigned tier throughout, with no escalation event logged → gate MUST NOT fire, per §11.4.201(1) the false-positive guard).

Canonical authority: constitution submodule `Constitution.md` §11.4.231.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--always-heaviest-model`, `--always-lightest-model`, `--skip-complexity-reassessment`, `--fixed-tier-per-session`, `--relax-review-pin`, `--relax-merge-conflict-pin`, `--per-request-tier-opt-in` flag exists.

Amended 2026-10-23: the ladder governs generative work only. Typed decisions are resolved by the §11.4.284 pre-ladder decision stage before this ladder is consulted. Nothing in this amendment adds a rung, reopens the ladder, or permits a decision endpoint in a chat-model configuration.

§11.4.232 — Anti-mess long-op orchestration: every long-op is registered, single-owned-per-purpose, liveness-PROVEN, handed-off-on-stop, safely-reaped, consistency-checked, and escape-bounded — the single source of truth for what is running, never inferred (research-derived, 2026-07-26).

Forensic anchor (genericised): in one cycle a verification sweep produced five distinct mess states — (1) multiple concurrent same-purpose sweeps ran unintentionally (no single-owner-per-purpose claim); (2) a sweep HUNG on a consumer-blocked-on-an-unclosed-producer pipeline and was undetected because liveness was inferred from process-alive (a §11.4.201(6) FALSE-NULL: a wedged op and a progressing op returned the identical "process alive" signal); (3) stopping the owning subagent SILENTLY KILLED its child long-op because ownership was implicit (no handoff-on-stop); (4) a stale/non-growing log was read as live progress (the same false-null, reader-side); (5) an un-completable tooling gate blocked a release for hours (no escape to a terminal state). Per §11.4.102 Phase 4.5 these are ONE architectural state-management flaw, not five bugs: **there was no single source of truth for what long-op is running, who owns it, its state, and its PROVEN liveness** — the §11.4.196(F) configured≠in-use gap generalised to project long-ops, so every mess state was un-observable, and an un-observable bad state is a §11.4/§11.4.1 bluff at the orchestration layer.

The mandate — a **long-op orchestration layer** binding every long-op (any op expected to exceed the runtime background-execution threshold per §11.4.89 — verification sweep, meta-test sweep, build, flash/deploy, test/QA suite, dispatched subagent), ALL of which hold:

(A) SINGLE SOURCE OF TRUTH — the long-op lifecycle registry. Every long-op MUST register BEFORE it runs and reach a terminal state (`complete|failed|reaped|handoff|blocked-escape`) when it ends, in a durable (never-tmpfs, write-temp-then-rename, §11.4.116 append-only event stream + atomic snapshot) registry recording `{op_id, purpose_key, owner, pid|pgid, cwd, state, last_heartbeat, heartbeat_seq, progress_offset, log_path, verdict, evidence_path}` . Success is read from the op's terminal verdict/ `result` , NEVER its process exit code. A long-op running with no registry record — or a record with no live op — is a FALSE-READY drift state.

(B) SINGLE-OWNER-PER-PURPOSE. No two same- `purpose_key` long-ops run at once; acquisition is a probe-race-safe claim keyed on `purpose_key` (§11.4.119). A live claim forces exactly ONE of REUSE (attach) or explicit SUPERSEDE (graceful handoff of the running owner, logged), never a silent second run.

(C) LIVENESS TRUTH-SOURCE. Progress is PROVEN, never inferred from process-alive: every long-op emits a monotonic heartbeat and/or an advancing progress offset; a watchdog reads the DELTA over a window and flags HUNG on no-advance past the op's declared no-progress budget (the §11.4.201(6)-(7)/§11.4.89 discipline — `kill -0` is a necessary-not-sufficient pre-filter, a stale heartbeat/non-growing log is NEVER "progressing"). A consumer blocked on an unclosed producer write-end (§11.4.201(12)) is caught by the flat heartbeat regardless of internal state.

(D) OWNERSHIP-HANDOFF-ON-STOP. Stopping/killing an owner MUST hand its long-ops to the registry (`handoff` , re-adoptable per §11.4.147) or deliberately reap them (`reaped` , reason logged) — NEVER silently orphan or accidentally cascade-kill; long-ops launch detached (own process-group) so an owner stop does not cascade-signal them, and the stop path records each owned op's disposition. The loop done-condition is not satisfied while a `handoff` op is un-adopted.

(E) SAFE REAPING. An orphaned/hung op is reaped ONLY on PROVEN staleness — holder PID dead (`kill -0` fails) OR no-progress-budget exceeded per (C) — resolved by the process's real `/proc/<pid>/cmdline` (§11.4.196(D)/§12.12), never a bare `pgrep` /substring that could match a

carrier or the reaper itself; a LIVE, ADVANCING op is NEVER reaped (§9.2); every REAPED/KEPT decision + resolved evidence is logged (§11.4.180).

(F) CONSISTENCY INVARIANT SWEEP. A fast invariant check runs before every gated transition (build/flash/tag) and on a standing cadence, asserting: no duplicate owner per purpose, no orphan, no stale lock (§11.4.180), no hung op (C), no registry-vs-reality drift (live-long-op set == registry live rows — the §11.4.196(F) check generalised). Any violation is surfaced (named, actionable) and the gated transition is REFUSED until resolved. The sweep is itself a guard (§11.4.201): golden-TRUE + golden-FALSE-with-carrier fixtures, conservative-safe REFUSE on an unresolvable signal, resolved evidence printed on every refusal.

(G) UN-COMPLETABLE-BLOCKER ESCAPE. A gate that cannot reach a terminal verdict MUST degrade, after a bounded budget, to an explicit evidence-backed decision — NEVER an indefinite silent block: §11.4.102 first (hung/(C) vs duplicate/(B) vs wrong-seam/§11.4.120 re-seat-at-activation); NOT-YET-RUNNABLE → §11.4.69 `artifact_not_yet_built` (non-blocking on an intermediate artifact, blocking only on the release candidate per §11.4.135); genuinely un-completable → §11.4.101 (autonomous reversible SKIP-with-reason + tracked §11.4.197 item, ELSE a bounded §11.4.66 operator decision) — the loop keeps progressing every other non-blocked item, it does not idle, and the escape NEVER fabricates a PASS.

Honest boundary (§11.4.6). This layer reduces orchestration mess; it does NOT make product bugs impossible, does NOT make a gate correct (§11.4.108/§11.4.201 still bind each gate), and does NOT remove the operator from genuine blocks — it converts an indefinite silent stall into a bounded, evidence-backed, operator-visible decision. A heartbeat proves ADVANCE not CORRECTNESS; the registry proves what CLAIMS to run, (F) proves registry==reality. **Reuse-not-rewrite:** it is the multitrack per-track agent registry (§11.4.147/§11.4.116/§11.4.187) generalised from per-track workers to any long-op and bound through, per §11.4.196(F) — no parallel mechanism is invented.

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their long-op set, purpose keys, no-progress budgets, and stop-hook wiring as DATA per §11.4.35. Composes §11.4.89 / §11.4.101 / §11.4.116 / §11.4.119 / §11.4.120 / §11.4.135 / §11.4.147 / §11.4.180 /

§11.4.187 / §11.4.196(D)(F) / §11.4.201(1)(6)-(12) / §11.4.207 / §11.4.69 / §12.6 / §12.12 / §9.2. Propagation gate `CM-COVENANT-114-232-PROPAGATION` (literal `11.4.232`) + recommended mechanism gates `CM-LONGOP-REGISTRY-SINGLE-SOURCE` (A) / `CM-LONGOP-SINGLE-OWNER-PER-PURPOSE` (B) / `CM-LONGOP-LIVENESS-HEARTBEAT` (C) / `CM-LONGOP-OWNERSHIP-HANDOFF-ON-STOP` (D) / `CM-LONGOP-SAFE-REAP-PROVEN-STALE` (E) / `CM-ORCHESTRATION-CONSISTENCY-SWEEP` (F) / `CM-UNCOMPLETABLE-BLOCKER-ESCAPE` (G) + paired §1.1 mutations (run two same-purpose sweeps with no reuse/supersede → (B) FAILs; infer liveness from `kill -0` alone / drop the heartbeat-delta check → (C) FAILs; a stop that leaves an owned op with no terminal/handoff record → (D) FAILs; reap a live-advancing op OR reap by bare-`pgrep` substring → (E) FAILs; a live long-op with no registry row passes the consistency sweep → (F) FAILs; an un-completable gate blocks with no escape/decision → (G) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6). No escape hatch — no `--allow-duplicate-longop`, `--infer-liveness-from-pid`, `--silent-orphan-on-stop`, `--reap-live-op`, `--skip-consistency-sweep`, `--block-without-escape`, `--longop-without-registry` flag.

EXTENSION landed 2026-07-26 — §11.4.182 model-in-label + flawless alias/provider↔model consistency + real-time always-in-sync (User mandate, 2026-07-26). Verbatim operator mandates (2026-07-26): "Extend our logging to present models used since we will now have dynamic models choice and switching! From (example) (T1/main - claude4) into: (T1/main - claude4 - model_used), model MUST be dynamically obtained so we present name equivalents - short names - like: haiku, opus, sonnet, fable. However we MUST fully support all Claude Toolkit providers aliases, so real model used names MUST BE obtained in this case(s) as well! Examples: (T1/main - claude4 - haiku), (T1/main - claude4 - fable), (T1/main - claude4 - opus), (T1/main - claude4 - deepseek), (T1/main - claude4 - xiaomi) ... Labels MUST BE everywhere ALWAYS up to date in real time and in sync!" and "Adding model names into labels considers that providers names / aliases will be up to date too and in relation to model used since models belong to providers / aliases !!! Consistency MUST BE flawless!" This EXTENDS the §11.4.182 label discipline (it does NOT re-mint the anchor — the literal `11.4.182` and its propagation gate `CM-COVENANT-114-182-PROPAGATION` are UNCHANGED and stay

GREEN; this carrier block cites §11.4.182, it is not a second §11.4.182 block-start per §11.4.227(B)). (1) **LABEL FORM EXTENDED** — the mandated §11.4.182 label is extended from `(T<N>/<branch> - <alias>)` to `(T<N>/<branch> - <alias> - <model> - <effort>)` (the trailing `<effort>` field added 2026-07-26 per §11.4.231 clause (F) — see the EFFORT-FIELD clause (7) below), `<model>` recording which model is CURRENTLY answering for that alias: for a native Claude alias, the SHORT name (`opus` / `sonnet` / `haiku` / `fable` , case-insensitive-substring-derived from the model id the agent platform reports for the most recent turn — DYNAMIC, obtained live at label-emission time, never a fixed/assumed value since native aliases have no persistent per-alias model and the serving model can legitimately switch mid-session per §11.4.231); for a claude-toolkit PROVIDER alias (`deepseek` , `xiaomi` , `kimi-for-coding` , `opencode` , ...) the REAL provider model id as declared in that provider's host-config registry (the SAME value the alias's own launch mechanism uses — a single source of truth, never re-derived). (2) **MODEL-FIELD DERIVATION (§11.4.6, never guessed)** — a closed two-case split on the SHAPE of the already-derived `<alias>` : (a) native-alias → read the model id from the CURRENT session's own transcript (the platform's per-project per-session transcript; LAST assistant-turn's reported model → short name); (b) provider-alias → read the model id from the host multi-account-toolkit provider registry (`CMA_PROVIDER_MODEL`). Either lookup that cannot resolve yields the honest `?` — NEVER a fabricated or last-known value; an id obtained but matching none of the four native short-names is printed VERBATIM (real-but-unrecognised is more honest than a false `?`). The 3-field legacy form (no `<model>`) remains a VALID, ACCEPTED label during migration — the enforcing guard-hook `LABEL_RE` admits both forms, and NEVER treats an honest `?` model, or an absent model field, as a block-cause. (3) **FLAWLESS ALIAS/PROVIDER↔MODEL CONSISTENCY INVARIANT (operator NOTE, load-bearing)** — the `<alias>` /provider field and the `<model>` field MUST be MUTUALLY CONSISTENT AT ALL TIMES: the model shown MUST be a model that genuinely BELONGS to the alias/provider shown (models belong to providers/aliases), and the alias/provider name MUST itself be current — never a stale alias beside a fresh model, never a model that does not belong to the shown alias. This is guaranteed BY CONSTRUCTION, not by coincidence: the native model is read from THAT alias's own live session transcript, and the provider model is read from THAT provider's own registry

entry — so the derived `<model>` is definitionally a model of the shown `<alias>`; a derivation that ever emits a model not belonging to the shown alias is a §11.4.6 / §11.4.182 consistency violation, never a valid label. (4) **REAL-TIME, ALWAYS-IN-SYNC (§11.4.229/§11.4.106(F))** — the label is STATELESS and RE-COMPUTED at EVERY emission (every human-invoked labeler run AND every synchronous PreToolUse guard-hook firing on every Agent/Task/TaskCreate dispatch) against the LIVE environment (current `CLAUDE_CONFIG_DIR`, current session id, current transcript tail, current provider `.env`) — so a mid-session model switch (`/model`, an autonomous §11.4.231 escalation) is reflected in the VERY NEXT emission with no cache to go stale: real-time BY CONSTRUCTION, not by a sync daemon. Honest boundary (§11.4.6): the live label has NO persisted artifact that could drift, so the Docs-Chain (§11.4.106) sync engine — which propagates PERSISTED documents/DBs — does not apply to the stateless label itself; the operator's "use/extend Docs Chain for real-time responsiveness" ask attaches to any PERSISTED label-history / model-switch LOG surface (a §11.4.197 follow-up to be tracked), which, if adopted, IS §11.4.106-Docs-Chain-synced and §11.4.208-request-history-adjacent — never faked as already-shipped. (5) **HONEST SUBAGENT-INTROSPECTION LIMIT (§11.4.6/§11.4.197)** — the native-alias transcript lookup is proven reliable when invoked from the DISPATCHING process's own shell (the guard-hook's actual execution context at dispatch time); it is NOT proven reliable when a background-dispatched subagent introspects its OWN model from inside its own subprocess on hosts where the subagent shares its parent's session-transcript with no subagent-vs-conductor marker — recorded as an open limitation to be tracked, never silently claimed solved. (6) **DIVERGENT-LABELER RECONCILIATION (§11.4.227)** — the LIVE labeler wired via `.claude/settings.json` is `constitution/scripts/multitrack/track_branch_label.sh`; the top-level `scripts/multitrack/` path is a thin pass-through WRAPPER that `exec`s that canonical copy (§11.4.177 inherited-by-reference), so it mirrors nothing, cannot drift, and emits byte-identical output by construction. The reconciliation LANDED in commit `a48e919f52d` and is gate-enforced at `pre_build_verification.sh:41911-41915` (the gate detects the `exec` wrapper and follows it to the canonical implementation), never a silent divergence. (7) **EFFORT FIELD (added 2026-07-26 per the operator IMPORTANT mandate — verbatim: "Besides dynamic determining proper**

models to use, we MUST decide the effort as well and switch between the effort settings same as we do now for chosen model! Labeling MUST BE extended for this details as well. Example: (T1/main - claude4 - opus - high)."). The 5-field label appends `<effort>` after `<model>`, recording which reasoning-EFFORT setting is CURRENTLY in force for that work-stream, drawn from the closed set `{low | medium | high | xhigh | max}` (the SAME effort ladder §11.4.231 clause (F) selects). (a) DERIVATION (§11.4.6, never guessed) — `<effort>` is read from whatever LIVE effort signal the execution path exposes (the Workflow-tool `agent()` `effort` argument, a future Agent-tool effort parameter, an env/config effort setting); a path exposing NO effort signal, OR any value outside the closed set, yields the honest `?` — EXACTLY like the `<model>` field's `?` fallback, NEVER a fabricated / assumed / last-known value. (b) HONEST HARNESS BOUNDARY (§11.4.6, load-bearing — do NOT overstate the mechanism) — effort-tiering is REAL where the execution path exposes it and a DOCUMENTED CAPABILITY-GAP where it does not: the current Claude Code **Agent tool (subagent dispatch) exposes a `model` parameter but **NO `effort` parameter**, so a subagent's effort is NOT currently settable via that path and its `<effort>` field degrades honestly to `?`; the **Workflow tool's `agent()`** DOES take an `effort` argument, so effort IS settable + derivable there — the label reports whichever path is live, never invents one. This is the SAME aspirational-vs-harness state the §11.4.209 / §11.4.211 Opus-`xhigh` review/merge pins already occupy: the pins mandate the effort, and whether a given dispatch path can SET it is a harness capability the label reports honestly and NEVER fakes. (c) LEGACY-FORM ACCEPTANCE (migration) — the 3-field `(T<N>/<branch> - <alias>)` and 4-field `(T<N>/<branch> - <alias> - <model>)` label forms remain VALID and ACCEPTED; the guard `LABEL_RE` admits all three shapes and NEVER treats an absent `<effort>` field, or an honest `?` effort, as a block-cause (§11.4.6 — mirroring the §11.4.182 track/alias/model honest-`?` rule). (d) CONSISTENCY — the effort field is selected TOGETHER WITH the model by §11.4.231 (both tiers chosen for one task) and never contradicts it; a stale effort beside a fresh model, or an effort outside the closed set, is a §11.4.6 / §11.4.182 consistency violation, never a valid label. (e) REAL-TIME, ALWAYS-IN-SYNC (§11.4.229) — like `<model>`, `<effort>` is STATELESS and RE-COMPUTED at EVERY label emission against the LIVE environment, so a mid-task §11.4.231 effort escalation is reflected in the VERY

NEXT emission with no cache to go stale. Propagation: `CM-TRACK-BRANCH-LABEL` is EXTENDED to also assert the labeler emits a parseable `<model>` field AND a parseable `<effort>` field, the guard `LABEL_RE` accepts the 3-field / 4-field / 5-field label forms (legacy 3-field and 4-field ALWAYS remain accepted during migration; a `?` effort is NEVER a block-cause), and the alias/provider↔model consistency holds; paired §1.1 mutation: strip the `<model>` derivation OR emit a model not belonging to the shown alias → the extended gate FAILs (gate-code = separate work item, NOT claimed shipped §11.4.6). No new escape hatch — the existing §11.4.182 "no escape hatch" list applies to the extended label unchanged.

§11.4.233 — Standing anti-mess control plane: a level-triggered, idempotent, invariant-catalogue reconciliation loop over the WHOLE persistent System state, orchestrating every per-domain enforcer, refusing gated transitions on un-reconciled drift (research-derived, 2026-07-26).

Forensic anchor (genericised): every existing consistency enforcer in a multi-track / multi-agent / multi-submodule / multi-upstream System is EDGE-TRIGGERED — a PreToolUse hook fires on an action, a check runs at a seam (lock-acquisition / merge-time / build-time / commit-time), a gate runs when a build runs — so a mess that arises WITHOUT an edge firing is un-observed until it causes harm (two live examples captured 2026-07-26: a large uncommitted-batch pile-up sat unreconciled through a session, and multiple provably-stale VCS/wrapper locks were held by dead holders with no acquisition attempt to reap them — the §11.4.205 "a hook never installed / a check that runs only when asked" forensic generalised to the whole persistent state). Per §11.4.102 Phase 4.5 the ~19 persistent-state mess classes (Plane R repository: uncommitted pile-up, stale/duplicate locks, orphaned worktrees, half-applied cross-repo commits, drifted submodule pointers, partial merges, divergent mirrors, mis-placed work; Plane G governance: divergent mirrors, pointer drift, gate-ledger drift, anchor-number collision; Plane S tracking: doc/DB drift, cross-track SSoT divergence, ledger drift, handoff-doc drift; Plane P runtime: the §11.4.232 long-op set + un-reaped streams + dangling review state) are ONE architectural flaw — there is no LEVEL-TRIGGERED control plane that re-derives the actual persistent state and reconciles it against a

declared clean state; an un-observed drift is a §11.4/§11.4.1 bluff at the organization layer ("the System is clean" while a mess silently accumulates).

The mandate — a standing **anti-mess control plane** (the desired-state/actual-state reconciliation-loop + control-plane/data-plane pattern of distributed systems), ALL of which hold:

(A) DECLARED INVARIANT CATALOGUE. A single machine-readable consumer-owned catalogue (DATA per §11.4.28/§11.4.35) enumerating every consistency invariant across the persistent planes, each carrying `{id, plane, desired-condition, detector, reconcile-class ∈ {auto-safe | operator-gated | owned-by <anchor>}, evidence-shape}` ; the catalogue IS the desired clean state; an un-catalogued mess class is an honest tracked GAP (§11.4.6/§11.4.197), never silently claimed covered.

(B) LEVEL-TRIGGERED LOOP. The loop re-derives the ACTUAL state (not an event) and diffs it against the catalogue BEFORE every gated transition (commit-batch/build/flash/merge-to-trunk/tag/constitution-pull) AND on a standing cadence (§11.4.94/§11.4.103) — catching drift regardless of how it arose (the Kubernetes level-triggered property); actual-vs-desired uses a §11.4.86 fingerprint diff drilling to the offending record (anti-entropy), never a whole-scan where a fingerprint suffices.

(C) IDEMPOTENT RECONCILE-OR-REFUSE. Each drift takes exactly one dedup-keyed action: RECONCILE-AUTO-SAFE (reversible + evidence-determinable + bounded-blast per §11.4.101 — reap a provably-dead lock §11.4.180, regen a stale derived doc §11.4.106, scope-group-commit a placement-clean batch §11.4.191, retire a merged worktree §11.4.167), idempotent (a no-op on a clean state); OR REFUSE-THE-GATE + a bounded operator decision (irreversible/high-blast/indeterminate — id-colliding SSoT §11.4.206, anchor collision §11.4.227, partial cross-repo merge, divergent mirror) per §11.4.101/§11.4.66; a gated transition NEVER proceeds while a catalogued invariant is violated and un-reconciled; the loop keeps progressing every other non-blocked item (park-one-not-the-loop).

(D) SAGA FORWARD-RECONCILE (no-loss). A partially-applied cross-repo operation is reconciled complete-forward under a §9.2 pre-op backup (finish the remaining legs / bump the pointer / complete the merge zero-markers-zero-drop §11.4.41 step 3 / ff the lagging mirror) — NEVER force-push/reset/ `-s`

`ours` (§11.4.113/§9.2); compensation is §9.2 backup-restore only; the saga reaches fully-completed, never rolled-back-with-loss.

(E) THE LOOP IS A GUARD (§11.4.201). Every detector ships golden-TRUE + golden-FALSE-with-carrier fixtures (§11.4.107(10)); a false-positive REFUSAL of a clean state is a §11.4.201(1) FAIL-bluff exactly as a false-negative PASS is a §11.4 PASS-bluff; every "no drift" null is control-needle-proven (§11.4.201(6)-(12)); conservative-safe REFUSE on an unresolvable signal, resolved evidence printed on every refusal.

(F) CONTROL-PLANE / DATA-PLANE SEPARATION. The control plane DETECTS + RECONCILES state and NEVER performs the data-plane work (the actual build/commit/merge/long-op — §11.4.232 + the build/flash/commit wrappers); the two never merge; the control plane's own state is durable (§11.4.205(6) temp→fsync→rename, never tmpfs, may reuse §11.4.207).

Honest boundary (§11.4.6). Converges the CATALOGUED invariants; does NOT make an un-catalogued mess class impossible (extensible catalogue, never "provably mess-free"), does NOT make a gate correct (§11.4.108/§11.4.201), does NOT auto-fix irreversible drift (operator-gated §11.4.101/§11.4.66), and is eventual-consistency-at-every-gate-and-cadence NOT a real-time daemon (§11.4.205(7)). **Reuse-not-rewrite:** it is the level-triggered COMPOSER over the existing per-domain enforcers (§11.4.180/§11.4.186/§11.4.106/§11.4.191/§11.4.206/§11.4.227/§11.4.232/§11.4.147/§11.4.167), inheriting each's detector — no parallel enforcer is invented (§11.4.227). It composes §11.4.232 (the RUNTIME data-plane long-op orchestrator — §11.4.232 clause (F) is Plane P of this catalogue) as the plane it does not touch.

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their invariant catalogue, plane detectors, gated-transition list, cadence, and reconcile-class bindings as DATA per §11.4.35. Composes §11.4.6 / §11.4.26 / §11.4.28 / §11.4.32 / §11.4.84 / §11.4.86 / §11.4.101 / §11.4.106 / §11.4.108 / §11.4.116 / §11.4.121 / §11.4.147 / §11.4.164 / §11.4.167 / §11.4.176 / §11.4.179 / §11.4.180 / §11.4.181 / §11.4.186 / §11.4.191 / §11.4.196(F) / §11.4.201 / §11.4.205 / §11.4.206 / §11.4.207 / §11.4.227 / §11.4.232 / §12.6 / §9.2 / §2.1. Propagation gate `CM-COVENANT-114-233-PROPAGATION` (literal `11.4.233`) + recommended

mechanism gates `CM-ANTIMESS-INVARIANT-CATALOGUE` (A) / `CM-ANTIMESS-LEVEL-TRIGGERED-LOOP` (B) / `CM-ANTIMESS-RECONCILE-OR-REFUSE` (C) / `CM-ANTIMESS-SAGA-FORWARD-NO-LOSS` (D) / `CM-ANTIMESS-LOOP-IS-A-GUARD` (E) / `CM-ANTIMESS-CONTROL-DATA-PLANE-SPLIT` (F) + paired §1.1 mutations (drop the standing-cadence trigger so the loop is edge-only → (B) FAILs on the pile-up/stale-lock fixture; let a gated transition proceed on a catalogued un-reconciled drift → (C) FAILs; a saga that force-pushes/resets to reconcile a mirror → (D) FAILs; a detector that REFUSES a clean golden-TRUE fixture, or PASSes a golden-FALSE-with-carrier → (E) FAILs; the control plane invokes a data-plane build/commit → (F) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). No escape hatch — no `--edge-triggered-only`, `--skip-consistency-catalogue`, `--proceed-on-drift`, `--reconcile-with-force-push`, `--unvalidated-detector`, `--control-plane-does-work`, `--assume-clean-without-reconcile` flag.

§11.4.234 — Dedicated hook-validation script: hook checks run as an explicit stage of a dedicated commit/push script, and the commit/push mechanism is ALWAYS unblocked (operator mandate, 2026-07-26).

Forensic anchor (genericised): a consumer project wired its full local CI gate as an automatic git `pre-push` hook (`core.hooksPath` → hook → multi-minute build/test/sweep stage on EVERY push). The result, captured 2026-07-26: routine pushes took many minutes each, and when one gate stage inside the hook failed or stalled, the push was rejected outright — the commit/push mechanism itself became blocked, with pending commits stranded locally and no operator-usable path forward short of a forbidden `--no-verify` bypass. The gate logic was sound; wiring it as an uninterruptible implicit hook on the commit/push code path was the architectural flaw — an edge-triggered enforcer (§11.4.233) placed directly on the operator's only forward path, violating the §2 single-entrypoint discipline by making the entrypoint unusable.

The mandate, ALL of which hold:

(A) DEDICATED SCRIPT AS THE SINGLE COMMIT/PUSH ENTRYPOINT. Every consumer MUST ship a dedicated, executable, idempotent script (Lava binding: `scripts/commit-push-all.sh`; consumers bind their own path as DATA per §11.4.35) that runs, in order: (1) submodule/workspace sync, (2)

commit of all pending state, (3) the hook validations as an EXPLICIT stage — the unmodified hook logic fed the real push range, Layer-cheap checks always, long gates stage-separated, (4) push to ALL configured upstreams (§2.1), (5) a closing green/clean verification (clean status + zero unpushed commits, recursive over submodules). The script prints per-stage evidence and is safe to re-run; direct `git commit` / `git push` on the main repo ceases to be the routine workflow (§2).

(B) HOOKS MUST NOT BLOCK THE MECHANISM. Automatic git hooks wired via `core.hooksPath` MUST NOT gate routine commit/push in a way that can leave the repository uncommittable or unpushable. Disconnecting means: `core.hooksPath` unset, the hook file itself preserved UNMODIFIED in the repo, and `--no-verify` NEVER used as the routine path. Validation relocates from the implicit hook to the explicit script stage (A) and to tag/release-time gates — it is never deleted.

(C) NO GATE IS LOST. Every check the disconnected hook performed MUST remain executed — by the dedicated script's validation stage, by the tag/release gate, or by a documented scheduled run. Dropping a check silently because the hook no longer fires is a §11.4 PASS-bluff at the process layer ("the gate still exists" while nothing invokes it); the script's stage list MUST name every check it runs and every check it defers, with the deferral target recorded.

(D) ALWAYS-UNBLOCKED INVARIANT. The commit/push mechanism MUST always be able to complete. A failing validation yields a clear per-check report plus a documented remediation path — never an opaque hung or rejected push. Long gates (multi-minute build/test/sweep stages) MUST be separable from the cheap checks and skippable via an explicit, documented flag (Lava binding: `LAVA_SYNC_SKIP_CI=1`), with every skip recorded in the commit message or evidence pack so the deferred gate is tracked, never forgotten. The escape hatch is a recorded deferral, not a bypass: the skipped gate remains owed and is caught at the next full run or tag gate.

Honest boundary (§11.4.6). This anchor relocates WHERE validation executes (implicit hook → explicit script stage + release gates); it does NOT weaken any substantive gate — the Sixth/Seventh-Law-equivalent anti-bluff covenant, the tag-time full gate, and §11.4.32's sweep-after-constitution-pull all

still bind at their own seams. It does NOT mandate deleting hooks (the hook file is preserved), does NOT authorise `--no-verify` as routine practice, and does NOT make a failing gate pass — it makes the mechanism reachable while the failure is remediated. Composes §2 / §2.1 (single entrypoint + all-upstreams push — this anchor keeps that entrypoint usable), §11.4.32 (sweep still runs, now as a named stage), §11.4.113 (no force-push history-rewrite — the unblocked path never rewrites), §11.4.201 (the script is a guard: a false GREEN verification is a FAIL-bluff), §11.4.227 (amendment discipline), §11.4.233 (the commit/push path is a gated transition the anti-mess control plane reconciles).

Classification: universal (§11.4.17) — no hardware/vendor/project literal; consumers supply their script path, upstream-remote list, cheap-vs-long gate split, and skip-flag name as DATA per §11.4.35. Propagation gate `CM-COVENANT-114-234-PROPAGATION` (literal `11.4.234`) + recommended mechanism gates `CM-DEDICATED-HOOK-VALIDATION-SCRIPT` (A) / `CM-HOOKS-NEVER-BLOCK-PUSH` (B) / `CM-NO-GATE-LOST-ON-HOOK-DISCONNECT` (C) / `CM-COMMIT-PUSH-ALWAYS-UNBLOCKED` (D) + paired §1.1 mutations (wire the full gate back as an automatic blocking pre-push hook → (B) FAILs; disconnect the hook AND drop a check from every stage → (C) FAILs; a validation failure that strands commits with no remediation report → (D) FAILs; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). No escape hatch beyond the recorded (D) deferral — no `--no-verify-routine`, `--skip-all-gates-silently`, `--delete-blocking-hook`, `--push-without-any-validation` flag.

§11.4.235 — Build-and-deploy the moment source fixes are proven-correct (test-side hardening is a parallel stage, never a build gate) + post-deploy version increment + per-group flash-every-eligible-target live verification with a decimal-carry version increment (operator mandate, 2026-07-27; clause (E) added 2026-10-09).

(A) BUILD-AND-DEPLOY THE MOMENT THE SOURCE FIXES ARE PROVEN-CORRECT — TEST-SIDE WORK IS A PARALLEL STAGE, NEVER A BUILD GATE. The moment the SOURCE fixes of a batch are done AND proven-correct — proven-correct = the mandatory independent SOURCE-correctness review has returned a clean GO (§11.4.125/§11.4.142/§11.4.194 on the source diff, on the §11.4.209 Opus-`xhigh` substrate, iterated to zero-

finding/zero-warning GO per §11.4.134) — the build MUST be triggered, pre-build + post-build tests EXECUTED (§11.4.108 layer-1 before, layer-2 after), and the artifact deployed. IN PARALLEL WITH the build AND the manual-QA session (§11.4.185), NEVER gating the build: (1) gate-instrumentation hardening + new-gate authoring (§11.4.110); (2) paired §1.1 mutations + four-layer §11.4.4(b) coverage + HelixQA Challenges; (3) "gates and checks of all kinds" (§11.4.96); (4) the re-review of THAT test-instrumentation work. The build's GO-SIGNAL is source-correctness alone; test-guard hardening (future-regression coverage) + its re-review are a deploy-independent parallel stage per §11.4.230(A). Holding the build sequentially — waiting for the test-instrumentation buildout + its re-review — when the source is already proven-correct is a §11.4.235(A) violation.

(B) POST-DEPLOY VERSION INCREMENT + MANUAL-QA-DEPLOY CYCLE BOUNDARY (extended 2026-07-28 — operator mandate, verbatim: "Increase upcoming release to 0.1.5 since new cycle starts every time we deploy release for manual QA testing! This is how we will be working this ALWAYS! Extend the constitution!"). The moment image(s)/artifact(s) deploy, the version code/version is INCREMENTED (next build targets the next version; every deployed artifact carries a distinct, monotonic, greppable id — §11.4.151/§11.4.108/§11.4.200). Two deployments sharing one version id defeats per-deploy artifact identity → §11.4.235(B) violation. EVERY deployment of a release FOR MANUAL QA TESTING (§11.4.185) is a DEVELOPMENT-CYCLE BOUNDARY: the manual-QA deploy CLOSES the outgoing cycle and STARTS a NEW development cycle, and the version code/version is incremented AT THAT POINT — the new cycle's work targets the NEXT monotonic version (§11.4.151 project-prefixed naming preserved) — ALWAYS, the STANDING working model per clause (C), never a per-request opt-in. The manual-QA deploy is the canonical clause-(B) increment trigger (a deploy-for-manual-QA IS a deploy), and it SHAPES the §11.4.126 release-cycle framing: fixes for QA findings against the deployed artifact land in the NEW cycle on the NEW version id, never patched onto the already-QA-deployed version id. Starting a new cycle's work still targeting the QA-deployed version id is a §11.4.235(B) violation.

(C) STANDING DEFAULT (§11.4.126/§11.4.230/§11.4.103), never a per-request opt-in.

(D) FINDING-LAYER CLASSIFIER — the decidable split clause (A) relies on, mandatory at every §11.4.125/§11.4.142/§11.4.194 review verdict (added 2026-10-03; research-derived from the §11.4.230 four-agent audit of this project's own SpecKit-004 T048 remediation saga). Every finding a code-review verdict records (§11.4.115(F) machine-written) MUST carry exactly one `finding_layer` from the closed set `{source-defect | test-instrumentation | process-doc}` (a field DISTINCT from any other per-finding classification the review record already carries — e.g. this project's own `review_record.py` pre-existing `"class"` field `{mechanical | judgment | false-positive}` is a different axis and is unaffected): `source-defect` — the finding shows the SHIPPED artifact's own behaviour wrong, unsafe or exploitable as it will actually run, OR invalidates the evidence that proves THIS batch's fixes correct (their §11.4.115 RED→GREEN guard, §11.4.108 runtime signature, or covering test per §11.4.125(6) — a proving guard that cannot fail or whose reviewer-authored mutation survives per §11.4.194(6)(d) means the source is NOT proven-correct); `test-instrumentation` — the finding concerns only future-regression coverage, mutation-guard robustness or detection completeness of a guard/gate/meta-test whose protected behaviour was ALREADY proven, with no production decision path shown wrong and no proof-of-this-batch evidence invalidated; `process-doc` — labels, comments, citations or prose with no behavioural effect. The layer is recorded by the REVIEWER (§11.4.240/§11.4.249 producer≠verifier) — the producer MAY contest it only via a fresh review round and MUST NOT lower it unilaterally. A finding that cannot be decidably classified defaults to `source-defect` (the conservative-safe default per §11.4.201(4); secondarily §11.4.101 safe-reversible). §11.4.134 is NOT relaxed: every finding of every layer is still driven to a zero-finding, zero-warning GO in its own loop; clause (D) decides only which GO is the build GO-signal — a batch/slice is BUILD-ELIGIBLE when its `source-defect` findings reach that GO, while outstanding `test-instrumentation` / `process-doc` findings continue as their own tracked §11.4.197 item on a §11.4.230(A)/§11.4.103 parallel stream. A BATCHED review (one review spanning many slices/files) MUST record its verdict PER SLICE/FILE, never one undifferentiated batch verdict, so one slice's unresolved `test-instrumentation` loop cannot withhold build-eligibility from the other slices. Clause (D) applies prospectively; recorded verdicts are never rewritten or reclassified (§11.4.6/§11.4.268). Honest boundary (§11.4.6): this clause does

not make classification easy or infallible — it makes it a required, recorded, reviewer-owned decision instead of an implicit one.

(E) GROUP-COMPLETE ⇒ BACKGROUND FLASHABLE BUILD ⇒ FLASH EVERY AVAILABLE TARGET ⇒ LIVE VERIFICATION OF SANITY + EVERY CONTAINED ITEM, WITH A DECIMAL-CARRY VERSION INCREMENT ON EVERY DEPLOYED IMAGE (operator mandate, 2026-10-09, `IMPORTANT`; added 2026-10-09). Verbatim operator mandate (2026-10-09): "As group by group of tasks from issues_01 gets done, we MUST create flashing image in the background, flash it to all available Orange Pi connected devices (see the list using adb devices command), and perform LIVE testing on devices of basic system major sanity and all workable items coming in into that particular build! We MUST after each flashing image we build and deploy (using our flashing scripts) regularly increase version or version code! Remember that once we get to 9 at some position we switch it to 0 and first left one we incresae by one! Example: 1.0.0-dev-0.0.9 -> 1.0.0-dev-0.1.0 !!! Make sure that all these mandatory RULES are universally applied acrtoss all projects and added into the constitution Submodule, commited and pushed to all upstreams!!!" (the project names and the device-listing command in that quote are the operator's own example, recorded verbatim; the rule below is generic and consumers supply those names as DATA per §11.4.35). The GAP this clause closes, stated precisely so it is not restated elsewhere (§11.4.227): clause (A) triggers the build on a source-correctness GO and clause (B) increments the version on every deploy; §11.4.230(B) exercises every available device concurrently during VALIDATION and §11.4.230(D.3) cuts a QA deliverable on a declared cadence; §11.4.200 target-isolates and read-back-verifies each single flash; §11.4.236 refuses a QA hand-off without a candidate-fingerprinted verdict — but none of them obliges a completed group's image to be FLASHED onto EVERY available target, none binds LIVE on-device verification of a basic major-sanity suite PLUS every workable item contained in that build, and none fixes HOW a version id is incremented (measured 2026-10-09 with a control-needled census over §11.4.235/.230/.236/.200/.119/.108/.151/.264: zero decimal-carry rule, zero "flash every available target" obligation, zero per-item live-verification binding; the only `carry` hits are the verb "carries"; positive control `version` = 18 hits in §11.4.235, negative control = 0). ALL of the following hold:

(E.1) TRIGGER + BACKGROUND BUILD. The moment a group / increment (§11.4.230(D.1) — a project-declared group of tracked items) holds its source-correctness GO per clauses (A)/(D), it is ELIGIBLE for a FLASHABLE image (the consumer's deployable artifact, §11.4.230(D.3)), and that image MUST be built in the BACKGROUND (§11.4.89 / §11.4.230(A) — the loop keeps progressing other items while it builds), through the consumer's official build entry point only, inside the §12 host-safety bounds (§12.6 / §12.11 / §12.12). WHEN the build is cut follows the consumer's declared §11.4.230(D.3) QA-build cadence and its coalescing rule — never a second, competing cadence: under the default `per-increment` trigger each GO'd group triggers a build; a build already in flight covers every group that reached GO before it started, a group reaching GO while it builds rolls into the NEXT build, and a build that fails covers nothing (its groups roll forward); a `periodic` / `whichever-first` consumer cuts at its declared trigger and every group that reached GO since the last covered build rides that build. A group without its GO is NOT built for flashing — clause (A) is unchanged.

(E.2) FLASH EVERY AVAILABLE TARGET. When the image exists it MUST be deployed, through the consumer's official flash/deploy tooling only, to EVERY AVAILABLE target device. A device is AVAILABLE for this image only when ALL THREE hold: (i) it is ENUMERATED fresh at flash time by the consumer-declared device-listing command (DATA per §11.4.35, e.g. `adb devices`), keyed by stable serial (§11.4.111), never a remembered or hand-typed list (§11.4.6); (ii) it is ELIGIBLE for this image — its product / hardware class matches the image's target per the consumer-declared eligibility map (DATA per §11.4.35; an image is never flashed onto a board of a different product line or hardware variant); and (iii) its device LEASE is acquired for this deploy under the §11.4.176(B) / §11.4.119 single-owner device lock (all-or-nothing, non-blocking, TTL-reaped — the consumer's lock mechanism). An enumerated device that cannot be exercised is recorded per device by one of TWO DIFFERENT mechanisms, never conflated, and in neither case is it reflashed out from under its owner or silently dropped. (1) An honest §11.4.69 SKIP-with-reason (§11.4.3) — used ONLY when the device is genuinely absent or ineligible; its reason code MUST be a member of the consumer's §11.4.69 CLOSED reason set (this clause mints NO new code and does NOT extend that set; a helper that enforces the set turns an out-of-set code into a FAIL, §11.4.69): no device enumerated ⇒ `hardware_not_present`; enumerated but

ineligible for this image ⇒ `topology_unsupported`. (2) DEFERRED — a device whose lease is held by another stream is NOT a §11.4.69 SKIP at all, because it IS present and eligible: recording it as `hardware_not_present` would assert that a present board is absent, exactly the misstatement §11.4.69 / §11.4.6 forbid. It is recorded in the DEPLOY RECORD as `DEFERRED`, with the deploy-record field `device_leased_by_other_stream` naming the holder. The specific cause always travels ALONGSIDE the code — in a deploy-record field, or in the SKIP's description text where the consumer's SKIP helper has no field for it (the §11.4.69 helper takes a description and a reason code and nothing else; no `detail` sub-field exists). A DEFERRED device is retried at the next covered build, never waited on unboundedly, and the retry is BOUNDED: after N consecutive DEFERRED builds of the same device (N is consumer DATA per §11.4.35 / §11.4.267(E)), the stall escalates per §11.4.267(D) — a tracked §11.4.197 item naming the device and the holder, surfaced in every §11.4.97 report until resolved, and routed to a recorded operator decision (§11.4.66) — never an indefinite silent re-deferral. Each flash is target-isolated and verify-after-write per §11.4.200 (the deployed identity read back FROM each intended target). Independent devices are tested CONCURRENTLY; genuinely shared exclusive resources (one flash bus, one single-eligible-target flash tool, one power controller, one sink) serialise per §11.4.119 / §11.4.230(B). Devices count as independent only when their CONTROL TRANSPORTS are independent as well: legs that share one host-side transport daemon whose recovery path restarts that daemon (one debug-bridge server that a recovery step kills and restarts, for example) share an exclusive resource, so a flash or recovery leg on that daemon serialises with every live-test leg on the same daemon — unless each device is given its own daemon instance AND that per-device isolation has been proven to work on that host with captured evidence before anything relies on it (§11.4.6); an unproven isolation is treated as shared. An available device that cannot be flashed or verified is recorded per device as a FAIL or an honest SKIP-with-reason (§11.4.3), never silently dropped; the deploy record lists the enumerated, eligible, leased, flashed, and read-back-verified serials, plus every per-device SKIP reason code with its cause and every DEFERRED device with its holder.

(E.3) LIVE VERIFICATION ON EACH FLASHED DEVICE. On each verified device run (a) the consumer-declared basic major-sanity suite (at minimum:

boot completed, core services up, no crash/hang loop — the suite named as DATA) AND (b) a LIVE test of EVERY workable item contained in that build — the item set recorded in the hand-off per §11.4.230(D.5) — each producing captured machine evidence on that device (§11.4.262 / §11.4.69 / §11.4.108 runtime layer); metadata-only, config-only, or grep-only PASS is forbidden (§11.4). An item whose live verification is genuinely device- or topology-gated is SKIP-with-reason PER ITEM (§11.4.3 / §11.4.69 reason code), never folded into a group-level PASS. A FAIL on any device is linked to the affected item; an item already in a terminal state is re-opened, while an item still open is only linked (§11.4.214 — 're-opening an open item' is undefined); a huge FAIL triggers §11.4.129.

(E.4) LATENT BINDING + NO AVAILABLE DEVICE ⇒ HONEST SKIP. The device legs — (E.2), (E.3), and the zero-device rule of this clause — bind every consumer that HAS an official flash/deploy entry point for a device-targeted artifact, or that has bound a deploy ledger for such an artifact to the §11.4.235 gates (the §11.4.96 / §11.4.275 latent-binding pattern); whether the consumer has declared its device-listing command does NOT decide the binding. A bound consumer MUST declare that command; a bound consumer that has not declared it is in a CONFIG ERROR — a blocking result, never N/A and never a reason to skip the device legs — because a project must not opt out of flashing its devices by leaving the command undeclared. Only a consumer with NO device-targeted flash/deploy entry point and NO deploy ledger bound for a device-targeted artifact (no flashable device tier at all) records the flash and live-verification legs as honest N/A (`feature_disabled_by_config`), never a fabricated device run, and the obligation binds the moment either condition becomes true. When a declared enumeration yields zero AVAILABLE targets (none enumerated, or every enumerated device ineligible or leased per (E.2)), the flash and live-verification legs record what (E.2) prescribes — a §11.4.69 SKIP-with-reason (`hardware_not_present` , cause `no_target_device_attached` carried as (E.2) describes, or `topology_unsupported` per ineligible device) and a DEFERRED record per leased device — plus a tracked §11.4.197 item for the owed live run — never a faked PASS and never a claim that the build was verified on hardware. The build itself still completes and is retained.

(E.5) DECIMAL-CARRY VERSION INCREMENT ON EVERY DEPLOYED IMAGE. Every image built and deployed under this clause carries a NEW version id per clause (B). SCOPE — universal in force, fenced in reach (a conservative-safe reading per §11.4.6 / §11.4.201(1), recorded as awaiting operator confirmation): the carry rule binds the dotted numeric group that the consumer's ONE version tool owns under its DECLARED version scheme (DATA per §11.4.35), and applies only where that declared scheme is POSITIONAL-DIGIT style (every non-leftmost position a single decimal digit, as in the operator's own example); a consumer on Semantic Versioning, Calendar Versioning, or any scheme whose positions are unbounded counters or dates declares that scheme and is NOT subject to the carry (`1.2.10` is correct SemVer) — that exemption exists ONLY by explicit declaration backed by a recorded operator decision (§11.4.66) naming the consumer — a declaration without that record is UNDECLARED (the gate requires the declaration to cite the record; whether the record exists and says so stays a review obligation the gate cannot verify, §11.4.6): once a consumer binds a deploy ledger, declaring its version scheme is MANDATORY, and a bound ledger with no declared scheme is UNDECLARED, a distinct result that a release seam consuming the gate MUST treat as blocking, exactly as a FAIL (§11.4.135) — until such a seam exists that is an obligation on the seam, tracked as a §11.4.197 item by each consuming project, not an enforced block — never a silent skip; a consumer that has bound no ledger at all records an honest SKIP whose result is mechanically distinct from a PASS (its own result code and machine-readable verdict, §11.4.1), BUT a consumer bound by (E.4) MUST bind a deploy ledger for its device-targeted artifact, so for such a consumer the no-ledger SKIP is a CONFIG ERROR at the release seam, never N/A — the SKIP stays legitimate ONLY for a consumer NOT bound by (E.4) — and enforcement over a deploy ledger applies only to rows AFTER the consumer-declared adoption marker (the §11.4.224(E) adoption-fence precedent) — ids minted before adoption are history and are never retroactively failed. Within that scope every version id obeys ONE POSITIONAL-DIGIT ID GRAMMAR, implemented identically by the consumer's version tool and by the gate: (i) in EVERY dotted numeric group of the id — any run of two or more dot-joined decimal positions, not only the owned one — every non-leftmost position is a single digit 0..9 and the leftmost has no leading zero (so a dotted suffix or build-metadata group can never hide a no-carry group: `…-0.0.10-rc.1.2` is in violation); (ii) no two

numeric runs are joined by a single dash (`0.0-9` , `0.0.1-1`), because which group is owned would be ambiguous; (iii) the OWNED group — the dot-joined positions that END the id after its last dash, or the whole id when it has no dash — has no leading zero in any position (a single-position owned group such as `-dev-7` may grow). The tool refuses to bump any id that violates (i)–(iii) or does not end in its owned group; the gate fails any in-scope id that violates (i) or (ii) or whose owned group violates (iii) — an id ending in a non-numeric suffix (`-rc1`) has no owned group and is held to (i) and (ii) alone. The next id is computed by the DECIMAL CARRY rule: in that owned dotted numeric group (the trailing dot-joined decimal positions after any tag prefix and pre-release separator), every position except the leftmost holds a single digit 0..9; incrementing adds 1 to the LAST position; a position that would exceed 9 becomes 0 and carries +1 into the next-left position; the carry propagates as far as needed; the leftmost position absorbs the final carry and may grow; the tag prefix (§11.4.151), pre-release separators, and every non-group character are preserved verbatim. Worked examples: `1.0.0-dev-0.0.8 → 1.0.0-dev-0.0.9` ; `1.0.0-dev-0.0.9 → 1.0.0-dev-0.1.0` ; `1.0.0-dev-0.9.9 → 1.0.0-dev-1.0.0` ; `0.9.9 → 1.0.0` ; `9.9.9 → 10.0.0` . A non-leftmost position above 9 (e.g. `0.0.10`) is the no-carry form and a violation. A rollover is a CARRY, not a reset and not a skip: `0.0.9 → 0.1.0` is the immediate successor, so the §11.4.151 "never reset, never skipped" discipline holds — no id is reused and none is jumped; where a consumer also derives an integer version CODE from the id, its encoding MUST be strictly monotone over carried ids (a positional-weight code is), and an encoder whose domain cannot represent the next carried id MUST refuse loudly rather than wrap or collide (§11.4.6) — widening that domain is tracked work, never a silent truncation. The consumer's ONE version tool (DATA per §11.4.35) is the single implementation (§11.4.28 / §11.4.251); a hand-rolled or hand-edited increment is a violation.

(E.6) HONEST BOUNDARY (§11.4.6). Clause (E) makes deploying and live-verifying every completed group mandatory and mechanical. It does NOT replace §11.4.185 manual QA, the §11.4.40 tag preconditions, or the §11.4.236 readiness verdict for a QA hand-off; it never flashes a build lacking its review GO (clause (A)); it never authorises bypassing the consumer's official flash tooling or power-cycling hardware outside its authorised recovery path (§11.4.133 / §11.4.144); and a live PASS on the available devices proves those

devices only — hardware variants not attached are an enumerated gap (§11.4.118), never an implied PASS.

Honest boundary for clauses (A)–(E) (§11.4.6): this anchor relocates the TEST-INSTRUMENTATION re-review + gate/check hardening OFF the build critical path; does NOT weaken any substantive gate. The SOURCE-correctness review (§11.4.125/§11.4.142/§11.4.194 Opus-`xhigh` §11.4.209 → GO §11.4.134) STILL PRECEDES + gates the build (it IS "source proven-correct"); pre/post-build tests (§11.4.108 L1-2) run at their seams; §11.4.185 manual-QA-final un-weakened; §11.4.129/§11.4.40 un-weakened. "Test-side work is parallel, never a build gate" is bounded to future-regression instrumentation — a check that proves the SHIPPED artifact wrong is a source/artifact-correctness gate (§11.4.108 L1-2), stays on the critical path. Clause (D) supplies the mandatory classification mechanism this boundary's own test presupposes — a review that never classifies its findings has not actually applied this honest boundary, it has merely stated it. Clause (E) adds deployment and live-verification obligations on top of (A)–(D) and weakens none of them: its own boundary (E.6) applies in full — no manual-QA, tag, or readiness gate is replaced, no un-GO'd build is flashed, no device is flashed outside its eligibility or lease, and a live PASS proves only the devices it ran on.

(Process note for future extenders: §11.4.235's next free clause letter is (F); never reuse (A)–(E).)

Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-235-PROPAGATION` (literal `11.4.235`) + recommended `CM-BUILD-ON-SOURCE-PROVEN-NOT-TEST-SIDE` (FAIL: build held past source-GO w/ unfinished test-instrumentation; FAIL: build started with NO source-review GO — the §11.4.201(1) dual-assertion guard) + `CM-VERSION-INCREMENT-ON-DEPLOY` (FAIL: new-artifact deploy shares a version id; FAIL: work of the cycle opened by a manual-QA deploy still targets the QA-deployed version id — the clause-(B) cycle-boundary increment skipped) + paired §1.1 mutations. `CM-BUILD-ON-SOURCE-PROVEN-NOT-TEST-SIDE` is IMPLEMENTED (`constitution/scripts/gates/cm_build_on_source_proven_not_test_side.sh` + its paired `_mutation_test.sh`), as is `CM-VERSION-INCREMENT-ON-DEPLOY` (`constitution/scripts/gates/cm_version_increment_on_deploy.sh` + its paired mutation test) — the prior "gate-code = separate work item" disclaimer

for those two was stale and is corrected here. NEW recommended gate `CM-REVIEW-FINDING-CLASS-RECORDED` for clause (D) (FAIL: a batched-review verdict covering more than one slice/file with no per-slice verdict or no per-finding `finding_layer` recorded; FAIL: a slice marked build-eligible while any `source-defect` finding in it has not reached GO; FAIL: a `finding_layer` lowered by the producer without a fresh review round) + paired §1.1 mutation — gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227), registered as a tracked deferral in `constitution/scripts/gates/gate_ledger_deferrals.tsv`; it binds ADDITIVELY by computing whether the existing `source_review_go` marker may be written, never by modifying `cm_build_on_source_proven_not_test_side.sh`'s own tested logic. Clause (E) adds two recommended gates. `CM-GROUP-DEPLOY-FLASH-ALL-AND-LIVE-VERIFIED` (binds per (E.4) — every consumer with an official flash/deploy entry point, or a bound deploy ledger, for a device-targeted artifact; FAIL: such a consumer with no declared device-listing command (config error); a per-device SKIP code outside the consumer's §11.4.69 closed set; a lease-held device recorded as a §11.4.69 SKIP instead of DEFERRED; a device DEFERRED for more than its declared N consecutive builds with no escalation item; a group holding its GO that no completed covered image was ever flashed for; an enumerated device absent from the deploy record with no per-device FAIL or SKIP reason; a device flashed while ineligible for the image or without its lease; a flashed device with no §11.4.200 read-back; a contained item with no per-device live verdict or per-item SKIP-with-reason; a zero-device run recorded as PASS; golden-FALSE per §11.4.201(1): every available device flashed and read-back-verified with sanity and per-item verdicts present, every ineligible device recorded as a reasoned closed-set SKIP and every leased device as DEFERRED with its holder, and a consumer with no device-targeted flash/deploy entry point and no such bound ledger recorded N/A, does not fire) + paired §1.1 mutation (drop one enumerated serial from the deploy record, strip one item's live verdict, or record a zero-device run as PASS → the gate MUST FAIL) — gate-code NOT claimed shipped (§11.4.6 / §11.4.227), registered as tracked deferral OWED-GATE-114 in `constitution/scripts/gates/gate_ledger_deferrals.tsv`. `CM-VERSION-DECIMAL-CARRY` (the carry check is armed only for a gate config declaring a positional-digit `version_scheme`, and binds only ledger rows after its declared `carry_adoption_marker` when one is set; FAIL: such a row that breaks the (E.5) positional-digit id grammar —

ANY dotted numeric group with a non-leftmost position that is not a single digit 0..9 or a leftmost leading zero (a suffix or build-metadata group included), two numeric runs joined by a single dash, or an owned group with a leading zero; UNDECLARED (distinct; a consuming release seam MUST treat it as blocking per §11.4.135 — no seam consumes the gate yet, an obligation each consuming project tracks per §11.4.197): a bound deploy ledger with no declared `version_scheme`, or a non-positional scheme with no `scheme_decision_ref`; golden-FALSE: `9.9.9 → 10.0.0`, opaque non-dotted ids, an EXPLICITLY declared non-positional scheme, and pre-marker rows do not fire; a declared marker absent from the ledger, a named config that is missing or unreadable, an unknown, duplicated or empty config key, or a ledger row that is not exactly two non-empty TAB-separated fields is a config error, never a silent pass or a SKIP) is IMPLEMENTED as an added check inside `constitution/scripts/gates/cm_version_increment_on_deploy.sh`, with its discriminators in the paired `cm_version_increment_on_deploy_mutation_test.sh` (RED observed before the check existed); every gate run prints machine-readable `VERDICT:` and `CARRY:` lines that callers MUST parse, and every VERDICT has its own exit code (0 PASS, 1 FAIL, 2 config error, 3 UNDECLARED, 4 SKIP) so a SKIP can never be read as a PASS — the exit code alone is ambiguous by design (0 covers CARRY PASS and CARRY SKIP; 1 covers a version-id collision and a carry failure), and the informational modes `--help` / `--print-example-config` exit 0 with NO verdict line and are refused unless used alone; honest scope: it proves no in-scope deployed id is in the no-carry form, not that each deploy used the immediate successor, and it SKIPs (exit 4, never the PASS code) only for a consumer that has bound no deploy ledger — which for a consumer bound by (E.4) is a config error at the release seam, per (E.5) — or whose bound ledger has no recorded deploys yet.

Non-compliance is a release blocker. No escape hatch — no `--build-behind-test-hardening`, `--test-side-gates-the-build`, `--skip-source-correctness-review`, `--deploy-without-version-increment`, `--not-default-speed-mode`, `--serialise-build-and-validate`, `--flash-one-device-only`, `--skip-live-item-verification`, `--zero-devices-is-pass`, `--hand-rolled-version-bump`, `--no-carry-version`. **§11.4.236 — QA-deploy-readiness**

gate: no manual-QA hand-off until the mandated validation produced a candidate-fingerprinted PASS verdict; a blocker MUST bind to a seam, never prose (research-derived, 2026-07-31). Compact summary: a release artifact MUST NOT be deployed / handed off for manual QA (§11.4.185) until the mandated autonomous validation(s) for THAT artifact produced a machine-written PASS verdict whose artifact fingerprint == the candidate (§11.4.115(F), read from the target at run time); ABSENCE of the verdict BLOCKS the deploy exactly as a FAIL does (§11.4.135); the refusal lives in the deploy TOOL (the seam), never in prose (§11.4.205/§11.4.226) — a source-gate asserting the harness merely EXISTS is necessary-not-sufficient; any operator-declared blocker (a §11.4.140 CRITICAL / "release-blocker") MUST bind to a mechanical seam at the point it gates, never satisfied by prose nor DOWNGRADED to a follow-up, and when its instrument bugs the response is §11.4.4/§11.4.120 (fix the instrument, never route around it); override only via an explicit, recorded operator flag (§11.4.234/§11.4.135), never a silent bypass; the gate is self-validated golden-good/golden-bad (§11.4.107(10)/§11.4.201, cannot false-allow) and the verdict-emitter reads the fingerprint from the target at run time, never a hardcoded per-release value (§11.4.111). Honest boundary §11.4.6 — the seam proves a candidate-matching PASS verdict EXISTS, it does NOT replace the §11.4.185 manual-QA sufficiency gate nor make the underlying validation honest (that stays §11.4.5/.69/.107). Classification: universal (§11.4.17). Composes §11.4.4/.107(10)/.108/.111/.115(F)/.120/.126/.135/.146(D3)/.185/.201/.205/.226/.234. Propagation gate `CM-COVENANT-114-236-PROPAGATION` (literal `11.4.236`) + recommended gate `CM-QA-DEPLOY-READINESS-GATE` + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.236. Non-compliance is a release blocker. No escape hatch — no `--deploy-without-verdict`, `--prose-blocker-suffices`, `--downgrade-blocker-to-followup`, `--skip-readiness-gate`, `--verdict-fingerprint-optional`, `--silent-override` flag.

§11.4.237 — Mandatory exhaustive context-and-spirit-aware translation review: every localized artifact independently reviewed for CONTEXT (source) + SPIRIT (target-language idiom/register) + technical/non-

translatable fidelity — accuracy necessary, never sufficient (operator mandate, 2026-08-05).

Forensic anchor (operator mandate, 2026-08-05): a translated artifact that is merely ACCURATE — every source token faithfully rendered — can still read as stilted, foreign, machine-literal, register-wrong, or idiom-broken to a native speaker of the target language, and a localization pass that silently mangles a technical term or "translates" a non-translatable product/proper name ships a defect worse than untranslated text; the recurring failure mode is a translation that scores green on an accuracy check while a native reader immediately perceives it as non-native. Accuracy is NECESSARY; it is NOT SUFFICIENT for a slick, polished, professional localization.

The mandate (ALL hold):

(A) EVERY LOCALIZED ARTIFACT IS INDEPENDENTLY REVIEWED — NEVER SELF-REVIEWED. Every translation and every translated artifact — UI strings, pages, documents, PDFs, manuals, marketing copy, captions/subtitles, help text, any localized content — MUST undergo a HEAVY, EXHAUSTIVE, independent review by a reviewer (agent or human) STRUCTURALLY SEPARATE from the translator that produced it (§11.4.70/§11.4.20 independence). The translator's own pass (§11.4.92 multi-pass self-review) PRECEDES this review and NEVER satisfies it (§11.4.142's no-self-review-substitution discipline applied to localization; §11.4.92 precedes, never satisfies). A localized artifact accepted without an independent context-and-spirit review is a §11.4 PASS-bluff at the localization layer.

(B) THE REVIEWER HOLDS AND APPLIES THREE DIMENSIONS. (a) FULL CONTEXT OF THE SOURCE MATERIAL — the original ideas, the subject/domain, the purpose, and the intended audience; the reviewer reads the source MEANING, not merely the source STRING, so intent, nuance, and tone are carried across, never lost. (b) THE SPIRIT OF THE TARGET LANGUAGE — its tradition, idiom, register, tone, and cultural conventions — so the result reads as NATIVE, natural, fluent, and MAXIMALLY POLISHED, never literal / word-for-word / calqued; a construction correct in the source that reads foreign in the target is REWRITTEN into the natural target-language form. (c) CORRECT TECHNICAL VOCABULARY AND VERBATIM PRESERVATION OF NON-TRANSLATABLE TERMS — domain/technical terms rendered with their established target-language equivalents, and NON-TRANSLATABLE tokens

(product names, brand names, proper nouns, technology/library/API names, code identifiers, trademarks) preserved VERBATIM — never "translated", never accidentally transliterated, never localized away.

(C) THE GOAL IS MAXIMALLY POLISHED LOCALIZATION — ACCURACY NECESSARY, NEVER SUFFICIENT. The bar is a slick, polished, professional, native-reading localization; fidelity to the source is the FLOOR, not the ceiling. A rendering that is accurate but stilted / foreign / register-wrong / idiom-broken is a FINDING, not a pass.

(D) ANTI-BLUFF — CAPTURED MACHINE-READABLE POSITIVE EVIDENCE + FAIL-LOOPS-TO-CLEAN. The review emits captured, machine-readable POSITIVE evidence per §11.4.5/§11.4.69 (`feature_class` = translation_review): a per-artifact verdict carrying a PER-DIMENSION rationale across the closed dimension set {fidelity, fluency/naturalness, spirit/register, terminology, completeness, non-translatable-preservation} — a PASS with no captured per-dimension rationale is a §11.4/§11.4.1 bluff. A FAIL on ANY dimension LOOPS BACK to re-translation/polish and re-review, iterating to a zero-finding clean verdict per §11.4.134 — a single pass that "looked fine" never closes the artifact.

Honest boundary (§11.4.6). This anchor REFINES the existing localization discipline from PASS/FAIL accuracy checking to CONTEXT-AND-SPIRIT native-polish review; it does NOT replace §11.4.255's canonical translation pipeline (the HelixTranslate-canonical-translation-pipeline sense of that anchor number) nor §11.4.256's independent per-language review (the independent-per-language-translation-review sense of that anchor number — a KNOWN operator-owned §11.4.54 anchor-number collision on `11.4.140` / `11.4.141`, re-mint still operator-owned; cited here strictly in those senses, introducing no new collision), it STRENGTHENS them (accuracy → native polish). It does not make a mistranslation impossible (a miscalibrated reviewer stays §11.4.107(10) golden-good/golden-bad fixture territory), and where a target language genuinely has no established term for a source concept, that gap is an honestly recorded finding (§11.4.3 SKIP-with-reason), never a silently invented word.

Classification: universal (§11.4.17) — no vendor/project/language literal; the consuming project supplies its language set, term glossary, non-translatable token list, and reviewer substrate as DATA per §11.4.35. Composes §11.4.255 (canonical translation pipeline) / §11.4.256 (independent per-language

translation review — this REFINES it from PASS/FAIL accuracy to context+spirit polish) / §11.4.6 / §11.4.69 / §11.4.92 / §11.4.134 / §11.4.142 / §11.4.209 / §1.1. Propagation gate `CM-COVENANT-114-237-PROPAGATION` (literal `11.4.237`) + recommended gate `CM-TRANSLATION-CONTEXT-SPIRIT-REVIEW` (every localized artifact carries an independent per-dimension context-and-spirit review verdict with captured rationale across {fidelity, fluency/naturalness, spirit/register, terminology, completeness, non-translatable-preservation}, FAIL looping to re-review per §11.4.134) + paired §1.1 mutation (a translation accepted on an accuracy-only check, OR self-reviewed by its own translator, OR carrying a golden-bad-seeded mistranslated idiom / a "translated" non-translatable product name → the gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.237.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-spirit-review`, `--accuracy-only-OK`, `--self-review-translation-OK`, `--literal-translation-OK`, `--translate-non-translatable-OK`, `--skip-per-dimension-rationale` flag exists.

§11.4.238 — Automated QA must be the DISCOVERER: manual QA finds nothing new, and anything it finds is a coverage escape (User mandate, 2026-08-08). Verbatim operator mandate: *"Helix QA full websites testing MUST find all such issues, not us! Once we do manual QA we shall not be able to find anything! THIS IS MANDATORY RULE / CONSTRAIN / GOAL to achieve with this and any other project and MUST BE part of the root constitution!"* Forensic case study (FACT, 2026-08-08): in one cycle five real defects were found — a health endpoint returning `{"status":"healthy","components":[]}` that was structurally incapable of ever reporting unhealthy; a gateway routing every completion at a port nothing served (HTTP 500 on the product's primary capability); two submodule pins on `main` that upstream answers with `not our ref`, so no fresh clone could check them out; and two hardware-profiler defects reading a key from a file where it does not exist. **Every one was found by an agent reading source or probing by hand; not one by the automated QA system**, which reported green throughout. The mandate (ALL hold): **(A) the automated QA system is the DISCOVERER, not the confirmer** — discovery by a human, an operator's manual pass, an agent

reading code, or an end-user report is a FAILURE of the QA system; the finding is still fixed, but the QA gap it exposes is a defect of equal standing and closing only the reported defect is the violation. **(B) manual QA must find NOTHING NEW** — by the time manual QA runs (§11.4.185) the automated regime must already have surfaced everything, so manual QA is a CONFIRMATION and sufficiency gate, never the discovery mechanism; the target is explicit and measurable — a manual pass discovering zero previously-unknown defects. **(C) every escape triggers the §11.4.138 loop, widened** to EVERY out-of-band channel: systematic-debug the defect (§11.4.102), produce a **coverage-escape audit** naming WHY the regime missed it cited to the specific missing check / unexercised path / absent assertion (never "we didn't think of it"), register a new or strengthened automated check that WOULD have caught it into the standing regime (§11.4.135) with a §11.4.115 RED capturing the escaped defect, and commit the audit as captured evidence. **(D) coverage is measured by what the regime can CATCH, not what it runs** — every surface falsifiable per §1.1 (a check whose paired mutation does not make it fail is decoration), exercising real user-reachable paths end-to-end (§11.4.98 / §11.4.143 / §11.4.136 / §11.4.158 / §11.4.159 / §11.4.160). **(E) directional and measured** — track the discovery-channel split (automated vs out-of-band) and drive out-of-band toward zero; honest boundary (§11.4.6): this does NOT claim a regime can be provably complete (§11.4.118) and does NOT license closing an escape by narrowing the definition of "defect". **EXTENSION — Discovery-channel record schema + escape-ratchet no-data-point rule (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26).** §11.4.238(C) states the CONCEPT (a coverage-escape audit citing the specific missing check) but fixes no record schema, and (E)'s "toward zero over successive cycles" names neither an input condition nor an observable pass/fail outcome — the ratchet is the decidable form. **(1) Discovery-channel record schema:** every recorded defect MUST carry a discovery channel from the CLOSED SET `{automated_seam, manual_qa, operator, end_user, agent_inspection}` (or the consuming project's declared equivalent per §11.4.35) AND a `should_have_been_caught_by` value naming a specific seam or gate, OR the literal `none` with a WRITTEN justification; a record missing either field is REFUSED. The record MUST be WRITTEN BY THE VERIFIER SEAM, never by the producer of the defect's fix (the §11.4.240 producer≠verifier separation applied to the record OF the escape, not only the record of the fix). **(2)**

Monotone-decreasing escape count: a cycle whose escape count EXCEEDS the recorded baseline is REFUSED at the release seam; a cycle whose count is lower moves the baseline DOWN. **(3) A manual-QA-not-run cycle contributes NO DATA POINT and MUST NOT lower the baseline** — the §11.4.201(6) false-null applied to this ratchet: an unmeasured cycle and a genuinely-clean cycle both read zero, and only the `manual_qa_ran` flag distinguishes them. **(4)** `should_have_been_caught_by: none` **records are counted and reported SEPARATELY**, so narrowing the definition of an escape is visible rather than silent. Honest boundary (§11.4.6): at the correct end-state — every escape caught by a seam — the count reaches 0, so the ratchet is a valid NECESSARY condition; it is NOT sufficient, since zero also results from nobody looking, which the no-data-point rule and the separate `none` tally exist to expose. Composes §11.4.201(6)-(8) / §11.4.135(5) / §11.4.261. Recommended mechanism gates `CM-DISCOVERY-CHANNEL-RECORD-COMPLETE` + `CM-ESCAPE-RATCHET-NO-DATA-POINT-HONEST` + paired §1.1 mutations (a record with no channel, or a `none` with no justification, or a producer-authored record → the schema gate MUST FAIL; a `manual_qa_ran` false cycle lowering the baseline → the ratchet gate MUST FAIL; golden-FALSE per §11.4.201(1): a genuine manual-QA cycle with a real zero-escape result MUST be allowed to lower the baseline); gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). Composes §11.4.27 / §11.4.138 (trigger set widened) / §11.4.185 (manual QA becomes confirmation, never discovery) / §11.4.118 / §11.4.135 / §11.4.115 / §11.4.98 / §11.4.136 / §11.4.143 / §11.4.158 / §11.4.159 / §11.4.160 / §11.4.102 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its QA system, surface inventory, and discovery-channel ledger per §11.4.35. Propagation gate `CM-COVENANT-114-238-PROPAGATION` (literal `11.4.238`) + recommended gates `CM-QA-IS-THE-DISCOVERER` / `CM-MANUAL-QA-FINDS-NOTHING-NEW` + paired §1.1 mutation (gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.238. Non-compliance is a release blocker. No escape hatch — no `--manual-qa-may-discover`, `--skip-coverage-escape-audit`, `--found-by-human-is-fine`, `--close-escape-without-new-check`, `--qa-green-is-enough`, `--discovery-channel-not-tracked` flag.

§11.4.239 — Critical-invariant work-class Definition of Done mandate — money/safety/availability/integrity code MUST include failure-path scenarios as gates, not merely happy-path passes (research-derived, 2026-08-15). Compact summary: every project's DoD MUST identify a closed CRITICAL-INVARIANT WORK CLASS (money movement, safety-of-life/property outputs, availability of the primary user-facing capability, integrity of the persistent record) and MUST require that every change touching this class carry EXPLICIT FAILURE-PATH SCENARIOS as passing tests BEFORE closure — a happy-path-only DoD on money/safety/availability/integrity code is the exact "green means done" bluff §11.4/§11.4.1 forbids (extraction line 49 [un-resolvable offset per the §11.4.239 CITATION NOTE below; verified locator `module_27:30`]: "a Definition of Done that stops at 'the code compiles and the happy-path test is green' is a machine for generating reopened tickets on the paths that move money"). (A) The four critical-invariant categories (money/safety/availability/integrity) with consumer-owned domain extensions per §11.4.35; (B) domain-appropriate MINIMUM failure-path scenario sets rendered as Given/When/Then acceptance criteria with "Then" observing real state deltas (§11.4.69), never "no error" — the canonical MONEY set: fresh / idempotent retry / same-key-different-payload / crash-in-the-middle (extraction lines 395-428 [un-resolvable offsets per the §11.4.239 CITATION NOTE below; verified locator `module_27:312`]); (C) mechanical precondition binds through the existing §11.4.146(D3) status-custody chain (the status-write seam refuses a closure without the scenario map cited, each scenario mapped to a §11.4.115(F) machine-written verdict pair); (D) composes with §11.4.108 four-layer / §11.4.129 huge-blocker / §11.4.185 manual-QA-sufficiency / §11.4.238 automated-QA-discovers — automation NECESSARY, manual QA still the terminal human oracle; (E) honest boundary — directional (forbids happy-path-only), not exhaustive (§11.4.118 discovery-pressure stays in force). **CITATION NOTE (2026-08-20 citation audit, §11.4.6):** the "extraction file" offsets cited throughout §11.4.239-§11.4.254 are absolute offsets into a consolidated extraction artifact that is NOT tracked in this repository and whose numbering does NOT resolve against the tracked corpus at `docs/research/ai_curriculum_2026_08/source_corpus/` — the highest cited offset (6490) exceeds the 5838-line total of ALL NINE module files concatenated. Each extraction offset is therefore retained ONLY as an un-resolvable provenance pointer, NEVER as a resolvable location, and is accompanied by a VERIFIED

`module_NN:LINE` locator measured against the tracked corpus with a control-needle-proven instrument per §11.4.201(7)(b). The corpus landed 2026-08-19, AFTER these anchors landed, so it corroborates them; it is not their derivation source. Full note: `constitution/Constitution.md` §11.4.239. Classification: universal (§11.4.17). Composes §11.4.15/.16/.33/.35/.43/.69/.108/.115(F)/.135/.146(D3)/.185/.205/.226/.238. Propagation gate `CM-COVENANT-114-239-PROPAGATION` + recommended gate `CM-CRITICAL-INVARIANT-FAILURE-PATH-DOD` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.239. Non-compliance is a release blocker. No escape hatch — no `--happy-path-suffices-for-money`, `--skip-failure-scenarios`, `--defer-failure-path-tests`, `--criticality-not-applicable-without-reason`, `--closure-without-scenario-map` flag.

§11.4.240 — Producer ≠ Verifier scope separation: the actor that produces a change/artifact/verdict cannot be the one that gates it — least privilege over cleverness (research-derived, 2026-08-15). Compact summary: every project MUST enforce STRUCTURAL SEPARATION between the actor that PRODUCES a change/artifact/verdict and the actor that GATES that same work — the producer cannot write, weaken, delete, or edit the acceptance tests, the oracle, the gate configuration, the verdict store, the paired §1.1 mutation, or any other check gating its own work; enforcement is by CAPABILITY (structurally cannot reach), never by INSTRUCTION (told not to reach), following the "least privilege over cleverness" principle (extraction line 1488 [un-resolvable offset per the §11.4.239 CITATION NOTE; MODULE CORRECTED 2026-08-20 — the quoted passage resolves to module 28, not module 27; verified locator `module_28:393`]: "the durable fix is least privilege over cleverness ... the load-bearing move is that the agent chasing the green gate cannot reach the gate, the oracle, or the evidence store"). (A) Four scopes MUST resolve to distinct actors — PRODUCER / ORACLE / GATE / VERIFIER; a two-scope-or-more collapse involving the producer is a §11.4.240 finding; (B) enforcement channels are filesystem scope / branch protection (§11.4.248 PROTECTED-SPEC + CODEOWNERS) / verdict-store isolation / clean-checkout execution isolation — NEVER prompts, policies, or social contracts (extraction line 5715 [un-resolvable offset per the §11.4.239 CITATION NOTE; module 34 attribution VERIFIED CORRECT 2026-08-20, verified locator `module_34:87`] catalogues "we told it in the prompt not to

hardcode" as the shallow / red-flag answer); (C) three forbidden collapses named: PRODUCER=ORACLE (tautological testing), PRODUCER=GATE (weakened thresholds), PRODUCER=VERIFIER (self-certification); (D) composes with the existing verifier layer — §11.4.115(F) machine-written verdicts, §11.4.125/.142/.209 independent review, §11.4.146(D3) status custody, §11.4.135 release verdict coverage, §11.4.165 independent verifier agent, §11.4.245 oracle independence; (E) honest boundary — separation forbids the CLASS of self-certification, does NOT prove verifier correctness (§11.4.107(10) still binds). **(F) Independence-tier vocabulary + Tier-C capability-separation triggers, degrading honestly on a single-uid host (extension, spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26).** Clauses (A)-(E) state the SEPARATION PRINCIPLE in the abstract; this clause supplies the CLOSED VOCABULARY for recording HOW MUCH separation was ACHIEVED, plus the two triggers that raise the required strength — because claiming "capability separation" where only "instance separation" is achievable is itself the theatre this anchor exists to prevent. **(1) Closed tier set:** every accepted verdict's recorded basis of independence MUST name its tier from the CLOSED SET `{instance, model, capability}` — `instance` (a separate agent/process instance sharing no in-flight state with the producer), `model` (a DIFFERENT model), `capability` (a genuine host-level access-control boundary separating the producer's write scope from the verifier's). **(2) Model diversity is PERMITTED, NEVER REQUIRED — the false-positive guard:** a verdict from an independent INSTANCE running the SAME model as the producer MUST be ACCEPTED on that basis alone; a check that refuses same-model independent verdicts is the exact class of false-positive refusal §11.4.201 forbids. This does NOT weaken the constitution's OTHER, narrower model-substrate pins — §11.4.256 (translation review runs on a DIFFERENT model/provider than the translator) and §11.4.209 (every mandatory independent code-review runs on Opus at `xhigh` as PRIMARY substrate, with a genuine-unavailability Sonnet fallback, amended 2026-10-04; Fable and Haiku remain forbidden in every role) both remain in force unmodified for their own domains. **(3) Tier-C trigger 1 — producer holds write access to the evidence/verdict store:** the verdict MUST be written through the MOST RESTRICTIVE separation genuinely available on the host, and the ACHIEVED separation (not the aspired-to one) MUST be reported. **(4) Tier-C trigger 2 — the three named high-blast-radius seams:** at `release-tag` , `qa-deploy`

or `manual-qa-handoff` (or the consumer's declared equivalents per §11.4.35) a producer-authored verdict is REFUSED regardless of the write-access analysis. **(5) No demand for capability outside the two triggers:** at a seam matching NEITHER trigger a separate agent INSTANCE MUST SUFFICE and capability separation MUST NOT be demanded — over-applying Tier-C strength to every seam is itself the same false-positive-refusal violation. **Honest boundary (§11.4.6):** on a SINGLE-UID host capability separation CANNOT be established by any mechanism an agent can set up — ownership defeats access-control lists, the append-only attribute requires privilege the agent lacks, and any key the producer can use it can also read — so such a host reports `instance` HONESTLY and never `capability`, until the operator takes the one host-boundary action agents are correctly forbidden from creating (§11.4.66/§12); the field records WHAT WAS ACHIEVED, never what was ASPIRED to, and honest degradation IS the compliant path. Recommended mechanism gate `CM-INDEPENDENCE-TIER-HONESTLY-RECORDED` + paired §1.1 mutation (record `capability` where no verified host boundary exists → MUST FAIL; refuse a same-model independent verdict at an ordinary seam → MUST FAIL; accept a producer-authored verdict at `release-tag` / `qa-deploy` / `manual-qa-handoff` → MUST FAIL; golden-FALSE per §11.4.201(1): an honest `instance`-tier verdict at an ordinary seam MUST NOT fire it); gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). Classification: universal (§11.4.17). Composes §11.4.6/.20/.70/.92/.107(10)/.115(F)/.125/.134/.135/.142/.146(D3)/.165/.194/.201/.205/.209/.245/ (mutual composes with §11.4.249 four-role + flight-recorder architecture per §11.4.209 reviewer MINOR-1 — §11.4.240 the separation principle, §11.4.249 the four-role taxonomy that instantiates it). Propagation gate `CM-COVENANT-114-240-PROPAGATION` + recommended gates `CM-PRODUCER-NOT-VERIFIER-SCOPE-SPLIT` + `CM-LEAST-PRIVILEGE-CAPABILITY-NOT-INSTRUCTION` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.240. Non-compliance is a release blocker. No escape hatch — no `--producer-may-write-tests`, `--self-certification-OK`, `--exhortation-suffices-as-enforcement`, `--producer-authored-verdict-accepted`, `--capability-limit-optional`, `--policy-document-is-enforcement`, `--single-actor-for-all-scopes` flag.

§11.4.241 — Illegal-state-unrepresentability preference (types > API-shape > lint > property-test > runtime-assert) (research-derived, 2026-08-15). Compact summary: every load-bearing invariant a change relies on MUST land at the STRONGEST enforcement rung the target language/toolchain supports, walking a closed ordered ladder — (1) TYPE SYSTEM (invalid state cannot be constructed: sum types / newtype wrappers / `NonEmpty` / opaque handles), (2) API SHAPE (only exposed constructor path enforces it: private constructors + smart factories, sealed hierarchies, capability tokens), (3) LINT (invalid pattern refused by tree before build), (4) PROPERTY-BASED TEST (invariant asserted across generator-driven input space), (5) RUNTIME ASSERTION (last-line guard raises when invalid state actually forms). Defect PREVENTABLE at rung N caught only at rung > N is a finding. Operator honest boundary 2026-08-15 — rungs 1-2 conditional on toolchain support (mypy present, TypeScript strict, Pydantic used), rungs 3-5 unconditional (every language has a linter + runtime). §1.1 mutation: drop the type constraint / relax `PositiveInt` to `int` / make private constructor public → compiler/type-checker/API gate FAILs to build; if it still builds, the rung was decoration. Composes §11.4.6 / §11.4.108 (pushes enforcement LEFT of Layer 1) / §11.4.115 / §11.4.142 / §11.4.194 / §11.4.201 / §11.4.224. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-241-PROPAGATION` + recommended gate `CM-ILLEGAL-STATE-LADDER-PREFERENCE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.241. Non-compliance is a release blocker. No escape hatch — no `--runtime-assert-suffices`, `--skip-type-preference`, `--dynamic-language-exempts-all-rungs`, `--property-test-in-lieu-of-type`, `--decorative-type-annotation-OK` flag.

§11.4.242 — Bisection-not-blame regression cause-finding + failure modes (research-derived, 2026-08-15). Compact summary: regression localisation FIRST-choice mechanism is BISECTION (`git bisect`, O(log N)) over the range from the last known-good tag (§11.4.114) to current HEAD, NOT blame-log walking (O(N) linear) NOR speculative root-cause hunting (unbounded). Bisection produces the introducing commit as CAPTURED EVIDENCE (§11.4.5) feeding §11.4.102 systematic-debugging's root-cause phase with the actual change in hand. MONOTONE-SEARCH INVARIANT: bisection valid only when observable is monotone across the range (broken stays broken); non-monotone (flaky, environment-dependent, timing-race)

MUST stabilise first — bisection over flaky signal produces WRONG "guilty" commit with same confidence as correct one. FAILURE-MODE CATALOGUE (closed): (1) flaky/non-monotone observable, (2) build-broken commits (mistaken for "bad", use `bisect skip`), (3) merge-commit introductions (defect in conflict resolution not either parent), (4) external/environmental changes (upstream dep, OS update), (5) delayed-fuse introductions (latent commit + later trigger), (6) test-infra regressions masquerading as product regressions. `git bisect run` script MUST use exit-code discipline (0=good, 1-124=bad, 125=skip); mapping every non-zero to "bad" collapses the taxonomy. Composes §11.4.5 / §11.4.6 / §11.4.69 / §11.4.102 / §11.4.114 / §11.4.115 / §11.4.201 / §11.4.224. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-242-PROPAGATION` + recommended gate `CM-BISECTION-BEFORE-BLAME` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.242. Non-compliance is a release blocker. No escape hatch — no `--blame-log-suffices`, `--skip-bisection`, `--flaky-observable-bisect-anyway`, `--run-script-exit-code-collapse-OK`, `--regression-guess-without-localisation` flag.

§11.4.243 — Characterization / golden-master safety net before behaviour-changing modification of legacy (research-derived, 2026-08-15). Compact summary: before ANY behaviour-changing modification to code that qualifies as legacy (lacks tests asserting observable behaviour, or written spec diverges from actual, or predates the constitution's TDD discipline, or the modifier cannot describe its behaviour from reading source), the modifier MUST FIRST author CHARACTERIZATION TESTS — tests asserting the CURRENT observable behaviour as GROUND TRUTH whether "correct" or not — and land them as a green baseline. Only AFTER the safety net exists may the modification proceed; characterization tests then act as the oracle catching unintended behaviour changes. Pattern originates in Feathers's WORKING EFFECTIVELY WITH LEGACY CODE + generalises GOLDEN-MASTER (record outputs across representative input space; assert future runs match). Characterization is DESCRIPTIVE not prescriptive — it captures WHAT IS not WHAT SHOULD BE; if current behaviour is subtly wrong, ASK operator per §11.4.66 whether modification should preserve or correct it (never silently). Composes §11.4.5 / §11.4.6 / §11.4.66 / §11.4.108 / §11.4.115 (RED-first — characterization is legacy counterpart) / §11.4.118 (sampled input space = honest gaps, not completeness) / §11.4.124 (investigate-before-remove — parallel discipline) /

§11.4.201 / §11.4.224 (test-first backfilled for legacy). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-243-PROPAGATION` + recommended gate `CM-CHARACTERIZATION-BEFORE-LEGACY-MODIFICATION` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.243. Non-compliance is a release blocker. No escape hatch — no `--modify-untested-legacy-directly`, `--skip-characterization`, `--claim-tests-not-needed-because-obvious`, `--decorative-assertion-suffices`, `--refactor-without-safety-net` flag.

§11.4.244 — Cross-boundary contract tests + can-i-deploy gate (research-derived, 2026-08-15). Compact summary: changes modifying a boundary between independently-deployable components (service↔consumer, library↔dependents, DB schema↔readers/writers, message topic↔subscribers) MUST be covered by CONSUMER-DRIVEN CONTRACT TESTS on both sides AND a `can-i-deploy` gate MUST verify proposed provider version compatibility against every currently-deployed consumer version (and vice versa) BEFORE the change ships. CDC pattern: consumers publish assumptions they make about the provider (Pact / Spring Cloud Contract / gRPC schema-evolution); providers verify each consumer's contract against actual implementation; provider change breaking a live consumer's contract fails provider's build. CAN-I-DEPLOY: at deploy attempt, gate queries broker for compatibility matrix (P × C1 × C2 × C3); single red cell BLOCKS deploy naming incompatible pair, NEVER warning-with-continue; MISSING contract = first-class refusal reason (§11.4.201, not implicit "assume compatible"). SemVer becomes MEASURABLE: PATCH bump breaking contract = mis-labelled version. Backward-compatibility window declared per §11.4.35 (support current + previous N minor versions, deprecate with M weeks notice); consumers outside window flagged for §11.4.66 operator decision. §11.4.244 MERGES what the task brief labelled contract-tests + can-i-deploy — they are useless in isolation. Composes §11.4.5 / §11.4.6 / §11.4.35 / §11.4.66 / §11.4.108 (per-side four-layer) / §11.4.135 (contract tests as permanent regression guards) / §11.4.185 (composed-system manual QA still needed) / §11.4.201 (gate honesty). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-244-PROPAGATION` + recommended gates `CM-CONTRACT-TESTS-BOTH-SIDES` + `CM-CAN-I-DEPLOY-GATE-WIRED` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.244. Non-compliance is a release blocker. No

escape hatch — no `--skip-contract-tests`, `--deploy-without-compatibility-check`, `--assume-compatible-when-broker-silent`, `--patch-may-break-contract`, `--contract-tests-optional-for-internal-boundaries` flag.

§11.4.245 — Oracle-problem-first test authoring: identify the oracle BEFORE authoring the test (research-derived, 2026-08-15). Compact summary: every test MUST FIRST identify its ORACLE — the mechanism deciding PASS vs FAIL for the input it drives — and the oracle MUST be DIFFERENT from the code under test AND from the test's own assertion mechanism. Authoring without explicit oracle identification produces the "test agrees with code" anti-pattern (test asserts what code happens to produce, so it passes as long as code produces something, never catches wrong-output). ORACLE STRATEGY CLOSED SET, each test's oracle NAMED in source (comment/docstring/tag): (1) SPECIFIED (spec/RFC/standard names expected output), (2) DERIVED (independent reference implementation), (3) METAMORPHIC (metamorphic relations between related inputs), (4) GOLDEN-MASTER / CHARACTERIZATION (past captured behavior — §11.4.243), (5) INVARIANT (universal property — sort output sorted + permutation of input; serialize-deserialize round-trip = original), (6) STATISTICAL (distribution properties for stochastic outputs, never specific value), (7) HUMAN (§11.4.185 manual QA). NO other classes. STRUCTURAL INDEPENDENCE: oracle's SOURCE of expected value must not depend on code being tested — different implementation / spec / previous run / mathematical property / inverse operation / human. Reviewer per §11.4.194 verifies (i) NAMED strategy, (ii) structural independence, (iii) paired §1.1 mutation flips the oracle. Composes §11.4.5 / §11.4.6 / §11.4.107(10) (self-validated analyzers — analyzer-as-oracle) / §11.4.115 / §11.4.194 / §11.4.201 / §11.4.224 / §11.4.243. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-245-PROPAGATION` + recommended gate `CM-ORACLE-STRATEGY-NAMED-AND-INDEPENDENT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.245. Non-compliance is a release blocker. No escape hatch — no `--test-agrees-with-code`, `--skip-oracle-identification`, `--oracle-is-the-code`, `--assertion-is-oracle`, `--implicit-oracle-OK` flag.

§11.4.246 — Reproducible + hermetic builds + supply-chain integrity at SLSA Build Level 2 as fleet-wide minimum (operator decision, 2026-08-15). Compact summary: every consuming project's build pipeline MUST produce REPRODUCIBLE + HERMETIC artifacts and meet SLSA Build Level 2 as fleet-wide MINIMUM (operator decision 2026-08-15 — L3+ tracked as backlog per §11.4.197). REPRODUCIBLE = same source at same time on any conformant host produces byte-identical artifacts (or diff-set fully accounted for by declared non-determinisms pinned to canonical values). HERMETIC = no undeclared inputs (every dep version-pinned + lockfile-hashed, every tool version declared, no unpinned public-network reach at build, no host-env leakage). SLSA-L2 = signed provenance describing source URI + commit hash + builder identity + invocation + artifact hash, on hosted tamper-resistant build platform (boba already satisfies ancillary: SSH-only §2.1, rootless podman §11.4.161, fixed-tag pins §11.4.30/§11.4.77). Every consumer MAINTAINS `docs/security/SLSA_LEVEL.md` (or equivalent per §11.4.35) recording CURRENT L level with cited evidence + TARGET L + tracked §11.4.197 upgrade items — CURRENT below L2 without tracked item is violation. SUPPLY-CHAIN EXTENSION: deps either vendored (checked in hash-verified), from provenance-attested source (npm/PyPI/crates.io as they add SLSA), OR mirrored through hash-pinning local registry — an unattested public source is integrity risk. Threat model NARROW: covers source→artifact chain, does NOT cover malicious commit landing in source (§11.4.142 review) nor compromised dep source (supply-chain extension) nor bug-free artifact (§11.4.108 four-layer + §11.4.185 manual QA). Composes §2.1 / §11.4.6 / §11.4.30 / §11.4.35 / §11.4.108 / §11.4.113 / §11.4.135 / §11.4.142 / §11.4.161 / §11.4.184 / §11.4.185 / §11.4.197 / §11.4.209. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-246-PROPAGATION` + recommended gates `CM-BUILD-REPRODUCIBILITY` + `CM-BUILD-HERMETICITY` + `CM-SLSA-L2-PROVENANCE-VERIFIED` + `CM-SUPPLY-CHAIN-DEPS-PROVENANCE-ATTESTED` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.246. Non-compliance is a release blocker. No escape hatch — no `--skip-reproducibility`, `--allow-network-during-build`, `--no-provenance`, `--slsa-optional`, `--below-l2-OK`, `--unpinned-dependency-OK`, `--host-env-may-leak-into-build` flag.

§11.4.247 — Composite-output layer-move must move every layer (research-derived, 2026-08-15). Compact summary: whenever a change MOVES a system-boundary layer (service migrates hosts, endpoint URL changes, API version bumps, DB schema shifts, message topic renames, ingress path re-routes, storage bucket moves), the change MUST move EVERY LAYER of the composite output atomically or via explicit compatibility window — NEVER piecemeal. Recurring failure: endpoint moves `/v1/foo` → `/v2/foo`; server handler + client call-site moved; load balancer + CDN + API gateway + OpenAPI spec + SDK stubs + integration tests + metrics + traces + logs + dashboards + alerts + docs still reference `/v1/foo`; operator declares victory; residuals silently break when `/v1/foo` shim retires. LAYER TAXONOMY per §11.4.35 baseline: (i) server (handler + route + OpenAPI/ proto/gRPC descriptor), (ii) client (call-sites + SDK stubs + integration tests), (iii) infrastructure (LB + ingress + API gateway + CDN + service-mesh + DNS), (iv) data stores (cached keys + DB URL rows + MQ subscriptions), (v) observability (metric labels + span names + log fields + dashboards + alerts + SLOs), (vi) documentation (user + dev + runbook + README + cookbook), (vii) tests (integration + e2e + contract per §11.4.244 + load + smoke). Consumer provides `scripts/audit/layer_move_completeness.sh` reporting residual references delta; change proposing move MUST show ZERO residuals OR explicit compatibility-window plan per residual with tracked §11.4.197 items. COMPATIBILITY-WINDOW: old value keeps working alongside new until stated date/version/event, deprecation warnings named per caller, tracked deadline, clean removal at expiry — no "clean up later." Composes §11.4.6 / §11.4.35 / §11.4.111 / §11.4.118 / §11.4.124 / §11.4.135 / §11.4.191 / §11.4.197 / §11.4.238 (residual = coverage escape) / §11.4.244. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-247-PROPAGATION` + recommended gate `CM-LAYER-MOVE-COMPLETENESS-CHECK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.247. Non-compliance is a release blocker. No escape hatch — no `--partial-layer-move-OK`, `--skip-completeness-check`, `--residuals-cleaned-up-later`, `--observability-layer-optional`, `--doc-move-can-lag` flag.

§11.4.248 — Flaky-test quarantine + protected-regression-spec gate via `[PROTECTED-SPEC: ATM-NNN]` tag + CODEOWNERS required reviewer (operator decision, 2026-08-15). Compact summary: flaky tests are the

SILENT ANTI-BLUFF DECAY vector (every flake conditions team to ignore red until real regression is dismissed as "probably flaky, re-run"). Two disciplines bound: (A) QUARANTINE — mechanical detection (test-history aggregator flags two-consecutive-run-different-verdict OR rerun-after-red pattern) moves flaky test to `tests/quarantine/` (per §11.4.35), opens tracked §11.4.197 stabilisation deadline, blocking suite runs WITHOUT quarantined test so green means green; deadline past = §11.4.66 extension or violation; (B) PROTECTED-REGRESSION-SPEC — regression tests codifying known-broken defect fixes (per §11.4.135 permanent-guard suite) MAY be tagged `# [PROTECTED-SPEC: ATM-NNN]` (target-language comment equivalent) referencing the workable-item ATM-NNN (§11.4.54) they guard, marking test as load-bearing regression guard whose modification requires elevated review. Tag applied deliberately per test (not blanket — that voids the discipline). CODEOWNERS-REQUIRED-REVIEWER GATE: consuming project's CODEOWNERS declares modifications to `[PROTECTED-SPEC]`-tagged tests OR `regression/**/*` path scope require designated required-reviewer approval BEFORE merge; composes with §2.1 SSH + §11.4.113 no-force-push (gate cannot be silently bypassed); audit trail in PR review thread + commit history. Gate CATCHES change so reviewer distinguishes STRENGTHEN (approve) vs FIX-DEFECT-IN-TEST (approve after verifying ATM-NNN's defect not silently re-introduced) vs WEAKEN/REMOVE (refuse or §11.4.66 escalate) — makes modification VISIBLE not preventable. Composes §2.1 / §11.4.6 / §11.4.35 / §11.4.50 (ideal empty quarantine) / §11.4.54 / §11.4.66 / §11.4.113 / §11.4.115 / §11.4.135 / §11.4.197 / §11.4.226. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-248-PROPAGATION` + recommended gates `CM-FLAKY-TEST-QUARANTINE-WIRED` + `CM-PROTECTED-REGRESSION-SPEC-GATE` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.248. Non-compliance is a release blocker. No escape hatch — no `--rerun-until-green`, `--quarantine-permanent`, `--protected-spec-review-optional`, `--codeowners-skip`, `--blanket-protect-everything`, `--tag-without-atm-id` flag.

§11.4.249 — Producer ≠ oracle ≠ gate ≠ verifier + flight recorder (research-derived, 2026-08-15). Compact summary: every load-bearing quality mechanism has FOUR distinct roles that MUST be structurally separate — (1) PRODUCER (code computing/emitting/building output), (2) ORACLE

(mechanism deciding correctness per §11.4.245), (3) GATE (seam consuming oracle verdict to allow/block/proceed), (4) VERIFIER (mechanism auditing gate's decision — paired §1.1 mutation proving gate can refuse a bad output). Collapsing any two produces specific bluff class: producer=oracle (§11.4.245 oracle-agrees-with-code); oracle=gate (refuses to say "cannot decide" defaults to allow, §11.4.201(6) FALSE-NULL); gate=verifier (self-auditing gate — §11.4.142 self-review anti-pattern applied). FLIGHT RECORDER: mechanism CAPTURING producer inputs + oracle verdict + gate decision + verifier audit as durable replayable artifact (structured JSONL event stream per §11.4.116, content-addressed per §11.4.207) so post-hoc "why did this go through" has evidence not shrug. Recorder is NOT the gate — a gate reading its own recorder to decide collapses gate + verifier + recorder. DETECTABLE COLLAPSES reviewer per §11.4.194 scans for: (1) producer=oracle (test's assertion calls code being tested), (2) oracle=gate (no "cannot decide" branch, defaults PASS), (3) gate=verifier (meta-test IS the gate — mutation making gate always-pass ALSO makes meta-test always-pass; meta-test MUST live OUTSIDE gate), (4) recorder=decision-substrate (gate uses recorder contents as decision input — precedent-based gating not authorised). Composes §11.4.5 / §11.4.6 / §11.4.107(10) / §11.4.116 / §11.4.142 / §11.4.194 / §11.4.201 / §11.4.207 / §11.4.240 (producer↔verifier separation principle — the WHY behind this four-role architecture) / §11.4.245. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-249-PROPAGATION` + recommended gates `CM-ROLE-SEPARATION-DECLARED` + `CM-FLIGHT-RECORDER-CAPTURED` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.249. Non-compliance is a release blocker. No escape hatch — no `--collapse-producer-oracle`, `--gate-self-audits`, `--skip-flight-recorder`, `--oracle-defaults-pass`, `--decision-without-recorded-rationale` flag.

§11.4.250 — Heuristic-tower signals a primitive defect: when N heuristic layers stack to compensate, the primitive is broken (research-derived, 2026-08-15). Compact summary: codebase accumulating TOWER of heuristics compensating for same underlying issue (retry-with-backoff + connection-pool-warmup + DNS-cache-flush + health-check-with-startup-grace — all guarding one flaky DB connection) IS the diagnostic: PRIMITIVE is defective, every layer papers over a manifestation without fixing cause. Recognition signals: (i) each layer justification "we added because sometimes X happens" — reactive, (ii)

removing any layer INCREASES failures BUT keeping all shows occasional residual, (iii) no single layer owns responsibility, (iv) layers added at different times by different authors per specific incidents, (v) monotone layer-count growth over time — debugging session for residual adds layer N+1 rather than fixing 1..N. DISTINCT from defense-in-depth: defense-in-depth is deliberate + each layer targets DIFFERENT failure class + designed together not accreted + removing layer produces SPECIFIC different failure not "same failure more often". SYSTEMATIC-DEBUGGING RESPONSE on tower depth ≥ 2: (1) STOP adding layers, (2) INVOKE §11.4.102 systematic-debugging on PRIMITIVE, (3) proceed root-cause / pattern / hypothesis / implementation phases, (4) fix primitive, (5) REMOVE layers one at a time verifying with §11.4.5 evidence, (6) commit each removal per §11.4.124 with git-history evidence citing primitive fix. Composes §11.4.5 / §11.4.6 / §11.4.102 / §11.4.124 / §11.4.194 / §11.4.201 (outer layer masks §11.4.201(6) false-null) / §11.4.226. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-250-PROPAGATION` + recommended gate `CM-HEURISTIC-TOWER-TRIGGERS-PRIMITIVE-DEBUG` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.250. Non-compliance is a release blocker. No escape hatch — no `--add-another-heuristic-layer`, `--skip-primitive-debug`, `--tower-is-defense-in-depth-without-justification`, `--reactive-mitigation-OK`, `--layer-per-incident` flag.

§11.4.251 — Byte-identical-fork prohibition + role-as-data-pack (research-derived, 2026-08-15). Compact summary: when project needs would be met by TWO NEAR-IDENTICAL CODE COPIES differing only in configuration / data / role-specific parameters (prod + staging with hardcoded hostnames; reader + writer with different flag values; web + mobile with different feature flags), copies MUST NOT be maintained as parallel forks. Instead ONE code copy retained + role-specific values extracted into DATA PACK (role-config file / env var set / capability descriptor / feature-flag bundle) consumed by the one code to instantiate as either role. Anti-pattern: FORK-AND-DIVERGE (copies drift as one patched not other, until drift = defect surface no one owns). DETECTION: reviewer or mechanical scan finds byte-identical fork when `diff` shows every difference is a config value (hostname/port/flag/label/env/credential-ref) NOT behavioral, common git-history ancestor, > 90% byte identity; per §11.4.35 threshold-DATA. EXTRACTION: (1) IDENTIFY configuration surface (closed value set differing), (2) EXTRACT to data pack

per role, (3) REFACTOR single code copy to consume at instantiation (constructor injection / factory / config-loader-at-startup), (4) DELETE one fork, (5) VERIFY per §11.4.5 both role instantiations produce same observable behavior their forks previously produced (§11.4.243 characterization before refactor), (6) commit extraction per §11.4.124 with git-history evidence. LEGITIMATE FORKS: temporary migration (tracked §11.4.197 end date), experimental / A-B (bounded end date), truly divergent behavior (copies differ in algorithm/protocol/data-model not just config). Composes §11.4.5 / §11.4.6 / §11.4.28 (decoupling — role-configurable code shared across consumers) / §11.4.35 (data-pack location) / §11.4.124 / §11.4.177 / §11.4.194 / §11.4.197 / §11.4.243. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-251-PROPAGATION` + recommended gate `CM-NO-BYTE-IDENTICAL-FORK` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.251. Non-compliance is a release blocker. No escape hatch — no `--fork-for-role-OK`, `--skip-data-pack-extraction`, `--copy-and-modify-config`, `--duplicate-code-per-environment`, `--near-identical-forks-are-fine` flag.

§11.4.252 — Fail-closed-on-dangerous-combination for mutating/credential surfaces (research-derived, 2026-08-15). Compact summary: every code path COMBINING ≥ 2 dangerous capabilities MUST FAIL CLOSED — refuse operation unless every capability's precondition verifiably satisfied — NEVER fail open on ambiguous/missing/unresolvable input. Security-engineering generalisation of §11.4.201 guard-honesty applied to code paths where false-negative blast radius dwarfs operator inconvenience of false-positive. DANGEROUS-COMBINATION TAXONOMY: (1) MUTATION of shared resource (DB / filesystem / container state / config), (2) UNTRUSTED INPUT (user string / network bytes / external env var / unverified file), (3) CREDENTIAL ACCESS (secret read / authenticated API call / signing key use), (4) EXTERNAL SIDE EFFECT (email / webhook / MQ publish / payment endpoint), (5) SHELL / EXEC (subshell / binary with user-influenced args / template eval with user values), (6) IRREVERSIBLE (delete / drop / truncate — §11.4.113 force-push already absolutely forbidden). Any path combining ≥ 2 = dangerous-combination, MUST fail closed. INVARIANT: (1) VERIFY every precondition (authorization + input shape + credential presence + target identity), (2) REFUSE when ANY unverifiable with ERROR EXIT naming specific unresolved precondition, (3) EMIT captured evidence per §11.4.5 of

refusal (log reason + resolved inputs + unresolved precondition + caller identity), (4) NEVER default to allow/proceed/retry-with-relaxed on ambiguity — §11.4.101 conservative-safe default = REFUSE. FAIL-OPEN ANTI-PATTERNS refused: `catch { /* ignore */ }`, `credential = credential || default`, `if (!valid) { log("warn"); return proceed(input); }`, `target = user_input || default_target`, unbounded `retry { dangerous(); }`. Distinct from graceful degradation (cache-miss returning cache-miss response is legitimate — NOT dangerous combination). Composes §11.4.5 / §11.4.6 / §11.4.10 (credentials specialisation) / §11.4.66 (operator escalation when refuse stuck) / §11.4.101 / §11.4.113 (max instantiation) / §11.4.194 / §11.4.201. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-252-PROPAGATION` + recommended gate `CM-DANGEROUS-COMBINATION-FAIL-CLOSED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.252. Non-compliance is a release blocker. No escape hatch — no `--fail-open-on-error`, `--swallow-exception`, `--default-missing-credential`, `--proceed-on-ambiguous-input`, `--retry-until-succeeds` flag.

§11.4.253 — Idempotency under retry + DB-level durable uniqueness guard (research-derived, 2026-08-15). Compact summary: every retryable operation (network calls with auto-retry, at-least-once MQ consumers, cron with overlapping schedules, HTTP endpoints with client-retry, DB transactions with retry-on-serialisation-failure, deploy pipelines with rerun capability) MUST BE IDEMPOTENT: N executions produce same observable end state as one. Idempotency ENFORCED at LOWEST LAYER where "same end state" is durably provable — typically DATABASE UNIQUENESS CONSTRAINT on caller-supplied idempotency key, NEVER only application-layer check-then-insert (races). Prevents DOUBLE-CHARGE / DOUBLE-EMAIL / DOUBLE-DELETE / DOUBLE-EFFECT bugs. IDEMPOTENCY-KEY DISCIPLINE: every retryable op accepts (or generates + passes forward) unique identifier naming THIS logical op; persistence layer has UNIQUE INDEX / CONSTRAINT on key column; second attempt REFUSED by DB at durable level not application logic reading-then-writing; refusal caught by caller as "duplicate — already succeeded" = SUCCESS (op happened, no further retry). ANTI-PATTERNS: (i) check-then-insert without unique index races, (ii) app-level dedup table read without spanning-transaction, (iii) key scoped to app instance (in-memory not shared), (iv) effect persisted BEFORE key (crash between = double-effect on

retry), (v) no key at all + "the effect is 'set X to V' so redoing is fine" (safe for pure sets, NOT for INSERTs / MQ emissions / external-system side effects), (vi) idempotency-key expires — converts to time-bounded-idempotent = §11.4.6 mis-labelling if caller expects unbounded. CHAOS VERIFICATION per §11.4.85: (1) SIMULTANEOUS RETRIES (same key from 2 concurrent callers), (2) MID-OPERATION CRASHES (kill receiver between "check exists" and "insert effect"), (3) RETRY AFTER APPARENT SUCCESS (caller believes fail, receiver persisted, retry MUST recognise duplicate). External-system side effects (Stripe/PayPal charging card + local DB write): idempotency key MUST pass through to external; if external does not support, op is not fully idempotent — label honestly per §11.4.6. Composes §11.4.5 / §11.4.6 / §11.4.85 / §11.4.201 / §11.4.252. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-253-PROPAGATION` + recommended gates `CM-IDEMPOTENCY-DB-LEVEL-UNIQUE-GUARD` + `CM-IDEMPOTENCY-CHAOS-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.253. Non-compliance is a release blocker. No escape hatch — no `--check-then-insert-OK`, `--application-dedup-suffices`, `--skip-idempotency-chaos-test`, `--claim-idempotent-without-key`, `--expire-idempotency-marker-silently` flag.

§11.4.254 — Boot-time invariant assertion + capability matrix (research-derived, 2026-08-15). Compact summary: every service / process / long-running executable MUST assert BOOT-TIME INVARIANTS at startup — required env vars present + valid; required config files parseable + declared schema version matches; required deps reachable; required credentials present + not-expired; required schema migrations applied; required feature-flag states declared; required host resources present (disk / memory / thread limits per §12.12 / port availability) — and FAIL FAST + LOUD if any violated, BEFORE accepting request / performing work. Prevents "service silently starts with broken invariant + serves requests that fail deep in request path hours after boot." INVARIANT CLASSES: (i) environment, (ii) configuration, (iii) dependencies, (iv) credentials, (v) schema, (vi) capabilities, (vii) host. FAIL-FAST-LOUD: (1) EMIT structured error to stderr + captured log naming specific failed invariant (env var name / config path / dep URL / credential file / schema version), (2) EXIT non-zero (traditionally 78 EX_CONFIG for config, consumer supplies per §11.4.35), (3) DO NOT accept requests / begin main event loop / bind listening port. Service booting partially + serving requests-that-will-fail is

worse than refusing to start — partial-boot masquerades healthy long enough for load balancer to route traffic. Dual to §11.4.252 fail-closed on dangerous combinations: both refuse to proceed on unresolvable preconditions. CAPABILITY MATRIX: consumer's `docs/services/CAPABILITY_MATRIX.md` (per §11.4.35) declares per service (i) boot-time invariants, (ii) runtime capabilities OFFERED (endpoints / topics / callable functions), (iii) runtime capabilities REQUIRED from other services. Regenerated / verified on every commit per §11.4.106 docs-chain + §11.4.86 fingerprint. Undeclared endpoints emitted = violation; silent upstream deps not declared = violation (invisible to §11.4.244 contract testing). Composes §11.4.5 / §11.4.6 / §11.4.35 / §11.4.86 / §11.4.106 / §11.4.108 Layer 3 (boot IS runtime-on-clean-target check) / §11.4.128 / §11.4.244 (contract tests consume matrix) / §11.4.252 / §12.12. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-254-PROPAGATION` + recommended gates `CM-BOOT-TIME-INVARIANT-ASSERTED` + `CM-CAPABILITY-MATRIX-PRESENT-AND-IN-SYNC` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.254. Non-compliance is a release blocker. No escape hatch — no `--skip-boot-time-assertion`, `--boot-partial-OK`, `--default-missing-env-silently`, `--capability-matrix-optional`, `--service-with-undeclared-dependency-OK` flag.

§11.4.257 — Comprehensive user-manual + guide + FAQ coverage: every component / service / feature ships an operator-usable manual, task-oriented guide, and FAQ (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Compact summary: every consuming project MUST cover the whole project (every component, service, subsystem, binary, container, plugin, feature, user-visible workflow) with three cooperating documentation classes — (a) complete USER MANUAL per component/service/product, (b) task-oriented GUIDES for every user-facing workflow, (c) properly-created FAQs derived from real operator/QA/end-user questions — always in sync (§11.4.44/§11.4.106), always exported (§11.4.65), always reachable from README (§11.4.212), always tracked when incomplete (§11.4.197); a capability that ships without its manual+guide+FAQ trio is a §11.4.257 violation of §11.4 PASS-bluff severity at the documentation layer. GENERALISES §11.4.18 + §11.4.153 to whole-project doc coverage; BINDS §11.4.212 / §11.4.106 / §11.4.65 / §11.4.44 / §11.4.99 / §11.4.194 / §11.4.238 into the always-in-sync guarantee. Honest boundary §11.4.6 — mandates

existence + sync + reachability, not doc semantic-correctness (that stays §11.4.99/§11.4.194/§11.4.238). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-257-PROPAGATION` + recommended gates `CM-USER-MANUAL-PER-COMPONENT` / `CM-GUIDE-PER-USER-WORKFLOW` / `CM-FAQ-FROM-REAL-QUESTIONS` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.257. Non-compliance is a release blocker. No escape hatch — no `--skip-user-manual`, `--guide-later`, `--faq-optional`, `--capability-without-docs-OK`, `--fake-doc-placeholder-OK`, `--source-of-truth-is-code-comments` flag.

§11.4.258 — Architectural + data-flow + state-machine + sequence-diagram coverage: every project ships accurate embedded diagrams across four canonical classes (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Compact summary: every consuming project MUST cover the whole project with properly created graphs/diagrams/schemes across FOUR canonical classes — (a) SYSTEM ARCHITECTURE (services, containers, ports, deployment topology), (b) DATA FLOW (information movement between components/storage/external), (c) STATE MACHINE (lifecycle of every stateful entity: workable items §11.4.15, long-ops §11.4.232, sessions §11.4.187, agents §11.4.147, deploys §11.4.108, builds), (d) SEQUENCE (per critical user workflow AND per critical cross-component protocol) — INCORPORATED at the point of use into the existing doc surface (README/manuals/guides/Status docs), NEVER an orphan gallery; every diagram open-format-diffable (Mermaid/PlantUML/D2/diffable SVG per §11.4.220), rendered non-blank and non-degenerate (§11.4.107(10) analyzer), reachable from README (§11.4.212), synced via §11.4.106 (a code change invalidating a diagram REFUSES commit); prefer derived-from-SSoT diagrams over hand-drawn where possible. STRENGTHENS §11.4.218/§11.4.219 (design-side) to whole-project doc coverage. Honest boundary §11.4.6 — mandates existence + accuracy + embedding + honest render; does not require every class for a project that lacks the class's subject. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-258-PROPAGATION` + recommended gates `CM-ARCHITECTURE-DIAGRAM-PRESENT-ACCURATE` / `CM-DATAFLOW-DIAGRAM-PER-SERVICE` / `CM-STATEMACHINE-DIAGRAM-PER-STATEFUL-ENTITY` / `CM-SEQUENCE-DIAGRAM-PER-CRITICAL-WORKFLOW` / `CM-DIAGRAM-EMBEDDED-NOT-ORPHAN` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.258. Non-compliance is a

release blocker. No escape hatch — no `--skip-architecture-diagram`, `--diagram-in-orphan-directory`, `--proprietary-master-only`, `--stale-diagram-OK`, `--dataflow-implied-from-code`, `--sequence-later` flag.

§11.4.259 — README quality-status badge row: comprehensive badges with a closed green→amber→red vocabulary at the top of README, each machine-derived, production-readiness gauge summarising deploy-readiness (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Compact summary: every consuming project's README MUST carry, at the TOP (immediately below the H1, above the introduction — reader's first visual signal), a comprehensive BADGE ROW: closed color vocabulary GREEN (ok) / AMBER (attention) / RED (not ok) / GRAY (N/A with reason), MINIMUM class floor covers build / test-breadth (§11.4.27 seven types) / code coverage (§11.4.224) / security posture (§11.4.184 + supply-chain) / documentation completeness (§11.4.257) / diagram completeness (§11.4.258) / live health / open defects (§11.4.15 with most-reopened highlighted §11.4.189) / supply-chain integrity level / zero-shortcomings verdict (§11.4.261) / machine-evidence coverage (§11.4.262) / PRODUCTION-READINESS GAUGE (composite; GREEN only when every clause and every §11.4.185 blocker clear; BINDING on release); every badge MACHINE-DERIVED (§11.4.6 no hand-typed colors), provenance embedded in tooltip or `docs/BADGES.md` (§11.4.86-fingerprint-driven sync via §11.4.106); badge-computer self-validated golden-good/golden-bad/negative-control per §11.4.107(10)/§11.4.201; stale = §11.4.229 violation. EXTENDS §11.4.57 with a symmetric badge section at the same first-visible-position privilege. Honest boundary §11.4.6 — mandates presence + machine-derivation + closed vocabulary + production gauge; does not claim green = bug-free (necessary-not-sufficient); GENUINELY N/A = GRAY with reason (silence-as-badge is §11.4.201(6) FALSE-NULL). Classification: universal (§11.4.17) — palette/placement/vocabulary/derivation universal; specific per-project badge set = consumer §11.4.35 DATA. Propagation gate `CM-COVENANT-114-259-PROPAGATION` + recommended gates `CM-README-BADGE-ROW-AT-TOP` / `CM-BADGE-CLOSED-COLOR-VOCABULARY` / `CM-BADGE-MACHINE-DERIVED-SOURCE` / `CM-PRODUCTION-READINESS-GAUGE` / `CM-BADGE-SELF-VALIDATED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.259. Non-compliance is a release blocker. No escape hatch — no `--badges-optional`, `--hand-type-color`, `--skip-

`production-gauge`, `--badge-not-derived-from-source`, `--silent-omit-red-badge`, `--badge-in-footer-OK` flag.

§11.4.260 — Cutting-edge enterprise quality + production-deployment readiness: every work product built for production + max stability + zero nasty surprises (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Compact summary: every work product MUST be built for CUTTING-EDGE ENTERPRISE QUALITY targeted at PRODUCTION DEPLOYMENT + MAXIMAL STABILITY + ZERO NASTY SURPRISES — standing default from first prompt (§11.4.126/§11.4.198/§11.4.183/§11.4.231), never "good enough" / "will do for now" / "MVP-first". "Cutting-edge" sourced via §11.4.99+§11.4.150+§11.4.8, never novelty-for-its-own-sake, genuine gaps honestly tracked (§11.4.197+§11.4.223 markers). Closed PRODUCTION-READINESS INVARIANT SET (10 clauses) every change satisfies: functional completeness (§11.4.108/§11.4.124), observability (§11.4.85/§11.4.128), resilience (§11.4.85 chaos), security (§11.4.184/§11.4.10 + fleet supply-chain minimum), performance (§11.4.85 measured), maintainability (§11.4.209 Opus-xhigh review → §11.4.134 GO), deployability (§11.4.161/§11.4.108/§11.4.200), documentability (§11.4.257/§11.4.258/§11.4.259 GREEN), testability (§11.4.27/§11.4.169 breadth + §11.4.224 floor + §1.1 mutations + §11.4.135 guards), operational runbook (§11.4.257 guide class). Zero nasty surprises: no known-broken ships (§11.4.238), no silenced gate (§11.4.120/§11.4.201), no disabled test w/o tracked follow-up (§11.4.197), no workaround-as-fix (§11.4.34), no flakiness tolerance (§11.4.50/§11.4.85), no silent-failure try/catch/swallow. `docs/PRODUCTION_READINESS.md` tracks per-component × invariant × evidence-source (§11.4.153 sibling); is the SoT for the §11.4.259 production gauge. STRENGTHENS §11.4.190 (web-only) to universal. Honest boundary §11.4.6 — mandates posture + invariants + tracker; does not claim provably-bug-free (§11.4.118 discovery-pressure remains); unmet invariant honestly red-badged, blocks release; genuine research-frontier gaps honestly documented as §11.4.8 original work. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-260-PROPAGATION` + recommended gates `CM-PRODUCTION-READINESS-TRACKER-PRESENT` / `CM-CUTTING-EDGE-POSTURE-ALWAYS-ON` / `CM-ZERO-NASTY-SURPRISES-AUDIT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.260. Non-compliance is a release blocker. No escape hatch — no `--mvp-quality-OK`,

`--good-enough`, `--will-fix-later`, `--skip-production-readiness-tracker`, `--production-readiness-optional`, `--acceptable-flakiness`, `--for-now-implementation` flag.

§11.4.261 — Zero shortcomings / gaps / weak spots / danger zones invariant with mechanical audit ratchet: standing invariant, monotone-decreasing findings ledger (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Compact summary: every consuming project MUST hold at ALL TIMES exactly ZERO shortcomings, gaps, weak spots, or danger zones across code / services / components / gates / docs / config — enforced by a MECHANICAL AUDIT SWEEP script inventorying findings from a CLOSED VOCABULARY (10 classes: shortcomings, gaps, weak spots, danger zones, TODOs/FIXMEs/"for now"/placeholders/dead-code §11.4.124, skipped tests w/o SKIP-OK, bluffs, unresolved §11.4.197 items, divergent/stale/orphan artifacts per §11.4.233, un-catalogued anti-patterns w/o §1.1 mutation), extensible via §11.4.35 DATA never subtractable; runs at pre-build + release + constitution-pull; emits a machine-readable ledger `{finding_id, class, file, line, description, status, tracker_ref, mitigation_evidence}` (append-only + snapshot §11.4.116, git-tracked §11.4.95); each finding class ships golden-good/golden-bad fixtures per §11.4.107(10). MONOTONE-DECREASING RATCHET (§11.4.135 pattern) — brownfield snapshot per §11.4.66/§11.4.224(E), per-class + total, never increases, may decrease; every finding EITHER CLOSED (evidence at defect layer per §11.4.226, code repaired, disappears from sweep) OR TRACKED (§11.4.197 item + §11.4.148 description + §11.4.223 marker + §11.4.101 mitigation + §11.4.135 regression guard). Silent absorption is §11.4 PASS-bluff. Honest "zero" is against the closed vocabulary (§11.4.118 discovery-pressure remains); narrowing vocabulary to declare zero is §11.4.6 fabrication + release-blocker; §11.4.238 escape triggers vocabulary extension + ratchet snapshot down. GENERALISES §11.4.124 + §11.4.118 + §11.4.238 to whole-project zero-finding invariant. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-261-PROPAGATION` + recommended gates `CM-ZERO-FINDINGS-AUDIT-SWEEP` / `CM-ZERO-FINDINGS-MONOTONE-RATCHET` / `CM-EVERY-FINDING-CLOSED-OR-TRACKED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.261. Non-compliance is a release blocker. No escape hatch — no `--skip-findings-audit`, `--allow-todo-without-tracker`, `--ratchet-may-increase`, `--`

`silently-absorb-finding`, `--narrow-vocabulary-to-declare-zero`, `--for-now-in-production` flag.

§11.4.262 — Machine-created evidence at every gate: every claim of works/passes/verified cites captured machine-derived machine-verifiable evidence, no operator eyeballing, no narrative-only PASS (BACKGROUND :: REMINDER :: IMPORTANT operator mandate, 2026-08-15). Compact summary: every claim by the System that something WORKS / PASSES / IS VERIFIED / IS PRODUCTION-READY MUST cite MACHINE-CREATED + MACHINE-VERIFIABLE evidence produced by the gate itself, cited by path + sha256 + timestamp (§11.4.207 content-addressed); operator eyeballing / narrative-only PASS / "looks fine" / "no error" / prediction / guessing / assumption are FORBIDDEN (§11.4/§11.4.1/§11.4.6). STRENGTHENS the §11.4 covenant preamble from CAPTURED to CAPTURED-AND-MACHINE-VERIFIABLE evidence, applying §11.4.107(10) self-validating analyzers UNIVERSALLY (not only AV playback). THREE-GATE COVERAGE: (1) CONFIRMATION (source review + static + clash §11.4.110), (2) VALIDATION (§11.4.108 four layers each with its own machine verdict + artifact), (3) VERIFICATION (§11.4.185 human + §11.4.236 machine sufficiency, each with captured evidence). MACHINE-CREATED four properties: produced by the gate not the tester, schema-parseable/machine-verifiable (JSON verdict, JSONL event stream, OCR/vision oracle per §11.4.107/§11.4.117, audio RMS/loudness), content-addressed, semantic-class-matched (§11.4.226 wrong-layer echo forbidden). Every evidence analyzer golden-good/golden-bad/negative-control validated per §11.4.107(10); analyzer that passes its golden-bad = bluff itself + release-blocker (§11.4.201). Four-layer binding with §11.4.108 — every layer PASS cites machine evidence PRODUCED AT THAT LAYER; higher layer citing lower layer only = §11.4.226 wrong-layer violation. Standing default from first prompt (§11.4.126/§11.4.198). Honest boundary §11.4.6 — mandates evidence + analyzer validation; does not claim machine evidence proves human satisfaction (§11.4.185 remains); does not license shipping when machine evidence disagrees with reality (§11.4.238 escape). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-262-PROPAGATION` + recommended gates `CM-MACHINE-EVIDENCE-AT-EVERY-GATE` / `CM-EVIDENCE-ANALYZER-SELF-VALIDATED` / `CM-NO-NARRATIVE-ONLY-PASS` / `CM-EVIDENCE-LAYER-MATCH-108` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md`

§11.4.262. Non-compliance is a release blocker. No escape hatch — no `--narrative-pass-OK`, `--operator-eyeball`, `--should-pass-suffices`, `--skip-evidence-analyzer-validation`, `--evidence-optional`, `--pass-without-machine-artifact`, `--predict-success` flag.

§11.4.263 — Process-group signal-safety mandate: NEVER signal pgid ≤ 1, NEVER trust a mock-derived pid/pgid — validate as int > 1 before every killpg / kill(-pid, sig) / pkill / killall call (BOB-126 forensic anchor, 2026-08-19). Compact summary: SEVEN forced-logout incidents on the operator's workstation over ~48h (BOB-116/120/123/124/125/126) traced by kernel audit trail to a single Python test — `AsyncMock().pid` unset → `MagicMock.__int__ == 1` default → `os.getpgid(1) == 1` → `os.killpg(1, SIGKILL)` → glibc → `kill(-1, SIGKILL)` = SIGKILL every UID-1000 process (systemd user@1000 manager, gnome-shell, tmux, ssh, browsers, Claude Code) — with `contextlib.suppress(Exception)` swallowing nothing because the syscall SUCCEEDED. Universal mandate: (A) NEVER signal `pgid ≤ 1` in any language — `killpg(1,sig)` == `kill(-1,sig)` == the disaster syscall on Linux/BSD/macOS; (B) validate `isinstance(pid,int) and pid > 1` AND `isinstance(pgid,int) and pgid > 1` before every process-group signal call (Python `os.killpg`, Go `syscall.Kill(-pgid,sig)`, Rust `nix killpg`, Bash `kill -<pgid>` / `pkill -g`, C `killpg` / `kill(-pid,sig)`); (C) TESTS mocking subprocess/proc objects MUST explicitly set `mock.pid = <int>` (never rely on mock library defaults) AND patch the killpg call itself AND assert killpg was called with `pgid > 1`; (D) DEFENSE-IN-DEPTH four-layer coverage per §11.4.4(b) — pre-build grep gate, post-build CodeGraph query, runtime kernel audit rule `syscall=kill a0=0xffffffff a1=9`, paired §1.1 mutation regression test; (E) honest boundary — hardens process-group signal APIs; does NOT prohibit legitimate `start_new_session=True` child-owned groups (guard still cheap); does NOT prevent kernel OOM-killer / systemd-oomd (§12.6 + §11.4.225 apply); (F) standing default from first prompt — every code review scans for killpg calls + verifies guard, every subagent writing subprocess-cleanup code includes the guard, every test-writing subagent includes explicit `mock.pid`. Classification: universal (§11.4.17). Composes §11.4.1/§11.4.4(b)/§11.4.5/§11.4.6/§11.4.85/§11.4.107(10)/§11.4.115/§11.4.126/§11.4.201/§11.4.225/§11.4.226. Propagation gate `CM-COVENANT-114-263-PROPAGATION` + recommended

gates `CM-KILLPG-PGID-GUARD` + `CM-TEST-MOCK-PID-EXPLICIT-INT` + paired §1.1 mutations. **Canonical authority:** constitution submodule `Constitution.md` §11.4.263. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-killpg-guard`, `--mock-pid-optional`, `--killpg-1-is-ok`, `--kill-minus-1-permitted`, `--suppress-catches-syscall`, `--test-may-not-explicit-pid` flag.

§11.4.27 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."

Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented system against real infrastructure. Production code MUST NOT

import mock paths. Gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` + paired mutation.

(B) 100% test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/ Challenges submodule fully incorporated), HelixQA (HelixDevelopment/ HelixQA submodule fully incorporated). HelixQA autonomous sessions drive end-to-end execution of every registered test bank with captured wire evidence per check.

Required dependency submodules (recursive per CONST-047):

- Challenges — `git@github.com:vasic-digital/Challenges.git`
- HelixQA — `git@github.com:HelixDevelopment/HelixQA.git`
- Any other functionality submodules under `vasic-digital/*` / `HelixDevelopment/*` orgs the project depends on.

Pointers bumped to upstream HEAD in same commit as cascade work (§11.4.26 step 7); pointer drift = §11.4.27 violation.

Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. Composes with §1, §7.1, §11.4.1–§11.4.26 (esp. §11.4.25 — this is its strict expansion into per-type-of-test territory). See Constitution §11.4.27 for full mandate.

§11.4.255 — Mandatory HelixTranslate canonical translation pipeline (User mandate, 2026-06-25). Compact summary: ALL translations of ANY content — UI strings, documents, articles, CVs, READMEs, marketing copy, in-app messages, release notes, legal text, ANY human-language string rendered in a non-source language — MUST be produced by **HelixTranslate** (`git@github.com:HelixDevelopment/HelixTranslate.git`) driven through the canonical universal translation pipeline shipped in the constitution submodule at `scripts/translation/` (`translate-pipeline.sh` + `render-articles.sh`) — a hard, non-negotiable constraint. FORBIDDEN as §11.4.255 violations: (a) **hand-translation** — no human-authored, manually edited, or "good enough" hand-written translation of any content, ever; the engine is the single source of every translated string; (b) **other engines** — no Google Translate, DeepL, raw ChatGPT/Claude prompting, OS translation

APIs, library i18n auto-translators, or any engine other than HelixTranslate's `unified-translator`; there is exactly one engine; (c) **silent fallback** — no quiet substitution of the source string, no empty/partial output passed off as a translation, no swallowed engine error, no absence-of-error PASS (§11.4/§11.4.1); on exhaustion of all configured providers the pipeline MUST FAIL LOUD (non-zero exit, destination NOT written) so the caller detects it, a missing translation being a hard error never a degraded-but-shipped string — provider-to-provider failover WITHIN HelixTranslate (primary → fallback, with logged, evidenced retries) is NOT a silent fallback and is permitted, while falling back to ANY non-HelixTranslate source is forbidden. **Mandatory validation** — every generated translation MUST be validated for accuracy with real (physical) captured evidence per §11.4.5/§11.4.69/§11.4.107, no metadata-only / config-only / grep-without-runtime PASS (§11.4/§11.4.1), no false results, no bluff of any kind (§11.4.6); the per-file run log (provider used, byte counts, retries, success/failure) IS the evidence artifact, and an unvalidated or unevidenced translation is treated as ABSENT. **Explicit provider routing** — provider + model routing MUST be explicit and recorded, never implicit/defaulted-in-the-dark; the canonical routing is provider `groq` / model `llama-3.3-70b-versatile` with documented failover to provider `mistral` / model `mistral-large-latest`, the provider that actually produced each output MUST be recorded in the per-file log, and any deviation from the canonical routing MUST be stated explicitly in configuration and logged. **Canonical pipeline location** — `scripts/translation/translate-pipeline.sh` (universal Markdown / plain-text + article-frontmatter-preserving translator) + `scripts/translation/render-articles.sh` (article-source → HTML fragment renderer); both reusable verbatim across projects, the engine binary supplied by the consuming project via `HELIX_TRANSLATE_BIN` and built from the HelixTranslate repository. **Rationale** — a single, evidenced, fail-loud translation engine guarantees every translated string in the fleet has a known provenance, a reproducible pipeline, and a captured accuracy proof, eliminating the silent drift, inconsistent quality, and untraceable "where did this string come from" failures that hand-translation and ad-hoc engines produce: one engine, one pipeline, one evidence trail. Classification: universal (§11.4.17) — a platform-neutral, language-neutral translation discipline reusable by ANY project; the consuming project supplies its concrete source files, target languages, and the HelixTranslate engine build per §11.4.35. Composes §11.4 / §11.4.1 / §11.4.5 /

§11.4.6 / §11.4.7 / §11.4.69 / §11.4.107. Propagation gate `CM-COVENANT-114-255-PROPAGATION` (literal `11.4.255`) + recommended gate `CM-MANDATORY-HELIXTRANSLATE-PIPELINE` + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.255. Non-compliance is a release blocker regardless of context. No escape hatch — no `--allow-hand-translation`, `--other-engine-OK`, `--skip-translation-validation`, `--silent-fallback` flag exists.

§11.4.256 — Mandatory independent per-language translation review (User mandate, 2026-06-25). Compact summary: every translated artifact produced under §11.4.255 MUST, BEFORE it is shipped / committed / deployed, be INDEPENDENTLY REVIEWED for quality by a per-language review agent — a hard, non-negotiable gate that composes with, and NEVER replaces, the §11.4.255 generation mandate. Requirements: (a) **Independence** — the reviewer MUST be a different model/provider than the one that produced the translation (e.g. translator `mistral/mistral-large-latest` → reviewer `groq/llama-3.3-70b-versatile`, or any distinct verified model); a model reviewing its own output is NOT an independent review and is forbidden. (b) **Per-language specialization** — the reviewer MUST act as a NATIVE speaker / linguistic reviewer of the SPECIFIC target language, evaluating that language's grammar, orthography, and correct script (Cyrillic for ru/be/kk, Arabic for ar/fa, Han/Kana/Hangul/Devanagari for zh/ja/ko/hi, Latin for sr/de/es/fr/tr, etc.). (c) **Full criteria** — each review MUST judge accuracy/fidelity, fluency/naturalness, completeness (nothing omitted or invented), correct script + orthography, and absence of untranslated source leftovers (proper nouns / URLs / code excepted). (d) **Strict machine-readable verdict + evidence** — each review MUST emit a structured PASS/FAIL verdict with per-criterion scores and an issues list, persisted as a physical evidence artifact (§11.4.5 / §11.4.69 / §11.4.107); a FAIL — or an ERROR / unavailable-reviewer — BLOCKS the content and only a real PASS may ship, with no metadata-only / absence-of-error PASS (§11.4 / §11.4.1) and no bluff (§11.4.6). (e) **Remediation loop** — a FAIL routes the artifact back through the §11.4.255 pipeline (re-translate / multipass-polish) and is re-reviewed, iterating until PASS, NEVER hand-patched to pass (hand-translation remains forbidden under §11.4.255). **Canonical tooling** — `scripts/translation/review-translations.sh` (per-language review driver) + `scripts/translation/`

`review_translation.py` (independent reviewer engine emitting strict JSON verdicts); reusable verbatim across projects, the consuming project supplying its source/translation file pairs and a reviewer provider DISTINCT from its translator. **Rationale** — §11.4.255 guarantees a translation's provenance and a fail-loud pipeline, but not its linguistic quality; an independent, native-level, per-language reviewer with a captured PASS/FAIL verdict is what turns "a string was produced" into "a string was verified correct", making translation quality an auditable fact rather than an assumption. Classification: universal (§11.4.17) — a platform-neutral, language-neutral translation-quality-assurance discipline reusable by ANY project that ships translated content. Composes §11.4.255 (the translation it gates) / §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.134 (rock-solid evidence). Propagation gate `CM-COVENANT-114-256-PROPAGATION` (literal `11.4.256`) + recommended gate `CM-MANDATORY-TRANSLATION-REVIEW` + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.256. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-translation-review`, `--self-review-OK`, `--accept-unreviewed-translation` flag exists.

§11.4.264 — Build once, promote ONE immutable content-addressed artifact through every environment; never rebuild per stage (research-derived, 2026-08-20). Compact summary: exactly ONE artifact is produced per release candidate and THAT artifact is promoted through every environment; rebuilding per stage silently BREAKS THE CHAIN OF EVIDENCE, because every gate that ran against the staging artifact measured a DIFFERENT binary than production runs — making each of those green verdicts a §11.4 PASS-bluff about the shipped artifact (corpus module 31: *"The thing you tested must be the thing you ship … Rebuilding per environment silently breaks the chain of evidence"*; red flag: *"'We rebuild for production to be safe.'"*). (A) BUILD ONCE per release candidate under §11.4.246 (reproducible + hermetic + SLSA-L2); determinism makes a rebuild detectably equivalent, it is never a licence to rebuild instead of promote. (B) IDENTITY IS A CONTENT ADDRESS — a digest, never a mutable tag (*"tags get re-pushed; digests cannot change without changing the digest"*; the attack defended is *"the same version, different bytes"*), composing §11.4.111 and §11.4.200 (identity read BACK from the intended target). (C) PROMOTION IS A VERIFICATION — *"verify this exact digest is authentic and approved for the next environment," not "make a new*

thing and hope it's equivalent"; signature/SBOM/provenance travel WITH the bytes. (D) ENVIRONMENT CONFIG IS INJECTED AT DEPLOY, NEVER BAKED AT BUILD (baking per-environment config re-creates the per-stage rebuild; §11.4.10 still forbids baking a secret). (E) ROLLBACK IS DETERMINISTIC BY CONSEQUENCE — point at the previous known-good digest, *"not a rebuild-from-source scramble under incident pressure"* — the precondition that makes §11.4.265's auto-abort a real undo. (F) REBUILD-TO-DODGE IS GATE-EVASION — *"re-running until green, or rebuilding to get a different result, launders a real failure into a pass"*; retries are counted and surfaced (§11.4.248). Honest boundary §11.4.6 — guarantees the tested bytes are the shipped bytes; does NOT prove them correct (§11.4.108/§11.4.40/§11.4.185), that the deploy reached the intended target (§11.4.200), or that injected config is right (§11.4.254). Classification: universal (§11.4.17); environment ladder, digest scheme and config-injection mechanism are consumer DATA per §11.4.35. Composes §11.4/.1/.6/.10/.35/.108/.111/.151/.200/.235(B)/.246 (its DEPLOY-side complement: §11.4.246 governs how the artifact is BUILT, §11.4.264 that it is not built AGAIN)/.248/.252/.265/§1.1. Propagation gate `CM-COVENANT-114-264-PROPAGATION` (literal `11.4.264`) + recommended gates `CM-BUILD-ONCE-PROMOTE-DIGEST` + `CM-DEPLOY-CONFIG-INJECTED-NOT-BAKED` + paired §1.1 mutations (add a second per-environment build, or swap a promoted digest for a mutable tag, or bake an environment endpoint into the build → the gates FAIL; golden-FALSE per §11.4.201(1): one build + N digest-referencing promotions MUST NOT fire either); gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.264. Non-compliance is a release blocker. No escape hatch — no `--rebuild-per-environment`, `--promote-by-tag`, `--bake-env-config-at-build`, `--rebuild-until-green`, `--skip-digest-pin`, `--deterministic-build-means-rebuild-is-safe` flag.

§11.4.265 — Progressive delivery: traffic-shifted rollout gated by automated analysis on SLO and business metrics, with automatic abort to stable (research-derived, 2026-08-20). Compact summary: a release reaches users by a bounded-blast-radius strategy — CANARY (gradual weighted traffic shift, smallest blast radius, old+new run concurrently so schema/compat must tolerate it) or BLUE-GREEN (full parallel stand-up, one-shot cutover, rollback

window; *"at the switch, all users hit the new version simultaneously"*) — never an ungated cutover; for §11.4.239 critical-invariant surfaces the corpus prefers canary with blue-green's instant-undo posture. (B) THE PROMOTE/ABORT DECISION IS AUTOMATED, NOT A HUMAN WATCHING A DASHBOARD — a controller compares the canary against the stable baseline over successive analysis windows and *"auto-promotes … or auto-aborts and rolls back all traffic to stable — no human needed"*; a manual approval click is explicitly rejected (*"Manual LGTM is a rubber stamp with no receipt; under agents it looks like oversight while providing none"*), composing §11.4.240 and §11.4.262. (C) THE LOAD-BEARING CLAUSE — THE ANALYSIS MUST INCLUDE BUSINESS METRICS, NOT ONLY INFRASTRUCTURE METRICS; the corpus names infra-only measurement *"the classic mistake"*: *"'HTTP error rate is fine' does not tell you the checkout-completion rate quietly dropped 2% … the failures that matter most are business failures that look technically green. So the analysis must include business KPIs, or it re-creates the 'green pipeline that proves nothing' at the deploy layer."* An infra-only canary is a §11.4 PASS-bluff wearing a rollout's uniform. (D) MULTIPLE CONSECUTIVE ANALYSIS WINDOWS, never a single sample (§11.4.50/§11.4.107). (E) POST-DEPLOY SMOKE IS A FAIL-CLOSED PRECONDITION OF TRAFFIC (§11.4.252/§11.4.254). (F) THE DEPLOY GATE IS INDEPENDENT OF THE MERGE GATE — *"'got it merged' is not 'got it shipped.'"* (G) THRESHOLDS AND WINDOW COUNTS ARE CONSUMER DATA, NEVER CONSTITUTIONAL LITERALS — the corpus writes its abort criterion with UNFILLED placeholders (*"below X% or p99 exceeds Y ms … (thresholds illustrative — set from your SLOs)"*) and warns *"A release gate that quotes a stale requirement is itself a bluff."* Honest boundary §11.4.6 — bounds blast radius and automates undo; does NOT prove the release correct, does NOT replace §11.4.185 manual QA, and does NOT apply where no traffic-shiftable surface exists (honest §11.4.3 SKIP-with-reason, never a fabricated canary). The corpus's progressive delivery is exclusively traffic-shifting; it is silent on feature flags / dark launch / kill switches / shadow traffic and none is mandated, and it contains no "a deploy you cannot roll back is not a deploy" claim. Classification: universal (§11.4.17). Composes §11.4/.1/.3/.5/.6/.35/.50/.69/.107/.185/.201/.235/.238/.239/.240/.252/.254/.262/.264/§1.1. Propagation gate `CM-COVENANT-114-265-PROPAGATION` (literal `11.4.265`) + recommended gates `CM-PROGRESSIVE-DELIVERY-STRATEGY-DECLARED` + `CM-CANARY-ANALYSIS-INCLUDES-BUSINESS-METRIC` + paired

§1.1 mutations (strip every business metric leaving infra-only, or replace the automated abort with a manual approval → the gates FAIL; golden-FALSE: a non-routable artifact with an honest SKIP-with-reason MUST NOT fire either); gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.265. Non-compliance is a release blocker. No escape hatch — no `--full-cutover-ok`, `--infra-metrics-suffice`, `--manual-approval-is-the-gate`, `--single-window-analysis`, `--skip-smoke-before-traffic`, `--merge-implies-deploy` flag.

§11.4.266 — Claim-vs-reality ledger: every ADVERTISED capability is a row, typed from the closed bluff-type vocabulary, and a capability with no passing challenge is a release blocker (research-derived, 2026-08-20). Compact summary: corpus module 35 names this the FIRST and cheapest of its portfolio gates — *"A shared claim-vs-reality ledger per repo … every advertised capability is a row with a Bluff type and a severity, and a capability with no passing challenge is a release blocker. This makes stubbed-core, byte-identical-fork, and config-present-but-unwired visible and blocking rather than discovered later."* THE DIRECTION OF ENUMERATION IS THE POINT and is what no existing anchor supplies: §11.4.25's coverage ledger, §11.4.52's autonomous-path column and §11.4.169's test-type column are all keyed on FEATURES the project knows it has; a claim-vs-reality ledger is keyed on what the project ADVERTISES (README, docs, API/CLI surface, §11.4.254 capability matrix, §11.4.259 badges), so a claim with no feature row is invisible to a feature-keyed ledger by construction — §11.4.266 adds a keying, not a restatement (§11.4.227). (A) ONE LEDGER PER REPOSITORY, claim-side; the advertised-surface set is consumer DATA (§11.4.35). (B) EACH ROW CARRIES A BLUFF TYPE FROM THE CLOSED SEVEN-MEMBER VOCABULARY — `green-but-broken` / `coverage-theater` / `rubber-stamp-verified` / `stubbed-core` / `doc-vs-code-drift` / `config-present-but-unwired` / `byte-identical-fork` — plus a severity (the corpus states NO severity scale; consumer DATA). (C) THE TYPE IS A ROUTING KEY TO A COUNTER-GATE — *"it turns 'another bug' into 'another instance of type T,' which is what lets you install one gate that kills the whole type"* — each type binding to the anchor that already owns its counter (§11.4.108/.262, §1.1/§11.4.224(C), §11.4.146(D3)/.240/.249, §11.4.27/.254, §11.4.106/.186/.227, §11.4.124/.196(F), §11.4.251); §11.4.266 makes the

ROUTING mandatory, it does not restate the counters. (D) THE RELEASE-BLOCKING RULE — a row whose challenge is absent, never executed, or executed only against a stale artifact fingerprint blocks exactly as a FAILING challenge does (§11.4.135/§11.4.236 absence-blocks-as-FAIL, applied to the claim side). (E) THE ESCAPE HALF is §11.4.238's, cited not duplicated. **PROVENANCE CORRECTION (§11.4.6):** secondary project documentation has called this material an " `FP-1 … FP-14` " catalogue, an " `L-1 … L-16` " list, and an " `F-1 … F-31` " bluff taxonomy; measured against the tracked corpus, **no `F-n` , `FP-n` or `L-n` identifier exists in any of the nine modules** (control-needle proven — the same scan returns the real `ATM-277` / `ATM-343` / `LVA-098` / `ITE-6` / `API-33` ids and 0 for a negative control). The real structures are a 14-row numbered failure-pattern table, a 6-bullet portfolio-gate list, and this 7-member bluff-type vocabulary; those labels MUST NOT be cited as corpus wording. Honest boundary §11.4.6 — proves every ADVERTISED capability has a passing challenge; does NOT prove the challenge strong (§11.4.245/§11.4.107(10)/§1.1), the capability bug-free (§11.4.118), or enumerate capabilities nobody advertised (claim-keyed and feature-keyed ledgers are complements; running one leaves an honestly-stated blind side). Classification: universal (§11.4.17). Composes §11.4/.1/.6/.25/.27/.35/.52/.106/.108/.115(F)/.118/.124/.135/.146(D3)/.169/.186/.196(F)/.197/.212/ §1.1. Propagation gate `CM-COVENANT-114-266-PROPAGATION` (literal `11.4.266`) + recommended gates `CM-CLAIM-REALITY-LEDGER-COMPLETE` + `CM-UNCHALLENGED-CAPABILITY-BLOCKS-RELEASE` + `CM-LEDGER-ROW-TYPED-FROM-CLOSED-VOCABULARY` + paired §1.1 mutations (advertise a capability with no row, blank a row's challenge, or retype a row to an invented type → the gates FAIL; golden-FALSE: a fully-populated ledger with fresh candidate-fingerprinted PASS verdicts MUST NOT fire any); gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.266. Non-compliance is a release blocker. No escape hatch — no `--ledger-optional` , `--unchallenged-capability-ok` , `--untyped-ledger-row` , `--invent-bluff-type` , `--feature-ledger-suffices` , `--advertise-without-row` flag.

§11.4.267 — Shared attempt record + converge-on-evidence / escalate-on-stall: a failed approach is never silently retried, and a non-converging loop is itself a signal (research-derived, 2026-08-20). Compact summary: corpus module 34 — *"No memory of what was tried → the loop repeats itself.*

Agents re-attempt discarded approaches. Fix: a shared record of attempts and outcomes (in the task state / PR thread), so a failed approach is not silently retried; and an escalation rule — after N failed iterations, stop and route to a human or a stronger model, rather than burning cycles." The premise that makes it structural: *"The agent has no memory of prior attempts. It will re-implement the same wrong fix because it does not remember the last three"* — so the record is externalised memory, not a convenience log. WHAT THIS IS NOT (the §11.4.227 boundary, stated first): §11.4.147 governs a CRASHED agent, §11.4.232(C) a HUNG long-op, §11.4.21 the TERMINAL exhaustion audit — none governs a LIVE, HEALTHY, PROGRESSING loop re-walking a dead end its predecessor already proved dead; §11.4.267 closes exactly that gap. (A) A SHARED, DURABLE ATTEMPT RECORD across every actor that may take the next attempt (successor agent, §11.4.147 respawn, §11.4.176 other track); the §11.4.93 DB and §11.4.116 event stream are the substrates and MUST be reused, not duplicated. (B) CONSULTED BEFORE ACTING — the record is READ before the next attempt; a recorded-failed approach is not silently retried, and a re-attempt is permitted only on a recorded material difference (changed precondition, corrected §11.4.199 sequence, new §11.4.115(F) RED); a written-but-never-read record is the §11.4.196(F) configured-≠-in-use failure applied to agent memory. (C) CONVERGE ON EVIDENCE — *"Each loop must monotonically reduce uncertainty"*, on a SINGLE SOURCE OF TRUTH (*"the verifier's build exit 1 beats the producer's fluent paragraph every time"*) with CONCRETE FAILURE FEEDBACK (*"feed the exact captured error (argv, exit code, failing assertion, the offending line) back as the next input"*). (D) STALL IS A DETECTED SIGNAL WITH A BOUNDED RETRY AND AN ESCALATION PATH — two named signatures (*"the same criterion red after N tries"*, *"oscillating diffs"*), the stall itself tracked and trended, and the forbidden shape explicit: *"an unbounded retry loop with no ground-truth feedback and no escalation — that is how agents thrash expensively while appearing busy."* Escalation composes §11.4.231 (re-tier), §11.4.101, §11.4.66/§11.4.21. (E) `N` IS CONSUMER DATA, NEVER A CONSTITUTIONAL LITERAL — the corpus leaves `N` symbolic in every occurrence and states no numeric, wall-clock, token or cost bound; this anchor mandates that a bound EXISTS and is DECLARED, never a value. (F) THE LOOP'S DONE-CONDITION — not satisfied while an attempt is in-flight with no outcome, and every escalation reaches a terminal disposition

(§11.4.87/.94/.97/.126/.147/.197). Honest boundary §11.4.6 — prevents the RECORDED dead end from being re-walked; does NOT guarantee convergence (an escalation is an honest outcome), prove any attempt competent, or substitute for §11.4.102. Classification: universal (§11.4.17). Composes §11.4.6/.20/.21/.35/.66/.70/.87/.93/.94/.97/.101/.102/.115(F)/.116/.126/.146/.147/.176/.187/.196(F §1.1. Propagation gate `CM-COVENANT-114-267-PROPAGATION` (literal `11.4.267`) + recommended gates `CM-ATTEMPT-RECORD-SHARED-AND-CONSULTED` + `CM-STALL-BOUNDED-AND-ESCALATED` + paired §1.1 mutations (dispatch blind without reading the record, remove the retry bound, or record an escalation with no terminal disposition → the gates FAIL; golden-FALSE: a loop converging on attempt 2 with both attempts recorded MUST NOT fire either); gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.267. Non-compliance is a release blocker. No escape hatch — no `--no-attempt-record`, `--record-without-reading`, `--unbounded-retry-loop`, `--retry-same-failed-approach`, `--stall-without-escalation`, `--invent-N-and-cite-corpus` flag.

EXTENSIONS landed 2026-08-20 (existing anchors strengthened — full text in `Constitution.md`; research-derived from the tracked AI-curriculum corpus `docs/research/ai_curriculum_2026_08/source_corpus/`, modules 27/30/31/32/33/34/35). Five anchors gain clauses rather than five new numbers being minted (§11.4.227 — a rule already covered is EXTENDED, never restated as a fresh anchor). **§11.4.233(G) — dependency-pointer preflight fails closed on THREE classes, not one:** clause (A)'s Plane-R catalogue named only *drifted* submodule pointers; the catalogue is extended with **ABSENT/UNINITIALISED** (declared but not checked out — an empty directory read as a satisfied dependency) and **UNFETCHABLE** (the recorded pointer resolves on NO configured remote — the `not our ref` condition, invisible in the checkout that created it and failing only on a fresh clone). Corpus module 35 row 5: *"The fix is real and merged in a submodule, but the top-level image ships the old pointer, so the bug returns from a clean checkout"*; its prescription is a HARD gate — *"A hard preflight gate that fails the build … 'the fix is in the submodule but the image shipped old code' must be impossible, not caught after"* — with four mechanical steps: *"resolves every submodule pointer, verifies it matches the intended ref, fails*

closed on a missing checkout, and — for the truly load-bearing components — asserts a runtime signature on the flashed/deployed artifact." General principle: *"fail closed when an input to your correctness is unverifiable … a missing thing silently read as a present, correct thing."* Gate `CM-DEPENDENCY-POINTER-PREFLIGHT-FAIL-CLOSED`. **§11.4.242 — delta debugging: bisection is not only over commits.** §11.4.242's own honest boundary limits it to *"the INTRODUCING COMMIT of a MONOTONE regression"*, leaving defects with no known-good commit and defects whose discriminating variable is not a commit. Corpus module 33 supplies the generalisation and its containment direction — **`git bisect` is delta debugging over commits**, not the reverse — with the enumerated axes *"Bisect a config … Bisect the data … Bisect the change set … Bisect dependencies"* (plus feature flags; STATE is NOT an enumerated axis and is not claimed as one). A non-monotone or non-commit-discriminated failure is narrowed to a MINIMAL failing case which becomes the §11.4.115 RED test's input, under a *"stable, fast, unambiguous test predicate"* run repeatedly where the failure is noisy (§11.4.50/§11.4.248). Gate `CM-DELTA-DEBUG-MINIMAL-CASE`. **§11.4.260(B)(2) — observability: the mechanism, not only the property.** The clause NAMED observability; three mechanical requirements were missing. Corpus module 33: the **three complementary signals** — structured logs (*"queryable key–value events (JSON), not prose strings"*), metrics, traces — plus **crash/ANR analytics**; a **correlation id** threading all of them that MUST survive async boundaries (*"I make sure the correlation ID survives the queue … or I lose the thread exactly where the hard bugs live"*), pivoting **metric → trace → log** under *"narrow before you theorize"*; and **crash-free users/sessions as a release gate** — *"the honest health signal I gate releases on … I treat a new signature after a deploy as a release-blocker even at low volume"* (binding §11.4.152 and §11.4.265). Thresholds are consumer DATA — the corpus states none and cautions that crash-free thresholds *"change over time and by version"*. CITATION DISCIPLINE (§11.4.6): the corpus does NOT use "observability triad" or "three pillars", and contains NO claim of the form *"a system you cannot observe in production cannot be verified in production"* — what it licenses is narrower: *"Observability turns 'sometimes' into a query … the system must be built to explain itself."* Gates `CM-CORRELATION-ID-THREADED-ACROSS-BOUNDARIES` + `CM-CRASH-FREE-RELEASE-GATE`. **§11.4.169 — suite shape: the shape is a consequence, and the generating rule is push-tests-down.**

§11.4.169 freezes WHICH KINDS of test must exist but not WHERE a behaviour is tested. Corpus module 30 REFUSES to prescribe a shape (*"the shape is a consequence, not a goal"*; the trophy-is-always-correct answer is explicitly marked WRONG; no numeric ratios are stated) and instead mandates the generating rule: *"push each test as far down as it will still catch the bug; if a high-level test fails with no lower-level test failing, add the lower-level one and consider deleting the redundant high-level one."* Two named anti-patterns bound it — the **ice-cream cone** (*"slow, flaky, still misses unit edges"*) at the top and **mock-heavy unit theater** (*"it never touched the system it claims to test"*, *"can hit 100% coverage … and still catch nothing"*) at the bottom, the latter already forbidden by §11.4.27(A) and §11.4.224(C). Gate `CM-SUITE-SHAPE-DECLARED-AND-PUSHED-DOWN`. **§11.4.148 — slicing: an item too big to test is an item whose "done" cannot be proven.** §11.4.148 governs an item's integrity once it exists and §11.4.239(B) its acceptance criteria; neither governs SIZE AND SHAPE, the upstream cause of the reopen they then have to catch. Corpus module 27: *"INVEST (Bill Wake) … Independent, Negotiable, Valuable, Estimable, Small, Testable"* — and, stated by the corpus itself, only TWO letters are defined: *"the two load-bearing letters are Small and Testable, and they are linked: an item too big to test end-to-end is an item whose 'done' you cannot prove, so it closes on partial evidence and reopens on the untested part."* Vertical slices only — the horizontal slice *"is never independently valuable or testable"*; three amigos and specification by example pin the criteria before code; and under agents *"a big fuzzy item is one an agent will report 'done' on with nothing to check it against"* (§11.4.240/§11.4.262 entering at the item-definition layer). CITATION DISCIPLINE (§11.4.6): the corpus names "SMART Tasks" once, in a source title only, never expands or mandates it — no SMART obligation may be grounded on it; and "small" is testability-in-isolation, NOT a point/hour/line count (the corpus states no size metric). Gate `CM-ITEM-SLICED-SMALL-AND-TESTABLE`. All five gates + their paired §1.1 mutations = gate-code, a separate work item, NOT claimed shipped (§11.4.6/§11.4.227). The literals `11.4.233` / `11.4.242` / `11.4.260` / `11.4.169` / `11.4.148` are unchanged — their propagation gates stay GREEN.

§11.4.268 — Tamper-evident evidence chain + periodic anchor record: deletion, reordering, and tail-truncation of an accepted evidence record are DETECTED, not merely content-mutation (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26)

Compact summary: every accepted evidence record (the §11.4.115(F) machine-written verdict store, the §11.4.116 event stream, or an equivalent evidence-bearing ledger a consuming project maintains) MUST be tamper-evident against THREE distinct attacks, not only the content-mutation attack §11.4.115(F)/§11.4.226 already cover: (1) deleting an entry, (2) reordering two entries, and (3) truncating the tail — and the mechanism that detects each MUST be named, because a forward-hash-chain alone detects (1) and (2) only when the chain is left internally inconsistent, and does NOT detect an attacker who deletes an entry and then RECOMPUTES the chain forward from that point (spec.md:132, FR-029's own measured boundary: *"an actor who deletes an entry and then recomputes the whole chain is NOT detected by the chain alone (0.43 s for 100k entries); that case is detected only against an anchor"*) — closing that gap requires a SEPARATE periodic anchor record the producer can append to but never rewrite (spec.md:137–139, FR-037–039). **The mandate (ALL hold): (A) EVERY STATE-CHANGE ENTRY WITHIN THE DECLARED COVERED-CALL SET, NO CITED-READ VOLUME CREEP.** Scope (§11.4.35): this clause binds the enforcement system's own tool-call ledger under §002-anti-slop-enforcement (spec.md:129, FR-026 is written against THAT ledger, not every write/exec/deploy on the host) — the consuming project declares its own concrete COVERED-CALL SET (which state-changing operations this evidence chain tracks) as DATA per §11.4.35; §11.4.268 does NOT itself mandate a fleet-wide "every write/exec/deploy anywhere produces a chain entry" obligation, only that within the declared set, coverage is complete and cited-vs-uncited is honoured. A state-changing call WITHIN the declared covered-call set (write, exec, deploy — spec.md:129, FR-026) MUST produce a matching chain entry naming it. An UNCITED read of the store — a read whose result backs no claim — MUST NOT be required to produce an entry (spec.md:130, FR-027: *"this guards against both volume creep and a gate that refuses correct behaviour"*); a read result CITED in support of a claim MUST have a chain entry (spec.md:131, FR-028). The distinction is CITED-vs-UNCITED, never present-vs-absent-in-storage. **(B)**

CHAIN VERIFICATION DETECTS DELETION + REORDER WITHOUT RECONSTRUCTION. Deleting one entry, or reordering two entries, WITHOUT reconstructing the chain that follows, MUST be reported by the integrity check (spec.md:132, FR-029) — mutation-detection of a single entry's content alone does NOT satisfy this; the check walks the FULL forward chain, recomputing every link, not spot-checking. This clause's own honest limit is stated in (C): a reconstructed (recomputed) chain defeats (B) alone. **(C) THE ANCHOR IS THE ONLY MECHANISM THAT CATCHES A RECOMPUTED-FORWARD DELETION OR A TAIL TRUNCATION.** A periodic ANCHOR entry — chain head digest AND entry count, recorded to a location the producer can APPEND to but NOT REWRITE (spec.md:137, FR-037: *"the count is what makes wholesale store deletion a detected absence rather than silence"*) — MUST be written on a declared interval. The anchor's STRENGTH is recorded honestly as `mechanism` (cryptographically or append-only enforced — a git commit history, a write-once object store, or the consuming project's declared equivalent per §11.4.35) or `policy` (procedurally enforced only); `mechanism` MUST NOT be claimed without evidence that rewriting is mechanically prevented, and absent that evidence the recorded strength is `policy` (spec.md:138, FR-038). Verification against the anchor detects a tail truncation that chain-alone verification PASSES cleanly on (spec.md:139, FR-039: *"the two results together are the evidence that the anchor, not the chain, carries this property"*) — so "the chain verified clean" is NEVER sufficient on its own; a verdict of tamper-free MUST name BOTH the chain-walk result AND the anchor cross-check result. **(D) VERIFICATION THAT CANNOT COMPLETE REFUSES — IT NEVER REPORTS THE CHAIN INTACT.** A chain truncated mid-verification, or an anchor location unreachable at verification time, yields REFUSAL (an UNVERIFIED status, per §11.4.201's conservative-safe-default-on-an-unresolvable-signal), never a "clean pass by default" (spec.md:133, FR-030). "Could not verify" and "verified clean" are never the same reported state. **Honest boundary (§11.4.6).** §11.4.268 detects deletion, reordering, and tail-truncation of ALREADY-RECORDED entries. It does NOT prove the first entry ever written was itself truthful — the entry's CONTENT is only as trustworthy as the harness that wrote it (§11.4.115(F) machine-written, §11.4.240 producer≠verifier for who may write it), and it does NOT make the underlying evidence CORRECT, only tamper-EVIDENT. Where a project already operates a content-addressed, atomically-committed store for a related

purpose (the §11.4.207 continuum resume engine), §11.4.268 REUSES that mechanism's chaining primitive rather than inventing a second one — a consuming project MUST NOT stand up a parallel, divergent chain implementation when one already satisfies this anchor's properties (§11.4.227 extend-don't-duplicate applied to mechanism reuse, not only to rule text). **Classification: universal (§11.4.17)** — a platform-neutral tamper-evidence discipline reusable by any project maintaining an evidence/verdict ledger; the consuming project supplies its concrete chain implementation, anchor location, anchor-interval, AND covered-call set (clause A — which state-changing operations this project's evidence chain tracks) as DATA per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.69 / §11.4.115(F) (the verdict entries THIS anchor chains) / §11.4.116 (the event stream this anchor may chain) / §11.4.201 (verification-cannot-complete → refuse, the conservative-safe default) / §11.4.205 (durable/atomic/ append-only construction discipline) / §11.4.207 (reuse its content-addressed store where one already exists) / §11.4.226 (evidence-class-at-closure — a chain entry's own class must match the defect layer it backs) / §11.4.262 (machine-created evidence at every gate — the chain entries this anchor makes tamper-evident ARE the machine-created evidence artefacts §11.4.262 requires; §11.4.268 is the integrity property that keeps those artefacts trustworthy AFTER the moment of PASS, not merely present at it) / §9.2 (a genuine tamper finding is itself a destructive-op-class event requiring the §9.2 backup-and-investigate protocol) / §1.1. Propagation gate `CM-COVENANT-114-268-PROPAGATION` (literal `11.4.268` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gates `CM-EVIDENCE-CHAIN-DELETE-REORDER-DETECTED` (chain-walk verification detects a deleted or reordered entry left internally inconsistent) + `CM-EVIDENCE-CHAIN-ANCHOR-CATCHES-RECOMPUTE-AND-TRUNCATION` (an anchor record exists on the declared interval, carries head-digest + entry-count + an honest `mechanism` / `policy` strength, and anchor-cross-check verification detects a recomputed-forward deletion and a tail truncation that chain-alone verification misses) + `CM-EVIDENCE-CHAIN-INCOMPLETE-VERIFICATION-REFUSES` (a chain or anchor that cannot be reached/completed at verification time yields REFUSAL, never a default clean pass) + paired §1.1 mutations (delete a middle entry and recompute the chain forward from that point using only internal chain data → chain-alone verification MUST still PASS

[proving the gap the anchor exists to close] while anchor cross-check MUST FAIL; truncate the tail and verify chain-alone → MUST PASS while anchor cross-check MUST FAIL; make the anchor location unreachable → verification MUST report UNVERIFIED, never PASS; golden-FALSE per §11.4.201(1): an untampered chain with a fresh, reachable, correctly-computed anchor MUST NOT fire any of the three gates). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.268 (pending landing). Non-compliance is a release blocker regardless of context. No escape hatch — no `--content-mutation-detection-suffices`, `--skip-anchor-record`, `--claim-mechanism-strength-unverified`, `--chain-truncation-passes-by-default`, `--reconstructed-chain-is-clean` flag.

---

§11.4.269 — Critic/consensus advisory-only ban AT THE EVIDENCE-ACCEPTANCE SEAM: an ungoverned confidence/consensus signal MAY inform but MUST NEVER substitute for a receipt or adjudicate producer-verifier disagreement; role recorded `advisory`, never entered into an evidence chain — the CONSTITUTION'S MANDATORY INDEPENDENT-REVIEW-VERDICT FAMILY (§11.4.125/.134/.142/.165/.209/.237/.256) IS UNWEAKENED (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26)

Compact summary: at the evidence-acceptance / claim-adjudication seam — where a producer's claim of DONE/PASS is accepted or refused, and where a producer-verifier DISAGREEMENT is adjudicated — an ungoverned second opinion (a confidence score, an LLM-judge output, or a consensus aggregation across multiple such opinions, none of them produced under this constitution's own independent-review discipline) is a real, adoptable capability, but it MUST be admitted in an ADVISORY-ONLY role at THAT seam, never as the mechanism that decides accept/refuse there. This project's own operator decision on the point (spec.md:232, A-009) states the rule directly: *"Current rules adjudicate producer-verifier disagreement on the recorded receipt, never on confidence or argument quality, and explicitly bar an LLM judge from being the gate."* An ungoverned critic signal that CAN flip an accept/refuse outcome

at that seam has re-created a self-certification collapse under a new name — it is a second oracle/gate sharing correlated blind spots with the producer (same model family, same training data, same failure modes as an LLM producing the work it is asked to judge), exactly the class §11.4.240/§11.4.249 already forbid a producer from occupying. **This anchor is NARROWER than "no second model may ever review a producer's work"** — it does NOT reach, weaken, override, or substitute for the constitution's SEPARATE, already-mandatory independent-review-verdict family (§11.4.125 code-review-before-build, §11.4.134 iterate-to-GO with rock-solid evidence, §11.4.142 universal every-change review, §11.4.165 independent verification agent, §11.4.209 Opus-`xhigh` review substrate, §11.4.237 translation context-and-spirit review, §11.4.256 independent per-language translation review). Those seven anchors mandate a STRUCTURALLY-SEPARATED verifier's verdict (§11.4.240 producer≠verifier, §11.4.249 oracle/gate roles) as a GENUINE receipt-generating gate on the work itself — a NO-GO from any of them BLOCKS the build (§11.4.125) and RE-ARMS the iterate-to-GO loop (§11.4.134) — and every one of the seven remains fully in force, unmodified, unrelaxed. Clause (F) below states this unweakened-list explicitly and mechanically. **The mandate (ALL hold): (A) TWO CLOSED TERM DEFINITIONS, EXCLUDING THE MANDATORY REVIEW FAMILY.** `critic` = a second (or Nth) model instance, confidence score, or LLM-judge that offers an OPINION on a producer's claim, diff, or artifact WITHOUT itself constituting a structurally-separated, receipt-generating review conducted under §11.4.125 / §11.4.134 / §11.4.142 / §11.4.165 / §11.4.209 / §11.4.237 / §11.4.256 — i.e. an ungoverned, ad-hoc, or supplementary second opinion, not the mandatory independent-reviewer verdict those anchors already require and gate on. `consensus` = a mechanism aggregating multiple such UNGOVERNED opinions (voting, averaging, an LLM-judge score) into one combined signal. Both are ADOPTABLE capability categories a project may wire in — §11.4.269 does not forbid their existence, only their misuse as a gate AT THE EVIDENCE-ACCEPTANCE / CLAIM-ADJUDICATION SEAM. A verdict produced by the constitution's own mandatory review family is NOT a "critic" for the purposes of this anchor — it is the receipt-generating verifier role §11.4.240/§11.4.249 already govern, and it continues to gate exactly as those seven anchors require. **(B) A CLAIM WHOSE ONLY SUPPORT IS AN UNGOVERNED CRITIC/CONSENSUS SIGNAL IS REFUSED.** Per spec.md:163 (FR-023):

"Given a claim whose only support is a critic consensus or confidence score, the seam MUST refuse it" — naming the missing receipt. A recorded receipt — a machine-written §11.4.115(F) verdict, a captured evidence-chain entry per §11.4.268, OR a mandatory independent review's own recorded GO/NO-GO under §11.4.125/§11.4.134/§11.4.142/ §11.4.165/§11.4.209/§11.4.237/ §11.4.256 — remains the ONLY basis on which producer-verifier disagreement is adjudicated; confidence or argument quality UNBACKED by such a receipt, however produced, is never a substitute. **(C) DETERMINISTIC WITH AND WITHOUT THE UNGOVERNED CRITIC.** Per spec.md:164 (FR-024): the SAME claim, evaluated once with an UNGOVERNED critic step present and once with it absent, MUST produce the IDENTICAL accept/refuse outcome. A pipeline in which disabling that ungoverned critic changes the verdict is, by definition, using it as a gate — the deterministic-outcome test is the mechanical proof that (B) is genuinely honoured, not merely stated. (This clause does NOT apply to a mandatory review under the seven-anchor family in clause (A) — skipping a REQUIRED §11.4.125/§11.4.134/§11.4.142 review is itself a violation of those anchors, not a deterministic-outcome test to pass; the test in this clause is scoped to the OPTIONAL, ungoverned critic only.) **(D) UNGOVERNED CRITIC OUTPUT IS RECORDED `advisory` IN A SEPARATE STORE, NEVER ENTERED INTO AN EVIDENCE CHAIN.** Per spec.md:165 (FR-025): ungoverned critic output MUST be recorded with a role of `advisory` and MUST NOT appear in any §11.4.268/§11.4.115(F) evidence chain — it is recorded in a separate advisory store, never intermixed with the accept/refuse evidence chain itself. A mandatory review's OWN recorded GO/NO-GO under the seven-anchor family continues to be entered into the evidence/verdict record as its own anchors require (§11.4.125's review-completed marker, §11.4.134's clean-GO record) — it is not retroactively downgraded to `advisory` by this clause. **(E) WHY THE UNGOVERNED-CRITIC COLLAPSE MATTERS.** Composes §11.4.240(A)/(C) and §11.4.249 — an ungoverned critic wired into the evidence-acceptance gate collapses ORACLE and GATE into a single mechanism that is NOT independent of the producer's own failure surface (correlated model errors, shared blind spots), which is precisely the class of collapse §11.4.240 names as maximum-severity. Advisory-only admission avoids re-creating that collapse under the label "review" while retaining the genuine usefulness of an ungoverned critic — surfacing likely issues earlier, feeding §11.4.102 systematic debugging,

prioritising a human reviewer's attention — none of which requires it to hold gating authority. The mandatory seven-anchor review family avoids the SAME collapse by a DIFFERENT mechanism entirely — structural separation of the reviewer per §11.4.240, not advisory-demotion of its verdict — which is exactly why that family's verdicts remain full receipts and this anchor never touches them. **(F) THIS BAN IS SCOPED TO THE EVIDENCE-ACCEPTANCE / CLAIM-ADJUDICATION SEAM — IT DOES NOT WEAKEN, OVERRIDE, OR SUBSTITUTE FOR THE MANDATORY INDEPENDENT-REVIEW-VERDICT FAMILY.** §11.4.125 (code-review-agent gate before pre-build + main build), §11.4.134 (iterate-to-GO with rock-solid evidence), §11.4.142 (universal code-review — every change reviewed, always), §11.4.165 (independent verification agent), §11.4.209 (code-review MUST run on the Opus model at `xhigh` effort), §11.4.237 (context-and-spirit translation review), and §11.4.256 (independent per-language translation review) each mandate a STRUCTURALLY-SEPARATED verifier (§11.4.240 producer≠verifier) whose VERDICT genuinely gates, and NONE of these seven anchors is relaxed, narrowed, made advisory, or made optional by §11.4.269. What §11.4.269 forbids is narrower and different: (i) treating an UNGOVERNED second opinion / confidence score — one NOT produced under any of those seven anchors' own independence-plus-evidence discipline — as a substitute receipt (clause B); and (ii) letting such an ungoverned signal, rather than a recorded receipt (including a receipt produced by the seven-anchor family), adjudicate a producer-verifier DISAGREEMENT (the A-009 scope, spec.md:232). A project MUST NOT cite §11.4.269 to justify skipping, weakening, downgrading-to-advisory, or making optional any of the seven listed mandatory reviews — doing so is itself a §11.4.269(F) violation, exactly the mis-scoping this clause exists to foreclose. **Honest boundary (§11.4.6).** §11.4.269 does not forbid using an ungoverned critic/consensus signal to prioritise human review, to surface candidate defects for §11.4.102 investigation, or as ONE input among many an independent human/verifier weighs when authoring (not executing) a gate. It forbids only its use as an automated GATING mechanism substituting for a real, receipt-backed oracle/gate/verifier chain. It does not claim critics are useless — only that their output is advisory context, never a verdict, UNLESS that output is itself produced under the mandatory independent-review-verdict family named in clause (F), in which case it is a receipt and this anchor does not touch it. **Classification: universal (§11.4.17)** — a platform-neutral

discipline reusable by any project that adopts, or is asked to adopt, a second-model/critic/consensus review capability OUTSIDE its own mandatory independent-review-verdict family; the consuming project supplies its concrete critic implementation and advisory-record location as DATA per §11.4.35, and its own set of already-mandatory review-verdict anchors (analogous to this project's §11.4.125/ .134/.142/.165/.209/.237/.256) as the fixed, unweakened-by-this-anchor list. Composes §11.4.6 / §11.4.69 / §11.4.115(F) / §11.4.125 / §11.4.134 / §11.4.142 / §11.4.165 / §11.4.209 / §11.4.237 / §11.4.256 (the seven mandatory review-verdict anchors clause (F) leaves unweakened) / §11.4.240 (the separation principle this anchor is a specific instantiation of) / §11.4.249 (the four-role architecture that names why an ungoverned critic-as-gate is a collapse) / §11.4.268 (the evidence chain the ungoverned critic's output MUST NOT enter) / §11.4.102 (the legitimate advisory use — feeding investigation) / §1.1. Propagation gate `CM-COVENANT-114-269-PROPAGATION` (literal `11.4.269` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gate `CM-CRITIC-CONSENSUS-ADVISORY-ONLY` (a claim supported only by an ungoverned critic/consensus/confidence signal, with no receipt, is refused; the accept/refuse outcome for a claim is identical whether an ungoverned critic step ran or was skipped; every stored ungoverned-critic output carries role `advisory` and appears in no evidence chain; a mandatory review under §11.4.125/.134/.142/.165/ .209/.237/.256 is NEVER downgraded to `advisory` or excluded from gating by this gate) + paired §1.1 mutation (wire an ungoverned critic's approve/reject value directly into the pass/fail decision → the gate MUST FAIL; store an ungoverned critic output with no role field, or role `verdict`, or entered into the evidence chain → the gate MUST FAIL; downgrade a §11.4.125/.134/.142/.165/.209/.237/.256 review's own GO/NO-GO record to `advisory`, or exclude it from gating, citing §11.4.269 as the reason → the gate MUST FAIL; golden-FALSE per §11.4.201(1): (i) an ungoverned critic present, its output stored with role `advisory`, absent from the evidence chain, and the deterministic outcome unchanged whether it ran or not → the gate MUST NOT fire; (ii) a §11.4.125 code-review's own NO-GO record blocking the build exactly as §11.4.125 requires → the gate MUST NOT fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.269 (pending landing). Non-compliance

is a release blocker regardless of context. No escape hatch — no `--critic-may-gate`, `--consensus-score-is-a-receipt`, `--llm-judge-is-the-gate`, `--critic-output-in-evidence-chain-OK`, `--outcome-may-differ-with-critic`, `--269-overrides-mandatory-review`, `--downgrade-mandatory-review-to-advisory` flag.

---

§11.4.270 — Dependency-existence-verdict register: every proposed dependency carries a closed-set existence verdict {VERIFIED, AMBIGUOUS, UNVERIFIED} with citable evidence; adoption is gated behind resolution and behind the wiring-sweep result (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26)

Compact summary: a project that adopts an external (or internal, cross-module) dependency to close a coverage gap MUST first record, for that dependency, a closed-set EXISTENCE verdict — VERIFIED, AMBIGUOUS, or UNVERIFIED — backed by a citable artifact or an explicit statement that none was found (spec.md:114, FR-013). Existence verification is NECESSARY but NOT SUFFICIENT for adoption (spec.md:226, A-012: *"Existing is not the same as working ... Several verified entries are at initial release versions, one declares a repository that returns 404, and one's 'tamper-evident' claim is on inspection only 'append-only' — which detects nothing if the appender is the party being checked"*) — a VERIFIED existence verdict is the FLOOR a dependency clears before it may even be considered adopted, never a claim that it is fit for the purpose it is being adopted for. **The mandate (ALL hold): (A) THE REGISTER, ONE ROW PER CLAIMED DEPENDENCY.** Every dependency a project proposes to adopt gets exactly one register row, keyed by a stable identifier (name + version + source per §11.4.111 resolve-by-stable-name). The row carries the verdict + its evidence field (spec.md:114, FR-013: *"MUST carry a verdict of VERIFIED, AMBIGUOUS or UNVERIFIED with a citable artifact or an explicit statement that none was found"*). **(B) VERDICT SEMANTICS, EVIDENCE-BASED, NEVER PROMOTED BY OMISSION.** UNVERIFIED is the honest default state — a claim about a dependency's existence with no independent confirmation attempted MUST NOT be silently treated as VERIFIED. AMBIGUOUS is assigned when evidence conflicts or a

name collision exists — spec.md:116 (FR-015): *"a dependency name that collides with an unrelated project" gets AMBIGUOUS, never VERIFIED* — the specific conflicting evidence is recorded, not merely a vague "unsure". VERIFIED requires an independently-reproducible check: a real resolve + checksum/hash match, a real successful call or import, or an equivalent control-needle-proven presence per §11.4.201(7)(b). **(C) ADOPTION REFUSED WHILE THE VERDICT IS NON-VERIFIED.** Per spec.md:115 (FR-014): *"Given a dependency whose verdict is UNVERIFIED or AMBIGUOUS, adoption MUST be refused."* Adoption is admitted ONLY after the row resolves to VERIFIED (or the AMBIGUOUS conflict is investigated and the row is re-verdicted). **(D) ADOPTION IS GATED BEHIND MAPPING TO A RECORDED UNCOVERED GAP + BEHIND THE WIRING-SWEEP RESULT.** Per spec.md:170 (FR-020): adoption is refused unless the dependency maps to one of the project's own recorded uncovered capability gaps (adopting a dependency for a capability the project already covers is itself refused, naming the covering rule — the §11.4.227 duplicate-coverage discipline applied to dependency adoption). Per spec.md:171 (FR-021): *"the adoption step MUST NOT be accepted before the wiring step reports its sweep result"* — a project does not decide "yes, adopt" before it has run the sweep that would reveal whether the dependency's claimed capability is genuinely reachable and wired (§11.4.124/§11.4.196(F) investigate-before-adopt, the mirror-image of investigate-before-remove applied to the opposite direction). **(E) ZERO ADOPTED WITHOUT VERIFIED — MONOTONIC.** Per spec.md:217 (SC-005): the count of external dependencies adopted without a VERIFIED existence verdict is ZERO, checked at the SAME seam that already refuses unwired-gate names per §11.4.227(A) (the named-gate ledger), generalised here to dependency-adoption claims. **Honest boundary (§11.4.6).** §11.4.270's register proves EXISTENCE/RESOLVABILITY was checked and recorded — it does NOT prove the dependency BEHAVES correctly for the purpose it is adopted for (that stays ordinary integration/contract testing per §11.4.244), and it does NOT prove the dependency is SECURE or well-maintained (that is §11.4.246's supply-chain territory). A-012's own finding — "existing is not the same as working" — is why this anchor is deliberately narrow: it closes the EXISTENCE-CLAIM bluff, not the fitness-for-purpose question, which stays governed by the constitution's ordinary testing/review family. **Classification: universal (§11.4.17)** — a platform-neutral existence-verification discipline reusable by

any project that adopts external or cross-module dependencies to close a capability gap; the consuming project supplies its register location, its recorded uncovered-gap set, and its wiring-sweep mechanism as DATA per §11.4.35. Composes §11.4.6 / §11.4.69 / §11.4.111 (stable-name keying) / §11.4.124 (investigate-before-adopt, the direction-mirror of investigate-before-remove) / §11.4.196(F) (configured ≠ wired, the same principle applied to a dependency's claimed capability) / §11.4.201(7)(b) (control-needle-proven presence checks) / §11.4.227(A) (named-gate ledger — this anchor generalises its zero-unwired-debt discipline to dependency claims) / §11.4.244 (fitness-for-purpose stays contract-testing territory, distinct from existence) / §11.4.246 (security/supply-chain, also distinct from existence) / §11.4.74 (submodule-catalogue-first discovery + extend-don't-reimplement — the register's VERIFIED-tier check IS the mandated catalogue-first discovery step, applied to any dependency, not only own-org submodules) / §11.4.31 (submodule-dependency-manifest — an owned-org dependency's `helix-deps.yaml` is itself citable evidence toward a VERIFIED verdict) / §11.4.150 (mandatory deep multi-angle web research before declaring a capability covered — the research pass that PRODUCES a register row's citable evidence, or its explicit none-found statement) / §1.1. Propagation gate `CM-COVENANT-114-270-PROPAGATION` (literal `11.4.270` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gate `CM-DEPENDENCY-EXISTENCE-VERDICT-REGISTER` (every claimed dependency has a register row carrying a closed-set verdict + evidence field; a name collision resolves AMBIGUOUS never VERIFIED; a shipped change relying on an UNVERIFIED or AMBIGUOUS row is refused; adoption refused when the dependency maps to an already-covered capability or precedes its wiring-sweep result) + paired §1.1 mutation (adopt a dependency with no register row → the gate MUST FAIL; leave a row UNVERIFIED and adopt anyway → the gate MUST FAIL; feed a name-collision fixture and assert the verdict resolves VERIFIED instead of AMBIGUOUS → the gate MUST FAIL; golden-FALSE per §11.4.201(1): a dependency VERIFIED with real, reproducible, control-needle-proven evidence, mapped to a genuinely-uncovered gap, adopted after its wiring sweep reported a result → the gate MUST NOT fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.270 (pending landing). Non-compliance is a release

blocker regardless of context. No escape hatch — no `--unverified-adoption-OK`, `--ambiguous-treated-as-verified`, `--skip-wiring-sweep-before-adopt`, `--adopt-already-covered-capability`, `--existence-proves-fitness` flag.

---

§11.4.271 — Waiver mechanism: the strict FORMALIZATION of allow-with-tracked-debt (unifying §11.4.234(D)'s recorded-deferral, §11.4.236(4)'s explicit-operator-override, and §11.4.248(A)'s deadline-bound quarantine into ONE roster + expiry + tracked-item schema), distinct from refusal (§11.4.21) and closure (§11.4.90) vocabulary (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26)

Compact summary: every project needs, on rare occasion, a mechanism to ALLOW a change to proceed despite a normally-blocking missing-evidence condition — never a silent, never a permanent bypass, but a tracked, expiring, human-authorised exception. **This is NOT an absent capability being introduced here** — re-verified in this remediation pass: §11.4.234(D) already permits an explicit, documented deferral flag ("The escape hatch is a recorded deferral, not a bypass: the skipped gate remains owed and is caught at the next full run or tag gate" — Constitution.md:10999); §11.4.236(4) already requires "an explicit, logged, operator-set flag" for any override of its deploy-readiness seam ("EXPLICIT, RECORDED OVERRIDE ONLY" — Constitution.md:11023); and §11.4.248(A) already ships a deadline-bound quarantine for one specific gate class (flaky tests, moved to `tests/quarantine/` with a tracked §11.4.197 stabilisation deadline). Those three are genuine allow-with-known-debt PRECEDENTS this anchor does not discover but FORMALIZES. What §11.4.271 genuinely adds — needled: `waiv*` returns ZERO hits anywhere in `constitution/Constitution.md` outside this draft, confirming the SCHEMA below is new even though the CATEGORY of mechanism is not — is the STRICT, DATA-BEARING, cross-gate schema those three precedents describe only informally and per-mechanism: a ROSTERED NON-PRODUCER authoriser (§11.4.234(D) and §11.4.248(A) name no authoriser; §11.4.236(4) names the operator as a role but no roster-resolvable identity schema —

§11.4.271 adds the roster resolution + producer exclusion), a MANDATORY UNELAPSED EXPIRY (§11.4.234's skip flag and §11.4.236's override flag carry no expiry field at all — an indefinite skip/override is currently legal under their text), and a MANDATORY TRACKED ITEM (neither §11.4.234(D) nor §11.4.236(4) requires one; §11.4.248(A) requires it for its one gate class — §11.4.271 makes it mandatory cross-gate). §11.4.271 turns three ad-hoc, per-anchor escape hatches into ONE cross-cutting mechanism any future gate can invoke by reference instead of re-inventing its own — this project's own `scripts/lib/critical_blocker_gate.sh` (built for the §11.4.236 seam) already implements `waive` / `override` operations by name, direct evidence this schema generalises code the project has already written, not a speculative addition. **The mandate (ALL hold): (A) THE WAIVER IS THE ONE FORMALIZED SCHEMA FOR AN ALLOW-WITH-KNOWN-DEBT PATH.** A waiver EXISTS at a seam that would otherwise refuse (a missing receipt, an unresolved dependency verdict, an unmet acceptance-criteria scenario, or any other normally-blocking condition this constitution's family of gates enforces) — the SAME class of seam §11.4.234(D)'s deferral flag, §11.4.236(4)'s override flag, and §11.4.248(A)'s quarantine already gate. It is invoked explicitly, never inferred, and it never disables the gate permanently — it grants a bounded, tracked exception to ONE candidate. A gate ALREADY implementing an allow-with-known-debt path under §11.4.234/§11.4.236/§11.4.248 is NOT required to reimplement one from scratch — it satisfies §11.4.271 by conforming its existing flag/quarantine mechanism to this clause's roster + expiry + tracked-item schema (clauses B–D below), the strict superset those three anchors' informal mechanisms currently lack. **(B) AUTHORISATION MUST RESOLVE TO A ROSTERED IDENTITY, NEVER THE PRODUCER.** Per spec.md:157 (FR-040): *"Given a waiver whose authorisation value does not resolve to an identity in the declared authoriser roster, the seam MUST refuse it"* — a free-text authorisation ("looks fine") is refused exactly as an absent one is; a waiver naming a genuinely rostered identity is accepted (the false-positive guard — a check that refuses EVERY waiver has replaced a bypass with a deadlock, not fixed anything). The authoriser roster is consumer-supplied DATA per §11.4.35, and the producer of the change under waiver MUST NOT appear as its own waiver's authoriser (composes §11.4.240 producer≠verifier — a producer cannot waive their own gate). **(C) EXPIRY IS REQUIRED AND MUST NOT HAVE ELAPSED.** Per spec.md:158 (FR-041): a waiver with NO expiry, or

whose expiry HAS elapsed, is refused; a waiver whose expiry is in the future is accepted. An absent expiry is NOT read as "valid forever" — the spec's own framing is exact: *"An expiry absent by omission is not 'valid for every candidate forever' — that is the same downgrade-by-omission the satisfaction path already refuses"* elsewhere in this family (§11.4.146(D3)/§11.4.115(F) fingerprint-absent-is-refused, the general pattern this clause instantiates for waivers). An elapsed waiver's gate reverts to blocking automatically — no grace period, no silent auto-renewal. **(D) A TRACKED ITEM NAMES THE DEBT; A WAIVER NAMING NONE IS REFUSED.** Per spec.md:159 (FR-042): *"Given a waiver, it MUST name a tracked item for the blocker that remains open, and a waiver naming none MUST be refused"* — because *"a waiver is the only ALLOW that leaves the defect unfixed, so it is the one path on which the owed work must stay traceable."* The tracked item is an ordinary §11.4.54-stable-id workable item bound to the SAME lifecycle discipline (§11.4.15/§11.4.16/§11.4.33/§11.4.197) as any other tracked work — a waiver whose tracked item never resolves is itself surfaced by the standing zero-shortcomings audit (§11.4.261) as an unresolved-waiver finding, never silently forgotten. **(E) COMPOSITION, NOT SUBSTITUTION — INCLUDING WITH THE THREE PRECEDENT MECHANISMS.** A waiver whose expiry lapses UNRESOLVED re-enters the §11.4.21 Operator-blocked class for its underlying condition (the block returns; nothing about the original blocking condition changed, only the temporary permission to pass did). A waiver's tracked item CAN close via §11.4.90 Obsolete if the underlying requirement genuinely changed. Neither §11.4.21 nor §11.4.90 substitutes for the waiver mechanism itself — a project attempting to express "allow despite missing evidence, tracked, expiring" via either of those two vocabularies is misusing them, per this anchor's own honest-boundary clause. Symmetrically, §11.4.234(D)'s deferral flag, §11.4.236(4)'s override flag, and §11.4.248(A)'s quarantine are NOT replaced by §11.4.271 — each remains the correct, narrower mechanism for its own seam class; §11.4.271 is the schema they conform TO when a project wants their allow-with-debt behaviour to carry a rostered authoriser, an expiry, and a tracked item, not a rival mechanism competing with them. **Honest boundary (§11.4.6).** §11.4.271 formalizes, it does not discover, the ALLOW-with-tracked-debt CATEGORY — §11.4.234(D), §11.4.236(4), and §11.4.248(A) already establish that category exists in this constitution; why §11.4.271 does not make waivers routine or easy is the SCHEMA's own shape (rostered-non-

producer authorisation, mandatory unelapsed expiry, mandatory tracked item) — deliberately strict, because a waiver is the ONE path this constitution's anti-bluff family permits to leave a defect genuinely unfixed while still shipping. It does not weaken any gate a waiver bypasses for OTHER candidates, and it does not authorise an indefinite or permanently-renewed waiver — an expiry that keeps getting pushed forward without the underlying debt closing is itself a §11.4.261 finding (a monotone-decreasing-findings-ledger violation, not a compliant waiver renewal pattern). **Classification: universal (§11.4.17)** — a platform-neutral, deliberately-strict allow-with-tracked-debt discipline reusable by any project whose gate family needs an occasional, bounded, evidence-honest exception; the consuming project supplies its authoriser roster, expiry mechanism, and tracked-item store as DATA per §11.4.35. Composes §11.4.6 / §11.4.15 / §11.4.16 / §11.4.21 (refusal vocabulary — distinct, cited not restated) / §11.4.33 (closure vocabulary) / §11.4.35 / §11.4.54 (stable-id tracked item) / §11.4.90 (closure vocabulary — distinct, cited not restated) / §11.4.146(D3) (the absent-field-is-refused pattern this anchor applies to expiry + authoriser + tracked-item) / §11.4.197 (the tracked item's completion obligation) / §11.4.234(D) (recorded-deferral precedent this anchor formalizes) / §11.4.236(4) (explicit-recorded-override precedent this anchor formalizes) / §11.4.248(A) (deadline-bound quarantine precedent this anchor formalizes for one gate class) / §11.4.240 (producer≠verifier — the authoriser exclusion) / §11.4.261 (an unresolved waiver is a standing-audit finding) / §1.1. Propagation gate `CM-COVENANT-114-271-PROPAGATION` (literal `11.4.271` present as a block-start, exactly-once per governance file, lockstep content-hash equality across the mirror set per §11.4.227(B)) + recommended mechanism gate `CM-WAIVER-ROSTERED-EXPIRY-TRACKED` (every active waiver resolves to a genuinely rostered, non-producer authoriser identity; a future, unelapsed expiry; and a named §11.4.54-stable-id tracked item; a waiver missing ANY ONE of the three is invalid and its waived gate reverts to blocking) + paired §1.1 mutation (submit a waiver with free-text or absent authorisation → the gate MUST FAIL; submit one with no expiry, or an elapsed expiry → the gate MUST FAIL; submit one naming no tracked item → the gate MUST FAIL; submit one where the authoriser identity equals the change's producer → the gate MUST FAIL; golden-FALSE per §11.4.201(1): a waiver naming a genuinely-rostered non-producer identity, a future expiry, and a real tracked item → the gate MUST NOT fire). Gate-code = separate work item, NOT

claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.271 (pending landing). Non-compliance is a release blocker regardless of context. No escape hatch — no `--free-text-authorisation-OK`, `--waiver-without-expiry`, `--elapsed-waiver-still-valid`, `--waiver-without-tracked-item`, `--producer-may-self-waive`, `--permanent-waiver-renewal` flag.

§11.4.272 — Dynamic, on-demand skill/extension activation: the active capability surface is the MINIMUM needed now, everything else stays DISCOVERABLE and one command away (User mandate, 2026-09-07). Compact summary: verbatim operator mandate ("We MUST use actively only Skills we need at particular moment! ... fully dynamic so token use is ALWAYS at the minimum and any of Skills and the extensions activated / deactivated as they are needed / used!"); forensic case study (measured 2026-09-07, Claude Code 2.1.263): a 955-skill host pool renders 29,153 B (~7,288 tok) of NAMES every turn, 101,279 B (~25,319 tok/turn) with description fields — a 3.5x render-mode cliff. Every project whose agent platform exposes a capability surface MUST keep the ACTIVE surface at the minimum genuinely needed, everything else activatable on demand: (A) minimal declared `core` set always active, everything else inactive-by-default; (B) discoverability NEVER traded for tokens — a deactivated capability stays findable via an always-available index, §11.4.102(B) binds unconditionally; (C) fail SAFE never silent (§11.4.201) — a failed activation exits non-zero + rolls back, never leaves a dangling/unusable claim; (D) concurrency-safe by reference counting (§11.4.119/§11.4.176) — one session's deactivation never strips a capability another session is using; (E) honest asymmetry (§11.4.6) — measured first-hand that activation hot-loads but deactivation does NOT hot-unload in the live session, so the saving is a minimal session-start baseline plus on-demand growth, never an in-session token-freeing claim; the platform's own `permissions.deny Skill(name)` blocks invocation without removing the rendered entry, saving ZERO tokens; (F) a budget gate guards BOTH the name-only and description-eager render modes; (G) works with AND without the constitution submodule vendored — an explicit resolution order ending in honest, loud degradation, never a silent no-op; (H) PRUNE-TO-CORE is the default (operator decision 2026-09-07) and is NOT a §11.4.122 silent removal — structurally reversible+visible by construction (LISTED in the core catalogue, ACTIVATABLE by one command, NOT DELETED — moved/linked not destroyed), with an explicit opt-out at

session+project scope, refusing loudly rather than risking content on any unsafe demotion. Honest boundary (§11.4.6): reduces per-turn capability-LIST cost only, never a skill's invoked body cost, never a substitute for §11.4.102(B) discovery, never a licence to hide a needed capability. Classification: universal (§11.4.17). Composes §9.2/§11.4.6/§11.4.28/§11.4.35/§11.4.74/§11.4.80/§11.4.102(B)/§11.4.119/§11.4.122/§11.4.141/§11.4.164/§11.4.174/§11.4.176/§11.4.177/§11.4.180/§11.4.201/§1.1. Propagation gate `CM-COVENANT-114-272-PROPAGATION` (literal `11.4.272`) + recommended gates `CM-SKILL-SURFACE-BUDGET` / `CM-SKILL-ACTIVATION-FAIL-SAFE` / `CM-SKILL-PRUNE-PRESERVES-CONTENT` + paired §1.1 mutation (squeeze the budget below the real surface, break an activation target, remove the engine without the distinct engine-absent exit code, or make a demotion destructive/strip its relocation-safety or other-live-session check → the gates MUST FAIL). Gate-code beyond the shipped scripts = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.272. Non-compliance is a release blocker. No escape hatch — no `--all-skills-always-on`, `--skip-activation-engine`, `--hide-inactive-skills`, `--silent-activation-failure`, `--ignore-surface-budget`, `--assume-skill-loaded` flag.

§11.4.273 — Measuring-instrument verification: a census that INFORMS a decision must be control-needled before its result is believed (research-derived, 2026-09-08). Compact summary: forensic anchor — FIFTEEN distinct ad-hoc-measurement instrument failures in one working session (2026-09-08), each returning a confident, plausible, actionable-looking result that was an artefact of the instrument, not a property of the system: a block-opener regex missing a second heading form manufactured 24 phantom dangling anchors; `comm` on non-lexicographically-sorted input reported all 34 anchors missing from a file containing none missing; `$?` read after a pipeline reported the pipeline's LAST stage, turning exit 1 into reported exit 0; a field-name typo returned `<none>` for every record; `grep -w` treated `-` as a word boundary and silently reclassified a record; a summary-line grep assumed field order that flips on failure, reading a real failure as merely quiet; `while read` silently dropped a final no-trailing-newline line; a mutation was applied via the WRONG env knob so the real (unmutated) tree was tested and the green was meaningless; a guard-probe used a command the guard does not target,

proving nothing either way; a test fixture aliased two columns to the same string so a real column-swap mutation survived 52/0 undetected; a positive control chosen OUTSIDE the instrument's declared scope reported a working instrument as broken; a positive control that WAS the very question under test could not discriminate anything; an `awk` range mis-escape reported 838 rows where the live system held 134 (6x wrong), nearly gating a destructive DB recreate; a denylist with TWO needles was searched with only ONE, so "zero remaining" was blind to the first while a live credential was still present in four copy-pasteable lines per file; `$?` was clobbered by a command substitution sitting earlier in the SAME statement, reporting a failed `pandoc` run (exit 64, nothing written) as exit=0. The gap: §11.4.201 governs guards/gates (block/pass), §11.4.107(10) governs calibrated analyzers — neither covers the far more common ad-hoc census (a grep, a set-diff, an exit-code read, a field lookup, a row count) whose output is a FACT a decision/fix/classification/report is then built on, written inline, used once, trusted implicitly, never itself tested. The mandate: any decision/claim/classification/report-bearing measurement MUST be control-needled BEFORE acting on or communicating its result — (a) POSITIVE control (a known-present value must be found, catching too-narrow patterns/wrong fields/wrong knobs/silent no-output stages); (b) NEGATIVE control (a known-absent fabricated needle must NOT be found, catching too-broad patterns/accidental substring-or-word-boundary matches); (c) an EMPTY result is NEVER a finding on its own — genuinely-absent / instrument-blind / wrong-location / wrong-output-shape are four distinct states an empty result cannot by itself distinguish, reportable only AFTER (a) proves the instrument can see a positive; (d) a result CONTRADICTING directly-observed evidence presumes the INSTRUMENT is at fault first; (e) the instrument that established a FACT is itself part of the evidence — captured claims cite the producing command; (f) when the criterion is a SET (a denylist, a taxonomy, a multi-column key), the control MUST cover EVERY member — needling one member proves only that member is searchable, the instance-14 failure mode, the most dangerous on the list because both controls PASS while the answer is wrong; (g) capture an exit status BEFORE any other expansion can overwrite it — assign `$?` to a variable on its own line immediately after the command, never read it after a pipeline or a same-statement command substitution. Honest boundary (§11.4.6): control needles prove an instrument distinguishes present-from-absent, never that it measures the RIGHT property, and never

that it is correct for inputs unlike the needles chosen. Classification: universal (§11.4.17). Composes §11.4.2/§11.4.5/§11.4.6/§11.4.107(10)/§11.4.118/§11.4.201/§1.1. Propagation gate `CM-COVENANT-114-273-PROPAGATION` (literal `11.4.273`) + recommended gate `CM-CENSUS-CONTROL-NEEDLED` (a decision-bearing census cites its positive and negative control) + paired §1.1 mutation (accept a decision-bearing census with no control → the gate MUST FAIL). Gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.273. Non-compliance is a release blocker. No escape hatch — no `--skip-control-needle`, `--empty-is-absent`, `--trust-the-grep`, `--census-needs-no-proof`, `--obvious-result-exempt`, `--one-needle-covers-the-set`, `--read-status-after-substitution` flag.

§11.4.274 — Mechanical work belongs in a script, not in an agent's context: extract it, test it, and re-scan for it on a cadence (User mandate, 2026-09-08). Compact summary: forensic anchor — in one working session five independent review agents consumed roughly 400,000–450,000 tokens EACH, with the dominant cost being MECHANICS (copy a tree to scratch, apply a one-line mutation, run a suite, read the summary, restore, verify the restore byte-identical, compare counts, repeat per mutation) — a deterministic, fixed-input/fixed-output, zero-judgment loop executed one tool call at a time by a language model at roughly four orders of magnitude the cost of a script, with a fresh transcription-error opportunity at every step (the session recorded ten such instrument errors, §11.4.273). The rule: work that is DETERMINISTIC, REPEATABLE and contains NO JUDGMENT MUST be delegated to a properly written script/utility and invoked, never performed turn-by-turn inside an agent's context — concretely mutation apply/run/restore/verify loops, suite fan-outs + result tabulation, before/after comparisons, scratch-tree setup/teardown, residue/quiescence scans, census/inventory queries, artifact fingerprinting, evidence collection/layout, and any already-fixed recipe. (a) EXTRACT — a mechanical sequence performed by hand more than twice is a tooling defect, not a task; the bar is behavioural (if the steps can be written down exactly, they can be executed exactly, and hand-execution is the wrong instrument). (b) THE EXTRACTED TOOL IS ITSELF GOVERNED — ships documentation (§11.4.18), an operator-facing guide where applicable, and tests proving its behaviour + failure paths + exit-code semantics (§11.4.201) + its own control needles (§11.4.273) with deterministic machine evidence (§11.4.50/§11.4.5); a

fast-but-unproven extraction is a regression, not an optimisation. (c) RE-SCAN ON A CADENCE — every project periodically scans itself (transcripts/logs/ repeated inline recipes/"run these commands in order" docs) for mechanical work still performed by agents and refactors it out; a STANDING recurring obligation, never a one-off cleanup. (d) HONEST BOUNDARY (§11.4.6) — JUDGMENT MUST NOT be scripted: root-cause determination (§11.4.102), review verdicts (§11.4.125/§11.4.142/§11.4.194), the decision a fix is correct, the choice of what to measure, whether captured evidence actually supports a claim — a script that renders a verdict is itself a bluff gate (§11.4.201); the division is SCRIPT gathers+executes deterministically and cheaply, AGENT decides what it means; cost is never a reason to move a decision into a script. Classification: universal (§11.4.17). Composes §11.4.5/§11.4.18/§11.4.50/ §11.4.124/§11.4.141/§11.4.201/§11.4.273/§1.1. Propagation gate `CM-COVENANT-114-274-PROPAGATION` (literal `11.4.274`) + recommended gates `CM-MECHANICAL-WORK-EXTRACTED` (a hand-performed mechanical sequence used more than twice has an extraction item) / `CM-EXTRACTED-TOOL-DOCUMENTED-AND-TESTED` (every extracted tool carries docs + tests + deterministic machine evidence) + paired §1.1 mutation. Gate-code = separate work item, NOT claimed shipped (§11.4.6/§11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.274. Non-compliance is a release blocker. No escape hatch — no `--skip-extraction`, `--agent-can-do-it`, `--one-off-is-fine`, `--script-the-review`, `--extract-without-tests` flag.

§11.4.275 — Semantic (Lumen) index + indexing-efficiency gate + universal agent/subagent accessibility (research-derived, 2026-09-24). Compact summary: every code index the project maintains — the structural CodeGraph index (§11.4.78) AND the semantic Lumen index, and any future code index — MUST (A) be reachable by EVERY agent AND every dispatched subagent through a server definition they can actually resolve (committed project config or installed plugin); a config that ENABLES a server nothing DECLARES is a violation; proof = a real tool call from a dispatched subagent returning an index-only fact (§11.4.78(4) unforgeable challenge), un-drivable agents = documented §11.4.3 SKIP; (B) for the semantic index: embedding model provisioned rootless (§11.4.161) and proven by a real embed call returning the declared dimensions, scope GENERATED from the same class source as the structural scope (`<constitution>/scripts/lumen/`

`gen_lumenignore.py`), health = files AND chunks non-zero + known-symbol hit + negative control + freshness within consumer TTL; (C) be called ready ONLY when pending work == 0, state complete, file count within consumer tolerance of the accepted count, schema indexes present — every count/empty result control-needled (§11.4.273); (D) earn its place by a DETERMINISTIC benchmark (fixed queries + golden answers, index route vs grep+read, N≥3 identical canonical hashes per §11.4.50, bytes/est-tokens/p50/p95/correctness per query), reduction computed ONLY over queries both routes answered correctly, wrong answers listed BY ID and never averaged away, thresholds = consumer DATA (§11.4.35); (E) route structural→CodeGraph, conceptual→Lumen before grep+read (§11.4.141) except measured-wrong query classes (fall back + tracked); (F) root-cause + patch performance defects in the version-keyed runner with RED→GREEN + paired mutation + byte-copy equivalence proof; (G) writes via the single writer entry (§11.4.80(5)), live reads read-only. Forensic FACTS (genericised): a configured semantic server reported 0 files / no model / never indexed; the structural index cut tokens 69.2% over 8 both-correct fixture queries yet answered 5 of 15 fixture queries WRONG (grep+read 2 of 15); one uncached full-table file-list query was 93.4% of resolve time; an FK-cascade stalled the bulk window; a launcher classifying `sync` by stored rows instead of pending backlog ran a tens-of-millions-reference bulk job stock + foreground + unsupervised; the vendor reaps locks by age not liveness and a writable open recreates dropped indexes without the lock; an errored instrument's empty result was nearly read as absence. Latent binding (§11.4.96 pattern): a project with no semantic index is bound only for the indexes it maintains; the semantic clauses bind the moment it adds one. Honest boundary (§11.4.6): proves reachable/complete/scoped/measured, not every answer correct; never licenses narrowing scope to hit a threshold (§11.4.66); the reference benchmark harness is still a prototype outside the constitution tree (landing = owed work). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-275-PROPAGATION` (literal `11.4.275`) + recommended gates `CM-CODE-INDEX-AGENT-ACCESSIBLE` / `CM-CODE-INDEX-COMPLETE-BEFORE-READY` / `CM-LUMEN-MODEL-PROVEN` / `CM-LUMEN-SCOPED-FRESH` / `CM-CODE-INDEX-BENCH` / `CM-CODEGRAPH-SCOPE-PROOF` / `CM-CODEGRAPH-UPGRADE-PROBE-GATED` / `CM-CODEGRAPH-WRITER-EXCLUSIVE` / `CM-CODEGRAPH-BULK-BY-BACKLOG` / `CM-CODEGRAPH-STALL-WATCHDOG` + paired §1.1 mutations (enabled-undeclared server, ready with pending > 0,

reduction without wrong-answer list, unloaded model, bulk-by-rows, dropped liveness check → each gate MUST FAIL; golden-FALSE: unchanged complete reachable index fires none). Gate CODE = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.275. Non-compliance is a release blocker. No escape hatch — no `--index-partial-ready`, `--enabled-is-declared`, `--main-session-only-index`, `--skip-model-proof`, `--reduction-without-wrong-answers`, `--bulk-by-row-count`, `--lock-age-is-liveness` flag.

§11.4.276 — Review-round budget: every independently-reviewed work item converges within a declared budget of at most 7 rounds (5–7, consumer-declared), by front-loading ground truth, class-exhaustive enumeration, real-path tests and a declared threat boundary, with a binding methodology change at round 3 (User mandate, 2026-10-05). Compact summary: every item passing through the §11.4.125/§11.4.134/§11.4.142/§11.4.194 review loop carries a round budget `R_max` in the closed range 5–7 (operator-mandated ceiling, verbatim: "all work ALWAYS gets done with 5 to 7 reviews max"; absent declaration resolves to 5); this is NOT the invented-`N` case §11.4.267(E) forbids — the consumer's `R_max` within the operator ceiling IS §11.4.267's `N` for the review loop. Forensic root causes (five, genericised from a reference project running items to 7/10+/30/39+ rounds in one week): (1) wrong-model ground truth — a fix and its OWN verifying gate shared a simplified model of an external tool's real composition path, caught only when a reviewer read the tool's source and rebuilt the fixture on the real path; (2) instance-at-a-time closure of a known class — a security gate hardened one command name per round for three consecutive rounds after a whole-file enumeration was explicitly declined for lack of a classifier; (3) fixer-authored tests that never drive the real code — regression tests re-implementing or text-extracting the guarded logic so the reviewer's own mutations survived the suite; (4) non-binding escalation signals — the heuristic-tower anchor (§11.4.250) invoked ~30 times and the stall anchor's (§11.4.267) undeclared `N` never stopped a loop; (5) scope drift — an out-of-scope tangent absorbed into a review's round counter. (A) declared round budget (a round = one independent review verdict; splitting/renaming does not reset the count except per (E)(3); a spun-out tracked item inherits the parent's consumed count except for an (E)(2)-boundary finding, recurrence-linked per §11.4.214). (B)

ground-truth probe (authoritative source + real-composition-path fixture + recorded behavioural model) required BEFORE the first fix whenever correctness depends on an unowned external system; a gate verified on a simplified model is wrong-layer evidence (§11.4.226) and cannot GO. (C) class-exhaustive enumeration on first instance — name the class, enumerate every member with a control-needled instrument (§11.4.201(7)(b)), close or explicitly scope-out every member; fixing only the reported instance when the class is nameable is itself a violation; building a missing classifier IS the remediation. (D) every remediation-round regression test drives the REAL artifact through its real invocation path (§11.4.224(A)) — a test that re-implements or text-extracts the guarded logic is a `source-defect` (§11.4.235(D)); every reviewer-surviving mutation is adopted verbatim and must fail the suite before the next review; the dispatch seam refuses to START a review missing that record, but a review that WAS dispatched and returned a verdict always counts. (E) at round 3+ the fixer records a convergence assessment; two-or-more findings sharing a (C)-named class (with ≥1 non-MINOR) / a reappearing finding / wrong-layer evidence forces a STRUCTURAL round (fresh fixer instance, re-derive (B), re-run (C) over the whole scope, replace rather than layer) unless an operator decision (§11.4.66) defers it; every review declares its scope/threat boundary before round 1, fixed except by recorded operator decision — narrowing it mid-loop to reach GO is a §11.4.6 fabrication; at `R_max` without GO the item STOPS (never runs `R_max` +1 under the same approach) and escalates to a recorded operator decision (re-scope via §11.4.148 / replace via a structural new item / §11.4.271 waiver for residual LOW only) — parked per §11.4.101/ §11.4.267(F), parking only that work unit while the autonomous loop (§11.4.126) continues everything else, surfaced in every §11.4.97 report until resolved. (F) nothing here lowers §11.4.134's zero-finding/zero-warning GO bar, §11.4.209's substrate, §11.4.235(D)'s finding-layer classification, or §11.4.262's machine-evidence requirement — the budget bounds rounds, not rigor; reaching `R_max` without GO never yields a GO or a silent close. EXTENDS (not restates, §11.4.227) §11.4.134 (re-run obligation now bounded per-approach by this anchor, escalating per (E)(3) at the ceiling) and §11.4.267(E) (a one-sentence carve-out: an operator-mandated ceiling recorded with its own verbatim mandate in its own anchor is the one permitted `N`-bound literal). Honest boundary (§11.4.6): does not promise every item converges within `R_max` — a genuinely hard problem still produces real

findings each round; what it removes is round inflation from re-discovering a wrong model, one member of a known class, a never-real test, an unacted stall signal, or scope drift; (E)(3) converts an open-ended loop into a bounded, evidence-backed operator decision, never a shallower review. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-276-PROPAGATION` (literal `11.4.276`, implemented in this change) + recommended gates `CM-REVIEW-ROUND-BUDGET` / `CM-REVIEW-ROUND3-CONVERGENCE-ASSESSMENT` / `CM-FIX-GROUND-TRUTH-AND-CLASS-INVENTORY` (registered as tracked deferrals OWED-GATE-111..113) + paired §1.1 mutations (round 8 with no operator decision / missing round-3 classification / a point-fix after a same-class trigger / a class closed with no inventory → each gate MUST FAIL; golden-FALSE: GO at round 4 with a clean round-3 assessment fires none). Gate CODE for the three recommended gates = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227). **Canonical authority:** constitution submodule `Constitution.md` §11.4.276. Non-compliance is a release blocker. No escape hatch — no `--unbounded-review-rounds`, `--fix-instance-not-class`, `--defer-ground-truth-probe`, `--narrow-threat-model-to-reach-go`, `--reset-round-count-on-refile`, `--defer-structural-fix-past-round-3` flag.

§11.4.277 — llmctl model validation gate: every model run through llmctl (Vision included) by a project or any of its components / services / submodules (recursively) passes deterministic known-answer validation BEFORE decision-bearing use; decision-bearing use runs with `LLMCTL_VALIDATE_STRICT=1` and gates on `llmctl require-validated <profile>`; models are acquired and run ONLY through the constitution-vendored llmctl (`submodules/llmctl`); a wrong answer means the model or runtime is defective and is investigated, never retried-until-green; llmctl units are host-safe by construction (User mandate, 2026-10-12). Compact summary: (A) UNDETERMINED is never a pass: no decision-bearing use until `llmctl require-validated <profile>` exits 0 (fresh PASS record), run with `LLMCTL_VALIDATE_STRICT=1`; `LLMCTL_VALIDATE_ON_START=0` is explicit-diagnostics only (then `require-validated` refuses unless a fresh PASS record for the same key already exists). Reference gate (llmctl `v3.2.2`, commit `5c3dea5`): exits 0 pass / 1 wrong result / 2 could not determine (75 = pressure refusal, mapped to 2; 98 = port in use, from the `ExecCondition` portcheck skip and, in the v3.2.2 pin, the

onnx server (a llama-server bind failure's exit status is UNKNOWN) — `RestartPreventExitStatus` covers the main process only); FAIL stops the profile, UNDETERMINED stops it only under STRICT; the hook runs on start/switch/auto/enable/restart, and the systemd waiter only when the port registry is active and only for catalog LLM profiles. (B) Acquire and run ONLY through the vendored llmctl; no ad-hoc `llama-server`, hand-written units or manual downloads; catalog SHA256 checked where the catalog carries one (small non-LFS files may have a null sha256, recorded as `none`); credentials per §11.4.10; STRUCTURE (amended 2026-10-21 by the §11.4.28(C) EXTENSION, superseding the 2026-10-13 mutual-reference decision): llmctl is a hub engine and MUST NOT nest this constitution or any own-org repository; it resolves the canon via the single §11.4.28(C) resolver; `CM-HUB-ENGINE-NO-OWNORG-NESTING` enforces that, `recursive_init: false` stays in `helix-deps.yaml`, and `CM-LLMCTL-SUBMODULE-NONRECURSIVE` stays as a narrower re-entry check (unresolvable pin = BLIND); honest boundary: the llmctl commit recorded at amendment time still nested `constitution` and `submodules/containers`; measured 2026-10-21 with the llmctl pin at `f42bad6` (staged gitlink; it nests only third-party `llama.cpp` and `colibri` and resolves the rest via `lib/resolve_deps.sh`), both gates PASS for llmctl. (C) Vision: screenshots with KNOWN content and a machine-readable expected-answer manifest prepared IN ADVANCE (manifest commit precedes the first run), negative controls, temperature 0 / fixed seed / top-k 1, a mechanical self-validated judge (never a model), determinism repeats (reference value one; consumers may require more); the reference suite `vision-golden-v1` is 11 generated fixtures (9 golden carrying 16 probes, 2 golden-bad controls carrying 2 control-self probes, cross-checked by 2 discrimination controls), 1 determinism probe, 19 judge self-test entries reported separately (the recorded run's 21 = 16 + 2 + 2 + 1), and claims nothing about CJK/RTL or real-application screens. (D) A wrong answer is a model/quant/ `mmproj` /template/ engine defect: §11.4.102 root cause + §11.4.115 RED baseline, change the cause, re-validate from clean; no retry-until-green, re-seed, temperature raise, threshold lowering or manifest edit after a failure. (E) Host safety (real incident 2026-10-12: `Restart=always`, `RestartSec=5`, `MemoryMax` = RAM, 8 OOM kills in about an hour, ~3.3 GB re-read per restart, load ≈160, mem PSI full 97–99 %, forced power-off); mechanisms mandatory, numbers are llmctl `v3.2.2` reference values and HOST-DEPENDENT (stricter allowed, looser

never): `Restart=on-failure` with backoff, `StartLimitBurst=3` / `StartLimitIntervalSec=900` in `[Unit]`, `RestartPreventExitStatus=SIGKILL SIGTERM 98` (OOM/SIGKILL never auto-restarted, main process only; `Restart=always` forbidden); per-profile (engine unit) `MemoryHigh` 1.25x / `MemoryMax` 1.5x footprint with floors (1024/1280 MiB; 5120/6144 MiB for unmeasured GPU-mode profiles), always below 90 % of RAM, `MemorySwapMax=0`; refuse to start an ENGINE unit (exit 75, no partial start; the unit gate waits up to 120 s for pressure to clear first) at memory PSI some avg10 > 10 or full avg10 > 2 or `MemAvailable` < footprint + 4 GiB, also gating restarts and boot (`switch` runs a PSI-only pre-stop gate and then the normal footprint gate for its target; the decision-gateway unit has no such gate and is capped at 20 %/25 % of RAM); co-residency is a live RAM/VRAM budget, not a count; measured data point: one host, run 2 of 2, `vision` profile peak 3005 MiB (committed in llmctl `v3.2.2` as `docs/qa/vision-live-2026-10-13-run2/`; one host, not a constant). (F) Evidence: llmctl keeps a 0600 evidence log and a JSON record per profile; NOT hash-chained and no host-class fingerprint in `v3.2.2` — consumer obligations (§11.4.268), UNKNOWN until built. (G) Other kinds: llmctl ships chat/coder/embedding suites; any kind without a closed-form oracle is `UNKNOWN` and stays unvalidated. (H) The record binds catalog model SHA256 list (incl. `mmproj`), file size/mtime, engine version line, launch fingerprint (argv minus host/port) and suite content hashes, and expires after `LLMCTL_VALIDATION_MAX_AGE` (reference 7 days); NOT bound: host class, driver/CUDA, OS/kernel, build flags beyond the version line, external chat template — consumers bind those or record the gap; any change voids the record. (I) Validation is per host; `llmctl validate --host URL` is evidence only; remote hosts via SSH keys, never passwords. (J) Consumer data: model inventory, extra fixtures, repeats, tightened thresholds, record age, remote hosts. Evidence honesty: v3.2.2's Vision evidence is TWO runs on ONE host of the `vision` profile (each pass 21 / fail 0 / undetermined 0 = 16 golden + 2 control-self + 2 discrimination + 1 determinism; judge self-test 19/19 separately); `vision-pro` and every other profile are not validated. Gates: `CM-COVENANT-114-277-PROPAGATION` (implemented) + implemented `CM-LLMCTL-SUBMODULE-NONRECURSIVE` + deferred `CM-LLMCTL-VALIDATED-BEFORE-DECISION-USE` / `CM-VISION-KNOWN-ANSWER-FIXTURES` / `CM-MODEL-ANSWER-COMPARATOR-SELF-VALIDATED` / `CM-MODEL-DETERMINISM-PINNED` / `CM-MODEL-WRONG-ANSWER-NOT-RETRIED` / `CM-`

`LLMCTL-ONLY-ACQUISITION` / `CM-LLMCTL-UNIT-HOST-SAFE` / `CM-LLMCTL-PREFLIGHT-PRESSURE-ABORT` / `CM-MODEL-VALIDATION-KEY-COMPLETE` / `CM-MODEL-VALIDATION-EVIDENCE`. Composes §11.4.5/.6/.10/.26/.28/.31/.74/.102/.107/.115/.117/.160/.163/.262/.268 and §12.3/.6/.12. Classification: universal (§11.4.17). Canonical authority: `Constitution.md` §11.4.277. No escape hatch.

§11.4.278 — Mandatory HelixQA UI QA with Vision eyes: every project with a UI on ANY platform or OS passes HelixQA full suites over all screens, flows, use cases and edge cases, driven and read by a Vision model validated under §11.4.277 that recognises the real content of every screen / page / component / widget, against deterministic expected-content oracles, with no skip flags and a per-frame evidence path for every verdict (User mandate, 2026-10-12). Compact summary: (A) every shipped platform/surface (web, desktop, mobile, TV, installer wizard, kiosk, TUI and so on) is covered; an un-drivable surface is recorded `structurally_impossible` / `topology_unsupported` / `operator_attended` with a tracked item, never omitted. (B) A closed, machine-readable screen-and-flow catalogue (screens, component × state, flows, edge cases); 100 % of non-waived entries read-verified; the catalogue is cross-checked against the build's real route/view inventory. (C) HelixQA drives: UI-driven and intent variants on the real artifact on a clean target, record → read → score, the §11.4.160 frame bridge, ALL test types (§11.4.169) — vision is added, never substituted. (D) The eyes are a Vision model with a fresh `llmctl require-validated <profile>` PASS under `LLMCTL_VALIDATE_STRICT=1` (§11.4.277); otherwise the vision step refuses and its checks are `not_validated`, and the §11.4.277(E) host-safety limits (host-dependent reference values) apply. (E) Per frame: screen identity, component/widget inventory and state, the actual content, health (no clipping, placeholder or frozen frame), consistency with backend data; plausibility is a FAIL. (F) The expected-content spec is written BEFORE the run; oracle order is accessibility tree / DOM > deterministic OCR/template > validated Vision (a `model-decided` assertion needs a corroborating second angle); a mechanical, self-validated comparator. (G) No skip flags, partial suites or silent skips; the only non-PASS outcomes are `FAIL`, `not_validated`, `topology_unsupported`, `operator_attended`, `structurally_impossible`, `waived`. (H) Per-frame evidence: PNG +

SHA256, OCR, Vision record, spec, matched/unmatched patterns, validation-record id, MP4 segment; machine-created, credentials redacted, no blind typing (§11.4.193). (I) An escape triggers the §11.4.138 audit, a §11.4.277(H) re-validation review and a §11.4.135 regression-bank entry. (J) Artifacts: catalogues, banks/challenges with §1.1 mutations, specs, the §11.4.277 record, the coverage ledger, manuals/guides/FAQ, diagrams, README badges, translations. (K) Adoption as data by each consumer. Status: PROPOSED (operator decision 2026-10-13: ship as proposed, evidence PENDING) — no HelixQA/consumer end-to-end evidence exists yet; a real HelixQA bank run through llmctl `vision` with STRICT + `require-validated` is a separate future work item and every mechanism gate below is a deferral. Gates: `CM-COVENANT-114-278-PROPAGATION` (implemented) + deferred `CM-HELIXQA-UI-SUITE-PER-PLATFORM` / `CM-UI-CATALOGUE-COVERAGE-100` / `CM-UI-CATALOGUE-MATCHES-BUILD-INVENTORY` / `CM-HELIXQA-VISION-VALIDATED` / `CM-UI-EXPECTED-CONTENT-SPEC-BEFORE-RUN` / `CM-UI-VERDICT-PER-FRAME-EVIDENCE` / `CM-UI-SCREEN-IDENTITY-NOT-PLAUSIBILITY` / `CM-UI-NO-SKIP-FLAGS` / `CM-UI-EDGE-CASE-MATRIX-COVERED` / `CM-UI-ESCAPE-TRIGGERS-REVALIDATION` / `CM-UI-QA-DOCS-DIAGRAMS-SYNC` / `CM-UI-QA-READINESS-GATE`. Composes §11.4.27/.52/.107/.117/.118/.158/.159/.160/.163/.193/.238/.262/.277. Classification: universal (§11.4.17). Canonical authority: `Constitution.md` §11.4.278. No escape hatch.

§11.4.279 — Decision-model detection-first mandate (User mandate, 2026-10-07). Compact summary: every project using the constitution MUST detect available decision models BEFORE the first generative-LLM invocation of a work item — `llmctl plan --json` (local `decision_instances`) plus `claude-providers sync-jev` (ranked endpoint cache: local decide gateway, LAN pins, OpenRouter, TypeSafe-hosted Jev) — and persist `jev-detection.json` (endpoints, model IDs, probe timestamps, absence reasons); no hosted network I/O at work-item time (TTL cache + local plan only); zero endpoints MUST yield an explicit `{"endpoints": [], "reason": "…"}` record (silent absence is a §7.1 bluff); §11.4.141 token claims not referencing the session's `jev-detection.json` are non-compliant. Classification: universal (§11.4.17); keys file, llmctl host and evidence path are DATA (§11.4.35). Gates: `CM-COVENANT-114-279-PROPAGATION`,

`CM-JEV-DETECTION`. **Canonical authority:** `Constitution.md` §11.4.279. Non-compliance is a release blocker. No escape hatch.

§11.4.280 — Per-agent decision-layer installation mandate (User mandate, 2026-10-07). Compact summary: for every agent platform detected on the host (Claude Code, Kimi Code, pi, opencode, and any platform SUPPORTED in the §11.4.228 compatibility matrix) the project MUST install the decision-layer artifacts that platform's verified mechanics allow — PreToolUse/UserPromptSubmit hook (`jev-gate.sh`), MCP registration (`jev_mcp.py`, `mcp/jev-decisions.json`), provider-alias env exports, plugin commands (`plugins/helix`), skill `skills/jev-decision-layer/` — via the per-agent installer `jev-setup.sh` invoked by the §11.4.281 wizard; install/update/removal follows §11.4.228 and §11.4.272; any call class a platform cannot intercept MUST be stated verbatim as a coverage gap (claiming full coverage is a §7.1 bluff). Gates: `CM-COVENANT-114-280-PROPAGATION`, `CM-JEV-WIRING`. **Canonical authority:** `Constitution.md` §11.4.280. Non-compliance is a release blocker. No escape hatch; uninstall is a logged operator act (`jev-setup --uninstall`).

§11.4.281 — Bootstrap wizard and post-update auto-trigger (User mandate, 2026-10-07). Compact summary: `scripts/jev/bootstrap.sh` is the single master wizard (detect → provision llmctl + claude_toolkit → detect decision models → install per-agent artifacts → configure → calibrate in shadow → enforce); interactive on a TTY, fully automatable with `--non-interactive` and `JEV_BOOTSTRAP_*` env (non-TTY without the flag is an error, never a hang); `scripts/post_update_hook.sh` MUST run it on every constitution pull that changes the Jev extension files — guarded (opt-out marker), idempotent (second run is a byte-stable no-op), deferrable (`--defer` writes a pending stamp); CLI-agent triggers `/jev_bootstrap` and `/jev_status` come from `actions/registry.yaml` rows (data, not code). Gates: `CM-COVENANT-114-281-PROPAGATION`, `CM-JEV-BOOTSTRAP-IDEMPOTENT`. **Canonical authority:** `Constitution.md` §11.4.281. Non-compliance is a release blocker. No escape hatch; deferral is bounded and logged.

§11.4.282 — Maximal decision-instance utilization (User mandate, 2026-10-07). Compact summary: decision capacity is COMPUTED from `llmctl plan --json` `decision_instances` and recorded in `jev-`

`detection.json`; when more than one decision endpoint or instance exists, decision traffic (gates, routing, scoring, calibration probes) MUST be distributed across them — local first, then LAN pins, then hosted — bounded per endpoint by the profile's `parallel` slots; count 1 or 0 is recorded with its reason and the §11.4.231 ladder proceeds unchanged; §9/§12 host-safety budgets are never overridden (headroom is the only legitimate limiter). Gates: `CM-COVENANT-114-282-PROPAGATION`; capacity fields via `CM-JEV-DETECTION`. **Canonical authority:** `Constitution.md` §11.4.282. Non-compliance is a release blocker. No escape hatch.

§11.4.283 — Decision-layer token-reduction proof (User mandate, 2026-10-07). Compact summary: every incorporating project MUST extend its §11.4.141 BEFORE/AFTER harness with a decision-layer arm — BEFORE = frozen workload with `JEV_GATE=off`, AFTER = the identical workload with the layer active, accounted on provider `usage` fields; pass = AFTER ≤ BEFORE (or the project's declared floor, DATA per §11.4.35) AND `decisions.jsonl` shows ≥1 decision served per gated tool-call class, with the §11.4.141 aggregate bar unchanged; `jev-stats.sh` figures are ESTIMATES, never a substitute for the measured harness. Gates: `CM-COVENANT-114-283-PROPAGATION`, `CM-JEV-TOKEN-REDUCTION`. **Canonical authority:** `Constitution.md` §11.4.283. Non-compliance is a release blocker. No escape hatch — no `--skip-token-efficiency` flag exists.

§11.4.284 — Decision layer as pre-ladder routing stage (amends §11.4.231) (User mandate, 2026-10-07). Compact summary: the decision layer runs BEFORE the §11.4.231 ladder — a judgment expressible as a typed decision (yes/no, choice, score) MUST be answered by a decision model, and only generative work enters the ladder; the closed ladder (`sonnet < opus`; `fable` not a rung; haiku context-floor prohibition) is neither extended nor weakened and decision models are not a rung, tier or chat model; below its §11.4.285 threshold a decision — and only that decision — escalates to the lightest capable rung; wiring a decision endpoint into any chat-model/provider configuration is a critical defect. §11.4.231 carries the dated amendment paragraph. Gates: `CM-COVENANT-114-284-PROPAGATION`; `CM-JEV-WIRING` asserts no decision endpoint in a chat-model config. **Canonical authority:** `Constitution.md` §11.4.284. Non-compliance is a release blocker. No bypass flag; zero endpoints is lawful only via the §11.4.279 detection record.

§11.4.285 — Traditional-LLM auto-tuning of decision thresholds, shadow-first (User mandate, 2026-10-07). Compact summary: the layer starts in shadow mode (decisions logged to `decisions.jsonl`, state hash only — never raw state); weekly and on demand (`jev-calibrate.sh --now`) a deterministic sample per question type is labeled by the project's main LLM (lightest capable §11.4.231 tier) and per-type thresholds are written to `jev-thresholds.json` (DATA); a type enters enforce mode ONLY when calibrated agreement meets the declared floor (absent: 0.90) and the sample meets the declared minimum (absent: 30); constitution updates and decision-model version changes trigger recalibration. Gates: `CM-COVENANT-114-285-PROPAGATION`; thresholds schema via `CM-JEV-WIRING`. **Canonical authority:** `Constitution.md` §11.4.285. Non-compliance is a release blocker. No escape hatch — `JEV_MODE=shadow` is the only alternative to calibrated enforcement.

§11.4.286 — llmctl and claude_toolkit as dependency submodules (User mandate, 2026-10-07). Compact summary: `llmctl` and `claude_toolkit` MUST be git submodules at `submodules/llmctl` and `submodules/claude_toolkit`, pinned in `helix-deps.yaml` (the single source of pin truth, §11.4.31); `scripts/jev/bootstrap.sh` installs/updates them from the checkouts by invoking each tool's own installer (never reimplementing it) and falls back to a managed clone at the same pinned refs; a pin advance marks the bootstrap pending (§11.4.281). Gates: `CM-COVENANT-114-286-PROPAGATION`; provenance record (path, ref, source) via `CM-JEV-DETECTION`. **Canonical authority:** `Constitution.md` §11.4.286. Non-compliance is a release blocker. No escape hatch — an unpinned, unprovenanced install is non-compliance.

EXTENSIONS landed 2026-10-21 (existing anchors strengthened — full text in `Constitution.md`; owner mandates of 2026-10-21). §11.4.28(C) — one source of truth, resolver instead of nested copies: an owned submodule (a hub engine included) MUST NOT declare as a nested submodule the constitution, the hub or any hub engine, or any owned repository the parent already provides flat; a nested `constitution` inside a hub engine is the canonical forbidden case (mutual reference + a second drifting canon copy). The engine finds the canon and its siblings with ONE documented resolver (env override -> parent hub at `<hub>/submodules/`

`<name>` -> `<project_root>/constitution` and `<project_root>/submodules/<name>` -> sibling -> `find_constitution.sh` host fallback), fails LOUDLY when nothing resolves, and the version of a shared dependency is the parent's pin (latest by default). Third-party repositories an engine really builds against stay legal nested submodules (third-party, never initialised recursively). Gate `CM-HUB-ENGINE-NO-OWNORG-NESTING` (`scripts/gates/cm_hub_engine_no_ownorg_nesting.sh` , IMPLEMENTED with a paired mutation test) reads each checked-out engine's `.gitmodules` at the recorded (or staged) gitlink and FAILs by NAME MATCHING on nested URLs (owner `vasic-digital` / `HelixDevelopment` , or a repository named `constitution` / `HelixConstitution` under any owner); exit 0 PASS / 1 FAIL / 2 BLIND / 3 SKIP (nothing inspected); it runs on every pull from `scripts/post_update_hook.sh` and generalises `CM-LLMCTL-SUBMODULE-NONRECURSIVE` . **§11.4.162 — OpenDesign:** consumed as a project dependency and never forked; the hub MAY carry ONE pinned REFERENCE checkout at `constitution/submodules/open_design` (third-party, Apache-2.0, `third_party: true` in `helix-deps.yaml` , excluded from the index/coverage/own-org review, never a product's consumption path); `design-toolkit` carries the token/brand/catalogue mechanisms. **§11.4.278(L) — Vision model identity and tiers:** the consumer declares an ordered ladder of at least two validated Vision tiers; EVERY verdict, per-frame record and run summary carries the model identity (llmctl profile, tier, model + `mmproj` sha256, quantisation, engine/version, launch fingerprint, validation-record id and age); a lower-tier PASS is never presented as a higher-tier one; recommended gate `CM-VISION-MODEL-IDENTITY-RECORDED` (OWED-GATE-143). Classification: universal (§11.4.17). No escape hatch — no `--nested-constitution-in-engine` , `--copy-instead-of-resolve` , `--vision-verdict-without-model-identity` , `--unlisted-vision-tier` flag.

EXTENSION landed 2026-10-21 — §11.4.31 a tracked `helix-deps.yaml` in EVERY own-org repository, one canonical engine, one gate (full text in `Constitution.md`). This carrier cites §11.4.31; it is not a second block-start for it (§11.4.227(B)). Operator mandate (2026-10-21, relayed, not verbatim): every repository MUST carry the list of its submodule dependencies in the §11.4.31 format at the expected locations (`<root>/submodules` , `<root>/constitution/submodules`), systematically, for every existing and new project. (A) A tracked `helix-deps.yaml` MUST exist at the root of the project

root (always), of every own-org repository the root or the `constitution` hub declares in `.gitmodules`, and of every checked-out own-org repository under `<root>/submodules/*` or `<root>/constitution/submodules/*`; own-org = remote organisation in the consumer's own-org list (DATA, default `vasic-digital`, `HelixDevelopment`); a third-party repository is `SKIP(third_party)`, never PASS; an untracked manifest does not count (§11.4.215). (B) Closed, strings-only schema: `schema_version: 1`, `deps` (list, `deps: []` when empty), `transitive_handling` (`recursive`, `conflict_resolution: operator-required`), `language_specific_subtree`; each dep `name`, `ssh_url` (`git@host:org/repo.git`), `ref`, `why`, `layout` (`flat` | `grouped` | `hub`), optional `path`, `recursive_init`, `third_party` (+ required `licence`); a non-own-org dep MUST be `third_party: true`; unknown/duplicate keys, duplicate names/URLs and UNKNOWN/TODO/TBD/FIXME placeholders are defects. (C) Root and hub declare every own-org submodule they carry (the hub itself is inherited by reference, not listed); a leaf nests no own-org or hub repository (§11.4.28(C)) — with `recursive_init: false` on the parent entry it is reported `KNOWN`, owned by `CM-HUB-ENGINE-NO-OWNORG-NESTING`. (D) Canonical engine `scripts/deps_manifest/verify_helix_deps_manifests.sh` (`--root`, `--validate-file`, `--scaffold <dir> [--write]` never overwriting and emitting placeholders the verifier rejects, `--selftest`), consumed BY REFERENCE (§11.4.28 / §11.4.177); a project script may only be a thin wrapper; exit 0 PASS / 1 FAIL / 2 BLIND / 64 usage. (E) Bootstrap: after adding the constitution, `--scaffold . --write`, fill every placeholder, `--validate-file`, `git add`; every own-org submodule add/remove/re-pin updates its `deps` entry in the same commit; a missing manifest is added in ITS OWN repository. (F) Gate `CM-HELIX-DEPS-MANIFEST-PRESENT` (`scripts/gates/cm_helix_deps_manifest_present.sh`, IMPLEMENTED with a paired mutation test) runs on every constitution pull from `scripts/post_update_hook.sh` against the consumer `PROJECT_ROOT`. Docs: `docs/guides/dependency_manifest_user_guide.md`, `docs/manuals/dependency_manifest_manual.md`, `docs/faq/dependency_manifest_faq.md`. Honest boundary (§11.4.6): presence, form and own-org coverage only — ref-vs-gitlink drift stays `CM-DEPENDENCY-POINTER-PREFLIGHT-FAIL-CLOSED` (§11.4.233(G)); measured at

landing on the constitution's own tree: pass 9 / fail 2 (`claude_toolkit` has no manifest; `llmctl` lists `superspec` without `third_party: true`) / skip_third_party 17, both owed upstream; re-measured with the llmctl pin at `f42bad6` (superspec marked `third_party: true`): pass 10 / fail 1 / skip_third_party 17; re-measured 2026-10-22 with the `claude_toolkit` pin at `1f6dad3` (flat, no own-org nests, a tracked `helix-deps.yaml`): pass 11 / fail 0 / blind 0 / skip_third_party 17 / known 0, and `CM-HUB-ENGINE-NO-OWNORG-NESTING` PASS for every hub engine. Classification: universal (§11.4.17). No escape hatch — no `--skip-manifest-check` , `--manifest-optional` , `--third-party-as-pass` , `--scaffold-overwrite` flag.

EXTENSION landed 2026-09-24 — §11.4.78 / §11.4.79 / §11.4.80 indexing mechanism is consumed BY REFERENCE; the consumer supplies only DATA (full text in `Constitution.md`). This carrier cites the three anchors; it is not a second block-start for any of them (§11.4.227(B)). **§11.4.78(7)-(10):** the scope renderer (`<constitution>/scripts/codegraph/scope_render.py` + `scope_enumerate.js` + `scope_baseline.txt` + template `scope.example.yaml`), the single writer entry (`codegraph_safe.sh` + helper/guard/disk tripwire), the version-keyed runner patcher (`fk_index_patch.py` + `runner_patches/`), the hazard probe (`fk_cascade_probe.py`), the progress-proven watchdog (`index_watch.py`), the scope proof (`codegraph_scope_guard.py`) and `scripts/lumen/gen_lumenignore.py` MUST live in the constitution submodule and be invoked BY REFERENCE (§11.4.28 / §11.4.80 / §11.4.177) — a consumer copy/fork/hand-rolled equivalent is a violation; each tool MUST be documented (§11.4.18) and test-first with paired mutations, and an untested tool never touches a live index — at the time of writing `index_watch.py` and `fk_cascade_probe.py` (no test references of their own — the writer entry's unit test substitutes a stub for the probe), the runner patches `lockfix1` and `datafrag1` (no tests of their own — excluded from the default patch set for that reason), and the §11.4.18 companion guide of EVERY tool named here (none exists yet) are OWED, tracked per §11.4.197; the consumer supplies ONE scope DATA file (own-org list, project rubbish, pathological inputs, re-include negations, reasoned class overrides, accepted file count + tolerance, efficiency thresholds) and never drops a baseline class; scope truth = the config the INSTALLED CLI version actually loads (from its own loader, §11.4.99), a config the CLI does not read is dead documentation, the effective

scope is GENERATED (hand edits refused, silent fall-back = FAIL); before "ready" the scope guard MUST enumerate through the runner's own discovery code and assert count within tolerance, 0 third-party files, 0 secret-class files, ≥1 file per own-org root, each control-needled — database size is NEVER evidence. Where this conflicts with the older clause-2 filter wording, the extension governs. **§11.4.79(6)-(8):** own-org vs third-party is DERIVED by walking `.gitmodules` recursively and matching remote-URL organisation against the consumer own-org DATA; third-party code nested inside an own-org submodule is third-party; a gitlink change without re-render makes the render stale and the guard FAILs; steps 2–3 now mean "edit scope DATA + re-render + re-index only through the single writer entry". **§11.4.80(4)-(8):** upgrades are gated on a compatibility probe (loader config filename/keys, bulk-window hazard probe, runner-patch applicability) and REFUSE loudly keeping the prior version when the patched runner cannot be re-proven (§11.4.201); a runner patch without its own RED→GREEN test + paired mutation + byte-copy equivalence proof never touches a live index; every init/index/sync goes through the ONE writer entry, which MUST refuse while any live process targets the root (real cmdline §11.4.196(D), whatever the lock's age), classify BULK by PENDING backlog never by stored rows, and run bulk only on the proven patched runner, detached, watchdog- and tripwire-supervised; long runs are progress-proven (§11.4.232(C)) — alive-but-not-advancing is a STALL; cadence is a real timer/hook + constitution-pull hook (§11.4.164) with a ledger entry each run, a missing validator = FAIL; the live `codegraph help` diff is authoritative over the embedded list, and telemetry MUST be off (per-host FACT, captured not assumed). The literals `11.4.78` / `11.4.79` / `11.4.80` are unchanged; no `CM-COVENANT-114-78/79/80-PROPAGATION` gate exists (none in `constitution/scripts/gates/`), so no propagation gate is claimed for them.

EXTENSION landed 2026-09-25 — §11.4.78 host-adaptive V8 heap budget + §11.4.275(D) benchmark run, both live-verified on this project (full text in `Constitution.md`). §11.4.78's item-7 single writer entry `codegraph_safe.sh` now computes its Node.js heap budget from measured `MemAvailable` (never hardcoded, §12.11/§11.4.6) instead of inheriting V8's roughly-4-GiB stock default — closing a real bulk-index OOM crash (rc=134) on this repo's 584K-file/49M-edge tree; landed with 2 new regression tests + live `/proc/<pid>/environ` verification + a zero-regression full-suite comparison

(commit `941841d3e7235a081e3199a4d3073b409d41753b`). §11.4.275(D)'s deterministic benchmark ran via the now-in-tree `constitution/scripts/lumen/lumen_verify.sh` (superseding this anchor's own note that the harness lived outside the constitution tree — it now has its own test file): a pre-declared 13-query golden set, N=3 byte-identical runs, recall 9/11=0.818, the 2 failing/unsupported query classes (`.sh` / `.kt`, unindexed by Lumen's chunker — confirmed via direct `sqlite3` + Lumen's own source, not guessed) reported honestly and tracked, never averaged away (commit `ac532a21df78a447c2aceec30f63476348e93077`, `docs/research/lumen_benchmark_20260925/RESULTS_v2.md`). Gate code for both anchors remains owed, unchanged (§11.4.6/§11.4.227).

EXTENSION landed 2026-09-25 (round 2) — §11.4.78 heap ceiling corrected to the REAL §12.6 60%-of-total-RAM bound (was an invented 64 GiB) + CodeGraph now wired as a genuine, project-tracked MCP tool via a new incident-hardened wrapper (full text in `Constitution.md`). Direct operator mandate mid-session: "do not set caps for codegraph and lumen … indexed space exposed and used by all agents" — audited, and both were genuinely under-delivering. (1) The earlier round's 64 GiB heap cap was an INVENTED ceiling (~25% of this 251 GiB host's total RAM), not a derivation of §12.6; fixed to `MemTotal * 60/100` computed fresh every run — the real, unweakened §12.6 bound, mathematically almost never binding an ordinary run (commit `8a5f47ab2e0e7a05d1c1652ba5a3475751652062`, new regression test U46). (2) Audited further and found `.mcp.json` was completely EMPTY — CodeGraph had ZERO agent-facing MCP presence in this repository despite shipping a full query toolset (`codegraph_explore` / `codegraph_node` /etc.) and its own installer; Lumen's own "prefer this over grep" nudge was found to be an UNTRACKED Claude Code PLUGIN hook, not anything this repository owns — neither tool's use was actually guaranteed for any future clone/agent/track. Fixed by registering `.mcp.json`'s `codegraph` server against a new, tested, `§11.4.177`-decoupled wrapper (`constitution/scripts/codegraph/codegraph_mcp_serve.sh`) that resolves the SAME validated patched runner `codegraph_safe.sh` itself uses (never bare stock-on-PATH). **A separate, more serious forensic incident surfaced while building this**: `serve --mcp`'s file watcher WRITES (auto-syncs) the database and the server daemonizes past its own parent's death — a single manual test against this project's own live checkout spawned a stray daemon that collided with an

in-flight bulk `sync`, crashing it with "database is locked" (a further ~85 minutes of progress lost, on top of the earlier heap-crash loss); recovered via a targeted `REINDEX` (3 rows, confirmed non-corrupting) + a bulk `sync` resume. Durable fix: the wrapper now ALWAYS passes `--no-watch` (never optional), making it a true non-writing reader safe to run permanently — a fact PROVEN, not assumed, by a real init'd scratch-project DB-mtime check (unchanged after a tracked-file edit with the server running). Six new regression tests land (`tests/test_codegraph_mcp_serve.sh`), including the incident itself replayed as a repeatable test (T04); two unrelated bugs found and fixed during this same investigation before landing (a `.sh`-stub force-fed to `python3`; a submodule-vs-parent project-root resolution bug). Honest boundary (§11.4.6): `.mcp.json` registration takes effect on a fresh session — this session could not self-restart to verify the MCP tools are end-to-end callable from inside it; that check is a tracked §11.4.197 follow-up, not claimed here. Gate code for all of the above remains owed, unchanged (§11.4.6/§11.4.227).

EXTENSION landed 2026-10-08 — §11.4.54 / §11.4.248 ticket-id key is per-project DATA (full text in `Constitution.md`). This carrier cites the two anchors; it is not a second block-start for either (§11.4.227(B)). The ticket-id KEY is per-project DATA, not the literal `ATM`: The spelling `ATM-NNN` in this corpus (the `[ATM-NNN]` heading token, the `[PROTECTED-SPEC: ATM-NNN]` tag, the three gate contracts `CM-ATM-TICKET-IDS-COMPLETE` / `CM-ATM-TICKET-IDS-UNIQUE` / `CM-ATM-TICKET-IDS-MONOTONIC`, the state-file field `atm_id`) is the constitution's canonical NOTATION and the legacy key of the project that originated the rule; read it as `<KEY>-NNN`, where `<KEY>` is the consuming project's own declared key (`ATM` remains the default notation). Concrete historical ids (for example `ATM-627`, `ATM-1117`) are NOT rewritten. A register's key MUST be DECLARED as consumer data (the `id_prefix` key already carried by the reporting/feature config, §11.4.202/§11.4.213, passed to `workable-items add --prefix`) and then used consistently: monotonic, unique, never renumbered, no gaps are per key; the engine's derived default (key from the §11.4.151 release prefix via `defaultKeyPrefix()` in `scripts/workable-items/cmd/workable-items/prefix.go`, last resort `WIT`) only BOOTSTRAPS a key and depends on the clone directory name and `HELIX_RELEASE_PREFIX`, so it is NOT a stable register key by itself. Switching a live register to a new key is a §11.4.66 operator decision that preserves every existing id. MEASURED ENGINE FACTS (WF26 independent review,

2026-10-08, `specs/001-full-project-audit-remediation/evidence/wp07/constitution-0108/` and its reproduction log `specs/001-full-project-audit-remediation/evidence/wp07/constitution-0108/fix-r2-exp6-reproduction.txt` — a file of the CONSUMER repository evidence pack, tracked with the consumer change and NOT part of this submodule tree, so the path resolves only in that consumer checkout; every figure below was RE-MEASURED first-hand 2026-10-12 with an id-level dump): the happy path is key-generic (with no `--prefix`, in a clone directory named `catalogizer`, the derived default `CAT` minted `CAT-001..003`; in a directory named `other-proj` it was `OTH`), and the heading parser accepts `^## [A-Z]{3}-[0-9A-Za-z]+` (`parse.go:21`) and the bracket form `\[[A-Z]{2,}-\d+\]` (`parse.go:32`); BUT the engine does NOT enforce a key grammar at mint or validate time: `add --prefix` minted `cat-001`, `CATS-001`, `C1T-001` and `AB-001`; `validate` reported "OK - 7 items" across 5 different keys (`CAT`, `cat`, `CATS`, `C1T`, `AB`); and a db to markdown to db round trip silently dropped 2 of the 7 items (`cat-001`, `CATS-001`), ending with 5 items and `validate` again reporting OK. These are OWED engine work (mint/validate-time key-grammar validation plus one-key-per-register validation), tracked per §11.4.197, not done here and not claimed fixed. The legacy NAMES `atm_id`, `new_atm_id`, `head_atm_id` (DB columns, Go fields, state-file field) and the `ATM-DERIVED-<hash>` legacy-import marker are NOT renamed: measured 2026-10-08, `atm_id` occurs in 67 Go files (20 non-test) under `scripts/workable-items` (68 with `AtmID` / `atmID`; 70 files of any type), and a rename (for example to `ticket_id`) needs a schema migration plus a compatibility view, so it is OWED engine work too. Whether the engine's mechanical last-resort key should be `ATM` instead of `WIT` is an owner decision (UNCONFIRMED). This EXTENSION spells out the three existing gate names in full and introduces no new CM-* token (ledger unimplemented count measured before and after the change, identical; see the evidence pack). PROVENANCE (§11.4.26 step 2, stated truthfully): the owner approved, in an interactive question on 2026-10-08 (relayed, not verbatim; recorded in the audit-remediation memory file `project_owner_decisions_2026_10_07.md`), that the agent commits this change set to the constitution repository fast-forward only; the decision ledger entry ODG-11 still records that the constitution wording needs separate

approval, so the WORDING of this EXTENSION is PENDING the independent review and owner sight.